

Undergraduate Topics in Computer Science

Peter Csaba Ölveczky

Designing Reliable Distributed Systems

A Formal Methods Approach Based on
Executable Modeling in Maude



 Springer

The Springer logo consists of a stylized chess knight piece facing left, positioned to the left of the word "Springer" in a serif font.

Undergraduate Topics in Computer Science

Series editor

Ian Mackie

Advisory Board

Samson Abramsky, University of Oxford, Oxford, UK

Karin Breitman, Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro, Brazil

Chris Hankin, Imperial College London, London, UK

Dexter C. Kozen, Cornell University, Ithaca, USA

Andrew Pitts, University of Cambridge, Cambridge, UK

Hanne Riis Nielson, Technical University of Denmark, Kongens Lyngby, Denmark

Steven S. Skiena, Stony Brook University, Stony Brook, USA

Iain Stewart, University of Durham, Durham, UK

Undergraduate Topics in Computer Science (UTiCS) delivers high-quality instructional content for undergraduates studying in all areas of computing and information science. From core foundational and theoretical material to final-year topics and applications, UTiCS books take a fresh, concise, and modern approach and are ideal for self-study or for a one- or two-semester course. The texts are all authored by established experts in their fields, reviewed by an international advisory board, and contain numerous examples and problems. Many include fully worked solutions.

More information about this series at <http://www.springer.com/series/7592>

Peter Csaba Ölveczky

Designing Reliable Distributed Systems

A Formal Methods Approach Based
on Executable Modeling in Maude

Peter Csaba Ölveczky
University of Oslo
Oslo
Norway

ISSN 1863-7310 ISSN 2197-1781 (electronic)
Undergraduate Topics in Computer Science
ISBN 978-1-4471-6686-3 ISBN 978-1-4471-6687-0 (eBook)
DOI 10.1007/978-1-4471-6687-0

Library of Congress Control Number: 2017947868

© Springer-Verlag London 2017

The author(s) has/have asserted their right(s) to be identified as the author(s) of this work in accordance with the Copyright, Designs and Patents Act 1988.

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Printed on acid-free paper

This Springer imprint is published by Springer Nature
The registered company is Springer-Verlag London Ltd.
The registered company address is: The Campus, 4 Crinan Street, London, N1 9XW, United Kingdom

To Cecilia, Roland, and Robert

Foreword

De facto, both individually and socially, all of us rely more and more on software-mediated systems and devices. However, as software disasters and successful cyber-attacks keep piling up, the crucial importance of software quality and reliability, and the sobering realization of how vulnerable our systems are, loom larger and larger. In areas such as avionics, railway systems, microprocessor design, and security protocols, the obvious consequence, namely, the need for *mathematical* methods providing high assurance beyond the insufficient assurance made possible by testing alone is well understood, so that formal methods are applied in practice in such areas. But this is far from being the case in general. In particular, since most systems nowadays are *distributed* systems, which are very hard to test and can have very subtle bugs, the necessary but insufficient role of testing is painfully felt; but the obvious need for stronger verification methods beyond testing is still not fully understood or appreciated in practice.

An important question is why this highly problematic state of affairs remains largely unresolved. It is certainly true that, although big advances in both scalability and automation of formal methods have been made and very important successful formal verification efforts have been carried out, *scalability* is still an important challenge. However, in my view two closely related problems, quite orthogonal to scalability, present a serious obstacle, namely: (1) verifying *designs*, as opposed to verifying *code*, is hindered in practice by the lack of suitable mathematical models for system designs; and (2) there is considerable ignorance about the *mathematical modeling* nature of programming made possible by *declarative languages*. The importance of solving problem (1) is one of effectiveness: design errors can be orders of magnitude more expensive than coding errors and in fact account for most of the critical errors in system development. This does not mean that verifying code is unimportant; however, correct-by-construction code generation from verified designs is a promising alternative to standard code verification and can be a considerably more cost-effective way of achieving code correctness. Problem (2) is quite serious and is self-inflicted. In many prestigious universities worldwide most

undergraduates now only learn to program in imperative languages like C, C++, or Java, and often do not even *know* that it is possible for a program to also be a *mathematical model* of the problem it solves.

The point is that problems (1) and (2) are closely related. A declarative program, that is, a program written in a computational logic and specified as a *theory* in such a logic has two key advantages: (i) it defines a *mathematical model* of the system it executes, which means that the distinction between design and code either evaporates or becomes reduced to one of refining and optimizing a high-level declarative program into a more efficient, yet equivalent, program; and (ii) since a system design specified as a declarative program is already a mathematical object, verifying its properties is typically much easier than verifying them for a program written in an imperative language. This all means that understanding the crucial role of declarative programs as formal executable specifications can greatly help in solving problems (1) and (2) at the same time.

An important distinction to be made is that between what I call *system specification* and *property specification and verification*. A computational system can obviously be programmed. By programming it in a declarative language, we obtain a *mathematical model* of the system thus programmed. But only by having a mathematical model of a system is it meaningful at all to *verify* its mathematical *properties*. Such properties need not be expressed in the computational logic of the declarative language in which the system in question has been specified. Indeed, many properties, for example, temporal logic properties or inductive theorems, need not be *executable* at all. This means that system properties to be verified *about* a system design may be specified in various logics in which such properties have a natural and easy expression. This also means that formal verification can be seen as the task of proving that the model defined by a formal, executable specification $\mathcal{S}$ —that is, by a declarative program $\mathcal{P}$ —*satisfies* a set $\{\varphi_1, \dots, \varphi_n\}$ of desired properties expressed as formulas $\varphi_1, \dots, \varphi_n$ in a suitable property specification logic.

All this brings us to the present book, that addresses the above problems (1) and (2) in an excellent and eminently practical way. One of its key contributions to undergraduate CS education is how well it shows students that programming as mathematical modeling in a declarative language such as Maude is: (i) quite easy, (ii) fairly intuitive, and (iii) actually fun to do. Once this is done through many well-chosen examples and exercises, students come to realize, almost as an after-thought, that they have been doing *mathematical modeling* all along. This happens just as for the man who suddenly realized that he had been speaking in prose all his life. This “aha moment” opens the door for discussing issues of *formal correctness* and *formal specification and verification* of system *properties*, so that property logics and their associated verification methods can be naturally introduced and explained.

In the first part of this book, all this is done for *deterministic systems* specified in equational logic as functional programs in Maude. Since the mathematical model defined by an equational program is the *initial algebra* of such a program as an equational theory, students are then introduced to the specification and verification of *inductive properties* satisfied by such initial algebras, and are shown how Maude itself can be used as a simple inductive theorem prover to verify such properties.

Since equational logic is a sublogic of rewriting logic, which is a natural and simple logic in which to specify *distributed systems*, the book then moves in a natural and seamless way from its first part focused on deterministic systems into its second and main part, focused on the executable specification of distributed systems as rewrite theories in Maude. Properties of distributed systems and their specification and verification are then explained. The same gentle and gradual approach is followed in this second part. This is achieved so well and with such a wealth of examples, that the book can also be used as a first introduction to distributed systems, their modeling, and their verification at the undergraduate level. The same gradual method of approach is also followed for the specification and verification of properties. First, the simplest of such properties, namely, *invariants*, are introduced, and explicit-state *reachability analysis* supported by Maude's `search` command is used to automatically verify such invariants, or to do so up to a given depth bound if the system is infinite-state. After this, a gentle, yet quite thorough, introduction to linear-time temporal logic (LTL) and its semantics is given, and many examples are given showing how Maude's LTL model checker can be used to automatically verify LTL properties of a distributed system formally specified as a rewrite theory in Maude. Finally, broader perspectives are opened up by explaining how additional topics such as the specification and verification of real-time and of probabilistic systems can be treated by corresponding extensions of rewriting logic by means of real-time rewrite theories and probabilistic rewrite theories; and at the property level by suitable real-time and probabilistic extensions of temporal logic. Each notion is again illustrated by means of well-chosen examples and exercises.

In summary, this book addresses an important and serious need in undergraduate CS education and, at the same time, the broader need of training a next generation of computer scientists who are well acquainted with both distributed systems and with the mathematical modeling and verification of such systems. Given the present state of affairs, both in the vulnerability of our systems and the serious gaps in mathematical modeling abilities in undergraduate CS education, the appearance of this book could not be more timely. I have been using earlier drafts of this book in a program verification course at the University of Illinois at Urbana-Champaign and plan to recommend the present book to my students as reading material for such a course in the years to come. I am sure that it will be of great help to many other persons teaching programming languages, formal methods, and distributed systems at the undergraduate level and, above all, to the students themselves.

Cabo Palos
June 2017

José Meseguer

Preface

The two main goals of this book are to:

1. provide an introduction to formal modeling and analysis of both data types and, in particular, distributed systems; and
2. provide an introduction to distributed computer systems and the challenges of designing and analyzing such systems.

The book is meant to be a first introduction to formal methods and therefore does not assume any previous knowledge about formal methods or distributed systems; it is based on a third-year course at the University of Oslo, but can equally well be taught at the second-year level. Some previous exposure to programming could be useful; likewise, experience with simple recursive functions is helpful but not necessary. There are no prerequisites on the mathematical side.

A distinguishing feature of this book is the significant use of the rewriting-logic-based Maude language and simulation and model checking tool for formally modeling both data types and distributed systems. Data types are specified using a functional programming style that students tend to like. Indeed, a valuable side effect of studying this book is training in writing recursive programs. For formally modeling distributed systems, Maude provides a simple yet intuitive and expressive modeling formalism that is particularly suitable for modeling distributed systems in an object-oriented way. Maude is by now a mature and well-established tool that is increasingly used around the world.

About the Content

As mentioned above, one main goal of this book is to gently introduce students to a wide range of concepts in formal methods, including:

- *verifying* properties about programs and (models of) systems; e.g., proving that a specification/program terminates for all possible inputs, and using equational logic to prove semantic properties;
- logics and inference systems; and
- automated model checking techniques to analyze properties for some—but not all—possible inputs/system configurations.

This book is divided into two parts. The first part deals with specifying the data types needed to model complex distributed systems. This part introduces classical algebraic specification and term rewriting theory, including reasoning about termination, confluence, and inductive equational properties.

The second part deals with formally modeling and analyzing distributed systems in rewriting logic using Maude. This part introduces rewriting logic and object-oriented modeling of distributed systems. It also introduces temporal logic to specify requirements that a system should satisfy. Such models are analyzed using Maude simulations, reachability analysis, and temporal logic model checking, thereby also giving the students a hands-on experience of the state-space explosion problem for distributed systems. As mentioned above, the second main goal of this book is to introduce the students to the problems of designing and analyzing distributed systems. Instead of giving theoretical explanations of these issues, the book tries to convey intuition about distributed systems and their design challenges through a range of examples/case studies in different domains, including: the dining philosophers problem, transport protocols like the alternating bit protocol and the sliding window protocol, classic distributed algorithms such as the distributed two-phase protocol for distributed database systems, distributed mutual exclusion and leader election algorithms, and the NSPK cryptographic protocol. Finally, the book briefly introduces two extensions of standard distributed systems: real-time systems and probabilistic systems.

The book is based on a course that has been given at the University of Oslo for more than 10 years, which implies that the book contains a wealth of exercises, both smaller ones and larger ones suitable for course projects, etc. Most of the executable code presented in this book, as well as other supplementary material, can be found at <http://peterol.at.ifl.uio.no/BOOK>.

I would like to thank José Meseguer, Dorel Lucanu, Narciso Martí-Oliet, and Ralf Sasse for many insightful and very helpful comments on earlier versions of this book, Indranil Gupta for discussions on distributed systems, Jon Grov for providing the figures used in this book, Si Liu for performing the statistical model checking experiments, Lars Kristiansen for discussions on logic, and Shiji Bijo, Antonio Gonzalez Burgueño, Benjamin Oliver, and Olaf Owe for pointing out mistakes in those earlier drafts. I also thank Hanne Riis Nielson and Ian Mackie for encouraging me to publish this book with Springer, and Simon Rees and Wayne Wheeler for their patience in waiting for it to be finished.

Contents

1	Introduction	1
1.1	Modeling	2
1.1.1	Models of Distributed Systems	2
1.1.2	From Model to System	3
1.2	The Maude Modeling Language and Analysis Tool	4
1.3	Why Maude?	5
1.4	Contents of the Book	6
1.4.1	Part I: Algebraic Specification and Term Rewriting	6
1.4.2	Part II: Dynamic Systems	7
1.4.3	Appendix: Mathematical Background	8

Part I Equational Specifications and Their Analysis

2	Equational Specification in Maude	11
2.1	Hello World: Our First Maude Specifications	12
2.1.1	Natural Numbers with Addition	13
2.1.2	The Boolean Values and Functions	14
2.1.3	Module Importation	15
2.2	Many-Sorted Equational Specifications	16
2.3	Requirements of Equational Specifications	19
2.3.1	One-to-one Constructor Basis	19
2.3.2	Termination: No Infinite Computations	20
2.3.3	Uniqueness of the “Result”	21
2.3.4	Definedness: The Result Should be a Constructor Term	21
2.3.5	Maude and the Requirements	22
2.4	Many-Sorted Specification of Data Types	22
2.4.1	Defining Functions: Getting Started	23
2.4.2	Expressiveness of Many-Sorted Equational Specifications	23
2.4.3	Maude Specifications of Some Data Types	24
2.5	Order-Sorted Equational Specifications	29

2.5.1	Examples of Order-Sorted Equational Specifications	30
2.6	Membership Equational Logic Specifications	33
2.7	Built-in Data Types	35
2.7.1	Booleans	35
2.7.2	Natural Numbers	36
2.7.3	Integers	38
2.7.4	Rational Numbers	38
2.7.5	Floating-Point Numbers	39
2.7.6	Strings	39
2.7.7	Random Numbers	40
2.8	Associativity and Commutativity: Lists and Multisets	41
2.8.1	Commutativity, Associativity, and Identity	41
2.8.2	Associativity and Identity: Lists	43
2.8.3	Associativity, Commutativity, and Identity: Multisets and Sets	44
2.9	Examples	47
2.9.1	Two Sorting Algorithms	47
2.9.2	Some NP-Complete Problems	49
2.10	* Some Other Maude Features	54
2.10.1	Parameterized Modules	54
2.10.2	Telling Maude how to Evaluate an Expression	56
2.10.3	Other Features	57
3	Operational Semantics of Equational Specifications	59
3.1	The Reduction Relation	59
3.1.1	Basic Definitions	60
3.1.2	The Reduction Relation	62
3.1.3	Some Derived Relations	62
3.2	Operational Properties	63
3.3	Conditional Equations and Matching with <i>assoc/comm</i>	64
3.3.1	Conditional Equations	64
3.3.2	* A-, C-, and AC-matching is NP-hard	65
4	Termination	67
4.1	Undecidability of Termination	68
4.2	Nontermination	72
4.3	Proving Termination Using “Weight Functions”	73
4.4	Simplification Orders	76
4.4.1	The Lexicographic Path Order	79
4.4.2	The Multiset Path Order and Other Variations of lpo	80
4.4.3	Comparing Weight Functions and Simplification Orders	81

5	Confluence	85
5.1	Unification	87
5.2	Checking Local Confluence	89
6	Equational Logic	93
6.1	Equational Logic	94
6.1.1	* Knuth-Bendix Completion	99
6.2	Inductive Theorems	101
6.2.1	Proving Inductive Theorems for Nat	103
6.2.2	Inductive Theorems for Other Data Types	105
7	Models of Equational Specifications	109
7.1	Many-Sorted Σ -Algebras	110
7.1.1	Homomorphisms and Isomorphisms	112
7.1.2	Term Algebras	115
7.2	(Σ, E) -Models: (Σ, E) -Algebras	116
7.2.1	Quotient Algebras	117
7.2.2	The Algebra $\mathcal{T}_{\Sigma, E}$	117
7.2.3	The Normal Form Algebra	118
7.3	Soundness and Completeness of Equational Logic	118
7.4	Intended Models: Initial Algebras	120
7.5	Empty Sorts and Many-Sorted Equational Logic	124

Part II Specification and Analysis of Distributed Systems in Maude

8	Modeling Distributed Systems in Rewriting Logic	127
8.1	Dynamic Systems	127
8.1.1	Properties of Dynamic and Distributed Systems	128
8.1.2	Behaviors of Distributed Systems	128
8.2	Modeling Dynamic Systems in Rewriting Logic	129
8.2.1	Rewrite Rules	130
8.2.2	Rewriting Logic Specifications	131
8.2.3	Examples	132
8.3	Concurrency	135
8.3.1	Sideways Concurrency	136
8.3.2	Nested Concurrency	138
8.4	Deduction in Rewriting Logic	139
8.4.1	Concurrent Steps	140
8.4.2	Termination and Confluence	142
8.5	* Frozen Operators	143
8.6	* Denotational Semantics	144
9	Executing Rewriting Logic Specifications in Maude	145
9.1	Executing One Sequential Rewrite Step	145
9.2	Simulating Single Behaviors	147
9.3	Search	150

10	Concurrent Objects in Maude	155
10.1	Modeling Concurrent Objects in Maude	155
10.1.1	Rewrite Rules for Objects	156
10.2	Concurrent Objects in Full Maude	162
10.2.1	Using Full Maude	162
10.2.2	Object-Oriented Modules in Full Maude	163
10.2.3	Subclasses	165
10.2.4	Search in Full Maude	168
10.2.5	Using Full Maude: Repetition	169
10.3	Example: The Dining Philosophers	170
10.3.1	Problem Description	170
10.3.2	Modeling the Dining Philosophers	171
10.3.3	Deadlock and Livelock	173
10.3.4	Fairness Issues	173
10.3.5	Version 2: A Deadlock-Free Solution	173
10.3.6	Version 3: A Deadlock-Free and Livelock-Free Solution	173
10.4	Randomized Simulations: Winning in Vegas	176
10.4.1	Blackjack	176
10.4.2	Modeling Blackjack Rounds	177
10.4.3	Further Guarantees	182
11	Modeling Communication in Maude	183
11.1	Synchronous Communication	184
11.2	Unordered Asynchronous Communication by Message Passing	185
11.2.1	Unordered Unicast	185
11.2.2	Multicast	188
11.2.3	Broadcast	189
11.2.4	Wireless Broadcast	190
11.2.5	Modeling Unreliable Communication	190
11.3	Ordered Asynchronous Communication using Links	193
11.3.1	Unreliable Links	195
11.3.2	Links with Limited Capacity	196
11.4	Asynchronous Communication Using Shared Variables	197
12	Modeling and Analyzing Transport Protocols	199
12.1	Reliable Communication Using Sequence Numbers	199
12.1.1	Maude Modeling	200
12.1.2	Formal Analysis	203
12.2	The Alternating Bit Protocol	204
12.3	The Sliding Window Protocol	206
12.3.1	Sliding Window with Links	209

13	Distributed Algorithms	211
13.1	Atomicity of Distributed Transactions: Two-Phase Commit.	211
13.1.1	The Two-Phase Commit Protocol	212
13.1.2	Abstraction	213
13.1.3	Assumptions	214
13.1.4	Specification and Analysis of 2PC in Maude.	214
13.2	Distributed Mutual Exclusion	221
13.2.1	Modeling the Central Server Algorithm.	222
13.2.2	Analyzing the Central Server Algorithm	224
13.3	Distributed Leader Election	226
13.3.1	A Ring-based Leader Election Algorithm	226
13.3.2	A Spanning-Tree-based Algorithm for Wireless Networks.	227
13.4	Consensus Algorithms	231
14	Analyzing a Cryptographic Protocol	233
14.1	Public-Key Cryptography	233
14.1.1	Digital Signatures	234
14.1.2	Symmetric-Key Cryptography	235
14.2	The Needham-Schroeder Public-Key (NSPK) Protocol.	235
14.3	Modeling NSPK in Maude	236
14.3.1	Executing the NSPK Specification.	240
14.4	Modeling Intruders	241
14.5	Analyzing NSPK with Intruders	244
14.6	Discussion	247
14.7	The Corrected Protocol.	248
15	System Requirements	249
15.1	State-based and Action-based Properties.	250
15.1.1	Actions/Events.	251
15.1.2	State Propositions	252
15.2	Temporal Properties	252
15.2.1	Invariance: “Nothing Bad Will Happen”.	253
15.2.2	Guarantee: “Something Good Must Eventually Happen”	254
15.2.3	Reachability: “Something Bad Could Happen”.	255
15.2.4	Response: “A Request Will Always be Answered”.	256
15.2.5	Stability.	256
15.2.6	Other Requirements.	257
15.3	Analyzing Invariants.	260
16	Formalizing and Checking Requirements	263
16.1	Linear Temporal Logic.	263
16.1.1	Behaviors	264

16.1.2	The Syntax of LTL	264
16.1.3	The Semantics of LTL	266
16.1.4	* Kripke Structures	268
16.2	Some LTL Formulas	269
16.2.1	Formalizing Classes of Requirements	269
16.2.2	Fairness Assumptions	271
16.3	Model Checking in Maude	273
16.3.1	Getting Started	273
16.3.2	Defining Atomic Propositions	274
16.3.3	Defining LTL Formulas	274
16.3.4	Performing Model Checking	275
16.3.5	Example: Analyzing Mutual Exclusion	277
16.4	* Some More Temporal Logic	281
17	Real-Time and Probabilistic Systems	283
17.1	Real-Time Systems	283
17.1.1	Specifying Real-Time Systems in Rewriting Logic	284
17.1.2	Timed Temporal Logics	291
17.1.3	Real-Time Maude	292
17.2	Probabilistic Systems	293
17.2.1	Probabilistic Rewrite Theories	294
17.2.2	Probabilistic Temporal Logics	296
17.2.3	PVestA Analysis	297
	Appendix A: Mathematical Preliminaries	299
	References	303
	Index	309

Our society increasingly depends on large and complex computer systems. Our cars, airplanes, banks, power plants, social interactions, shopping activities, etc., are all controlled and/or mediated to a large extent by computer systems. Most computer systems these days are *distributed* systems, consisting of multiple computers, or processors, of various kinds that collaborate to achieve some goal.

Unfortunately, distributed systems are quite complex and significantly harder to get right than single-threaded sequential programs, because:

- any component in the system may perform an action at any time,
- it may be hard to know *whether*, or *when*, a message will be delivered, and
- it may be hard to predict the behavior of other components in the system.

Example 1.1. A prerequisite for banking is *mutual authentication*: (i) you know that you are communicating with your bank and not with some impostor, and (ii) *the bank* knows that the person pretending to be you actually *is* you. In a physical bank, you know that you are in a bank by the imposing building, and the bank clerk asks you to show some photo identification to be sure that you are who you claim to be. In online banking and commerce, authentication *protocols* (“programs for distributed systems”) are used to ensure mutual authentication. One of the most well-known authentication protocols is the Needham-Schroeder public key protocol (NSPK) [88] that was published in 1978 by leading experts in the field. It is typically written as follows:

Message 1.	$A \rightarrow B : A.B.\{N_a.A\}_{PK(B)}$
Message 2.	$B \rightarrow A : B.A.\{N_a.N_b\}_{PK(A)}$
Message 3.	$A \rightarrow B : A.B.\{N_b\}_{PK(B)}$

Chapter 14 explains what all this means; essentially, *A* and *B* are the *agents* that want to establish mutual authentication (e.g., you and the bank), and the protocol consists of sending three encrypted messages: first one message ($A.B.\{N_a.A\}_{PK(B)}$) is sent from *A* to *B*; then *B* responds by sending a message ($B.A.\{N_a.N_b\}_{PK(A)}$) back

to A ; finally, A sends a message $(A.B.\{N_b\}_{PK(B)})$ to B . After these messages have been sent and received, A should know that it communicated with B , and vice versa.

This protocol was studied, used, and assumed correct until 1995, when Gavin Lowe used techniques very similar to those in this book to break the protocol. $\diamond$

This example shows that even a *three-line* distributed “program” can be really hard to get right. However, our lives and economy depend crucially on the correctness of considerably more complex distributed systems. How can we develop correct distributed systems and ensure that they indeed are correct?

1.1 Modeling

Let us consider an analogy. Thousands of years ago, building a hut for yourself was pretty easy and could be done right away without much elaboration. If the hut collapsed, you could rebuild it in a few hours. Just like you could start coding the programs in your introductory programming course without further ado. However, buildings have become much more complex in the last 1000 years. How are buildings constructed these days? You typically do not start building a large building with only a faint idea of what you want. You first build (or draw) a *model* of your building. A first model may be quite rough, but can be developed quickly and allows the architect and the person commissioning the building to get an idea of whether this is what they want. Once the main design is agreed upon, a more detailed model should be used to *infer* properties of the model: will the bridge collapse? can the proposed skyscraper withstand strong winds, floods, and earthquakes? The point is that:

1. such models are developed reasonably cheaply and quickly *before* starting to build the building; and
2. one should be able to use the model and the laws of physics to predict quite accurately whether the building to be built will satisfy certain desired properties.

It may be hard to compute by hand whether your skyscraper will withstand the winds/earthquakes/floods in the region. Computers should do that!

When advanced models have been developed and analyzed, impressive modern-day engineering technology can “easily” construct the building from the models. It may not be a coincidence that we know Gustave Eiffel, Oscar Niemeyer, and Frank Gehry, but have absolutely no clue about who actually *built* the Eiffel Tower, the Museum of Contemporary Art in Niteroi, and the Guggenheim Museum in Bilbao.

1.1.1 Models of Distributed Systems

In the same way, we need *models* of distributed systems before implementing them: you do not want to implement your new avionics system directly on an Airbus A380 and have one plane crash for each mistake in your code, or to deploy some new

e-commerce algorithm before you are really confident that your design is correct. The model should be reasonably quick to develop and should focus on the “essence” of the design and should *abstract* away inessential details. For example, a model of a distributed algorithm could focus on what happens when a message is successfully received or is lost in transmission, but can often abstract away details about how a packet is sent from one computer to another.

A model you can only look at is not very useful. We would like to both *simulate* the model and *infer properties* from it: can the flight control system deadlock? can your authentication protocol be broken by malicious agents? does the e-commerce protocol also work well if a crucial server goes down? Just like the architect should be able to use the laws of physics to predict properties about the building to be built, so should a system designer be able to analyze her model of a distributed system.

To reason about consequences of a design, its model must have a clear and precise meaning, and there must be some laws/rules that allow us to infer consequences of the model. Therefore, the model must be a *mathematical object* with precise, mathematical, rules of how one can infer properties from the design. Such a mathematical model of a computer system is called a *formal model*.

Specifications can be divided into *system* specifications (or models) and *requirement* specifications. System models specify the system, which means the computations performed by the system, whereas a requirement specification specifies the requirements or properties that a system should satisfy. For example, in the NSPK protocol, the three lines in Example 1.1 define the system model, which specifies the computation steps that the participants should perform (namely, sending encrypted messages, either to start a session or as a response to receiving a message). The corresponding requirement specification states the requirement(s) that the system should satisfy: “when an agent *A* thinks that it has established a connection with an agent *B*, then it indeed has a connection with *B* and not with some other agent.”

The main goal is to prove that all possible behaviors of the system (model) satisfy the system’s requirement specification. Furthermore, it would be great if computers could do this analysis, just like the architect wants to use computers to analyze consequences of her design. This is only possible if both the system model and the requirement specification are mathematical objects, and there are explicit mathematical rules that allow us to analyze whether or not a system satisfies its requirements.

The formal system model should preferably be *executable*; that is, the model can directly be executed. This would allow for a range of automated computer analyses, for example by *simulating* single behaviors of the system being modeled, or by *model checking* analyses that analyze many, or all, possible behaviors of the system.

This book focuses on developing and analyzing—by computer and by hand—executable formal models of distributed computer systems. It also deals with formalizing requirements of distributed systems using *temporal logic*.

1.1.2 From Model to System

The ultimate goal is not to have a nice model for its own sake, but to build a correct system. However, just like modern engineering technology and companies are

very good at constructing even very large buildings from correct models, modern programmers and programming environments and methodologies are quite good at implementing systems from correct specifications. There are also commercial *code generation* tools that can automatically generate code from high-level models.

Developing correct models is therefore a crucial task in the system development process. When the task at hand is well understood, the actual implementation is “just” programming and hardware engineering. In an early example illustrating the importance of developing correct system models, it turned out that only *three* of the 197 critical defects identified during integration and testing of the Voyager and Galileo spacecrafts were due to coding errors [74,99]. Most faults arose in requirements and difficult design problems related to distribution [99]. Furthermore, not only are defects more likely to be introduced in the early stage of system development; it is also much cheaper to catch errors early in the development process, since design errors can be orders of magnitude more expensive to fix than coding errors.

1.2 The Maude Modeling Language and Analysis Tool

This book uses the Maude [21] modeling language to define executable formal models of distributed systems, and uses the Maude analysis tool to analyze the models. In Maude, a distributed system is formalized as a theory in *rewriting logic* [16,80]. Maude and rewriting logic were both developed by José Meseguer and his research group at the Computer Science Laboratory at SRI International. (Meseguer now works at the University of Illinois at Urbana-Champaign.)

In rewriting logic, the data types of the system are defined algebraically by *equations*. In essence, defining data types amounts to defining functions in a functional programming style. The dynamic behavior of a distributed system is defined by *rewrite rules*, which describe how a part of the state can change in one step. Maude supports object-oriented programming, including multiple inheritance, and asynchronous communication through message passing, in a natural way.

The Maude interpreter evaluates an expression in an *equational* Maude program by applying the equations “from left to right” until no equation can be applied, thereby computing the normal form (or “value”) of the expression.

Since rewriting logic theories model distributed systems, they are typically *non-deterministic*, meaning that there may be many different behaviors from the same initial state of the system. A first form of analysis provided by Maude is to *simulate one* of those behaviors by *rewriting*, which applies rewrite rules to the state, either until no rule can be applied or until a user-given upper bound on the number of rewrites has been reached. (The equations are applied to reduce each intermediate state to its normal form before a rewrite rule is applied.) To analyze *all* possible behaviors *from a given initial state* one can use Maude’s *search* capabilities to check whether certain (un)desired states can be reached from the initial state.

Not only can we specify the system in Maude; we can also define the requirements the system should satisfy in Maude as linear temporal logic formulas. Maude’s

high-performance *model checker* can then be used to decide whether all possible behaviors from a given initial state satisfy the requirements, provided that the set of states reachable from the initial state is a finite set.

The Maude system, including a user manual, the source code, etc., is available free of charge at <http://maude.cs.illinois.edu> for various Unix/Linux platforms. Maude can also be compiled and run on Windows under Cygwin.

1.3 Why Maude?

There are a number of reasons why I think that Maude is a good choice for an introduction to formal modeling and analysis of distributed systems:

Simple and intuitive formalism. Maude models basically consist of equations that define functions recursively, and rewrite rules that specify how the states evolve dynamically. That's all! There are no tricky constructs for concurrency or communication. This functional programming style tends to appeal to students.

Expressive formalism. The modeling formalism is fairly expressive, which makes it easy to define models of complex systems. This is in contrast to simpler, e.g., automaton-based, approaches which either require a significant amount of work to specify larger systems, or cannot model such a system at all due to the system's infinite-state nature. Maude also provides a natural model of concurrent objects, which is ideal for modeling distributed systems. Together, this means that we can easily model a wide range of distributed systems, as illustrated in this book.

Active area of research. A number of leading research groups perform research on rewriting logic and apply Maude to state-of-the-art systems. A recent bibliography [76] lists about 1000 published scientific papers involving rewriting logic and Maude. Some applications of Maude include:

- Researchers at Microsoft and the University of Illinois at Urbana-Champaign (UIUC) modeled aspects of web browsers and their interface in Maude, and used Maude search to discover many previously unknown address bar and status bar spoofing attacks in web browsers [19]. Maude has also been used to formally specify and analyze a new secure web browser developed at UIUC [100].
- Modeling and analysis of a number of complex security and network communication protocols, including 50-page multicast protocols, protocols developed by the IETF, etc. (see, e.g., [51, 69, 94, 95]).
- Most modeling and programming languages do not have a well-defined *precise* meaning (or *semantics*); the meaning of a model may be unclear or ambiguous, and the meaning of a program may depend on the compiler being used. This is of course unacceptable for safety-critical systems. Furthermore, the lack of a formal meaning makes it impossible to deduce properties about such models, and hence to build tools for their analysis. Due to its expressiveness and simplicity, Maude is well suited to define the mathematical meaning of a model or a program, and has been used to define the semantics of a wide range of modeling and programming languages [86, 87], including subsets of the avionics (aircraft software) industrial

modeling standard AADL [91], the PLEXIL language developed at NASA for spacecraft operations [31], the most complete formal semantics of the C and Java languages [14, 39], and so on. Having a Maude semantics also means that models/programs in such a language can be analyzed using Maude. There is also an efficient tool for analyzing multi-threaded Java programs [43].

- Finding several bugs in embedded software used by major car makers.
- Programs developed at NASA to determine the position of objects in space.
- Formalization, analysis, and development of cloud computing systems [53, 71].
- Modeling of cell biology to simulate and analyze biological reactions [37, 38].

The survey paper [84] gives an overview of some applications of Maude.

Mature and efficient. Maude is a fairly mature, robust, and high-performance tool, publicly released in 1998, and is still under active development. It is also open-source and easy to install.

1.4 Contents of the Book

A model of a distributed system consists of (at least) two parts: (i) the definition of the data types (integers, Booleans, lists, sets, and so on) needed to define the states; and (ii) the definition of the *dynamic behavior* of the system. This is reflected in the structure of this book, which is divided into two parts.

Part I deals with defining data types by *equational specifications*, and analyzing both the meaning and the operational properties of such equational specifications. Part II deals with defining the dynamics of a distributed system using rewriting logic, and of manually and automatically analyzing such models. Since a closely related objective of this book is to introduce distributed systems, Part II also introduces examples of such systems from different domains, including communication protocols, distributed algorithms, and cryptographic (or “security”) protocols.

1.4.1 Part I: Algebraic Specification and Term Rewriting

This part covers classic topics in algebraic specification and term rewriting.

Chapter 2 introduces equational specification in Maude; we define in Maude the usual data types: natural numbers, integers, lists, binary trees, and multisets. We define the usual functions on these data types, including the quicksort and mergesort algorithms on lists, as well as some classical *NP-complete* problems.

Chapter 3 introduces some operational properties that equational specifications should satisfy. To exemplify how to formally reason about specifications, I focus on reasoning about *termination*. Chapter 4 provides some intuition and more concrete techniques to *prove* that your specification does not contain an infinite loop for any input. We study the theoretical basis for the concept of *simplification orders*, and use the standard path orders to prove termination. Chapter 5 shows how to verify

that specifications are *confluent*; that is, that the result of evaluating an expression is independent of the order in which Maude chooses to apply the equations.

Chapter 6 shows how to use *equational logic* to reason about the “meaning” of a specification. In particular, we focus on how induction techniques can be used to prove that certain desired properties “follow logically” from a specification.

In formal modeling, the precise meaning of a specification/program is given by the mathematical object defined by the program. Chapter 7 explains how an equational Maude program defines a mathematical object, namely, an algebra. Chapter 7 also proves Birkhoff’s Completeness Theorem: an equality holds in *all models* satisfying a set of equations E if and only that equality can be proved in equational logic.

1.4.2 Part II: Dynamic Systems

Chapter 8 introduces *rewriting logic* and explains how *rewrite rules* can be used to specify the possible concurrent behaviors of a system.

Chapter 9 explains how rewriting logic models can be analyzed in Maude by *simulating* one possible behavior of the system and by *searching* for (un)desired states. Chapter 10 then introduces Maude’s model of *concurrent objects*; all the larger examples in this book are modeled in an object-oriented style. Chapters 8 to 10 illustrate the concepts on simple examples, such as various small “games” and modeling the “lives” of persons, and end with the well-known dining philosophers problem and with randomized simulations to evaluate different blackjack strategies.

Chapter 11 shows how different forms of communication can be modeled at a high level of abstraction in Maude. These techniques are used in Chapter 12 to model a TCP-like transport protocol that uses sequence numbers to achieve reliable and ordered message communication when the network infrastructure is unreliable and only supports unordered message delivery. We then modify this protocol to the alternating bit protocol when we can assume ordered but unreliable links in the network. These two protocols are then generalized to two versions of the *sliding window protocol*, which is supposedly the best-known algorithm in computer networking [96].

We are then ready for some larger examples. Chapter 13 deals with modeling and analyzing a number of classic distributed algorithms, including the *two-phase commit* protocol for distributed database transactions, distributed mutual exclusion algorithms, and distributed leader election and consensus algorithms.

Chapter 14 shows how Maude can be used to model and analyze the aforementioned Needham-Schroeder security protocol, whose goal is to let Alice and Bob establish a communication between them so that Alice can be sure she’s communicating with Bob and not with the malicious intruder Walker. Is the security protocol up to this task, or can Maude show that Walker can impersonate Bob?

Chapter 15 introduces invariants and other kinds of requirements that our systems may have to satisfy, and discusses both how Maude can be used to analyze such system properties, and how they may be analyzed “by hand.”

These requirements are then formalized using *temporal logic* in Chapter 16, which also explains how Maude’s model checker can be used to check whether a system model satisfies its requirements.

Finally, Chapter 17 briefly discusses how the following kinds of systems can be modeled and analyzed in (extensions of) Maude:

1. *Real-time systems*, where the amount of time of/between events plays a crucial role and must be taken into account in the model.
2. *Probabilistic systems*, where certain events/values are chosen probabilistically.

1.4.3 Appendix: Mathematical Background

This book aims to be a self-contained introduction to formal methods. The little mathematical background needed is provided in Appendix A. It is worth mentioning that Section 4.4 and Chapter 7 might be the most mathematically advanced parts of the book. However, the book is written so that you can ignore these parts in a first reading, or in a more practically oriented course based on this book.

Part I
Equational Specifications and Their
Analysis

This chapter describes how data types can be defined in Maude as *equational specifications*. Section 2.1 introduces specification and execution in Maude with some simple “Hello World!” examples specifying the natural numbers and the Boolean values. Section 2.2 defines many-sorted equational specifications and explains how Maude computes with equations. Section 2.1.3 describes important requirements that an equational specification should satisfy. Section 2.4 shows the Maude specifications of other data types, including lists, multisets, and binary trees, and discusses the expressiveness of many-sorted equational specifications.

Data types are often related; for example, the natural numbers are a subset of the integers. Such subset relationships are captured in equational specifications by *subsorts*, which are treated in Section 2.5, and by *sort memberships* (Section 2.6). For convenience and performance, efficient versions of basic data types (natural numbers, Booleans, integers, rationals, floating-point numbers, and strings) are built-in in Maude as explained in Section 2.7. Section 2.8 introduces *functional attributes* that can be used to define lists and multisets elegantly in Maude. Section 2.9 shows Maude specifications of the well-known sorting algorithms quicksort and mergesort, and of solutions to some classic NP-complete problems. Finally, Section 2.10 briefly discusses other Maude features, including parameterized programming.

Maude specifications are *declarative* programs, which specify *what* to compute, whereas *imperative* programs, such as Java programs, give a step-by-step description of *how* to compute something. Declarative languages have some attractive features, including the following:

- Declarative languages do not have pointers, aliasing, and side effects, which make imperative programs very hard to understand and reason about.
- Declarative programs are easier to specify and modify. The constructs are more “powerful,” making it easier to specify complicated tasks, and to modify programs, as there are no side effects.

- Specification *is* programming: Instead of having to worry about all the intricate details of, say, quicksort or insertion sort, declarative programming allows you to *specify* what quicksort means, and you get a quicksort program for free.
- The meaning of an imperative program is usually given at a low level by how the program changes the values of memory cells in the CPU. It is hard to reason at that level, and it is often difficult to know what a program *really* does. Since a Maude specification specifies a mathematical object, it can be analyzed quite easily by following mathematical rules. For example, one can *prove* properties of programs such as “the program will never enter an infinite loop” and “quicksort returns a sorted list for *any* input list.” Properties like these cannot be guaranteed by *testing* a program, no matter how extensive the testing (we cannot test quicksort for *all* lists). Furthermore, while a Maude specification defines a single mathematical object, the meaning of a C or Java program may depend on the compiler/interpreter used, so that the same program can behave differently on different machines, which is of course unacceptable in safety-critical systems.

Imperative programs manipulate the store quite directly through assembly-like “low-level” instructions. In declarative programs you do not have to mess around with such details; however, this also means that you have much less control over the memory management and the execution. Declarative programs may therefore use more memory and time during execution than an optimized imperative program. Maude tries to minimize this disadvantage by a very sophisticated implementation which can perform millions of rewrites per second.

2.1 Hello World: Our First Maude Specifications

In this section we write and execute our first Maude specifications, defining the natural numbers and the Boolean values. Such data types are defined as *many-sorted equational specifications*, which consist of a set of *sorts*, where each sort roughly corresponds to a data type, a set of *function symbols* (also called *operators*)—some of which are used to construct the “values” of the data types, and others which are ordinary functions on those values—, and *equations* defining the functions.

In Maude, an equational specification is called a *functional module*, and is introduced with the following syntax:

```
fmod MODULENAME is
  BODY
endfm
```

where *MODULENAME* is the name of the module being introduced, and *BODY* is a set of declarations of sorts, function symbols, mathematical variables, and equations. The order of the declarations does not matter, since *BODY* is a *set* of declarations. A *comment* starts with ***** or *---* and goes until the end of the line, or it starts with **** (* or *--- (* and lasts until the first matching occurrence of *)*.

2.1.1 Natural Numbers with Addition

The following Maude module `NAT-ADD` specifies the natural numbers and a function ‘+’ on the natural numbers:

```
fmod NAT-ADD is
  sort Nat .
  op 0 : -> Nat [ctor] .
  op s : Nat -> Nat [ctor] .
  op _+_ : Nat Nat -> Nat .

  vars M N : Nat .

  *** Define the addition function recursively:
  eq 0 + M = M .
  eq s(M) + N = s(M + N) .
endfm
```

This module declares a sort `Nat` and three function symbols (or *operators*): `0`, which does not take any arguments (such function symbols are called *constants*) and gives an element of sort `Nat`; `s`, which takes an element of sort `Nat` as argument and gives an element of `Nat`; and `+`, which takes two elements of sort `Nat` as arguments and “returns” a `Nat`-value. The underscore (‘_’) tells where the arguments should be placed in “mix-fix” notation. If there are no underscores (as is the case for `s`), then the function symbol must be written using standard “prefix” notation.

The function symbols define the *expressions*, or *ground terms*, in our system; some of the terms of sort `Nat` are `0`, `s(0)`, `s(s(0))`, ..., `0 + 0`, `s(0) + s(0)`, The function symbols `0` and `s` are declared to be data *constructors* (`ctor`). The ground terms built up by the constructors, `0`, `s(0)`, `s(s(0))`, `s(s(s(0)))`, ..., denote the *data values* of `Nat`, and intuitively represent the numbers 0, 1, 2, 3, ...

After declaring two variables `M` and `N` of sort `Nat`, the module defines the function `+` recursively by two equations. The variables `M` and `N` are *mathematical variables* as we know from equations such as $(x+y)^2 = x^2 + 2xy + y^2$; they are *not* “program variables” in the imperative programming sense that can be assigned values. Just like an equation $(x+y)^2 = x^2 + 2xy + y^2$ is usually applied *from left to right* to *simplify* an expression, Maude also applies the equations from left to right to simplify an expression until it cannot be further simplified. The variables in the equations say that the equations hold for *all possible* values for `M` and `N`. The equations define a *recursive function* for computing the sum $m+n$ of two numbers m and n : if m is 0, apply the first equation and we are done; if m has the form $s(m')$, i.e., is greater than 0, the second equation recursively computes $m'+n$ and adds one to this sum.

Assuming that you have installed Maude according to the instructions given at <http://maude.cs.illinois.edu/>, you can start Maude, and should then get a greeting from Maude that looks like

```

\|||||/
--- Welcome to Maude ---
/|||||
Maude 2.7 built: Mar  3 2014 18:07:27
```

Copyright 1997–2014 SRI International
Sat May 20 03:48:00 2017

Maude>

You now need to enter the module `NAT-ADD` into Maude. This can be done either by typing the specification directly on Maude’s command line (not recommended) or by writing the module in some file, say `nat-add.maude`, and then let Maude read this file by using the `in` command:¹

Maude> `in nat-add.maude`

Maude will then reply with:

```
=====
fmod NAT-ADD
Maude>
```

If you get some error message(s) you should be aware of the following:

- Maude is case-sensitive. The sorts `Nat` and `nat` are not the same.
- Each declaration should end with a space followed by a period (‘.’). However, there should *not* be a period after `endfm`.
- For infix symbols such as `+` there should be a space before and after `+`. The equation should be written `eq 0 + M = M .`, not `eq 0+M = M .`
- There should be no space between ‘`_`’ and ‘`+`’ in the declaration of `+`.

To exit Maude, give the command `q` (or `quit`).

Maude’s `red` (or `reduce`) command computes the “value” of a given expression, such as `2 + 3`, by using the equations “from left to right” to “replace equals for equals” until no equation can be applied:

Maude> `red s(s(0)) + s(s(s(0))) .`

(Note the trailing period.) Maude answers with

```
reduce in NAT-ADD : s(s(0)) + s(s(s(0))) .
rewrites: 3 in 0ms cpu (0ms real) (1000000 rewrites/second)
result Nat: s(s(s(s(s(0)))))
```

The last line gives the result `s(s(s(s(s(0))))` (representing the number 5) and states that this result has sort `Nat`.

2.1.2 The Boolean Values and Functions

The following module `BOOLEAN` defines a data type for the Boolean values. The “values” in this data type are “true” and “false,” which we represent by two constructor constants `true` and `false`. We also declare the Boolean functions `not` (negation), `and` (conjunction), and `or` (logical disjunction) as follows:

¹ The command `load nat-add` does the same thing, but does not print the list of modules.

```
fmod BOOLEAN is
  sort Boolean .
  ops true false : -> Boolean [ctor] .
  op not_ : Boolean -> Boolean [prec 53] .
  op _and_ : Boolean Boolean -> Boolean [prec 55] .
  op _or_ : Boolean Boolean -> Boolean [prec 59] .

  var B : Boolean .
  eq not false = true .          eq not true = false .
  eq true and B = B .            eq false and B = false .
  eq true or B = true .          eq false or B = B .
endfm
```

The actual names of sorts and operators do not matter; we can equally well use the sort name `Bool` or `TruthValues` instead of `Boolean`, and the constructors `1` and `0` (or `T` and `F`) instead of `true` and `false`.

In first-order logic there is a *precedence* between the function symbols, where e.g. negation *binds tighter* than conjunction, so that $\neg x \wedge y$ is read $(\neg x) \wedge y$. We can tell the Maude parser to impose a similar precedence on the function symbols by adding an attribute `prec n` to the function symbol declaration, where n is a natural number. The *lower* the number of an operator, the tighter its binding. What matters is the relationship between the numbers: instead of 53, 55, and 59 we could have chosen 1, 2, and 3 with the same effect. A term `true and not true or false` is understood as `(true and (not true)) or false`.

2.1.3 Module Importation

A module may import another module that has already been entered into Maude using the keyword `protecting` or `including`.² The following module imports both our previous modules to define the “less than” function on natural numbers:

```
fmod NAT< is
  protecting NAT-ADD . protecting BOOLEAN .
  op _<_ : Nat Nat -> Boolean .
  vars M N : Nat .
  eq 0 < s(M) = true .
  eq M < 0 = false .
  eq s(M) < s(N) = M < N .
endfm
```

Exercise 1 Write the module `NAT-ADD` in a file, let Maude read the file with the specification, and use Maude’s `red` command to compute $2 + 4$ and $(2 + 3) + 4$.

² Although `protecting` and `including` have different mathematical meaning (see [21] for details), the Maude system treats them in the same way.

2.2 Many-Sorted Equational Specifications

In algebraic specifications we use *sorts* to distinguish different kinds of values, such as integers, strings, the Boolean values, and so on. In Maude sorts are declared using the keywords `sort` and `sorts`:

```
sort Int .
sorts Nat Boolean List .
```

The sorts are just *names* and do not contain *a priori* any associated values. Instead, we use *function symbols* (also called *operator symbols*) to define the “elements” or “values” of each sort, and to define functions on their domains of values. A declaration of a function symbol has the form

```
op f : s1 ... sn -> s .
```

for $n \geq 0$, where f is the introduced function symbol, and $s_1, \dots, s_n$, and s are sorts. The list $s_1 \dots s_n$ is the *arity* of f , and s is its *value sort*. Multiple function symbols with the same arity and value sort can be declared in one declaration:

```
ops f g h : s1 ... sn -> s .
```

We will use the terms “function symbol”, “function”, “operator symbol”, “operator”, and “operation” interchangeably.

Example 2.1. In the module NAT-ADD, the function symbol `0` has the empty list as its arity and `Nat` as its value sort, the function `s` has arity `Nat` and value sort `Nat`, and the symbol `+` has arity `Nat Nat` and value sort `Nat`. $\diamond$

A function symbol whose arity is the empty list (i.e., $n = 0$) is called a *constant*.

A *many-sorted signature* consists of a set of sorts and a set of function symbol declarations (where an element $w \in S^*$ is a finite sequence of S -elements):

Definition 2.1 (Signature) A many-sorted signature (S, Σ) consists of a set S , whose elements are called sorts, and an $S^* \times S$ -sorted family $\{\Sigma_{w,s} \mid w \in S^*, s \in S\}$ of function symbols. ($\Sigma_{w,s}$ is the set of function symbols with arity w and value sort s .) We often write $f : w \rightarrow s \in \Sigma$ for $f \in \Sigma_{w,s}$.

Example 2.2. The many-sorted signature $(\{\text{Nat}\}, \Sigma)$ defined by the module NAT-ADD has $\Sigma = \{\Sigma_{w,\text{Nat}} \mid w \in \{\text{Nat}\}^*\}$ where $\Sigma_{\varepsilon,\text{Nat}} = \{0\}$, $\Sigma_{\text{Nat},\text{Nat}} = \{s\}$, $\Sigma_{\text{Nat Nat},\text{Nat}} = \{+\}$, and $\Sigma_{w,\text{Nat}} = \emptyset$ for any other w . (The empty list is denoted ε .) The only constant in this signature is `0`. $\diamond$

The *ground terms* define the “expressions” we can talk about. A ground term is built by constants and other function symbols in a “sort-correct” way:

Definition 2.2 (Ground terms) Given a many-sorted signature (S, Σ) , the S -sorted set $\mathcal{T}_\Sigma = \{\mathcal{T}_{\Sigma,s} \mid s \in S\}$ of ground terms are defined inductively as follows:

1. $\Sigma_{\varepsilon,s} \subseteq \mathcal{T}_{\Sigma,s}$; that is, every constant of sort s is a ground term of sort s .
2. If $f \in \Sigma_{s_1 \dots s_n, s}$, and $t_1 \in \mathcal{T}_{\Sigma, s_1}, \dots, t_n \in \mathcal{T}_{\Sigma, s_n}$, and $n \geq 1$, then $f(t_1, \dots, t_n) \in \mathcal{T}_{\Sigma, s}$. That is, a function symbol “applied” to ground terms of the appropriate sorts gives another ground term.
3. In addition, each set $\mathcal{T}_{\Sigma, s}$ is the smallest set satisfying the above conditions. That is, only “things” which can be built from constants and the application of function symbols to ground terms of the right sorts are ground terms.

Notation. I sometimes use type-writer font and write ‘,’ ‘(’, and ‘)’ instead of ‘,’ ‘(’, and ‘)’, so that a term $f(a, b)$ will also be written $f(a, b)$.

Example 2.3. The set $\mathcal{T}_{\Sigma_{\text{NAT-ADD, Nat}}}$ of ground terms of sort Nat contains the ground terms $0, s(0), s(s(0)), 0 + 0, s(0) + 0, s(0) + (s(0) + 0), \dots$ $\diamond$

Example 2.4. Given the signature

```

sorts s s' .
ops a b : -> s .      op f : s -> s' .      op g : s s' -> s .

```

Then $a, b, g(a, f(b))$, and $g(g(a, f(b)), f(a))$ are all ground terms of sort s ; and $f(a), f(b)$, and $f(g(a, f(b)))$ are ground terms of sort s' . $g(a, b), f(a, b)$, and $q(, , \dots)$ are all ill-formed terms that have no sort whatsoever.

When a definition mentions “all terms of the form $f(t_1, \dots, t_n)$ for $n \geq 0$,” then this also includes all the constants (i.e., when $n = 0$).

As already mentioned, *constructor* functions (such as 0 and s) define the elements of the data type: the *data elements* of a sort are the ground terms consisting only of constructor functions. The other functions (such as $+$), called *defined functions*, are ordinary functions on those elements, and are defined by equations.

Mathematical variables of different sorts are needed to define equations:

Definition 2.3 (Variables) *Given a many-sorted signature (S, Σ) , a variable set X is an S -sorted family $X = \{X_s \mid s \in S\}$ of pairwise disjoint sets (that is, no variable has two different sorts: $s \neq s' \implies X_s \cap X_{s'} = \emptyset$), also disjoint from Σ (that is, nothing can be both a variable and a function symbol). We often write $x : s$ for $x \in X_s$.*

In Maude, the keywords `var` and `vars` are used to declare variables. However, variables of the form `var:sort` can also be used on-the-fly without explicit declaration, so that the following two specification fragments are equivalent:

```

vars M N : Nat .      eq 0 + M = M .      eq s(M) + N = s(M + N) .

```

and

```

eq 0 + M:Nat = M:Nat .      eq s(M:Nat) + N:Nat = s(M:Nat + N:Nat) .

```

(“Non-ground”) terms can contain variables: The set $\mathcal{T}_{\Sigma}(X)$ of *terms* in a signature (S, Σ) w.r.t. a set of variables X are all the “things” that can be built in a sort-consistent way from constants, variables, and the application of functions:

Definition 2.4 (Terms) Given a many-sorted signature (S, Σ) and a variable set $X = \{X_s \mid s \in S\}$, the S -sorted set of terms $\mathcal{T}_\Sigma(X) = \{\mathcal{T}_{\Sigma,s}(X) \mid s \in S\}$ is defined inductively by the following conditions:

1. $X_s \subseteq \mathcal{T}_{\Sigma,s}(X)$ for $s \in S$; that is, a variable of sort s is also a term of sort s .
2. $\Sigma_{\varepsilon,s} \subseteq \mathcal{T}_{\Sigma,s}(X)$ for $s \in S$; that is, a constant of sort s is also a term of sort s .
3. $f(t_1, \dots, t_n) \in \mathcal{T}_{\Sigma,s}(X)$ if $f \in \Sigma_{s_1 \dots s_n, s}$ and $t_i \in \mathcal{T}_{\Sigma,s_i}(X)$ for each $1 \leq i \leq n$.
4. $\mathcal{T}_\Sigma(X)$ is the smallest S -sorted set satisfying the above conditions.

Non-constructor functions are defined recursively by (unconditional and conditional) equations:

Definition 2.5 (Equations) Given a many-sorted signature (S, Σ) , a $(\Sigma-)$ equation is a triple (X, t, t') , written $(\forall X) t = t'$, where X is an S -sorted variable set disjoint from Σ , and t and t' are terms of the same sort; i.e., $t, t' \in \mathcal{T}_{\Sigma,s}(X)$ for some $s \in S$.

A conditional $(\Sigma-)$ equation is a $2(n+1)+1$ -tuple $(X, u_1, v_1, \dots, u_n, v_n, t, t')$ for $n \geq 1$, written

$$(\forall X) u_1 = v_1 \wedge \dots \wedge u_n = v_n \implies t = t',$$

such that there are sorts $s_1, \dots, s_n, s$ in S with $t, t' \in \mathcal{T}_{\Sigma,s}(X)$ and $u_i, v_i \in \mathcal{T}_{\Sigma,s_i}(X)$ for each $i \in \{1, \dots, n\}$.

Definition 2.6 (Many-sorted equational specifications) A many-sorted equational specification is a tuple (S, Σ, E) where (S, Σ) is a many-sorted signature and E is a set of Σ -equations and conditional Σ -equations.

In Maude, equations are written with syntax

$$\text{eq } t = t' .$$

and conditional equations are written with syntax

$$\text{ceq } t = t' \quad \text{if } u_1 = v_1 \wedge \dots \wedge u_n = v_n .$$

The meaning of an equation $(\forall X) t = t'$ is that t and t' are equivalent for all values of the variables X . $(\forall X) u_1 = v_1 \wedge \dots \wedge u_n = v_n \implies t = t'$ means that if $u_1 = v_1$ and $\dots$ and $u_n = v_n$ for some values of the variables in X , then t equals t' for those same values of the variables. For example, the following conditional equations define a function `max` on natural numbers in a module extending `NAT<`:

$$\begin{aligned} \text{ceq } \text{max}(M, N) &= N \text{ if } M < N = \text{true} . \\ \text{ceq } \text{max}(M, N) &= M \text{ if } M < N = \text{false} . \end{aligned}$$

The *operational* meaning describes how Maude's `red` command computes with equations. For example, if we ask Maude to compute the “value” of a ground term such as e.g., `s(s(0 + s(0))) + 0`, then the following happens:

1. Maude checks whether *some* equation can be applied *somewhere* in the term. That is, it checks whether the *lefthand* side of an equation “matches” the term somewhere. It then applies the equation by “replacing equal by equal.” For example, the equation `0 + M = M` could be applied to the term `s(s(0 + s(0))) + 0`,

reducing it to $s(s(s(0))) + 0$. If more than one equation can be applied, and/or if an equation can be applied in more than one place in a term, then Maude chooses (*pseudo-*)arbitrarily what equation to apply and where to apply it. For example, in addition to the previous application, the equation $s(M) + N = s(M + N)$ could be applied to $s(s(0 + s(0))) + 0$, giving $s(s(0 + s(0)) + 0)$.

2. The above process is repeated on the resulting term as long as there is some equation which can be applied.
3. When no equation can be applied anywhere, Maude outputs the “current” term.

Example 2.5. The term $s(s(0 + s(0))) + 0$ reduces to $s(s(s(0))) + 0$ in NAT-ADD. In the next step, only the equation $s(M) + N = s(M + N)$ can be applied, giving $s(s(s(0)) + 0)$. In the next step, only this same equation can be applied, giving $s(s(s(0) + 0))$. This equation can then be applied again, giving $s(s(s(0 + 0)))$. Now, only the equation $0 + M = M$ can be applied, giving the term $s(s(s(0)))$. Since no equation can be applied anymore, the result is $s(s(s(0)))$. The sequence $s(s(0 + s(0))) + 0 \rightsquigarrow s(s(s(0))) + 0 \rightsquigarrow s(s(s(0)) + 0) \rightsquigarrow \dots \rightsquigarrow s(s(s(0)))$ is called a *derivation*, a *computation*, or a *reduction sequence*. $\diamond$

The command `set trace on .` will make Maude show each step in the derivations.

Exercise 2 Overloading a function symbol means that the same function symbol can have different arities and/or value sorts. This can be quite convenient, since a constant ‘0’ could be both a bit value, a Boolean value, and a natural number:

```
sorts Bit Boolean Nat .
ops 0 1 : -> Bit .      ops 0 1 : -> Boolean .      op 0 : -> Nat .
```

1. Is such overloading allowed according to Definition 2.1?
2. If it is allowed, how can you modify Definition 2.1 to disallow such overloading?

Exercise 3 In the signature in Example 2.4, is $f(f(a))$ a ground term of sort s , sort s' , or neither?

Exercise 4 Show a derivation from $s(s(0 + s(0))) + 0$ where the equation $s(M) + N = s(M + N)$ is applied in the first step.

2.3 Requirements of Equational Specifications

This section introduces four requirements that an equational specification should satisfy to make Maude computations meaningful. Chapters 4 and 5 explain how two of these requirements, termination and confluence, can be analyzed.

2.3.1 One-to-one Constructor Basis

A *data type* consists of a set of elements (the *domain*) and a set of functions on those elements. Examples of domains are the set $\mathbb{N}$ of natural numbers, the set of all lists of natural numbers, the set of all binary trees of a certain kind, and so on.

In Maude, the elements in a data type are represented by the ground terms built by the constructor function symbols. For this to make sense: (i) each element in the domain we want to model must be represented by a constructor ground term; (ii) each element is only represented by *one* constructor ground term, or by a single equivalence class of such terms when there are equivalences on constructor ground terms (such as in the case of sets); and (iii) there are no “junk” constructor ground terms that do not represent elements in our domain.

For the natural numbers and their Maude representation in the module `NAT-ADD` we have the desired one-to-one correspondence: each number $n \in \mathbb{N}$ is represented by a constructor ground term $\underbrace{s(s(\dots(s(0))\dots))}_n$; and a constructor ground term of sort `Nat` is either 0 (representing the number 0) or has the form $\underbrace{s(s(\dots(s(0))\dots))}_m$, for $m \geq 1$, which represents the number m .

2.3.2 Termination: No Infinite Computations

To use Maude to compute the value of an expression, the computation of any expression must *terminate*; i.e., there should not exist infinite computations from any ground term. For example, in the module `NAT-ADD`, no matter how the equations to apply are chosen, each computation always ends up with a term to which no equation applies. However, in the specification

```
sort s .          ops a b : -> s .
eq a = b .        eq b = a .
```

Maude would “simplify” `a` to `b` using the first equation, and then `b` would be simplified to `a` using the second equation, and then `a` would again be simplified to `b` using the first equation, and so on, giving an infinite computation

$$a \rightsquigarrow b \rightsquigarrow a \rightsquigarrow b \rightsquigarrow \dots$$

starting from `a`. Similarly, adding the equation $M + N = N + M$ to `NAT-ADD` would lead to infinite computations such as $s(0) + 0 \rightsquigarrow 0 + s(0) \rightsquigarrow s(0) + 0 \rightsquigarrow \dots$.

A specification is called *terminating* if it does not allow any infinite computation.

A simple rule of thumb is that the value in *some* argument position in the recursive calls must decrease in some way³; other arguments may become larger. Furthermore, an equation can have multiple recursive calls, as long as the appropriate argument decreases in *all* recursive calls. For example, the module `NAT-ADD` extended with a function `op f : Nat Nat -> Nat` defined by

```
eq f(0, M) = s(s(M)) .
eq f(s(M), N) = f(M, M + N) + f(M, N) .
```

³ A “decrease” typically means that the number of function symbol occurrences in a constructor ground term must decrease.

is terminating, since the *first* argument of f decreases in each recursive call. However, if the second equation is replaced by

$$\text{eq } f(s(M), N) = f(M, M + N) + f(N, M) \text{ .}$$

then the specification would no longer be terminating (see Exercise 6).

2.3.3 Uniqueness of the “Result”

By definition, a function $f : A \rightarrow B$ assigns a *single* value $b \in B$ to each $a \in A$. Therefore, since we are computing the value of functional expressions, the “result” of an expression should be the *same*, no matter *how* Maude applies the equations. For example, any computation of $s(s(0 + s(0))) + 0$ should always end with the result $s(s(s(0)))$, and not with $s(s(s(0))) + 0$ or $s(0)$ or anything else. (Since we have no control over the application of equations, it would be unsatisfactory if the result of computing the value of an expression would depend on how Maude chooses which equations to apply.)

Example 2.6. In the terminating specification

```
sort s .           ops a b c : -> s .
eq a = b .         eq a = c .
```

the result of evaluating the expression a could be either b or c . ◇

A result of a computation of a term t is called a *normal form* of t . If it is in addition unique, then this unique normal form is written $t!$. For example, the normal form of $s(s(0 + s(0))) + 0$ is $s(s(s(0)))$ in the module NAT-ADD.

The property that all possible computations of an expression (in a terminating specification) give the same result, no matter how the equations are applied, is formalized as the *confluence* property in Chapter 5.

2.3.4 Definedness: The Result Should be a Constructor Term

We want to compute the *value* (i.e., a constructor ground term) of a functional expression (i.e., a ground term). Each expression should therefore be reducible to a constructor ground term. For example, if we “forget” the equation $0 + M = M$ in NAT-ADD, then $s(s(0 + s(0))) + 0$ reduces to $s(s((0 + s(0)) + 0))$, which cannot be further reduced, and which is not the result we really wanted.

This is the same as requiring that a non-constructor function is “defined” on *all* constructor ground terms. For instance, for natural numbers, $n_1 + n_2$ is defined for all values/constructor ground terms n_1 and n_2 , since n_1 (and n_2 as well for that matter) should have the form 0 or $s(n)$ for some n . In the first case, the equation $0 + M = M$ will apply, and in the second case $s(M) + N = s(M + N)$ can be applied.

Functions are often defined by one equation for each constructor, although sometimes we need fewer, and sometimes more, equations:

```
op double : Nat -> Nat .      var N : Nat .      eq double(N) = N + N .
```

The above equation covers all arguments of `double`. A function `minusTwo` which decreases any number greater than one by two can be defined by three equations:

```
op minusTwo : Nat -> Nat .      var N : Nat .
eq minusTwo(0) = 0 .            eq minusTwo(s(0)) = 0 .
eq minusTwo(s(s(N))) = N .
```

For any constructor ground term n , some equation can be applied on `minusTwo`(n).

The function $<$ in Section 2.1.3 is defined for all pairs (m, n) of constructor ground terms m and n ; this can be checked by considering all possible values for this pair: $(0, 0)$, $(0, s(n))$, $(s(m), 0)$, and $(s(m), s(n))$. In each of these cases, an equation defining $<$ can be applied.

A more precise name for the definedness property is *sufficient completeness*: the result of simplifying a ground term should be a constructor ground term.

2.3.5 Maude and the Requirements

Maude does *not* check whether your specification satisfies these requirements. The first one obviously cannot be checked, since Maude cannot know what domain you are trying to represent. The other three requirements are in general *undecidable*: there is no algorithm that can look at any user module and tell whether the module satisfies the requirements or not. However, Maude has (external) termination checkers [32], confluence checkers [33], and sufficient completeness checkers [57] that can *often* be used to check the corresponding requirements.

You must make sure that the above requirements are satisfied independently of how Maude is implemented. Since we have no control over the application of equations, it would be unsatisfactory if the result of computing a term would depend on how the Maude system chooses which equations to apply.

Exercise 5 Explain why there are no infinite computations in NAT-ADD and NAT- .

Exercise 6 Explain why there could be an infinite computation in NAT-ADD extended with the equation $\text{eq } f(s(M), N) = f(M, M + N) + f(N, M)$.

2.4 Many-Sorted Specification of Data Types

This section explains how data types can be defined as many-sorted equational specifications.

2.4.1 Defining Functions: Getting Started

Although there is no automatic way to define functions, one hint to help get you quickly started is to define a function $\text{op } f : S \rightarrow S'$ by one (or more) equation(s) for each constructor for S . For example, if the constructors for the sort S are two constants a and b , one unary operator g (i.e., a function taking one argument), and one binary operator h (i.e., a function taking two arguments), then one could first try to define f by four equations of the form

$$\text{eq } f(a) = \dots \quad \text{eq } f(b) = \dots \quad \text{eq } f(g(x)) = \dots \quad \text{eq } f(h(x, y)) = \dots$$

for variables x and y of appropriate sorts. For the sort Nat , we can follow this scheme to define the function `double`, which doubles its argument, also without using `+`:

$$\text{eq } \text{double}(0) = 0 \quad . \quad \text{eq } \text{double}(s(N)) = s(s(\text{double}(N))) \quad .$$

If the function f takes two arguments, you can define f by “cases” on the constructors for one of the arguments, or for both. `NAT-ADD` defines addition by “cases” on the first argument, but it could equally well have used the second argument. We can use this technique to define multiplication by “cases” on the first argument:

```
fmod NAT-MULT is protecting NAT-ADD .
  op _*_ : Nat Nat -> Nat .
  vars M N : Nat .
  eq 0 * N = 0 .
  eq s(M) * N = N + (M * N) .
endfm
```

For binary functions (or more generally, n -ary) functions, sometimes such case definitions only work for one of the arguments (like list concatenation in Section 2.4.3.1). Sometimes we may need to do a “case” on *both* arguments. For less-than on natural numbers, we need to consider both arguments: the first argument is 0 or has the form $s(m)$, and the second argument is either 0 or has the form $s(n)$:

$$\begin{aligned} \text{eq } 0 < 0 &= \text{false} . & \text{eq } s(M) < 0 &= \text{false} . \\ \text{eq } 0 < s(N) &= \text{true} . & \text{eq } s(M) < s(N) &= M < N . \end{aligned}$$

Again, this is just to help get you started; once you have defined your function, you should make its definition more elegant: the upper two equations can be combined into the single equation $M < 0 = \text{false}$, yielding the definition in Section 2.1.3. While this is a useful starting point, sometimes you need more elaborate definitions, such as for the function `minusTwo` above.

An important thing discussed next is that it is often convenient, or even necessary, to introduce *auxiliary functions* in order to define a given function.

2.4.2 Expressiveness of Many-Sorted Equational Specifications

Bergstra and Tucker show in [12] that it is *impossible* to define the *square* function on natural numbers in Maude *without* using other functions than 0 and `s`. And try

to define exponentiation *without* using other functions than addition! However, both the square function and exponentiation are easily defined if you introduce (addition and) multiplication as auxiliary functions:

```
fmod NAT-EXP is protecting NAT-MULT .
  op square : Nat -> Nat .
  op _^_ : Nat Nat -> Nat .
  vars M N : Nat .
  eq square(N) = N * N .
  eq M ^ 0 = s(0) .
  eq M ^ s(N) = M * (M ^ N) .
endfm
```

What does this difficulty of defining simple functions without introducing auxiliary functions say about the expressive power of *terminating* and *confluent* finitary⁴ many-sorted equational specifications? It turns out that by adding auxiliary functions, you can define whatever you want in this way. (The expressiveness of equational specifications is also indicated in Section 4.1, which shows that Turing machines can be simulated by equational specifications. However, the corresponding specifications are not necessarily terminating and/or confluent).

Formally, any recursive (i.e., computable) function on finite products of natural numbers can be defined by a terminating and confluent finitary many-sorted equational specification (see, e.g., [105, Section 3.2]). Furthermore, Bergstra and Tucker prove the following remarkable result in [11, 12] (see also the discussion in [85]):

Theorem 2.1 *Any computable algebra⁵ can be specified by a finitary terminating and confluent many-sorted equational specification.*

This means that anything you can do in your favorite programming language, you can also do in Maude! Just add auxiliary functions (new sorts are not needed).

2.4.3 Maude Specifications of Some Data Types

This section shows the Maude specification of some well-known data types.

2.4.3.1 Lists of Natural Numbers

How can lists of, say, natural numbers, be represented in a many-sorted equational specification? A constructor for the empty list is obviously needed:

```
sort List .
op nil : -> List [ctor] .
```

⁴ That is, using only a finite number of sorts, functions, and equations.

⁵ A computable algebra is one whose domains are recursive sets (i.e., we can decide whether an element is a member of the set) and whose functions are recursive (i.e., computable) functions.

A natural way of constructing lists is by appending an element to an existing list:

```
op app : List Nat -> List [ctor] .
```

In this case, a list “1 2 3” is represented by the constructor term

```
app(app(app(nil, s(0)), s(s(0))), s(s(s(0)))).
```

A more appealing way of representing lists is to let the append function instead be denoted by a mix-fix function symbol:

```
op _+_ : List Nat -> List [ctor] .
```

The list “1 2 3” can then be written `nil ++ s(0) ++ s(s(0)) ++ s(s(s(0)))`.

We can further shorten the representation of lists by removing the “++” part from the above append function; i.e., by using mix-fix *empty* syntax:

```
op __ : List Nat -> List [ctor] .
```

The list “1 2 3” is now represented by the term `nil s(0) s(s(0)) s(s(s(0)))`. The following module defines lists of natural numbers and some functions on them:⁶

```
fmod LIST-NAT1 is protecting NAT1 . protecting BOOLEAN1 .
sort List .
op nil : -> List [ctor] .
op __ : List Nat -> List [ctor] .
op length : List -> Nat .          *** # of elements in a list
op concat : List List -> List .    *** Concatenate two lists
op insertFront : Nat List -> List . *** Insert element first
ops first last : List -> Nat .      *** First/last element
op empty? : List -> Boolean .       *** Is the list empty?
op rest : List -> List .            *** Remove first element.
op reverse : List -> List .          *** Reverse list
op _occursIn_ : Nat List -> Boolean .
op remove : Nat List -> List .       *** Remove element(s)
op max : List -> Nat .               *** Largest element in list
op isSorted : List -> Boolean .      *** Is the list sorted?

vars N N' : Nat .    vars L L' : List .
```

The `length` function, giving the number of elements in the list, can be defined using the techniques suggested above; i.e., by recursion on the argument w.r.t. the constructors `nil` and `__`:

```
eq length(nil) = 0 .
eq length(L N) = s(length(L)) .
```

To define the list concatenation function `concat`, it turns out that doing the recursion on the second argument works:

```
eq concat(L, nil) = L .
eq concat(L, L' N) = concat(L, L') N .
```

⁶ The modules `NAT1` and `BOOLEAN1` are defined in Exercise 9.


```

    = insertFront(N, *** Root first, then left and right subtree:
                    concat(preorder(BT), preorder(BT')))) .
eq size(empty) = 0 .
eq size(bintree(BT, N, BT')) = s(size(BT) + size(BT')) .
...
endfm

```

The functions `preorder`, `inorder`, and `postorder` list the elements in a tree in the order they are encountered in, respectively, a preorder, an inorder, and a postorder traversal of the tree. `weight` gives the sum of the elements in the tree, `size` gives the number of elements, and `isSearchTree` returns `true` if and only if the tree is a *binary search tree*; that is, an inorder traversal (“from left to right”) encounters the elements in increasing (or at least non-decreasing) order. The function `reverse` reverses the tree; i.e. “flips it” around its vertical axis, and then does the same recursively for each subtree.

2.4.3.3 What About Sets?

Sets and multisets (which are essentially “sets,” but where the number of occurrences of each element matters) are important data types. However, since the sets $\{a, b\}$ and $\{b, a\}$ are the *same* sets, it is hard to define a one-to-one constructor basis. For example, using constructors

```
op empty : -> Set [ctor] .      op _;_ : Set Nat -> Set [ctor] .
```

the *same* set $\{0, 1\} = \{1, 0\}$ could be represented by the two *different* constructor ground terms `empty ; 0 ; s(0)` and `empty ; s(0) ; 0`. Section 2.8.3 defines sets so that each set is represented by one equivalence class of constructor ground terms.

Exercise 7 Define a function `square : Nat -> Nat` that computes the square of a number, without using any other function except `s`, `0`, `+`, and `square` itself.

Exercise 8 Explain why parentheses are not needed when using the constructors `nil` and `__` for lists. That is, show that expressions such as `nil s(0) s(s(0)) s(s(s(0)))` only can be parsed in one way.

Exercise 9 1. Define a module `NAT1` that extends `NAT<` with the functions

```

op half : Nat -> Nat .
ops _monus_ diff min : Nat Nat -> Nat .
ops odd even : Nat -> Boolean .
ops _<=_ _>_ _>=_ _==_ : Nat Nat -> Boolean .

```

`half` is “integer division by 2,” `m monus n` is “minus down to 0,” i.e., $\max(m-n, 0)$, `diff` is the difference between two numbers, `min` computes the smallest of two numbers, and `odd` and `even` return `true` if its argument is an odd, resp. even, number. The other functions are the usual comparison operators.

2. Define a module `BOOLEAN1` that extends `BOOLEAN` with the following functions:

```
op _implies_ : Boolean Boolean -> Boolean [prec 61] .
op if_then_else_fi : Boolean Boolean Boolean -> Boolean .
```

where x implies y is false only when x is true and y is false.

Test your specifications in Maude. Two things to remember is that: (i) since there are also built-in Boolean values in Maude, you must give the Maude command `set include BOOL off .` before entering the specifications into Maude; and (ii) you must have loaded the files containing the modules that you import. This can be achieved by starting your file as follows (for the appropriate file names):

```
set include BOOL off .
load nat-mult.maude
load boolean.maude
load less-than.maude
```

Exercise 10 Define the other functions in the module `LIST-NAT1`.

Exercise 11 Lists of natural numbers can be compared lexicographically. A list l is greater than a list l' if there is a number k such that

- the k -th element of l exists, and it is greater than the k -th element in l' or the k -th element in l' does not exist; and
- for all $j < k$, the j -th element of l is the same as the j -th element of l' .

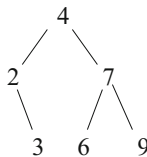
In short, l is greater than l' if both lists are the same until either l' stops or until an element in l is greater than the corresponding element in l' . For example, the list “4 5 6” is greater than both “3 4 5 6 7”, “4 5”, and “4 5 2 10”.

1. Show (by an example) that there is an infinite sequence $l_0 > l_1 > l_2 > l_3 > \dots$ of lists such that l_i is greater than l_{i+1} for each i .
2. Explain informally why there is no infinite sequence $l_0 > l_1 > l_2 > l_3 > \dots$ of lists of the same length such that l_i is greater than l_{i+1} for each i .
3. Define a function

```
op _greaterThan_ : List List -> Boolean .
```

which compares two lists lexicographically, and test your definition in Maude.

Exercise 12 Represent the following binary tree as a term of sort `BinTree`.



Exercise 13 Define the remaining functions in the module `BINTREE-NAT1` in Maude.

Exercise 14 1. Define a sort `Bits` of lists of bits 0 and 1.

2. Define a function `neg : Bits -> Bits` that “flips” each bit in the list.

3. Define a function `_+_ : Bits Bits -> Bits` that adds two binary numbers (represented as `Bits`). For example, `(nil 1 0 1 1) + (nil 1 1 0)` (11+6) should return `nil 1 0 0 0 1` (17).

2.5 Order-Sorted Equational Specifications

Different sorts are not related in the many-sorted world. This hardly seems practical. For example, it is natural to have a sort `Nat` for the natural numbers and a sort `Int` for the integers. Using *only* the sort `Int` and forgetting about `Nat` is not very elegant, since some functions, such as the factorial function, are *partial* functions on the integers that do not take negative numbers as arguments. We have seen other partial functions, such as `first`, `last`, and `rest` on lists, which should only be defined on non-empty lists. To have two unrelated sorts `Int` and `Nat` is unsatisfactory as well, since it requires functions used both on natural numbers and integers to be defined twice, and does not allow the use of a natural number in place of an integer.

Maude supports *order-sorted specifications* (see e.g. [50,82]), in which a sort may have *subsorts*. Intuitively, a subsort declaration

```
subsort s' < s .
```

means that the sort s' is “included” in the sort s , in the sense that each element of s' is also an element of s . For example, since the natural numbers are a subset of the integers, it is natural to have `Nat < Int`. Multiple subsort declarations can be combined into a single one: `subsorts Nat Neg < Int .`, which states that both `Nat` and `Neg` are subsorts of `Int`. (A subsort declaration does *not* declare the sorts, so the above sorts must also be declared as usual).

Formally, the set of sorts is equipped with a *partial order* $\leq$ (see Appendix A). The subsort relation $\leq$ induces a subsort relation $\leq$ on lists of sorts of the same length, where $s_1 \dots s_n \leq s'_1 \dots s'_n$ holds if and only if $s_i \leq s'_i$ for each $1 \leq i \leq n$.

If `Nat` is a subsort of `Int`, a function which takes `Int` arguments will also accept `Nat` arguments, since any `Nat` value is also an `Int` value. For example, a function

```
op _+_ : Int Int -> Int .
```

also applies to natural numbers. One *could* add a declaration

```
op _+_ : Nat Nat -> Nat .
```

to tell Maude that the value of $m + n$ has sort `Nat` if both m and n have sort `Nat`. As explained in Section 2.6, such declarations of *subsort overloaded* functions are only needed for constructors, to ensure that each (sub)sort has the desired domain.

An *order-sorted signature* is a many-sorted signature with a partial order $\leq$ on the sorts:

Definition 2.7 (Order-sorted signature) *An order-sorted signature $(S, \leq, \Sigma)$ consists of a set S (of sorts), a partial order $\leq$ on S , and an $S^* \times S$ -sorted family $\Sigma = \{\Sigma_{w,s} \mid w \in S^*, s \in S\}$ of “function symbol declarations.”*

Terms are defined as expected: if $s' \leq s$, then a term of sort s' is also a term of sort s .

Definition 2.8 (Terms in order-sorted signatures) *Given an order-sorted signature $(S, \leq, \Sigma)$ and a variable set $X = \{X_s \mid s \in S\}$, the S -sorted set of terms $\mathcal{T}_\Sigma(X) = \{\mathcal{T}_{\Sigma,s}(X) \mid s \in S\}$ is defined by “adding” the following condition*

0. $\mathcal{T}_{\Sigma, s'}(X) \subseteq \mathcal{T}_{\Sigma, s}(X)$ if $s' \leq s$; that is, a term of a subsort s' is also a term of the supersort s .

to Definition 2.4, which defines the terms in a many-sorted signature.

The set of ground terms is defined as expected: $\mathcal{T}_{\Sigma} = \{\mathcal{T}_{\Sigma, s} \mid \mathcal{T}_{\Sigma, s} = \mathcal{T}_{\Sigma, s}(\emptyset), s \in S\}$.

The following example shows that the sort of a term could be ambiguous in the sense of the term having completely unrelated sorts, which is of course undesired:

```
sorts s1 s2 s12 u1 u2 .
subsorts s12 < s1 s2 .
op a : -> s1 .      op b : -> s2 .      op c : -> s12 .
op f : s1 -> u1 .    op f : s2 -> u2 .    op h : u1 -> u1 .
```

What is the sort of the term $f(c)$? Since c is an element of sort $s1$, the term $f(c)$ should have sort $u1$, but since c is also an element of sort $s2$, the term $f(c)$ should have sort $u2$. Such ambiguity is undesirable since $u1$ and $u2$ are unrelated (is, e.g., $h(f(c))$ a term?). Maude therefore requires that each non-constant term has a *unique least sort* as explained below. The specification would be OK if we added

```
sort u12 .      subsorts u12 < u1 u2 .      op f : s12 -> u12 .
```

since the smallest sort of $f(c)$ would then be $u12$.

Definition 2.9 (Least sort) A term $t \in \mathcal{T}_{\Sigma, s}(X)$ has a unique least sort if the set $\{s \mid t \in \mathcal{T}_{\Sigma, s}(X)\}$ of sorts of t has a unique smallest element w.r.t. $\leq$, in which case this unique least sort of t is denoted $LS(t)$.

Order-sorted signatures should be *preregular*, which ensures that each non-constant term has a unique least sort.

Definition 2.10 (Preregular signature) An order-sorted signature $(S, \leq, \Sigma)$ is pre-regular if for any function symbol declaration $f : s_1 \dots s_n \rightarrow s \in \Sigma$ with $n \geq 1$, and any sequence $s'_1 \dots s'_n$ with $s'_i \leq s_i$ for all i , the term $f(x_1, \dots, x_n)$, where x_i is a variable of sort s'_i for each i , has a unique least sort.

An order-sorted equational specification consists of an order-sorted signature and a set of unconditional and conditional equations, where the sorts of the terms t and t' in an equation $t = t'$ must be in the same *connected component*⁷ of the partially ordered set $(S, \leq)$ of sorts, and analogously for conditional equations [50]. (Intuitively, s and s' are in the same connected component of $(S, \leq)$ if there is a “path” from s to s' when you draw the partially ordered set $(S, \leq)$ as an undirected graph.)

2.5.1 Examples of Order-Sorted Equational Specifications

This section shows some uses of order-sorted specifications.

⁷ A connected component of $(S, \leq)$ is an equivalence class in the transitive and symmetric closure of $(S, \leq)$.

2.5.1.1 Partiality

We have not defined division on the natural numbers. The reason is that division is a *partial* function on the natural numbers, since $n/0$ is undefined for any n . The point is that we can define a subsort `NzNat`, for the *nonzero* natural numbers, of `Nat`, so that division is well-defined on (the domain defined by) the subsort.

```
sorts NzNat Nat .          subsort NzNat < Nat .
```

The constructors must be declared so that the constructor ground terms of sort `NzNat` are exactly all the nonzero positive numbers:

```
op 0 : -> Nat [ctor] .    op s : Nat -> NzNat [ctor] .
```

The division operator can then be declared to have only nonzero denominators:

```
op _/_ : Nat NzNat -> Nat .
```

A subsort `NeList` of `List` for *non-empty* lists can be defined in the same way, so that `first`, `last`, `rest`, and `max` become total functions on that subdomain:

```
sorts List NeList .        subsort NeList < List .
op nil : -> List [ctor] .  op __ : List Nat -> NeList [ctor] .
```

The first three of the above functions can then be defined as follows:

```
ops first last : NeList -> Nat .
op rest : NeList -> List .

var N : Nat .    var L : List .    var NEL : NeList .
eq first(nil N) = N .          eq first(NEL N) = first(NEL) .
eq last(L N) = N .
eq rest(nil N) = nil .          eq rest(NEL N) = rest(NEL) N .
```

Likewise, as mentioned, in the context of *integers*, a number of functions, such as the factorial and the Fibonacci functions, are partial functions that are only defined on the natural numbers, which leads us to the next topic.

2.5.1.2 Constructors for the Integers

Without subsorts it is fairly tricky to represent the *integers* so that each integer corresponds to exactly one constructor ground term, and vice versa. However, it is easy to have this desired one-to-one correspondence using the sort hierarchy

```
sorts Zero NzNat NzNeg Nat Neg Int .
subsorts Zero < Nat Neg < Int .
subsort NzNat < Nat .
subsort NzNeg < Neg .
```

`Zero` is the sort for 0; `NzNat` and `NzNeg` denote the nonzero natural and negative numbers, respectively; `Nat` and `Neg` all natural, respectively negative, numbers, including 0; and `Int` denotes all integers. The sort `NzInt` for nonzero integers is added to deal with division:

```
sort NzInt .      subsorts NzNat NzNeg < NzInt < Int .
```

We use the following well-known constructors for the natural numbers:

```
op 0 : -> Zero [ctor] .      op s : Nat -> NzNat [ctor] .
```

There are two intuitive ways of constructing the negative numbers. One is to *negate* a natural number to get a negative number (so that $-s(s(0))$ represents -2):

```
op _- : NzNat -> NzNeg [ctor prec 15] .
```

The other option is to use a “predecessor” function p , where $p(x)$ is the predecessor of x (that is, $x - 1$), just as $s(n)$ is the successor of n . Such a constructor is declared

```
op p : Neg -> NzNeg [ctor] .
```

In this case, -2 is represented by $p(p(0))$. In either case, it should be possible to see that each constructor term represents exactly one integer, and vice versa.

Addition and subtraction on the integers (using the constructor $-$ for the negative numbers) can then be defined as follows.

```
ops _+_ _- : Int Int -> Int [prec 33] .
```

```
vars M N : Nat .      var I : Int .      var NEG : Neg .
var NZNEG : NzNeg .   var NZN NZN' : NzNat .
```

First, addition on the natural numbers is defined in the usual way:

```
eq 0 + I = I .      eq s(M) + N = s(M + N) .
```

Subtraction on the naturals is defined as follows:

```
eq I - 0 = I .      eq 0 - NZN = - NZN .
eq s(M) - s(N) = M - N .
```

Addition on all integers can then be defined:⁸

```
eq - NZN + (- NZN') = - (NZN + NZN') .
eq M + (- NZN) = M - NZN .
eq (- NZN) + N = N - NZN .
```

Finally, we define subtraction on all integers:

```
eq 0 - (- NZN) = NZN .      eq (- NZN) - (- NZN') = NZN' - NZN .
eq M - (- NZN) = M + NZN .  eq (- NZN) - N = - (NZN + N) .
```

2.5.1.3 Elements in a List

Our lists have the form $\text{nil } n_1 \dots n_k$. It is possible to get rid of nil from this list by saying that a natural number is also a (non-empty) list:

```
sorts Nat NeList List .      subsort Nat < NeList < List .
op nil : -> List [ctor] .    op __ : NeList Nat -> NeList [ctor] .
```

⁸ The extra parentheses in the following equations are not needed, due to the precedence on the operators. They are just added for readability.

2.5.1.4 “Undefined” Values

An additional “error” or “uninitialized” value must sometimes be added to a sort. The following `supersort DefNat` adds such a constant `noNat` to the natural numbers:

```
sort DefNat .
subsort Nat < DefNat .
op noNat : -> DefNat [ctor] .
```

Exercise 15 Consider the following signature:

```
sorts s1 s2 s3 s4 .      subsorts s2 s3 < s4 .      subsort s2 < s1 .
op a : -> s3 .           op b : -> s2 .
op g : s3 s2 -> s1 .     op g : s2 s1 -> s2 .      op g : s1 s1 -> s4 .
```

1. Is the signature preregular?
2. Can you list at least 4 ground terms of sort s_4 ? Of sort s_1 ?
3. What is the least sort of the terms a and $g(b, g(b, g(a, b)))$?
4. Explain why we cannot add a declaration `op g : s4 s4 -> s4` and still have a preregular signature.

Exercise 16 Define the integer division function $/$, the multiplication function, and the functions in `NAT1` (see Exercise 9) on the integers.

Exercise 17 Define the integers and the above functions when the predecessor function is used as the constructor for the nonzero negative numbers.

Exercise 18 An attempt to define the comparison function $\leq$ could be

```
eq NEG <= N = true .      eq (- NZN) <= (- NZN') = NZN' <= NZN .
eq N <= NZNEG = false .   eq s(M) <= s(N) = M <= N .
```

Explain why these equations do not define $\leq$ for all pairs of integers. Then add the “missing” equation(s).

2.6 Membership Equational Logic Specifications

Order-sorted specifications have some limitations:

1. An important subset of binary trees are binary *search* trees,⁹ with functions, such as `insertSorted`, which inserts an element in the right place in a search tree, that only make sense for search trees. A subsort `SortedList` of sorted lists, with functions `insertSorted` and `merge`, would also be useful. Unfortunately, such “semantic” subsorts cannot be defined as order-sorted specifications.

⁹ These are binary trees where, for each subtree, the root element of the (sub)tree is greater than or equal to all elements in its left subtree and is less than or equal to all elements in its right subtree.

2. The subsort `NzInt` for *non-zero* integers was defined to avoid problems with division by 0, so that $s(s(0)) / 0$ is not a (well-formed) term. A side effect is that an expression like $s(s(0)) / (s(0) - 0)$ (i.e., $2/(1-0)$), which denotes a well-defined mathematical expression, is *not* a term, since the least sort of $s(0) - 0$ is `Int`. Likewise, we use a subsort `NeList` for non-empty lists to avoid problems with `first` and `last` of an empty list. However, this means that a sensible expression like `first(rest(nil s(0) s(s(0))))` is not a well-formed term, since `rest(nil s(0) s(s(0)))` is not a term of sort `NeList`.

Membership equational logic [82] is an elegant generalization of order-sorted specifications that solves these problems by allowing us to define *membership axioms*

`mb t : s .                      and                      cmb t : s if cond .`

stating that the term t (of some supersort of the sort s) is also a term of sort s (provided that the condition *cond*, consisting of a conjunction of memberships $t' : s'$ and equalities $u = u'$, holds in the case of *conditional* membership axioms). The subsort `SortedList` of `List` can then be defined as follows:

```
fmod SORTED-LIST-NAT1 is protecting LIST-NAT1 .
  sort SortedList .                      subsort SortedList < List .
  var L : List .
  cmb L : SortedList if isSorted(L) = true .
endfm
```

A term `nil 0 s(0)` is also a term of sort `SortedList`, whereas `nil s(0) 0` is not.

Considering our second problem, membership equational logic allows expressions like $s(s(0)) / (s(0) - 0)$ and gives them “the benefit of doubt.” Such an expression does not have a sort like `Int` but an “error sort” `[Int]`. The term $s(s(0)) / (s(0) - 0)$ is evaluated by computing wherever possible, and is reduced to $s(s(0)) / s(0)$ using the equations for `-`. This latter term is a well-formed term of sort `Int` and the computation can proceed to give the expected result:

```
Maude> red s(s(0)) / (s(0)) - 0) .
result NzNat: s(s(0))
```

The term $s(0) / (s(0) - s(0))$ is also given the benefit of doubt and is reduced to $s(0) / 0$, which does not have a sort and cannot be further reduced, and is therefore a term of “error sort” `[Int]`:

```
Maude> red s(0) / (s(0) - s(0)) .
result [Int]: s(0) / 0
```

The formal explanation of this possibility of giving a term “the benefit of doubt” is that each connected component of the partially ordered set $(S, \leq)$ of sorts has a *kind* in membership equational logic. We write $[s]$ for the kind of the connected component of sort s . Terms which do not have sorts and only have a kind can be seen as “error terms.” Maude automatically adds a declaration

`op f : [s1] ... [sn] -> [s] .`

for each declaration $\text{op } f : s_1 \dots s_n \rightarrow s$ in the specification. This means that our Maude specification of the integers (implicitly) also contains a declaration

$\text{op } _/_ : [\text{Int}] [\text{NzInt}] \rightarrow [\text{Int}]$.

Since $s(0) - s(0)$ is a term of sort Int , and therefore also of kind $[\text{Int}]$, the term $s(0) / (s(0) - s(0))$ has kind $[\text{Int}]$, due to the implicit declaration above and the fact that $[\text{NzInt}] = [\text{Int}]$. Since $s(0) / (s(0) - s(0))$ is a “well-kinded” term, it can be further reduced to the term $s(0) / 0$ of kind $[\text{Int}]$. This term cannot be reduced any further, and, although well-kinded, has no *sort*.

Exercise 19 Define a subsort *BinSearchTree* of *BinTree* for binary search trees and define a function *insertSorted* : *BinSearchTree* *Nat* $\rightarrow$ *BinSearchTree* which inserts an element in the right place in the tree.

2.7 Built-in Data Types

The representation of the natural numbers and integers we have seen so far is not very convenient for computing with large numbers. Maude therefore provides *built-in* versions of the natural numbers, the integers, the rational numbers, and the IEEE-754 double precision floating-point numbers, in addition to strings and Boolean values. These built-in modules provide the standard notation for numbers and strings, such as 2017, -273, 22/7, and “Maude”, and the expected operations on numbers and strings *efficiently* implemented in C++. In contrast to many programming languages, Maude provides an efficient implementation of *unbounded* natural numbers, integers, and rational numbers, instead of only 32-bits or 64-bits numbers.

These built-in data types are defined in the file `prelude.maude` which is read when you start Maude. You can modify this file if you feel like redefining the built-in modules or giving commands which should be executed when Maude starts. Only the built-in Booleans are included automatically into any user module; to import Maude’s natural numbers, you need to explicitly import the module `NAT` into your module in the usual way. To automatically include `NAT` into all your modules, just add the Maude command `set include NAT on .` to the file `prelude.maude`. This section briefly introduces some of Maude’s built-in modules; see the file `prelude.maude` for more details about these and other built-in modules.

2.7.1 Booleans

The module `BOOL` defines the Boolean values and some useful functions:¹⁰

¹⁰ Parts of the specifications are omitted and replaced by ‘...’.

```

fmod TRUTH-VALUE is
  sort Bool .
  op true : -> Bool [ctor special (id-hook SystemTrue)] .
  op false : -> Bool [ctor special (id-hook SystemFalse)] .
endfm

fmod BOOL-OPS is protecting TRUTH-VALUE .
  op _and_ : Bool Bool -> Bool [assoc comm prec 55] .
  op _or_ : Bool Bool -> Bool [assoc comm prec 59] .
  op _xor_ : Bool Bool -> Bool [assoc comm prec 57] .
  op not_ : Bool -> Bool [prec 53] .
  op _implies_ : Bool Bool -> Bool [gather (e E) prec 61] .
  vars A B C : Bool .
  eq true and A = A .          eq false and A = false .
  ...
endfm

fmod TRUTH is protecting TRUTH-VALUE .
  op if_then_else_fi : Bool Universal Universal -> Universal
    [poly (2 3 0) special (...)] .
  op _==_ : Universal Universal -> Bool
    [prec 51 poly (1 2) special (...)] .
  op _/= _ : Universal Universal -> Bool
    [prec 51 poly (1 2) special (...)] .
endfm

fmod BOOL is protecting BOOL-OPS . protecting TRUTH . endfm

```

The `special` attribute says that the function is a built-in operator/function implemented in C++. The attributes `assoc` and `comm` mean that the function is, respectively, associative and commutative; these attributes are explained in Section 2.8. We ignore the `gather` attribute (see [21] for an explanation of this parsing issue). The `poly` attribute states that the corresponding arguments (of sort `Universal`) may have *any* sort. The operator `if_then_else_fi` behaves as expected, $x == y$ equals `true` if and only x and y are equal (that is, reduce to the same term), and conversely for the inequality operator.

A condition $b = \text{true}$ in an equation can be written just b :

```
ceq M monus N = 0 if M <= N .
```

Finally, $t :: s$ is a term of sort `Bool` which is `true` if and only if the term t has sort s .

2.7.2 Natural Numbers

Maude provides the following module for *arbitrarily large* natural numbers, whose implementation uses the GNU GMP library [48].¹¹

¹¹ Multiple declarations of the same non-constructor function are usually not needed, since equations will reduce a term to a constructor term of the right sort. However, in built-in modules, operators such as `+` have multiple declarations, since it is a built-in function *not* defined by equations.

```

fmod NAT is protecting BOOL .
  sorts Zero NzNat Nat .          subsort Zero NzNat < Nat .
  op 0 : -> Zero [ctor] .
  op s_ : Nat -> NzNat [ctor iter special (...)] .
  op _+_ : NzNat Nat -> NzNat [assoc comm prec 33 special (...)] .
  op _+_ : Nat Nat -> Nat [ditto] .
  op sd : Nat Nat -> Nat [comm special (...)] .
  op _*_ : NzNat NzNat -> NzNat [assoc comm prec 31 special (...)] .
  op _*_ : Nat Nat -> Nat [ditto] .
  op _quo_ : Nat NzNat -> Nat [...] .
  op _rem_ : Nat NzNat -> Nat [...] .
  op _^_ : Nat Nat -> Nat [...] .
  op gcd : NzNat Nat -> NzNat [...] .
  op lcm : Nat Nat -> Nat [...] .
  op min : Nat Nat -> Nat [...] .
  op max : Nat Nat -> Nat [...] .
  ...
  op _<=_ : Nat Nat -> Bool [...] .
  op _>_ : Nat Nat -> Bool [...] .
  op _divides_ : NzNat Nat -> Bool [...] .
endfm

```

The constructors for `Nat` are `0` and `s`, so the natural numbers are represented by the terms `0`, `s 0`, `ss 0`, For convenience, we can also write `0`, `1`, `2`, ...:

```

Maude> red s s 0 + s s s 0 .
result NzNat: 5
Maude> red 1234567 * 89 .
result NzNat: 109876463

```

There is no subtraction function on the natural numbers (why?). Instead, the function `sd` denotes the (symmetric) difference between two numbers.

Example 2.7. The factorial function can be defined by induction on the constructors:

```

fmod FACTORIAL is protecting NAT .
  op _! : Nat -> Nat .
  var N : Nat .
  eq 0 ! = 1 .          eq (s N) ! = s N * (N !) .
endfm

```

or using the “standard” natural numbers and replacing the above equations with

$$\text{eq } N ! = \text{if } N == 0 \text{ then } 1 \text{ else } N * (\text{sd}(N, 1) !) \text{ fi} . \quad \diamond$$

The function `quo` defines division, `rem` the *remainder* function, `^` exponentiation ($m^n = m^n$), `gcd` denotes the greatest common divisor, `lcm` the least common multiple, and `<`, `<=`, `>`, and `>=` are the usual comparison operators. The module `NAT` also has bit manipulating functions such as bitwise and (`&`), bitwise or (`|`), bitwise xor (`xor`), right shift (`>>`), and left shift (`<<`).

Subsort overloaded operators must have the same attributes, except for `ctor`. The attribute `ditto` stands for *all attributes except* `ctor` in previous declarations of the same (subsort overloaded) function symbol.

2.7.3 Integers

The integers are constructed from the natural numbers using the constructor $-$, so that negative numbers can be written as $-s\ 0$, -2009 , ..., and also as -1 , -2009 , The built-in efficient implementation of (unbounded) integers are given in the following module (where many functions are not shown):

```
fmod INT is protecting NAT .
  sorts NzInt Int .          subsorts NzNat < NzInt Nat < Int .
  op _- : NzNat -> NzInt [ctor special (...)] .
  op _- : NzInt -> NzInt [ditto] .
  op _- : Int -> Int [ditto] .
  op _+_ : Int Int -> Int [assoc comm prec 33 special (...)] .
  op _- : Int Int -> Int [prec 33 gather (E e) special (...)] .
  op abs : Int -> Nat [...].
  ...
endfm
```

(abs gives the *absolute value* of a number.) The function $-$ is a constructor only on NzNat, and is a non-constructor on NzInt and Int.

2.7.4 Rational Numbers

The *rational numbers* are defined in the module RAT, which defines the sorts NzRat (non-zero rational numbers), PosRat (non-zero positive rational numbers), and Rat (all rational numbers), with all the expected functions:

```
fmod RAT is protecting INT .
  sorts PosRat NzRat Rat .
  subsorts NzInt < NzRat Int < Rat .
  subsorts NzNat < PosRat < NzRat .
  op _/_ : NzInt NzNat -> NzRat [ctor prec 31 ... special (...)] .
  op _/_ : NzNat NzNat -> PosRat [ctor ditto] .
  op _/_ : PosRat PosRat -> PosRat [ditto] .
  op _/_ : NzRat NzRat -> NzRat [ditto] .
  op _/_ : Rat NzRat -> Rat [ditto] .
  ...
  ops trunc floor : PosRat -> Nat .
  ops trunc floor ceiling : Rat -> Int .
  op ceiling : PosRat -> NzNat .
  op frac : Rat -> Rat .

  var I : NzInt .      var N M : NzNat .      var K : Int .
  eq trunc(K) = K .      eq trunc(I / N) = I quo N .
  eq floor(K) = K .      eq floor(N / M) = N quo M .
  eq floor(- N / M) = - ceiling(N / M) .
  eq ceiling(K) = K .    eq ceiling(N / M) = ((N + M) - 1) quo M .
  eq ceiling(- N / M) = - floor(N / M) .
  eq frac(K) = 0 .      eq frac(I / N) = (I rem N) / N .
endfm
```

2.7.5 Floating-Point Numbers

The built-in module `FLOAT` implements 64-bits IEEE-754 double precision *floating-point numbers* with all the expected functions such as `sqrt` (for square root), the trigonometric functions, the logarithm function, and so on.¹²

```
fmod FLOAT is protecting BOOL .
  sorts FiniteFloat Float .          subsort FiniteFloat < Float .
  op <Floats> : -> FiniteFloat [special (id-hook FloatSymbol)] .
  op <Floats> : -> Float [ditto] .
  ...
  op sqrt : Float ~> Float [...] .
  op log : Float ~> Float [...] .
  op sin : Float -> Float [...] .   op cos : Float -> Float [...] .
  op asin : Float ~> Float [...] .  op acos : Float ~> Float [...] .
  ...
endfm
```

The syntax `<Floats>` means that the constructors are built-in as a set of constants such as 1.0, -9.87654321, and -1.23e+14 (for $-1.23 \cdot 10^{14}$). The sort `Float` also contains two constants `Infinity` and `-Infinity` that denote out of range values:

```
Maude> red 3.45e+223 * 2.99e+210 .
result Float: Infinity
```

2.7.6 Strings

The built-in Maude module `STRING` defines the sort `String` of *strings* of the form "this is a string". Strings of length 1 are constants of a subsort `Char`.

```
fmod STRING is protecting NAT .
  sorts String Char FindResult .
  subsort Char < String .          subsort Nat < FindResult .
  op <Strings> : -> Char [special (id-hook StringSymbol)] .
  op <Strings> : -> String [ditto] .
  op notFound : -> FindResult [ctor] .

  op ascii : Char -> Nat [...] .   op char : Nat ~> Char [...] .
  op _+_ : String String -> String [...] .
  op length : String -> Nat [...] .
  op substr : String Nat Nat -> String [...] .
  op find : String String Nat -> FindResult [...] .
  op rfind : String String Nat -> FindResult [...] .
  op _<=_ : String String -> Bool [...] .
  ...
endfm
```

¹² The arrow `~>` means that the function is a *partial* function.

The function `ascii` gives the ASCII value of a character, `char` does the opposite, `+` denotes string concatenation, and `length` returns the length of a string. `substr(s,p,l)` returns the substring of *s* which starts at character *p* + 1 and is *l* characters long. `find(s1,s2,p)` finds the starting position (minus 1) of the substring *s*₂ in *s*₁, starting at character number *p* + 1 in *s*₁ (and returns `notFound` if *s*₂ is not such a substring of *s*₁). `rfind` does the same, but starts looking “from the right.” The comparison operators `<`, `<=`, `>`, and `>=` compare strings lexicographically.

The module `CONVERSION` defines functions for converting between numbers and strings, and between rational numbers and floating-point numbers. For example, `string(r,n)` takes a rational number *r* and a base *n* (between 2 and 36), and displays the number as a `String` in the given base. That is, `string(123,10)` equals `"123"` and `string(5,2)` equals `"101"`. The function `rat` does the opposite.

2.7.7 Random Numbers

The Maude module `RANDOM` provides a function `random`, where `random(k)` gives the *k*-th “pseudo-random” number as a number between 0 and $2^{32} - 1$. Since `random` is a *function*, `random(k)` gives the same result for the same *k*.

```
fmod RANDOM is protecting NAT .
  op random : Nat -> Nat [special (...)] .
endfm
```

To restrict the range of the “random” number, e.g., to a number between 1 and 100, we can use the expression `(random(k) rem 100) + 1`:

```
Maude> red random(1) .
result NzNat: 2546248239
Maude> red (random(2) rem 100) + 1 .
result NzNat: 34
```

Exercise 20 Define a function `isPrime : NzNat -> Bool` which returns `true` if and only if its argument is a prime number (that is, a number which is not divisible by any number except 1 and itself). Test your specification on 14091 (not a prime), 2 (prime), 31 (prime), and 135727 (?).

Exercise 21 Explain what the functions `trunc`, `floor`, `ceiling`, and `frac` in the module `RAT` are supposed to compute.

Exercise 22 American sports scores have the form `"49ers 39 Giants 38"`, while Europeans prefer the notation `"49ers - Giants 39-38"`. Define a function `europify : String -> String` which transforms a score from American format to European format. You may assume that there are no blanks in the name of a team.

Exercise 23 Define a function `binary : Nat -> Nat` . which gives the “binary” value of a natural number, so that e.g. `binary(7)` equals the number 111.

Exercise 24 Define a sort for Roman numerals (lists of I , V , X , L , C , D , and M), and functions *roman* and *decimal* that convert between Roman and decimal numbers smaller than 3500.

2.8 Associativity and Commutativity: Lists and Multisets

This section defines some *equational attributes*, such as associativity and commutativity, that enable us to define lists and (multi-)sets in a nice way, and that can make the definition of certain functions more elegant.

2.8.1 Commutativity, Associativity, and Identity

The (multi)sets $\{a, b\}$ and $\{b, a\}$ are the same, and therefore their representations should be equivalent. More generally, it is sometimes needed or useful to define a function f (such as, e.g., set union) to be commutative:

$$f(x, y) = f(y, x).$$

However, this equation leads to infinite loops $f(x, y) \rightsquigarrow f(y, x) \rightsquigarrow f(x, y) \rightsquigarrow \dots$. The Maude solution to having both commutativity and termination is to declare that “ f is commutative,” so that Maude always “keeps in mind” that f is commutative. We can declare that a function $\mathfrak{f}$ is commutative by giving it an attribute `comm`:

```
fmod COMM1 is
  sort s .      op f : s s -> s [comm] .      ops a b c : -> s .
  eq f(a, b) = b .
endfm
```

When a function is declared to be commutative, computations are no longer performed on *terms*, but on *C-equivalence classes of terms*, where C is the commutativity axiom $f(x, y) = f(y, x)$. In `COMM1`, the function $\mathfrak{f}$ is declared to be commutative, and one therefore works on the set $\mathcal{T}_{\Sigma, C} = \mathcal{T}_{\Sigma} / C$ of equivalence classes of terms

$$\mathcal{T}_{\Sigma, C} = \{[t]_C \mid t \in \mathcal{T}_{\Sigma}\}$$

modulo commutativity with $[t]_C = \{u \mid t \sim_C u\}$, where C is the equation $\{\mathfrak{f}(x, y) = \mathfrak{f}(y, x)\}$, and $t \sim_C u$ holds if and only if t and u are equal up to commutativity of $\mathfrak{f}$: that is, there are zero or more simplification steps $t \rightsquigarrow_C \dots \rightsquigarrow_C u$ from t to u using the above commutativity equation. For example,

$$[\mathfrak{f}(a, \mathfrak{f}(b, c))]_C = \{\mathfrak{f}(a, \mathfrak{f}(b, c)), \mathfrak{f}(a, \mathfrak{f}(c, b)), \mathfrak{f}(\mathfrak{f}(b, c), a), \mathfrak{f}(\mathfrak{f}(c, b), a)\}.$$

Notation: To avoid too many symbols, I most often write t for $[t]_C$.

$f(b, a)$ can be reduced to b in `COMM1`, since by $f(b, a)$ we mean $[f(b, a)]_C$ and

$$[f(b, a)]_C = [f(a, b)]_C \rightsquigarrow [b]_C.$$

The point is that one-to-one constructor bases, termination, confluence, definedness, etc., are now defined on C -equivalence classes of terms instead of on terms.

Example 2.8. A function `minimum` which returns the smallest of two integers can be elegantly defined by a single equation:

```
fmod MIN1 is protecting INT .
  op minimum : Int Int -> Int [comm] .
  vars I J : Int .
  ceq minimum(I, J) = I if I <= J .
endfm
```

◇

A binary function f is *associative* if

$$f(f(x, y), z) = f(x, f(y, z))$$

holds for all x, y, z . Addition on the integers is associative since $(x + y) + z = x + (y + z)$. We can declare a function to be associative using the `assoc` attribute:

```
op f : s s -> s [assoc] .
```

A term t is considered to be equivalent to a term u if they are equivalent modulo the associativity axiom; that is, if you can perform zero or more simplification steps to go from t to u using the associativity axiom *in both directions*. For example, the term $f(f(a, b), f(c, d))$ is considered the same as $f(a, f(b, f(c, d)))$, $f(f(f(a, b), c), d)$, $f(f(a, f(b, c)), d)$, and $f(a, f(f(b, c), d))$ when f is declared associative. Since the parentheses can be rearranged for associative operators, they are no longer needed for f and we can write $f(a, b, c, d)$ instead of the above terms. Likewise, if an infix symbol $+$ is declared associative, we can write $1 + 2 + 3$.

Although the associativity axiom $f(f(x, y), z) = f(x, f(y, z))$ does *not* cause non-termination, there are some good reasons to treat associativity in this way:

- Specifications of data types such as lists and sets/multisets are more elegant as we may omit parentheses and can define functions on such types more naturally.
- Although associativity by itself does not lead to nontermination, if used as an equation from left to right to simplify an expression, it leads to nontermination if the function is declared commutative: The specification

```
op f : s s -> s [comm] .
vars X Y Z : s .
eq f(f(X, Y), Z) = f(X, f(Y, Z)) .      *** Associativity
```

is nonterminating modulo commutativity since there is an infinite derivation

$$\begin{aligned} [f(f(a, b), c)]_C &\rightsquigarrow [f(a, f(b, c))]_C = [f(f(b, c), a)]_C \rightsquigarrow [f(b, f(c, a))]_C \\ &= [f(f(c, a), b)]_C \rightsquigarrow [f(c, f(a, b))]_C = [f(f(a, b), c)]_C \rightsquigarrow \dots \end{aligned}$$

Therefore, if f is declared commutative, associativity of f must be taken care of by adding `assoc` as an attribute:

```
op f : s s -> s [assoc comm] .
```

A binary function can also be defined to have an *identity element* t :

```
op f : s s -> s [id: t] .
```

which means that computations are performed modulo the equations

$$f(t, x) = x \quad \text{and} \quad f(x, t) = x.$$

That is, any term u of sort s is considered to be identical to $f(u, t)$ and $f(t, u)$. For example, in

```
sort s .
ops a b e : -> s [ctor] .      op f : s s -> s [id: e] .
vars X Y : s .                  eq f(X, Y) = a .
```

the term b reduces to a , since b is the same as $f(b, e)$. However, be careful with termination; even the seemingly terminating equation above is nonterminating, since it has an infinite computation $[a]_I = [f(a, e)]_I \rightsquigarrow [a]_I = [f(a, e)]_I \rightsquigarrow [a]_I = \dots$.

2.8.2 Associativity and Identity: Lists

Section 2.4.3.1 defines lists using a constructor `__` : `List Nat -> List` and a constant `nil`. All lists have the form $(\dots(((\text{nil } n_1) n_2) n_3) \dots) n_k$ (even though the parentheses may be omitted since there is only one way to parse a term). However, it is more natural to view lists as “flat” structures; this suggests the following representation of lists, in which an integer is also a list (of one element):

```
sort List .          subsort Int < List .
op nil : -> List [ctor] .
op __ : List List -> List [ctor assoc] .
```

Both `4` and `7` are terms of sort `List`, since `Int` is a subsort of `List`. These two lists can be concatenated using the concatenation operator `__`, so that `4 7` is also a term of sort `List`. This list can be concatenated with the list `11`, which gives a term `(4 7) 11`, which can be concatenated with the list `99` to get the list `((4 7) 11) 99`. Or, the two lists `4 7` and `11 99` can be concatenated into `(4 7) (11 99)`. Since `__` is declared to be associative, these two lists are the same list, and we can ignore parentheses: `4 7 11 99`.

Unfortunately, since `nil` is a term of sort `List`, also `nil 4` and `7 nil` are `Lists`, and so is their concatenation `nil 4 7 nil`. The good thing is that we can “eliminate” these `nils` by declaring `__` to have `nil` as its identity element:

```
op __ : List List -> List [ctor assoc id: nil] .
```

`nil 4` and `4` are now exactly the same list (i.e., $[\text{nil } 4]_{AI} = [4]_{AI}$), and so are therefore `nil 4 7 nil` and `4 7`. This gives the desired one-to-one correspondence between (equivalence classes of) constructor ground terms modulo associativity and identity of the list concatenation constructor and the set of all lists of integers.

A list is now either the empty list `nil` or has the form `i l`, for `i` an integer and `l` a list (since the one-element list `i` is identical to `i nil`) or, equivalently, the form `l i`. This is reflected in the definitions below, which are much simpler than the corresponding ones in Section 2.4.3.1:

```
fmod LIST-INT is protecting INT .
  sorts List NeList .          subsorts Int < NeList < List .

  op nil : -> List [ctor] .
  op __ : List List -> List [assoc id: nil ctor] .
  op __ : NeList NeList -> NeList [assoc id: nil ctor] .

  op length : List -> Nat .      ops first last : NeList -> Int .
  op empty? : List -> Bool .    op rest : NeList -> List .
  op reverse : List -> List .   op _occursIn_ : Int List -> Bool .
  op max : NeList -> Int .      op isSorted : List -> Bool .

  vars I J : Int .              var L : List .
  eq length(nil) = 0 .           eq length(I L) = 1 + length(L) .
  eq first(I L) = I .           eq last(L I) = I .
  eq I occursIn nil = false .
  eq I occursIn J L = (I == J) or (I occursIn L) .
  ...
endfm
```

2.8.3 Associativity, Commutativity, and Identity: Multisets and Sets

A *multiset* over a set S is a “set” of S -elements where the number of occurrences of each element matters: while the sets $\{a, b\}$ and $\{a, a, b\}$ are the same, the *multisets* $\{a, b\}$ and $\{a, a, b\}$ are different. (Formally, a multiset m over S is a function $m : S \rightarrow \mathbb{N}$ where $m(s)$ denotes the multiplicity (the number of occurrences) of s in m . A *finite multiset* is a multiset m whose *support* $\{s \mid s \in S \wedge m(s) > 0\}$ is a finite set.)

To compare two multisets over totally ordered sets like the integers, just remove (equally many of) the common elements in the multisets until no common elements remain; the one with the largest remaining element is the largest multiset (a non-empty multiset is greater than the empty multiset). For example, $\{2, 2, 1\}$ is greater than $\{1, 1, 0, 1, 2\}$, and $\{28099, 3, 8\}$ is greater than $\{28099, 7, 6, 5, 7, 5, 5, 6, 0, 1\}$.

A multiset can be seen as a “list” where the order of the elements does not matter. Finite multisets can therefore be understood as lists where the *multiset union* operator `__` is also commutative:

```
fmod MSET-INT is protecting INT .
  sorts Mset NeMset .          *** Multisets/non-empty multisets
  subsorts Int < NeMset < Mset .
```

```

op none : -> Mset [ctor] .          *** Empty multiset
op __ : Mset Mset -> Mset [ctor assoc comm id: none] .
op __ : NeMset NeMset -> NeMset [ctor assoc comm id: none] .

op size : Mset -> Nat .             *** # of elements in a multiset
op mult : Int Mset -> Nat .         *** Multiplicity of an element
op delete : Int Mset -> Mset .      *** Remove ONE occurrence
op _in_ : Int Mset -> Bool .        *** Is element in multiset?
op max : NeMset -> Int .            *** Largest element
op _>mul_ : Mset Mset -> Bool .    *** Multiset comparison

var I : Int .      var MS : Mset .
eq delete(I, I MS) = MS .
ceq delete(I, MS) = MS if not I in MS .
eq I in MS = mult(I, MS) > 0 .
...
endfm

```

A *set* is essentially a multiset where the multiplicity of elements does not matter. Sets of integers can therefore be defined as multisets of integers with the extra axiom $\text{eq } I \text{ } I = I$. (for I a variable of sort `Int`) which removes duplicates.¹³

Exercise 25 For each of the (equivalence classes of the) terms $f(f(b, a), a)$ and $f(b, b)$ and $f(f(a, b), f(b, a))$ and $f(c, a)$, compute its normal form in `COMM1` “by hand” and using Maude’s `red` command.

Exercise 26 Complete the module `LIST-INT` by defining the functions `empty?`, `rest`, `reverse`, `max`, and `isSorted`.

Exercise 27 Define the functions

```

op comesBeforeIn : Int Int List -> Bool .
op _>lex_ : List List -> Bool .

```

such that `comesBeforeIn(i, j, l)` is true if and only if there are elements i and j in the list l , and where the first occurrence of i comes before the first occurrence of j in l ; and where $l_1 >_{\text{lex}} l_2$ is true if l_1 is lexicographically greater than l_2 (see Exercise 11 for the definition of lexicographic comparison).

Exercise 28 1. Define a sort `String` for lists of characters $a, b, \dots, z$.

2. Define a function `isPal : String -> Bool` so that `isPal(s)` returns true if and only if s is a palindrome, that is, reads the same backwards and forwards. For example, `anna` and `bob` are palindromes, whereas `peter` is not.
3. Define a function `_prefixOf_ : String String -> Bool` that checks whether the first argument is a prefix of the second argument.
4. Define a function `_substringOf_ : String String -> Bool` that checks whether the first argument is a substring of the second argument.

¹³ Maude has an idempotency attribute, but currently it cannot be used with the `assoc` attribute.

5. Define a supersort *Pattern of String* for strings that may contain the symbol ‘?’, which is a “wild card” that matches any single character.
6. Define functions `_prefixOf_ : Pattern String -> Bool` and `_substringOf_ : Pattern String -> Bool` that check whether the first argument “matches” a prefix, respectively, a substring, of the second argument. For example, `b ? d e ? g substringOf a b c d e f g h` should return `true`.
7. (Trickier?) Repeat the exercises above for patterns that may contain the symbol ‘*’ that can stand for any sequence of characters.

Exercise 29 Explain why `delete(2017, 1 2 2017 3)` returns the multiset `1 2 3`.

Exercise 30 Define the functions `size`, `mult`, `max`, `empty?`, and the multiset comparison operator `>mul` in the module `MSET-INT`.

Exercise 31 Show that for any multiset m_0 over the natural numbers, there is no infinite sequence

$$m_0 > m_1 > m_2 > m_3 > \dots$$

of multisets $m_0, m_1, m_2, m_3, \dots$ such that each m_i is greater than m_{i+1} .

Exercise 32 Assume that we have already defined two sorts *Obj* and *Msg*. Define a sort *Mset-ObjMsg* whose elements are multisets of *Obj* and *Msg* elements (that is, a multiset may contain both *Obj* and *Msg* elements).

Exercise 33 Define a data type of sets of integers with functions `in` (does the given number belong to the set?), `delete` (remove an element from a set), `card` (the cardinality (number of distinct elements) of a set), `setMinus` (set difference), and `intersect` (the intersection of two sets). Make sure that your specification is confluent. `delete(1, 0 1 2 1)` should give `0 2` no matter how the equations are applied. Similarly, the cardinality of the set `0 1 2 1 1` is 3.

Exercise 34 1. Define a data type *StringList* of lists of strings. In this exercise you will use both lists and sets, and you are therefore advised to use a symbol other than `__` (such as `_:`) for list concatenation. Only define the functions you will need in this exercise.

2. Define a data type *Set-StringList* of sets of lists of strings.
3. Define a function `perm : StringList -> Set-StringList` which takes a list of strings and returns the set of all permutations of this list. (A permutation of a list is a list where the elements are the same but are “rearranged.”) For instance, the set of all permutations of the list `"a" : "b" : "c"` is

`("a" : "b" : "c") ("a" : "c" : "b") ("b" : "a" : "c")`
`("b" : "c" : "a") ("c" : "a" : "b") ("c" : "b" : "a")`

Hint: It might be helpful to use an auxiliary function p , where $p(L_1, L_2, L_3)$ generates all permutations of $L_1 : L_2 : L_3$ which start with L_1 , and where the next string is taken from the list L_2 . The L_3 -elements have already been used.

2.9 Examples

This section shows how the sorting algorithms *quicksort* and *mergesort*, as well as solutions to classic NP-complete problems like *subset sum* and *Hamiltonian circuit*, can be formally specified in Maude. Such a specification has a number of benefits:

- In contrast to prose and pseudo-code (and even an imperative program), a Maude specification gives a precise, un-ambiguous specification of the algorithm.
- The *specification* is also at the same time a *program*, defined in a simpler and less error-prone way than, e.g., a Java implementation.¹⁴
- It is possible to reason mathematically about the Maude specification; it is also much easier to reason informally about the correctness of the Maude program than about the Java program, since we can focus on checking the correctness of *single* equations, instead of having to reason about the entire program.

2.9.1 Two Sorting Algorithms

2.9.1.1 Quicksort

The quicksort algorithm sorts a list L as follows:

1. Select a specific element N , called the *pivot*, in the list L .
2. Recursively sort the list of all elements in L smaller than N .
3. Recursively sort the list of all elements in L greater than N .
4. Concatenate the following lists: the list obtained in step 2, the list of all elements in L equal to N , and the list obtained in step 3.

The pivot could be *any* element in the list. The textbook [52] says that “For instance, let the pivot N be the last element.” The following Maude definition is a more general *specification* than the textbook description, since it chooses the pivot N nondeterministically instead of being forced to choose “for instance” the last element.

```
fmod QUICK-SORT is protecting LIST-INT .
  op quicksort : List -> List .

  vars L L' : List .      vars M N : Int .
  eq quicksort(nil) = nil .
  eq quicksort(L N L') = quicksort(smallerElements(L L', N))
                        equalElements(L N L', N)
                        quicksort(greaterElements(L L', N)) .
```

where `smallerElements( $l, n$ )` contains the elements in l that are smaller than n :

¹⁴ Is it `i=0` or `i=1`? `j=i` or `j=i+1`? `i++` or `++i`? Until `j>k` or `j>=k`? A `-1` or `+1` missing somewhere?

```

ops smallerElements greaterElements
   equalElements : List Int -> List .

eq smallerElements(nil, N) = nil .
eq smallerElements(N L, M) = if N < M then
    (N smallerElements(L, M))
    else smallerElements(L, M) fi .
eq equalElements(nil, N) = nil .
eq equalElements(N L, M) = if N == M then (N equalElements(L, M))
    else equalElements(L, M) fi .
eq greaterElements(nil, N) = nil .
eq greaterElements(N L, M) = if N > M then
    (N greaterElements(L, M))
    else greaterElements(L, M) fi .

endfm

```

2.9.1.2 Mergesort

The mergesort algorithm for sorting a list L works as follows [52]:

1. If L has at least two elements (otherwise, there is nothing to do): split the list L into two “sublists” L_1 and L_2 of (about) equal size.
2. Recursively sort the (sub)lists L_1 and L_2 .
3. *Merge* the two lists obtained in step 2.

Mergesort can be specified in Maude as follows:

```

fmod MERGE-SORT is protecting LIST-INT .
op mergeSort : List -> List .
op merge : List List -> List [comm] .

vars L L' : List .   vars NEL NEL' : NeList .   vars I J : Int .

eq mergeSort(nil) = nil .
eq mergeSort(I) = I .
ceq mergeSort(NEL NEL') = merge(mergeSort(NEL), mergeSort(NEL'))
if length(NEL) == length(NEL') or length(NEL) == s length(NEL') .

eq merge(nil, L) = L .
ceq merge(I L, J L') = I merge(L, J L') if I <= J .

endfm

```

The *raison d'être* for mergesort is that its execution time is $O(n \log n)$. The above specification may be less efficient, since splitting a list into two halves is done by matching. The usefulness of this kind of specification is that it is a precise description of a complex algorithm, and that it is at the same time a *prototype* of your algorithm that can be developed *quickly* to *test* and further analyze your algorithm *before* a detailed and efficient algorithm is implemented in all its glory.

2.9.2 Some NP-Complete Problems

Some problems, such as deciding whether or not a given equational specification is terminating, are in general *undecidable*: there is no algorithm that can always determine whether or not a given specification E is terminating. However, even if a problem *is* decidable, it may not have an *efficient* solution. For example, unless “ $P = NP$ ” (one of the Millennium Prize Problems), the *NP-complete* problems cannot be solved by deterministic algorithms in time that is a polynomial (like $5n^3 + 12n^2$) of the length n of (a reasonably efficient encoding of) the problem instance [22, 47]. Algorithms with exponential running times, performing, e.g., order of magnitude 2^n or $n!$ operations for some input parameter n , quickly become hopelessly inefficient once n grows.¹⁵ It takes more than 40 trillion years (almost 3000 times the age of the universe) to perform 2^{100} operations if you could do a billion operations per second.

Maude is not well suited for complexity analysis—where the run time is defined as the number of “machine instructions” performed during the execution¹⁶—since its powerful programming mechanisms abstract away the “machine steps” performed in a computation. This section nevertheless specifies some classical NP-complete problems, both to get acquainted with some iconic problems in computer science and to illustrate how Maude can specify prototypical combinatorial algorithms: one that amounts to finding a suitable subset (Subset Sum) and one that amounts to finding an appropriate sequence of n nodes (Hamiltonian Circuit).

If your decision problem is “more difficult” than a known NP-complete problem, then your problem is also NP-complete (or worse), and hence has no efficient solution unless $P = NP$. More precisely, if your problem P can be solved by a function

```
op P : "Problem Instance" -> Bool .
```

and a known NP-complete problem Q can be solved by an equation

```
eq Q(I) = P(f(I)) .
```

where f transforms (in polynomial time) an instance I of the problem Q to an instance $f(I)$ of your problem P , then P is also an NP-complete problem.

2.9.2.1 Canonical NP-Complete Problems

The original NP-complete problem identified by Stephen Cooke in 1971 [22] is **Boolean Satisfiability**: Given a Boolean expression B involving a number of variables, is there a substitution σ such that $B\sigma$ is `true`? For example, the Boolean expression $(x \vee \neg y) \wedge (\neg x \vee y \vee z) \wedge (\neg y \vee \neg z)$ is satisfiable with substitution $\{x \mapsto \text{true}, y \mapsto \text{true}, z \mapsto \text{false}\}$, but $\neg x \wedge (x \vee z) \wedge (\neg z \vee \neg y) \wedge (x \vee y)$ is *not* satisfiable.

¹⁵ There are 2^n different subsets of an n -element set, and $n!$ different permutations of n elements.

¹⁶ The complexity of an algorithm can be precisely defined in a machine-independent way as the number of steps performed by a Turing machine implementing the algorithm.

Thousands of other problems have been identified to be NP-complete (see, for example, [47]). Some of the best known are the following:

Partition. Given a multiset of natural numbers, can we partition (all) the numbers into two multisets with the same sum? For example, the multiset $\{8, 7, 11, 17, 5\}$ can be partitioned into two equal-summed multisets $\{8, 11, 5\}$ and $\{7, 17\}$, whereas the multiset $\{8, 7, 11, 19, 5\}$ cannot be so partitioned.

Subset Sum. Given a multiset M of natural numbers and a natural number K , can we pick numbers from M with sum K ? For example, it is possible to pick elements from $\{8, 7, 11, 17, 5\}$ with sum $K = 22$, but not with sum $K = 21$.

Hamiltonian Circuit. Given an undirected graph G , is there a circuit/loop/path from some node n to itself that visits *all* (other) nodes exactly once? For example, in the graph on the left in Fig. 2.2, there is such a Hamiltonian circuit $a \rightarrow b \rightarrow d \rightarrow f \rightarrow e \rightarrow c \rightarrow a$, while there is no such circuit in the graph on the right.

Clique. Given a graph G and a number K , is there a *clique*, i.e., a subset of the nodes where each node in the subset has an edge to each other node in the subset, with at least K nodes? The graph on the left in Fig. 2.2 has two cliques of size 3 but no clique of size 4, whereas the graph on the right has a clique of size 4.

Subgraph Isomorphism. Given graphs G and H , can we remove nodes and edges from G so that the resulting graph is *isomorphic* to (i.e., has the same structure as) H ? For example, if we add edges $a \leftrightarrow f$ and $b \leftrightarrow e$ to the graph on the left in Fig. 2.2, then a subgraph of this graph would be isomorphic to the graph on the right.

Traveling Salesman. Given a set of cities, the cost associated with traveling between any two cities, and a total budget K , can the traveling salesman visit all cities exactly once, and come home again, for a total cost less than or equal to K ?

Knapsack. We are going on a round-the-world trip and want to maximize the value of the stuff that we can carry in our backpack. Formally, given a set of items, each with a weight and a value, a weight limit K , and a value limit L , can we pick items with total weight $\leq K$ and total value $\geq L$?

Multiprocessor Scheduling. Given a set of (non-preemptive) tasks, each of which takes a certain amount of time, a number of processors, and a deadline, can we distribute the tasks to the processors so that all tasks finish before the deadline?

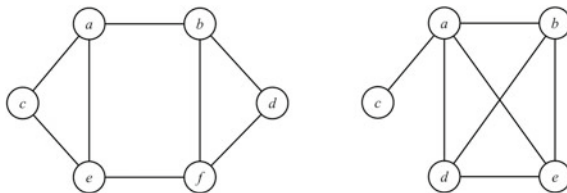


Fig. 2.2 Two undirected graphs

It is fairly easy to see that some of these problems are NP-complete, if we know that some other problems are NP-complete. For example, if Partition is NP-complete, then it follows that Subset Sum is NP-complete, since Partition is just a special case of Subset Sum (why?).

2.9.2.2 Specifying Subset Sum in Maude

A brute-force Maude solution to Subset Sum considers the elements one-by-one, and either picks the current element or does not pick it. This is a common solution to many problems consisting of finding a suitable subset of a given set of elements.

We first define multisets of non-zero natural numbers in the standard way:

```
fmod SUBSET-SUM is protecting NAT .
  sort MSet .      subsort NzNat < MSet .
  op none : -> MSet [ctor] .
  op __ : MSet MSet -> MSet [ctor assoc comm id: none] .
```

We define the function `subsetSum`, so that `subsetSum(numbers, K)` is true if and only if there is a subset of *numbers* with sum *K*:

```
op subsetSum : MSet NzNat -> Bool .
```

The following equations take care of the two base cases: (i) there are no (remaining) elements to choose from, and the (remaining) desired sum is a positive number *NZN*; and (ii) the desired (remaining) sum is *NZN* and there is a number *NZN* in the multiset:

```
vars NZN NZN1 NZN2 : NzNat .      var S : MSet .
eq subsetSum(none, NZN) = false .
eq subsetSum(NZN S, NZN) = true .
```

In the recursive case, we are left with some numbers *NZN1 S* and the desired sum *NZN2*. From those elements, there is a subset with sum *NZN2* if and only if either:

1. `subsetSum(S, NZN2 - NZN1)` holds; i.e., there is a subset of the elements *S* with sum *NZN2* - *NZN1*, which means that there is a subset of the elements *NZN1 S* with sum *NZN2*; or
2. there is a subset of the elements *S* with sum *NZN2*, which of course implies that there is also a subset of *NZN1 S* with sum *NZN2*.

The corresponding Maude specification is:

```
ceq subsetSum(NZN1 S, NZN2)
  = subsetSum(S, sd(NZN2, NZN1))    --- pick element NZN1
  or subsetSum(S, NZN2)             --- or don't pick NZN1
  if NZN2 > NZN1 .

ceq subsetSum(NZN1 S, NZN2) = subsetSum(S, NZN2) if NZN1 > NZN2 .
--- cannot pick element NZN1
endfm
```

Let us check our specification:

```
Maude> red subsetSum(7 3 5 12, 18) .
result Bool: false
Maude> red subsetSum(7 3 5 12, 15) .
result Bool: true
```

2.9.2.3 Specifying Hamiltonian Circuit in Maude

Since Hamiltonian Circuit is a graph problem, this section first shows one way of representing graphs in Maude.

The following module represents a graph as a set of nodes `node: n` `nbs: nbs`, where n is the name of the node and nbs is the (names of the) neighbors of n . An undirected edge between nodes n_1 and n_2 must be represented twice: n_2 must be in the set of neighbors of node n_1 , and vice versa:

```
fmod GRAPH is
  sort NodeId .          --- application-specific node names
  sort NodeIdSet .       subsort NodeId < NodeIdSet .
  op none : -> NodeIdSet [ctor] .
  op _ : NodeIdSet NodeIdSet -> NodeIdSet
                                [ctor assoc comm id: none] .

  sort Node .
  op node:_nbs:_ : NodeId NodeIdSet -> Node [ctor] .

  sort Graph .          --- multiset of nodes
  subsort Node < Graph .
  op emptyGraph : -> Graph [ctor] .
  op _;_ : Graph Graph -> Graph [ctor assoc comm id: emptyGraph] .
endfm
```

The graph on the left in Fig. 2.2 is represented as the term

```
(node: a nbs: b c e) ; (node: b nbs: a f d) ; (node: c nbs: a e) ;
(node: d nbs: b f) ; (node: e nbs: a c f) ; (node: f nbs: b d e) .
```

The brute-force way to solve the Hamiltonian Circuit problem goes as follows:

1. Select any node as the starting node, and also as the “current node.”
2. For each neighbor n of the “current node”: either that neighbor n is the next node in the circuit, in which case n becomes the “current” node, or the neighbor n is *not* the next node in the circuit.
3. When all nodes are included in the path, check whether there is an edge from the last (“current”) node to the starting node. If so, there is a Hamiltonian circuit.

The function `hamiltonianCircuit : Graph -> Bool` that checks whether an undirected graph has a Hamiltonian Circuit can then be defined as follows in Maude; this solution assumes that a graph always has at least three nodes.

```
fmod HAMILTONIAN-CIRCUIT is including GRAPH .
  op hamiltonianCircuit : Graph -> Bool .
```

This function calls the function `hCircuit`, where `hCircuit (startNode, currNbs, remainingNodes)` holds if and only if there is a path from one of the nodes in `currNbs` that visits all nodes in `remainingNodes` once and there is an edge from the last node in that path back to `startNode`. In other words, `startNode` is the start node, we have built a path which may or may not be the beginning of a Hamiltonian circuit

from *startNode*, this path includes all the nodes that are *not* in *remainingNodes*, the “current” node (the last node in the path we are building) has neighbors *currNbs*, and *remainingNodes* are the nodes not yet in the path.

The following equation takes an arbitrary node *N* as the starting node:

```
vars N N1 : NodeId .      vars NBS NBS2 : NodeIdSet .
var NS : Graph .          var NODE : Node .

eq hamiltonianCircuit((node: N nbs: NBS) ; NS) = hCircuit(N,NBS,NS) .
op hCircuit : NodeId NodeIdSet Graph -> Bool .
```

In the following equation, the “current” node has (remaining) neighbors *N1 NBS*, and there is a node *N1* that has not yet been visited in the path. There are now two choices: either *N1* is the next node in the path, in which case we remove the node *N1* from the remaining nodes and update the “current neighbors” to *N1*’s neighbors *NBS2*, or *N1* is *not* the next node in the path, in which case we “forget” *N1* from the current neighbors and try the other neighbors *NBS*:

```
eq hCircuit(N, N1 NBS, (node: N1 nbs: NBS2) ; NS)
  = hCircuit(N, NBS2, NS)          --- try N1 as the next node
  or hCircuit(N, NBS, (node: N1 nbs: NBS2) ; NS) . --- or not
```

A neighbor *N1* of the “current” node is ignored if it has already been visited:

```
ceq hCircuit(N, N1 NBS, NODE ; NS) =
  hCircuit(N, NBS, NODE ; NS) if not (N1 in NODE ; NS) .

op _in_ : NodeId Graph -> Bool .
eq N in ((node: N1 nbs: NBS2) ; NS) = (N == N1) or (N in NS) .
eq N in emptyGraph = false .
```

If there are nodes yet to be visited but there is no (remaining) edge from the current node, then the current path cannot be extended into a Hamiltonian circuit:

```
eq hCircuit(N, none, NODE ; NS) = false .
```

If there are no unvisited nodes, the current path can be extended to a Hamiltonian circuit if and only if the starting node is a neighbor of the “current” (last) node:

```
eq hCircuit(N, NBS, emptyGraph) = (N in NBS) .

op _in_ : NodeId NodeIdSet -> Bool .
eq N in none = false.   eq N in N1 NBS = (N == N1) or N in NBS .
endfm
```

Exercise 35 Define a version of quicksort which, for lists of at least two elements, will look at the first and the last element in the list, and choose as pivot element the number $\frac{\text{first} + \text{last}}{2}$. (It is possible that such a number is not an element in the list, but that does not matter.) Explain also why your specification is terminating.

Exercise 36 Modify the specification of mergesort so that *mergeSort* is called recursively only when the list has at least three elements.

Exercise 37 Specify the insertion sort algorithm in Maude. Insertion sort works as when you get some cards and have to sort them: you take the (unsorted) cards one by one, and put them into the right place in your hand, which always remains sorted.

Exercise 38 In the Unbounded Subset Sum problem we can use each number in the given multiset as many times as we want in order to achieve the desired sum. Specify a function `op unboundedSubsetSum : MSet NzNat -> Bool` which solves this problem. Is it easy to see that the new problem is NP-complete?

Exercise 39 Consider the Traveling Salesman problem, where the cost of a trip between two cities is given by a function `cost : City City -> NzNat [comm]`. Exercise 125 shows an example of such a `cost` function.

1. Specify a function `travelingSalesman : Cities NzNat -> Bool` so that `travelingSalesman(cities, budget)` is `true` if and only if there is a tour visiting all cities in `cities` (once) that does not cost more than `budget`.
2. Show that Traveling Salesman is NP-complete by showing that a solution for it easily can solve one of the other NP-complete problems in Section 2.9.2.
3. It is sometimes more expensive to travel directly between A and B than to travel from A to C and then to B. Specify a solution to the traveling salesman problem when the salesperson may visit a city more than once if needed.

Exercise 40 Explain how you can use a solution to the Subgraph Isomorphism problem to solve two other NP-complete problems (which ones?) in Section 2.9.2.

Exercise 41 Define a function `clique : Graph NzNat -> Bool` so that `clique(G, K)` is `true` if and only if `G` contains a clique with (at least) `K` nodes.

- Exercise 42**
1. Show that Knapsack and Multiprocessor Scheduling are NP-complete problems.
 2. Define a solution to the Knapsack problem in Maude, and then define a solution to the Integer Knapsack problem, where each item can be used multiple times.

2.10 * Some Other Maude Features

Maude has a number of useful features that will not be mentioned elsewhere in this book; the reader is referred to the Maude book [21] or the Maude manual for details.

2.10.1 Parameterized Modules

Instead of defining a data type, such as lists, from scratch for each kind of list (lists of integers, lists of strings, lists of lists of . . . , and so on), we can define *parameterized modules*. Assume that we want to define a *generic* mergesort function that can sort all kinds of lists, as long as we can compare the elements in the list. A parameter

for this generic function must have a sort for the elements and a total order on those elements. The “formal parameter” for this function is defined as the *theory*

```
fth TOTAL-ORDER is protecting BOOL .
  sort Element .
  op _le_ : Element Element -> Bool .
  vars E E1 E2 E3 : Element .
  --- reflexivity, anti-symmetry, transitivity, and totality:
  eq E le E = true [nonexec] .
  ceq E1 = E2 if (E1 le E2) and (E2 le E1) [nonexec] .
  ceq E1 le E3 = true if (E1 le E2) and (E2 le E3) [nonexec] .
  eq (E1 le E2) or (E2 le E1) = true [nonexec] .
endfth
```

This theory defines an “interface” or formal parameter `TOTAL-ORDER` that any actual parameter must “satisfy.” That is, an actual parameter must interpret the sort `Element` and the function symbol `le` (the comparison operator), so that the four equations for a total order are satisfied.

The parametric mergesort module is then given as follows:

```
fmod PARAM-SORT{X :: TOTAL-ORDER} is protecting INT .
  sorts List NeList .          subsort X$Element < NeList < List .
  op nil : -> List [ctor] .
  op __ : List List -> List [ctor assoc id: nil] .
  op __ : NeList NeList -> NeList [ctor assoc id: nil] .

  op #_ : List -> Nat .      eq # nil = 0 .      eq # (E1 L) = 1 + # L .
  op mergeSort : List -> List .
  op merge : List List -> List [comm] .
  vars L L' : List .          vars NEL NEL' : NeList .
  vars E1 E2 : X$Element .
  eq mergeSort(nil) = nil .      eq mergeSort(E1) = E1 .
  ceq mergeSort(NEL NEL') = merge(mergeSort(NEL), mergeSort(NEL'))
    if (# NEL == # NEL') or (# NEL == s ( # NEL')) .
  eq merge(nil, L) = L .
  ceq merge(E1 L, E2 L') = E1 merge(L, E2 L') if E1 le E2 .
endfm
```

The module defines lists of the sort `Element` of the parameter `X`. The rest is our mergesort function, with the comparison operator `le` used to compare elements.

Views define how the actual parameter module interprets the formal parameter module. A view maps the sorts (resp. operators) of the formal parameter to sorts (resp. operators or even expressions) in the actual parameter. For example, the following view `Int<=` says that we want to use `INT` as the actual parameter, with the sort `Element` mapped to the sort `Int`, and with the function `le` mapped to `<=`:

```
view Int<= from TOTAL-ORDER to INT is
  sort Element to Int .
  op _le_ to _<=_ .
endv
```

The following module `SORT-INT<=` then defines lists of integers and the mergesort function w.r.t. the comparison operator `<=`:

```
Maude> fmod SORT-INT<= is protecting PARAM-SORT{Int<=} . endfm
Maude> red mergeSort(5 2 11 23 -4 8) .
result NeList: -4 2 5 8 11 23
```

We can use the view

```
view Int>= from TOTAL-ORDER to INT is
  sort Element to Int .
  op _le_ to _>=_ .
endv
```

to sort lists of integers in decreasing order instead:

```
Maude> fmod SORT-INT>= is protecting PARAM-SORT{Int>=} . endfm
Maude> red mergeSort(5 2 11 23 -4 8) .
result NeList: 23 11 8 5 2 -4
```

We can also define lists of strings using the view

```
view String<= from TOTAL-ORDER to STRING is
  sort Element to String .
  op _le_ to _<=_ .
endv
```

which allows us to sort lists of strings according to `<=` on strings:

```
Maude> fmod SORT-STRINGS is protecting PARAM-SORT{String<=}. endfm
Maude> red mergeSort("Hi" "how" "are" "you" "today") .
result NeList: "Hi" "are" "how" "today" "you"
```

2.10.2 Telling Maude how to Evaluate an Expression

Consider a standard definition of the factorial function:

```
eq N ! = if N > 0 then N * sd(N,1) ! else 1 fi .
```

Although `if_then_else_fi` is a built-in function, we could assume that it is explicitly defined by the following equations:

```
eq if true then X else Y fi = X .
eq if false then X else Y fi = Y .
```

The specification of `!` is nonterminating since we have the following derivation:

```
0 !  ~>  if 0 > 0 then 0 * sd(0,1) ! else 1 fi
      ~>  if 0 > 0 then 0 * 1 ! else 1 fi
      ~>  if 0 > 0 then
            0 * (if 1 > 0 then 1 * sd(1,1) ! else 1 fi) else 1 fi
      ~>  if 0 > 0 then
            0 * (if 1 > 0 then 1 * 0 ! else 1 fi) else 1 fi
      ~>  ...
```


Since the derivation started with $0!$ and has reached a term containing $0!$, the specification is nonterminating. The point is that we *assume* that `if_then_else-fi` first computes the value of its first argument, and then evaluates “itself” using the `if_then_else-fi`-equations above. However, a term `if b then t else u fi` *could* equally well be evaluated by *first* evaluating t , as happened above.

To avoid such undesired computations, and to increase the efficiency of Maude computations, we can tell Maude how to evaluate a term by defining an *evaluation strategy* of a function using the attribute `strat`. For example, a declaration

```
op f : s1 s2 s3 -> s [strat (2 0 1 3 0)] .
```

tells Maude to first evaluate the second argument (2), then the whole term (0), then the first argument (1), and so on. That is, an expression $f(t_1, t_2, t_3)$ will be evaluated by first reducing t_2 as much as possible to t'_2 , and then simplify the term $f(t_1, t'_2, t_3)$ “at the top” using f -equations. If the resulting term still has the form $f(u_1, u_2, u_3)$, then u_2 is again evaluated, and so on. For example, `if_then_else-fi` should have the attribute `strat (1 0 2 3 0)` (or even `strat (1 0)`), which states that the test is computed first, followed by the application of an `if_then_else-fi`-equation.

Maude’s default evaluation strategy of a function is $(1\ 2\ \dots\ n\ 0)$. This strategy, in which all subterms are evaluated *before* the entire term is evaluated, is called *eager* evaluation. A strategy that starts with 0 denotes *lazy* evaluation, since subterms are not computed before the entire term is evaluated.

The choice of evaluation strategy can have a significant impact on the efficiency. For example, efficient evaluation strategies for a function f defined by $f(x, y, z) = y$ are $(2\ 0)$ or (0) , whereas the strategy $(1\ 3\ 2\ 0)$ is very inefficient (why?).

2.10.3 Other Features

owise Equations. An equation of the form $f(\dots) = t$ with the `owise` (for “other-wise”) attribute can only be applied if no other equation for f can be applied. This greatly simplifies the definition of some functions, as shown below:

```
vars I J : Int .      vars L L1 L2 L3 : List .
var N : Nat .         vars MS MS' : Mset .
eq I occursIn L1 I L2 = true .
eq I occursIn L = false [owise] .

ceq isSorted(L1 I L2 J L3) = false if I > J .
eq isSorted(L) = true [owise] .

eq I in I MS = true .
eq I in MS = false [owise] .
```

It is worth remarking how easily the NP-complete Subset Sum problem can be solved using the `owise` attribute and `assoc` and `comm` symbols:

```
ceq subsetSum(MS MS', N) = true if sum(MS) == N .
eq subsetSum(MS, N) = false [owise] .
```

As explained in [21], the `owise` construct is not an extra-logical feature of Maude: any specification can be reduced to an equivalent one *without* `owise` equations.

Formatting of Terms. Large terms can sometimes be hard to read. Maude therefore provides an operator attribute `format` that can be used to control how terms are printed, e.g., with different colors and indentations.

Tracing and Debugging. Maude provides features for tracing the computations and gathering statistics about the number of executions of each statement, as well as an advanced debugger. They are all described in [21, Chapter 22].

Exercise 43 What is the most efficient evaluation strategy for the functions f , g , and h in the specification

$$\{f(x) = x + x + x + x, \quad g(x, y, z) = a, \quad h(x, y, z) = k(y, y)\}?$$

Exercise 44 The Boolean tests `&&` and `||` evaluate their second argument only if necessary in languages like C and Java, so that b_2 is not evaluated in $b_1 \ \&\& \ b_2$ if b_1 evaluates to “false.” The built-in functions `and` and `or` evaluate both their arguments in Maude:

```
Maude> red 0 > 0 and (5 / 0 > 4) .
result [Bool]: false and 5 / 0 > 4
```

Define two Boolean functions `and-then` and `or-else` which work more like the C conjunctions and disjunctions.

Chapter 2 shows how to write equational specifications in Maude, but without explaining their precise *meaning*. This chapter, and Chapter 7, define the meaning (or *semantics*) of equational specifications in different ways.

The *operational semantics* describes the “computational meaning” of a specification, namely, how the specification can be “executed.” This chapter describes the operational semantics of an equational specification by defining precisely what it means that a ground term t *reduces* in one step to a term t' using some equation in the specification. To keep the exposition simple, I assume in this chapter that—unless stated otherwise—our specifications are *unsorted* (or *one-sorted*), meaning that there is only *one* sort in the specification, that the equations are *unconditional*, and that the function symbols do not have any attributes such as `assoc` or `comm`.

3.1 The Reduction Relation

This section defines what it means that a term *reduces*¹ in one step using an equation in an unsorted specification without functional attributes. Function symbols are not declared explicitly, but their declarations can be inferred from the context. Constants are denoted $a, a', b, c, \dots$, non-constant function symbols $f, g, h, \dots$, terms $t, t_1, t', u, \dots$, and variables $x, x', x_1, y, z, \dots$. Therefore, a specification

$$\{f(a, g(b, x), y) = f(a, b, y), \quad h(c, c, z) = h(a, b, c)\}$$

¹ Such reduction is often called *rewriting* (or (equational) *simplification*). To avoid confusion with non-equational rewriting in rewriting logic, I use *reduction* when equations are applied, and *rewriting* for the application of (“non-equational”) rewrite rules in rewriting logic. Similarly, I use the symbol $\rightsquigarrow$ instead of the more common arrow $\longrightarrow$ for equational reduction/simplification.

denotes the same equational specification as the Maude module

```
fmod M is
  sort s .
  ops a b c : -> s .    ops f h : s s s -> s .    op g : s s -> s .
  vars x y z : s .
  eq f (a, g (b, x), y) = f (a, b, y) .    eq h (c, c, z) = h (a, b, c) .
endfm
```

When a set of equations E defines a one-sorted equational specification (Σ, E) in this way, I often write E also for the induced equational specification (Σ, E) .

3.1.1 Basic Definitions

A term has a tree structure in the absence of equational attributes such as `assoc` and `comm`. For example, the term $f(h(a, b, g(x)), f(y, f(z, b)))$ can be seen as the tree in Fig. 3.1a. A *position* in a term is a string of numbers (with ε denoting the empty string) as seen in Fig. 3.2. The set of (legal) positions in a term can be defined formally by induction on the structure of the term as follows:

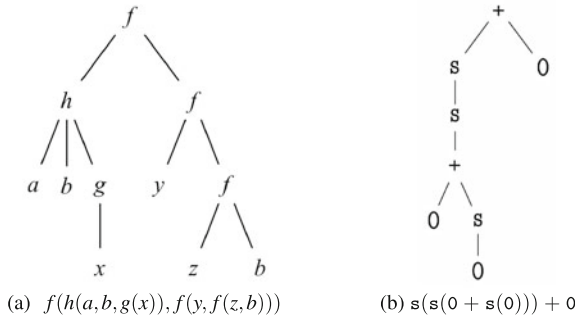


Fig. 3.1 The tree structure of two terms

Definition 3.1 (Position) *The set $Pos(t)$ of positions in a term t is the following set of strings of non-zero natural numbers:*

- if t is a variable or a constant, then $Pos(t) = \varepsilon$
- if $t = f(t_1, \dots, t_n)$, then $Pos(t) = \{\varepsilon\} \cup \bigcup_{i=1}^n \{i.p \mid p \in Pos(t_i)\}$.

A term with infix function symbols can also be written in prefix form, so that the term $s(s(0 + s(0))) + 0$ has the tree structure shown in Fig. 3.1b.

If p is a position in a term t , we denote by $t|_p$ the *subterm of t at position p* .

Definition 3.2 *The subterm of t in position $p \in Pos(t)$, written $t|_p$, is defined by*

$$t|_{\varepsilon} = t$$

$$f(t_1, \dots, t_n)|_{i.p} = t_i|_p.$$

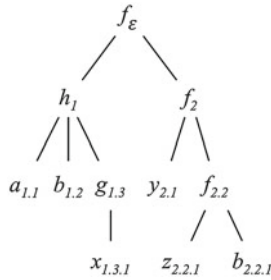


Fig. 3.2 The positions in the term $f(h(a, b, g(x)), f(y, f(z, b)))$

If $p \neq \varepsilon$, then $t|_p$ is called a proper subterm of t .

Example 3.1. The subterm of $h(a, b, g(x))$ at position 3 is $g(x)$ and $h(a, b, g(x))|_{3,1}$ is x . The subterms of $h(a, b, g(x))$ are $h(a, b, g(x))$, a , b , $g(x)$, and x . The last four are proper subterms. $\diamond$

The term $t[u]_p$ is t with $t|_p$ replaced by u . That is, we put u into t at position p in t :

Definition 3.3 If t and u are terms, and $p \in \text{Pos}(t)$, then $t[u]_p$ is defined as follows:

$$\begin{aligned} t[u]_\varepsilon &= u \\ f(t_1, \dots, t_i, \dots, t_n)[u]_{i,p} &= f(t_1, \dots, t_i[u]_{p, \dots}, t_n). \end{aligned}$$

Example 3.2. $f(a, f(x, g(y)))[b]_2$ is $f(a, b)$, and $f(a, f(x, g(y)))[c]_\varepsilon$ is just c , and $f(a, f(x, g(y)))[c]_{2,2,1}$ is $f(a, f(x, g(c)))$. $\diamond$

$\text{vars}(t)$ denotes the set of variables in t ; e.g., $\text{vars}(f(a, g(x, f(b, z)))) = \{x, z\}$.

A *variable substitution* (or just *substitution*) maps variables to terms, and is usually written explicitly as $\{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}$, where variables that are mapped to themselves are not mentioned. If σ is a substitution $\sigma : X \rightarrow T_\Sigma(Y)$, we also denote by σ its (homomorphic) extension $\sigma : T_\Sigma(X) \rightarrow T_\Sigma(Y)$ which takes a term and *simultaneously* replaces each variable x in the term with $\sigma(x)$. We often write substitutions in “postfix” notation. For example, if σ is $\{x \mapsto a, y \mapsto g(x, y), z \mapsto h(z, z)\}$ and t is the term $f(x, x, f(x, y, z))$, then $t\sigma$ is $f(a, a, f(a, g(x, y), h(z, z)))$.

A *ground substitution* maps each variable to a *ground* term.

Definition 3.4 (Matching) A term t matches² a term u if there is a substitution σ such that $t\sigma = u$. In this case u is called an instance of t .

Example 3.3. $f(x, y, z)$ matches $f(a, g(x), h(z))$ since $(f(x, y, z))\sigma = f(a, g(x), h(z))$ for the substitution $\sigma = \{x \mapsto a, y \mapsto g(x), z \mapsto h(z)\}$. $\diamond$

² Some authors write that u matches t in this case.

3.1.2 The Reduction Relation

A *reduction step* (or *simplification step*) is the application of an equation $l = r$ to a term t , so that l matches some subterm of t (which could be t itself); this subterm is replaced by the appropriate instance of r . For example, if $g(x) = h(x)$ is an equation, then $f(a, g(b))$ reduces in one step to $f(a, h(b))$.

Definition 3.5 (Reduction relation) *Given a set of equations E (with each equation “directed” from left to right). A term t reduces (in one step) to a term u , written $t \rightsquigarrow_E u$, if and only if there is an equation $l = r$ in E , a position p in t , and a substitution σ such that $t|_p = l\sigma$ and $u = t[r\sigma]_p$. That is, $t = t[l\sigma]_p \rightsquigarrow_E t[r\sigma]_p = u$. (I often write $\rightsquigarrow$ instead of $\rightsquigarrow_E$ when E is given by the context or is unimportant.)*

Example 3.4. If $E = \{f(x, y, z) = g(y)\}$, then we have both $f(a, b, b) \rightsquigarrow g(b)$ and $h(g(b), f(a, g(x), h(z))) \rightsquigarrow h(g(b), g(g(x)))$. $\diamond$

3.1.3 Some Derived Relations

We define some relations derived from $\rightsquigarrow_E$ as follows:

- $\Leftarrow_E$ such that $t \Leftarrow_E u$ holds if and only if $u \rightsquigarrow_E t$ holds.³
- $\rightsquigarrow^*_E$ such that $t \rightsquigarrow^*_E u$ holds if and only if $t \rightsquigarrow_E u$ or $u \rightsquigarrow_E t$ (or both) hold.⁴
- $\rightsquigarrow^+_E$ such that $t \rightsquigarrow^+_E u$ holds if and only if t reduces to u in *zero or more* steps. That is, either t and u are the same term, or $t \rightsquigarrow_E u$, or there are n terms $t_1, \dots, t_n$ such that $t \rightsquigarrow_E t_1 \rightsquigarrow \dots \rightsquigarrow_E t_n \rightsquigarrow u$.⁵
- The relation $\rightsquigarrow^*_E$ can be defined similarly; $t \rightsquigarrow^*_E u$ holds if and only if there is a “path” from t to u using $\rightsquigarrow_E$ -steps.⁶
- The relation $\rightsquigarrow^+_E$ (and analogously $\Leftarrow^+_E$) is defined by $t \rightsquigarrow^+_E u$ if and only if t reduces to u in *one or more* steps.⁷

Exercise 45 1. What is $f(a, b) \mid_2$, and what is $f(h(c), g(d, g(a, f(a, b)))) \mid_{2.2.1}$?

2. What is $(s(s(0 + s(0))) + 0) \mid_{1.1.1}$?

3. What is $f(h(c), g(d, g(a, f(a, b)))) \mid_{2.2}$?

Exercise 46 Let t be $f(x, x, f(x, y, z))$ and σ be $\{x \mapsto a, y \mapsto g(x, y), z \mapsto h(z, z)\}$. What is $(t\sigma)\sigma$?

Exercise 47 1. Does $g(x)$ match $h(g(a))$? Why/why not?

2. Does $f(x, x)$ match $f(a, b)$? Does it match $f(a, z)$?

³ This is called the *inverse relation*.

⁴ This is called the *symmetric closure* of $\rightsquigarrow_E$.

⁵ This is called the *reflexive-transitive closure* of $\rightsquigarrow_E$.

⁶ This is the *reflexive-symmetric-transitive closure* of $\rightsquigarrow_E$.

⁷ This is the *transitive closure* of $\rightsquigarrow_E$.

3. Does $f(x, y)$ match $f(g(a), g(a))$?
4. How many subterms of $f(f(a, a), f(a, a))$ are matched by $f(x, x)$?
5. Does $s(x) + y$ match any subterm of $s(s(0 + s(0))) + 0$?
6. Does $0 + x$ match any subterm of $s(s(0 + s(0))) + 0$?

Exercise 48 For each reduction step in Example 3.4, find the equation, the position, and the substitution used, and show that the step is indeed a reduction step.

3.2 Operational Properties

We introduce some more terminology:

- t is *reducible* if there is a term u such that $t \rightsquigarrow u$.
- t is *irreducible* if and only if t is not reducible.
- u is a *normal form* of t if and only if $t \rightsquigarrow^* u$ and u is irreducible. If u is the *unique* (that is, the *only*) normal form of t , we write $t!$ for this unique normal form u . This unique normal form u is sometimes also called the *canonical form* of t .
- u is a *successor* of t if and only if $t \rightsquigarrow^+ u$.
- A *derivation*, or *reduction sequence*, in a specification E is a finite sequence

$$t_1 \rightsquigarrow_E t_2 \rightsquigarrow_E \cdots \rightsquigarrow_E t_n$$

or an infinite sequence

$$t_1 \rightsquigarrow_E t_2 \rightsquigarrow_E t_3 \rightsquigarrow_E \cdots$$

of reduction steps $t_i \rightsquigarrow_E t_{i+1}$ in E .

- A *computation* in E is either an infinite derivation in E , or a finite derivation in E which cannot be extended (that is, the last term in the derivation is irreducible).

The following definitions formalize the notions of termination and confluence introduced informally in Chapter 2.

Definition 3.6 (Termination) A specification E is *terminating* if and only if there is no infinite derivation in E .

Definition 3.7 (Confluence) A specification is *confluent* if and only if for all terms t, t_1, t_2 such that $t \rightsquigarrow^* t_1$ and $t \rightsquigarrow^* t_2$, there is a term u such that $t_1 \rightsquigarrow^* u$ and $t_2 \rightsquigarrow^* u$.

Confluence, together with termination, essentially means that the result obtained by a computation in Maude is independent of how/which equations are applied.

Theorem 3.1 Let E be a terminating specification. Then each term t has a unique normal form if and only if E is confluent.

Proof. We first prove the “if” direction. Assume that E is confluent but that some term t does *not* have a unique normal form. If this leads to a contradiction, then each term has a unique normal form. If some term t has at least two *distinct normal forms* u_1 and u_2 , we have $t \rightsquigarrow^* u_1$ and $t \rightsquigarrow^* u_2$. But then, according to the definition of confluence, there must be a term u such that $u_1 \rightsquigarrow^* u$ and $u_2 \rightsquigarrow^* u$. Since $u_1 \neq u_2$, and we must have $u_1 \rightsquigarrow^* u$ and $u_2 \rightsquigarrow^* u$, it means that either $u_1 \xrightarrow{+} u$ or $u_2 \xrightarrow{+} u$ (or both). But this is impossible, since both u_1 and u_2 are normal forms, and therefore cannot be reduced in *one or more* steps.

To prove the “only if” direction, assume that each term has a unique normal form but that E is *not* confluent. If E is not confluent, then there are terms t, t_1, t_2 such that $t \rightsquigarrow^* t_1$ and $t \rightsquigarrow^* t_2$, but there is no term u such that $t_1 \rightsquigarrow^* u$ and $t_2 \rightsquigarrow^* u$. Since each term has a unique normal form, t_1 and t_2 have the respective normal forms $t_1!$ and $t_2!$. If $t_1! = t_2!$, then t_1 and t_2 have such a common successor term u (namely, $t_1!$), and the system is confluent. If $t_1! \neq t_2!$, then $t_1!$ and $t_2!$ are two different normal forms of t , which contradicts the assumption that each term has a unique normal form. $\square$

Analyzing whether a specification is terminating and confluent is the topic of the next two chapters. Not only are these crucial properties by themselves, but Maude assumes that your specifications are both terminating and confluent. Maude will not check this for you, for reasons that will be clear soon.

3.3 Conditional Equations and Matching with `assoc/comm`

This section briefly discusses the operational semantics of conditional equations and the computational complexity of matching (and hence of applying an equation) with operators that are declared to be associative and/or commutative.

3.3.1 Conditional Equations

Maude applies a conditional equation

$$l = r \text{ if } t_1 = u_1 \wedge \dots \wedge t_n = u_n$$

with substitution σ by checking whether $(t_i\sigma)!$ equals $(u_i\sigma)!$ for each $1 \leq i \leq n$.

More formally, $\rightsquigarrow_E$ is the smallest relation satisfying that $t \rightsquigarrow_E u$ if there is an equation as above (with $n = 0$ for unconditional equations), a position p and a substitution σ with $t = t[l\sigma]_p$ and $u = t[r\sigma]_p$, and such that the normal form of $t_i\sigma$ in $\rightsquigarrow_E$ equals the $\rightsquigarrow_E$ -normal form of $u_i\sigma$ for each i .

The evaluation of a term in a conditional equational specification may go on forever in Maude, even though the specification might be terminating according to the usual definition. Consider for example the specification E given by

$$\{a = b \text{ if } a = b\}.$$

This specification is terminating, since it does not have an infinite sequence of $\rightsquigarrow_E$ -reductions (indeed $\rightsquigarrow_E = \emptyset$). However, if we try the Maude command `red a .`, the system will check whether the equation can be applied by checking whether $a! = b!$, which is done by checking whether the equation can be applied, and so on. While E is terminating according to our definition, it is not *operationally terminating* [32].

3.3.2 * A-, C-, and AC-matching is NP-hard

Checking whether an equation $l = r$ can be applied to a term t with associative and/or commutative operators amounts to checking whether l matches a subterm of t modulo associativity and/or commutativity.

Matching modulo associativity, commutativity, or associativity and commutativity (denoted, respectively, A-matching, C-matching, and AC-matching) may produce more than one match. For example, $f(x,y)$ matches $f(a,b)$ with both $\{x \mapsto a, y \mapsto b\}$ and $\{x \mapsto b, y \mapsto a\}$ when f is commutative, and $g(x,y)$ matches $g(g(a,b),c)$ with both $\{x \mapsto g(a,b), y \mapsto c\}$ and $\{x \mapsto a, y \mapsto g(b,c)\}$ when g is associative. In our mergesort function, the pattern `NEL NEL'` in the main equation matches *any* partition of the list into two non-empty lists (why?); however, only one of the matches satisfies the condition of that equation.

Finding all matches modulo A, C, and AC is always possible, but it may not be very efficient. Even checking whether *one* match exists is an NP-complete problem in all three cases [10], which means that there is no algorithm that can *always* solve the matching problem efficiently (unless $P = NP$). A/C/AC-matching will therefore be exponential (i.e., very slow) for *some* terms.

This can be proved by showing that another NP-complete problem, *Positive 1-in-3-SAT* (1-3-SAT), can be solved easily *if* we can solve the matching problem.

Theorem 3.2 *AC-matching is an NP-complete problem.*

Proof. Following [6, 10], we show that 1-3-SAT, which is an NP-complete problem [47], can be solved easily by AC-matching. A 1-3-SAT instance is a set

$$\{(p_i \vee q_i \vee r_i) \mid 1 \leq i \leq n\}$$

of clauses where the p_i , q_i , and r_i are propositional variables. The problem is to decide whether there exists a valuation of all the propositional variables (to `true` or `false`) such that *exactly one* of the propositional variables p_i , q_i , and r_i is `true` for each clause $(p_i \vee q_i \vee r_i)$.

Each propositional variable p corresponds to a variable x_p in the corresponding matching problem. We have two constants `true` and `false`, and an AC-operator $\vee$. An instance of the 1-in-3-SAT problem is a “yes” instance if and only if the set

$$\{x_{p_i} \vee x_{q_i} \vee x_{r_i} \text{ “matches” } \text{true} \vee \text{false} \vee \text{false} \mid 1 \leq i \leq n\}$$

of matching problems has a solution. (A set $\{t_i \text{ “matches” } u_i \mid 1 \leq i \leq n\}$ of matching problems can be seen as *one* matching problem

$$f_n(t_1, \dots, t_n) \text{ “matches” } f_n(u_1, \dots, u_n)$$

for a new symbol f_n . We can use an ordinary binary operator f if we want finite signatures, and the problem becomes whether $f(t_1, f(t_2, f(\dots, t_n)) \dots)$ matches $f(u_1, f(u_2, f(\dots, u_n)) \dots)$. For example, the 1-3-SAT problem for $\{(p_1 \vee p_2 \vee p_3), (p_2 \vee p_3 \vee p_4)\}$ amounts to checking whether $f(x_{p_1} \vee x_{p_2} \vee x_{p_3}, x_{p_2} \vee x_{p_3} \vee x_{p_4})$ matches $f(\text{true} \vee \text{false} \vee \text{false}, \text{true} \vee \text{false} \vee \text{false})$. Since the latter term is AC-equal to $f(\text{false} \vee \text{true} \vee \text{false}, \text{true} \vee \text{false} \vee \text{false})$, there is such a match $\{x_{p_1} \mapsto \text{false}, x_{p_2} \mapsto \text{true}, x_{p_3} \mapsto \text{false}, x_{p_4} \mapsto \text{false}\}$. $\square$

Proving that A-matching and C-matching are NP-complete can be done in a similar way [6, 10]. Although these results indicate that computing with functions that have the attributes `assoc` and/or `comm` may be very inefficient, the Maude developers have put a lot of effort and ingenuity into making the A-, C-, and AC-matching algorithms really fast for most patterns occurring in practice [36].

Exercise 49 What matching problem solves the 1-3-SAT problem for $\{(p_1 \vee p_2 \vee p_3), (p_2 \vee p_3 \vee p_4), (p_1 \vee p_2 \vee p_4), (p_1 \vee p_3 \vee p_4)\}$? Is there such a match?

Termination (the absence of infinite computations) is a crucial property for both equational specifications and programs in general. Maude requires equational specifications to be terminating, but does *not* check it (for reasons that will be apparent). We must therefore be able to analyze whether or not a specification is terminating.

Recall the definition of termination:

Definition 4.1 *An equational specification E is terminating if there is no infinite derivation $t_0 \rightsquigarrow_E t_1 \rightsquigarrow_E t_2 \rightsquigarrow_E \dots$ for any term t_0 .*

This means that *each* derivation from *every* term must be finite.

For simplicity, we again assume unsorted specifications with at least one ground term, and without operator attributes and conditional equations. It is easy to see that if $t_0(x) \rightsquigarrow_E t_1(x) \rightsquigarrow_E t_2(x) \rightsquigarrow_E \dots$ is an infinite derivation of terms possibly containing a variable x , then there is also a corresponding infinite derivation $t_0(t) \rightsquigarrow_E t_1(t) \rightsquigarrow_E t_2(t) \rightsquigarrow_E \dots$ where each occurrence of x has been replaced with some ground term t . A specification E is therefore terminating if it does not allow any infinite derivation $t_0 \rightsquigarrow_E t_1 \rightsquigarrow_E t_2 \rightsquigarrow_E \dots$ of *ground terms* $t_0, t_1, \dots$.

It is obvious that $\{f(x) = g(x)\}$, $\{f(g(x)) = h(x)\}$, and $\{a = b, b = c\}$ are terminating, and that $\{f(x, y) = f(y, x)\}$, $\{f(x) = f(g(x))\}$, and $\{a = b, b = c, b = a\}$ are *not* terminating (the latter is *weakly terminating*, since each term has a normal form). But what about the specification $\{f(g(x, y)) = g(g(f(f(x)), y), y)\}$? And $\{f(g(g(x))) = f(f(g(f(g(f(x))))))\}$, $f(f(f(x))) = f(g(f(x)))$ and $\{f(a, b, x) = f(x, x, x), g(x, y) = x, g(x, y) = y\}$?

It would obviously be good to have a (terminating!) algorithm of the form

```
bool terminates (specification E) {
    ...
    if <E is terminating> return true; else return false;
}
```

which, for *any* specification E , can figure out whether or not E is terminating. It is well known that it is impossible to have such a function for both standard programming languages and for *Turing machines*. Section 4.1 explains how any Turing

machine M can be modeled by an equational specification $\tilde{e}(M)$ which is terminating if and only if M is terminating. It follows that it is in general *undecidable* whether or not an equational specification is terminating, and therefore no algorithm *terminates* of the above form exists.

Although we do not have a (finite collection of) method(s) that can *always* be used to decide termination, we can *quite often* either

- prove that a specification is nonterminating by finding an infinite derivation, or
- prove that a specification is terminating for all input/initial terms.

Section 4.2 discusses how to prove that a system is *nonterminating*. Section 4.3 presents a method for proving termination of a system by assigning a “weight” in a well-founded domain to each ground term t , and then showing that each step $t \rightsquigarrow u$ is “weight-decreasing.” It requires some ingenuity to find good weight functions; Section 4.4 therefore presents some powerful simplification orders that can be automated, as well as the underlying theory of simplification orders.

This chapter is based on Dershowitz’s papers [26, 27] and the handbooks [6, 105].

4.1 Undecidability of Termination

According to the Church-Turing thesis, every algorithmically computable function can be computed by a *Turing machine*. It is *undecidable* in general whether a Turing machine is terminating for all inputs (the “uniform halting problem”). This section shows that any Turing machine M can be simulated by an equational specification $\tilde{e}(M)$, so that M is terminating if and only if $\tilde{e}(M)$ is terminating.¹ It is then in general undecidable whether an equational specification is terminating. Otherwise we could decide termination of a Turing machine M by checking if $\tilde{e}(M)$ is terminating.

Theorem 4.1 *It is undecidable whether a specification is terminating.*

A Turing machine is defined as follows:

Definition 4.2 *A (nondeterministic) Turing machine M is a triple (Q, S, Δ) , where*

- Q is a finite set $\{q_0, \dots, q_n\}$ of states;
- $S = \{\square, s_1, \dots, s_m\}$ is the alphabet, with $S \cap Q = \emptyset$ and where $\square$ is the special symbol “blank”; and
- Δ is a relation $\Delta \subseteq Q \times S \times Q \times S \times \{\text{left}, \text{right}\}$, called the transition relation.

M is deterministic if it has at most one transition $(q, s, q', s', \text{dir})$ for each pair (q, s) .²

¹ Turing machines are a model of computation and not a data type. Equational specifications are therefore not well suited for modeling such machines, which can instead be naturally modeled in rewriting logic (see Exercise 126). We show how the computations of a Turing machine can be simulated by equational simplification steps only to prove undecidability of termination of equational specifications.

² Our results carry directly over to deterministic Turing machines.

A Turing machine has a *tape* which is infinite in both directions. This tape is divided into infinitely many squares. Each square contains one symbol from S , but there are only a *finite* number of non-blank symbols on the tape. At any time, the machine is in one of the states $q_0, \dots, q_n$, and has a (read/write) *head* that points to some square on the tape. The machine operates by performing transitions as long as possible: either until no transition can be taken, or forever. More precisely, if the machine is in state q , with its head pointing to a square that contains the symbol s , and there is a transition $(q, s, q_{next}, s_{next}, right)$ in Δ , the machine can perform this transition, in which case it writes s_{next} in the square on the tape where its head is (thereby erasing s from that square), goes to the new state q_{next} , and moves the head one position to the right on the tape (if the transition is $(q, s, q_{next}, s_{next}, left)$, the head is instead moved one position to the left).

Example 4.1. Two configurations (i.e., state, position of the head, and tape content) of a Turing machine $(\{q_1, q_2\}, \{\square, a, b\}, \{(q_1, b, q_2, a, right), \dots\})$ are:



In the left-hand side, the machine is in state q_1 and its head points to a square containing the symbol 'b.' The right-hand side shows the configuration resulting from performing the transition $(q_1, b, q_2, a, right)$ on the left-hand side configuration. $\diamond$

Example 4.2. The Turing machine $(\{q_{init}, q_{stop}\}, \{\square, 1, 2\}, \Delta)$ that changes every '1' to '2', and every '2' to '1', until it reaches a blank—when the initial state is q_{init} and the machine reads towards the right—has the following transitions Δ :

$(q_{init}, \square, q_{stop}, \square, right)$	blank read: stop!	
$(q_{init}, 1, q_{init}, 2, right)$	change 1 to 2 and continue	
$(q_{init}, 2, q_{init}, 1, right)$	change 2 to 1 and continue	$\diamond$

We can represent a configuration of a Turing machine in a simple way as a *finite list* of the form $[s_{i_1} \dots s_{i_k} q s_{i_{k+1}} \dots s_{i_l}]$. The delimiters '[' and ']' are used to represent an infinite tape as a finite list; the tape outside the delimiters contains only blanks. We add the current state q to the list; the position of q denotes the position of the read/write head: the head points to the square to the *right* of q . In the above configuration, the head points at the square with the symbol $s_{i_{k+1}}$.

Example 4.3. The left-hand side (resp., right-hand side) configuration in Example 4.1 is represented as the term $[a q_1 b b a b]$ (resp., $[a a q_2 b a b]$). $\diamond$

If a transition $(q, s_{i_{k+1}}, q_{next}, s_{next}, right)$ is performed when the machine is in the configuration represented by the term $[s_{i_1} \dots s_{i_k} q s_{i_{k+1}} s_{i_{k+2}} \dots]$, then the next configuration term is $[s_{i_1} \dots s_{i_k} s_{next} q_{next} s_{i_{k+2}} \dots]$. If the configuration was represented by $[s_{i_1} \dots s_{i_k} q s_{i_{k+1}}]$ (that is, the head points to the last square represented

in the list), the next configuration will be represented by $[s_{i_1} \dots s_{i_k} s_{next} q_{next} \square]$, where the list has been extended with a blank. Moving to the left is symmetric.

The Maude representation $e(M)$ of a Turing machine M is defined as follows:

```

sorts State Symbol Delimiter Tape .
ops q0 ... qn : -> State [ctor] .
ops s1 ... sm □ : -> Symbol [ctor] .
ops [ ] : -> Delimiter [ctor] .
subsort State Symbol Delimiter < Tape .      --- non-empty list
op _ : Tape Tape -> Tape [ctor assoc] .

```

A transition $(q, s, q', s', left)$ is translated into the two equations

```

var SYMBOL : Symbol .
eq SYMBOL q s = q' SYMBOL s' .
eq [ q s = [ q' □ s' .

```

where the second equation takes care of the case when the head points to the leftmost square represented in the list. In the first equation, the head points to the square containing s . The content of this square is changed to s' , and the new state q' jumps to the left, so that it now points to `SYMBOL`. The second equation inserts a blank ($\square$) at the left end of the list and makes the head point to this new blank. A transition $(q, s, q', s', right)$ that moves the head to the right is represented in the same way:

```

eq q s SYMBOL = s' q' SYMBOL .
eq q s ] = s' q' □ ] .

```

It should be fairly obvious that $e(M)$ can simulate each step of the Turing machine M . An infinite computation in M is therefore simulated by an infinite derivation in $e(M)$, so that $e(M)$ is nonterminating if M is nonterminating.

But hold the horses, there are two problems here:

1. A Turing machine is represented by an *order-sorted* specification with an `assoc` operator, whereas we were supposed to reason about the termination of *unsorted* specifications without such attributes.
2. $e(M)$ should be terminating *if and only if* M is terminating. We have only shown that *if* M is not terminating, then $e(M)$ is also nonterminating. Can $e(M)$ be nonterminating even when M is terminating? Remember that for $e(M)$ to be terminating, it must be terminating for *all* possible initial terms t_0 , *even those that do not represent legal Turing machine configurations*. Could $e(M)$ loop on some “junk terms” even when M is terminating?

Addressing the first problem is easy. The above model was chosen for simplicity of explanation. A list/string rewrite system of this form can be represented by a term rewrite system where each (state, alphabet, and delimiter) symbol except ‘`]`’ is represented by a *unary* function³ symbol with the same name. For example, the list $[\square s_1 \square s_2 s_5 s_1 q \square s_3]$ can be represented by the (unsorted) term $[(\square(s_1(\square(s_2(s_5(s_1(q(\square(s_3())))))))))]$. Translating the above system to such an unsorted system is fairly easy and is left as Exercise 50.

³ A unary function is a function that takes one argument.

The second issue is trickier. In such an *unsorted* representation, there are terms with multiple q 's, *de facto* representing multiple Turing machine “instances” on the same tape. For example, the term $[\square s_1 q_1 \square s_2 q_2 s_5 q \square q_5 s_3]$ does not represent any Turing machine configuration. Can the translation $e(M)$ of a terminating Turing machine M be nonterminating because it is not terminating on such junk terms?

Consider the following Turing machine M_{ab} :

- If M_{ab} initially reads ‘ a ’, it wants to ensure that the square *to the right* also contains ‘ a ’. If the square to the right contains ‘ b ’, then M_{ab} writes ‘ a ’ there, goes one position left, and then one position right, to *really* ensure that the square to the right position still contains ‘ a ’. If so, it is done. If not, then M_{ab} again writes ‘ a ’ and goes left, and then right, and repeats the confirmation process.
- If M_{ab} starts by reading ‘ b ’, it wants to ensure that *the current* square always contains ‘ b ’. That is, it jumps to the right, then jumps back left, and if the original square still contains ‘ b ’, it is done. If the original square contains ‘ a ’, it writes ‘ b ’ there, jumps to the right, then back to the left, and repeats the process.

The machine M_{ab} is obviously terminating (for *any* initial machine state q_i). However, if you “combine” two versions of M_{ab} on the same tape; that is, if you start with a “junk term” $[q_{init} a q_{init} b]$, you get a nonterminating system: the “first” head reads ‘ a ’ and remembers this; the “second” head then reads the ‘ b ’ and jumps to the right; the first head then reads that ‘ b ’ and turns it into an ‘ a ’, goes left and remembers to check for ‘ a ’; the second head goes back and checks whether its initial square still contains ‘ b ’, and since it does not, it sets that square to ‘ b ’ and moves right; and so on. It is an easy exercise (Exercise 53) to formalize M_{ab} and show that its translation $e(M_{ab})$ is nonterminating from the above term.

If M is terminating, it would not be a problem if many “instances” of M work at the same time *independently of each other*, since each instance would terminate sooner or later. The problem occurs when these different instances *interact*, which is exactly what happens in M_{ab} : the “left head” insists on having ‘ a ’ in the second tape position above, while the “right head” insists on having ‘ b ’ in this same location.

The solution is to ensure that the different “instances” *cannot* interact. Baader and Nipkow [6] achieve this by using two different representations $\overleftarrow{s}$ and $\overrightarrow{s}$ of each alphabet symbol s , with the arrow pointing to the head to which the symbol “belongs.” A transition only considers symbols pointing towards the head, and symbols generated by the head will always point towards it. Hence one head cannot use symbols generated by another head. Modifying our translation in this way (see Exercise 54) leads to an (unsorted and unconditional) equational specification $\tilde{e}(M)$ that is terminating if and only if M is terminating. (The representation in [105] avoids such unfortunate interactions between different Turing machine instances by representing a configuration $list_1 q list_2$ as a term $q(reverse(list_1), list_2)$.)

Exercise 50 Show how a Turing machine M can be represented as an unsorted equational specification (without attributes such as *assoc*) by transforming the order-sorted specification $e(M)$ into an unsorted one as suggested above. (Note that

the first equation above gives rise to an equation $s_i(q(s(x))) = q'(s_i(s'(x)))$ for each symbol s_i , since variables range over terms and not over function symbols.)

Exercise 51 Define a Turing machine over the alphabet $\{\square, 1\}$ that loops forever if there is an odd number of consecutive 1's on the tape (moving to the right from where the head points initially), and that stops if the number of consecutive 1's is an even number. Then define a terminating Turing machine over the alphabet $\{\square, 1, \text{odd}, \text{even}\}$ that stops by writing odd or even, depending on whether the “number” on the tape is odd or even. Which are the initial states?

Exercise 52 Define a Turing machine over $\{\square, 0, 1\}$ that adds 1 to the “binary number” on the tape.

Exercise 53 Define the Turing machine M_{ab} formally and show that its translation $e(M_{ab})$ has an infinite derivation from the term $[q_{\text{init}} a q_{\text{init}} b]$.

Exercise 54 Modify the translation of a Turing machine according to Baader's and Nipkow's idea (it is sufficient to modify the simpler order-sorted specification), and show that the modified equational specification does allow an infinite derivation from $[q_{\text{init}} a q_{\text{init}} b]$ in the translation of M_{ab} .

Exercise 55 In this exercise we define an interpreter for deterministic (and terminating) Turing machines in Maude.

1. Define a sort `TuringMachine` for representing Turing machines and a sort `TMConfig` for representing Turing machine configurations.
2. Define a subsort `DetTuringMachine` for deterministic Turing machines.
3. Define a Turing machine interpreter in Maude as a function

```
op interpret : DetTuringMachine TMConfig -> TMConfig .
```

so that `interpret(M, initConfig)` returns the configuration resulting from running the deterministic Turing machine (represented by the term) M with initial configuration `initConfig`.

4. Run your Turing machine interpreter on the terminating Turing machines you defined in Exercises 51 and 52.

Since any computable function can be defined by a deterministic Turing machine, there is a Turing machine that mimics the behavior of the function `interpret`. Such a Turing machine that can simulate the steps of any Turing machine it gets as input on any initial configuration for that machine is called a universal Turing machine.

4.2 Nontermination

A specification E is *looping* if there are terms t and u such that $t \xrightarrow{+}_E u$ and t is a subterm of u . A looping specification is nonterminating, since the steps from t to u can be repeated from (the subterm t inside) u .

Example 4.4.

- The specification $\{f(x) = f(f(x))\}$ has a reduction $f(x) \rightsquigarrow f(f(x))$ which is a looping derivation since $f(x)$ is a subterm of $f(f(x))$. The specification is therefore nonterminating: $f(x) \rightsquigarrow f(f(x)) \rightsquigarrow f(f(f(x))) \rightsquigarrow f(f(f(f(x)))) \rightsquigarrow \dots$.
- The specification $\{f(x,y) = f(y,x)\}$ has a looping derivation $f(x,y) \rightsquigarrow f(y,x) \rightsquigarrow f(x,y)$, where these steps can be repeated forever. $\diamond$

A specification is nonterminating if the righthand side of an equation contains a variable that does not occur in the lefthand side, since the new variable can be instantiated with anything, including the term being reduced. Therefore, no equation should introduce a new variable in its righthand side:

Example 4.5. $\{f(x) = g(x,y)\}$ has an infinite (and looping) derivation

$$f(x) \rightsquigarrow g(x, f(x)) \rightsquigarrow g(x, g(x, f(x))) \rightsquigarrow \dots \quad \diamond$$

To make the picture more complicated, there are also nonterminating systems which are not looping:

Example 4.6. The system $\{f(x) = f(g(x))\}$ is not looping, but is nonterminating:

$$f(x) \rightsquigarrow f(g(x)) \rightsquigarrow f(g(g(x))) \rightsquigarrow f(g(g(g(x)))) \rightsquigarrow \dots \quad \diamond$$

Exercise 56 Show that the specification

$$\{f(a,b,x) = f(x,x,x), g(x,y) = x, g(x,y) = y\}$$

is nonterminating. Hint: Start with the term $f(a,b,g(a,b))$.

This is a noteworthy specification by Toyama [106], where the union of the two terminating specifications $\{f(a,b,x) = f(x,x,x)\}$ and $\{g(x,y) = x, g(x,y) = y\}$, which do not have any function symbol in common, is nonterminating.

4.3 Proving Termination Using “Weight Functions”

This and the next section present some techniques that can be used to *prove* that a specification is terminating.

The specification $\{f(x) = g(x)\}$ is obviously terminating, but how would you *prove* that it does not have an infinite derivation for *any* start term t_0 ? Probably you would say that the number of (occurrences of) the function symbol f in the term decreases in each simplification step, and since it cannot be less than 0, the system must be terminating. Otherwise there would be an *infinite* sequence $t_0 \rightsquigarrow t_1 \rightsquigarrow t_2 \rightsquigarrow \dots$ which would lead to an *infinite* sequence $\#f(t_0) > \#f(t_1) > \#f(t_2) > \dots$ of decreasing natural numbers (where $\#f(t)$ denotes the number of f s in t), which is impossible, no matter how large $\#f(t_0)$ is. More

generally, we can prove termination of a specification by giving a natural number “weight” to each ground term and show that each simplification step $t \rightsquigarrow u$ is weight-decreasing:

Proposition 4.1 *A specification (Σ, E) is terminating if there is a function*

$$\text{weight} : \mathcal{T}_\Sigma \rightarrow \mathbb{N}$$

mapping a ground term to a natural number such that, for all ground terms t and u ,

$$t \rightsquigarrow u \quad \text{implies} \quad \text{weight}(t) > \text{weight}(u).$$

In the example above, the “weight” (or “progress”) function weight was $\#f$.

One problem is the need to consider all *contexts*: if $t \rightsquigarrow u$, then there are also simplification steps $f(t) \rightsquigarrow f(u)$, $f(f(t)) \rightsquigarrow f(f(u))$, $f(f(f(t))) \rightsquigarrow f(f(f(u)))$, $f(g(t)) \rightsquigarrow f(g(u))$, and so on, which must all be proved weight-decreasing. We can avoid having to consider all contexts if the weight function is *monotonic*.

Definition 4.3 *A function $w : \mathcal{T}_\Sigma \rightarrow \mathbb{N}$ is monotonic (w.r.t. to the relation $>$) if and only if, for each function symbol f , all ground terms t and u , and all lists $\mathbf{t}_1$ and $\mathbf{t}_2$ of ground terms,*

$$w(t) > w(u) \quad \text{implies} \quad w(f(\mathbf{t}_1, t, \mathbf{t}_2)) > w(f(\mathbf{t}_1, u, \mathbf{t}_2)).$$

What remains is to prove that each instance of an equation is weight-decreasing:

Proposition 4.2 *A specification (Σ, E) is terminating if there is a monotonic function $\text{weight} : \mathcal{T}_\Sigma \rightarrow \mathbb{N}$ such that $\text{weight}(l\sigma) > \text{weight}(r\sigma)$ for each equation $l = r$ in E and each ground substitution $\sigma : (\text{vars}(l) \cup \text{vars}(r)) \rightarrow \mathcal{T}_\Sigma$.*

Example 4.7. Consider again the specification $\{f(x) = g(x)\}$ and let $\text{weight}(t)$ be “the number of occurrences of f in t .” To prove termination we need to prove that

1. weight is monotonic, and
2. $\text{weight}(f(x)\sigma) > \text{weight}(g(x)\sigma)$ for each ground substitution σ .

For monotonicity, assume $\text{weight}(t) > \text{weight}(u)$ and prove $\text{weight}(f(t)) > \text{weight}(f(u))$ and $\text{weight}(g(t)) > \text{weight}(g(u))$. Since $\text{weight}(f(t)) = 1 + \text{weight}(t)$ and $\text{weight}(f(u)) = 1 + \text{weight}(u)$, the assumption $\text{weight}(t) > \text{weight}(u)$ gives the desired

$$\text{weight}(f(t)) = 1 + \text{weight}(t) > 1 + \text{weight}(u) = \text{weight}(f(u)).$$

Monotonicity for g , $\text{weight}(g(t)) > \text{weight}(g(u))$, follows from the assumption $\text{weight}(t) > \text{weight}(u)$ since $\text{weight}(g(t)) = \text{weight}(t)$ and $\text{weight}(g(u)) = \text{weight}(u)$.

For the second property, we have

$$\text{weight}(f(x)\sigma) = 1 + \text{weight}(x\sigma) > \text{weight}(x\sigma) = \text{weight}(g(x)\sigma)$$

for any ground substitution σ . ◇

A weight function is often defined recursively:

Example 4.8. The specification $\{f(x) = g(x), g(b) = f(a)\}$ can be proved terminating using the weight function

- $\text{weight}(a) = 1$,
- $\text{weight}(b) = 88$,
- $\text{weight}(f(t)) = 4 + \text{weight}(t)$, and
- $\text{weight}(g(t)) = \text{weight}(t)$.

weight is monotonic: $\text{weight}(t) > \text{weight}(u)$ implies $\text{weight}(f(t)) > \text{weight}(f(u))$ and $\text{weight}(g(t)) > \text{weight}(g(u))$. Each equation instance reduces the weight:

1. $\text{weight}(f(x)\sigma) > \text{weight}(g(x)\sigma)$, and
2. $\text{weight}(g(b)) > \text{weight}(f(a))$.

(1) holds since $\text{weight}(f(x)\sigma) = 4 + \text{weight}(x\sigma) > \text{weight}(x\sigma) = \text{weight}(g(x)\sigma)$, and (2) holds since $\text{weight}(g(b)) = 88 > 5 = \text{weight}(f(a))$. $\diamond$

Example 4.9. The specification $\{f(g(x)) = g(f(x))\}$ can be proved terminating using the weight function defined by

- $\text{weight}(a) = 2$ for each constant a (there is always at least one constant),
- $\text{weight}(f(t)) = (\text{weight}(t))^3$, and
- $\text{weight}(g(t)) = 2 \cdot \text{weight}(t)$.

Monotonicity is easy. The weight of each equation instance decreases, since

$$\text{weight}(f(g(x\sigma))) = (2 \cdot \text{weight}(x\sigma))^3 > 2 \cdot (\text{weight}(x\sigma))^3 = \text{weight}(g(f(x\sigma)))$$

holds for all $\text{weight}(x\sigma)$, since the weight of a ground term is at least 2. $\diamond$

Example 4.10. The system $\{f(f(x)) = f(g(f(x)))\}$ can be proved to be terminating using the *non-monotonic* weight function $\text{weight}(t) =$ “the number of “adjacent” pairs of f ’s in t ”, since $t \rightsquigarrow u$ implies $\text{weight}(t) > \text{weight}(u)$. However, it is hard to define a *monotonic* weight function which proves termination of this system. $\diamond$

It is sometimes more convenient, or even necessary, to use weights other than natural numbers. Any domain S and weight comparison $\succ$ can be used as long as $t \rightsquigarrow u$ implies $\text{weight}(t) \succ \text{weight}(u)$, and there is no infinite sequence $s_1 \succ s_2 \succ s_3 \succ \dots$ of $\succ$ -decreasing S -elements.

Recall that a *strict partial order* $\succ$ on a set S is a relation $\succ \subseteq S \times S$ which is

- irreflexive: there is *no* $s \in S$ such that $s \succ s$, and
- transitive: for all $s_1, s_2, s_3 \in S$, $s_1 \succ s_2$ and $s_2 \succ s_3$ imply that $s_1 \succ s_3$.

Definition 4.4 (Well-founded strict partial order) A strict partial order $\succ$ on S is well-founded if there is no infinite sequence

$$s_1 \succ s_2 \succ s_3 \succ \dots$$

of S -elements $s_1, s_2, s_3, \dots$

Example 4.11. The greater-than relation $>$ is a strict partial order on both the natural numbers $\mathbb{N}$ and the integers $\mathbb{Z}$, but is only well-founded on $\mathbb{N}$. $\diamond$

If $\succ_1$ and $\succ_2$ are well-founded strict partial orders on S_1 and S_2 , respectively, then the lexicographic comparison $\succ_{1,2}^{lex}$, defined as expected by $(s_1, s_2) \succ_{1,2}^{lex} (s'_1, s'_2)$ if and only if either $s_1 \succ_1 s'_1$ or both $s_1 = s'_1$ and $s_2 \succ_2 s'_2$, is also a well-founded strict partial order on the set $S_1 \times S_2$. This again implies that $\succ_{1,2,3}^{lex}$ is a well-founded strict partial order on $S_1 \times S_2 \times S_3$ if $\succ_3$ is a well-founded strict partial order on S_3 . Therefore, the lexicographic comparison of lists of the same length is well-founded if the comparison on each single domain is well-founded. A special case is that $\succ^{lex}$ is a well-founded strict partial order on k -tuples of natural numbers.

A well-founded strict partial order $\succ$ on S can also be extended to a well-founded strict partial ordering $\succ^{ms}$ on finite multisets of S , where $m_1 \succ^{ms} m_2$ holds if m_1 contains the largest element when all the common elements in m_1 and m_2 are removed.

Exercise 57 Prove termination of $\{f(h(x, y)) = h(x, x)\}$ using weight functions.

Exercise 58 Explain why the weight function in Example 4.10 is not monotonic.

Exercise 59 1. Explain why the following program terminates for any m and n :

```
int x := m;   int y := n;
while (x>2 and y>0) {
  if x>y then {x := x-1;  y := x+y;}  else y := y/2;
}
```

2. Explain why the following “Euclidean” algorithm for computing the greatest common divisor of two natural numbers terminates for all m and n .⁴

```
int gcd(int m, int n) {  // m, n > 0
  int x := m;   int y := n;   int r := x % y;
  while (r>0) {x := y;  y := r;  r := x % y;}
  return y;
}
```

Exercise 60 Use weight function techniques to prove termination of

$$\begin{aligned} \{f(g(h(x))) = f(f(x)), \quad f(g(h(x))) = g(g(x)), \quad f(g(h(x))) = h(h(x)), \\ f(x) = g(x), \quad g(x) = h(x)\}. \end{aligned}$$

4.4 Simplification Orders

Since finding suitable weight functions may require some clever ideas, the weight function method is not suitable for proving termination automatically. This section

⁴ $m \% n$ gives the remainder when m is divided by n .

introduces the theory of *simplification orders*, due to Dershowitz (see, e.g., [26]), and some powerful simplification orders which can be automated.

We start with some terminology. A term t *embeds* a term u if u is contained “inside” t , in the sense that if we remove some function symbols from t we get u .

Definition 4.5 (Embedding) A term t embeds a term u , denoted

$$t \triangleright u,$$

if and only if $t \xrightarrow{+}_{EMB} u$ in the specification EMB given by

$$EMB = \{f(x_1, \dots, x_m) = x_i \mid 1 \leq i \leq m\} \cup \{g(x_1, \dots, x_n) = x_i \mid 1 \leq i \leq n\} \cup \dots$$

for all non-constant function symbols $f, g, \dots$. We define $t \triangleright u$ if and only if $t \xrightarrow{*}_{EMB} u$, and write $t \triangleleft u$ (resp., $t \trianglelefteq u$) for $u \triangleright t$ (and $u \trianglerighteq t$, respectively).

Each equation $f(x_1, \dots, x_i, \dots, x_m) = x_i$ in EMB “removes” an f and some of its arguments, and only leaves its i -th argument.

Example 4.12. $f(g(f(a))) \triangleright f(f(a))$ holds since $f(g(f(a))) \xrightarrow{*}_{EMB} f(f(a))$, using the equation $g(x_1) = x_1$ in EMB . We also have $f(a, g(h(b, f(c, d)), e)) \triangleright f(a, h(b, d))$ and $f(a, g(h(b, f(c, d)), e)) \triangleright g(b, e)$. However, neither $f(a, g(h(b, f(c, d)), e)) \triangleright f(a, h(b, e))$ nor $f(a, g(h(b, f(c, d)), e)) \triangleright g(b, d)$ holds. $\diamond$

The following fundamental result says that some “patterns” must be repeated in an infinite sequence of ground terms constructed by a finite set of function symbols:

Theorem 4.2 (Kruskal’s Tree Theorem) If Σ has a finite set of function symbols, then any infinite sequence

$$t_1, t_2, \dots, t_j, \dots, t_k, \dots$$

of ground terms in $\mathcal{T}_\Sigma$ contains two terms t_j and t_k , with $j < k$, such that $t_k \trianglerighteq t_j$.

This theorem implies that if a *finite* specification does *not* have any *self-embedding* derivation, i.e., a derivation of the form

$$t_1 \rightsquigarrow t_2 \rightsquigarrow \dots \rightsquigarrow t_j \rightsquigarrow \dots \rightsquigarrow t_k \rightsquigarrow \dots$$

where $t_k \trianglerighteq t_j$ for some $k > j$, then it must be terminating! Therefore, if there is a strict partial order $\succ$ on $\mathcal{T}_\Sigma$ such that $t \triangleright u$ implies $t \succ u$, and $t \rightsquigarrow u$ implies $t \succ u$, then the specification is terminating! Why? Because if it did *not* terminate, there would be an infinite sequence

$$t_1 \succ t_2 \succ \dots \succ t_j \succ \dots \succ t_k \succ \dots$$

By Kruskal’s Theorem, $t_k \trianglerighteq t_j$, and therefore either $t_k = t_j$ or $t_k \triangleright t_j$. The case $t_k = t_j$ is impossible, since $t_j \succ t_k$ (because $\succ$ is transitive), and then it cannot be that $t_k = t_j$

because $\succ$ is irreflexive. $t_k \triangleright t_j$ is also impossible: Since $t_k \triangleright t_j$ implies $t_k \succ t_j$ (by the assumption on the definition of $\triangleright$), we have both $t_j \succ t_k$ and $t_k \succ t_j$. Since $\succ$ is transitive we get that $t_j \succ t_j$, which is impossible since $\succ$ is irreflexive.

Any strict partial order which includes $\triangleright$ and is monotonic (so that we do not have to worry about contexts) can therefore be used to prove termination:

Definition 4.6 (Simplification order) *A monotonic strict partial order $\succ$ on ground terms is a simplification order if it satisfies the subterm property*

$$f(t_1, \dots, t_n) \succ t_i$$

for all ground terms $f(t_1, \dots, t_n)$ and each $i \leq n$.

Proposition 4.3 *$t \triangleright u$ implies $t \succ u$ for all simplification orders $\succ$ and ground terms t and u .*

The main result follows trivially from the above facts:

Theorem 4.3 *A specification with a finite number of function symbols and/or a finite number of equations is terminating if there is a simplification order $\succ$ such that $l\sigma \succ r\sigma$ holds for each ground substitution σ for each equation $l = r$ in the specification.*

Proof. $l\sigma \succ r\sigma$ and the fact that $\succ$ is monotonic imply that $t \rightsquigarrow u \implies t \succ u$, and, since $\succ$ is transitive, we also have $t \rightsquigarrow^+ u \implies t \succ u$. Assume that the specification is *not* terminating. Then there is an infinite derivation

$$t_0 \rightsquigarrow t_1 \rightsquigarrow \dots \rightsquigarrow t_j \rightsquigarrow \dots \rightsquigarrow t_k \rightsquigarrow \dots$$

where all terms are built from a *finite* set of function symbols. (If the signature contains an *infinite* set of function symbols, but a finite set of equations, then all terms in the above derivation are constructed from the function symbols appearing in t_0 and in the right-hand sides of the equations. Given a finite set of equations, there is only a finite number of distinct function symbols in these right-hand sides.) Therefore, Kruskal's Tree Theorem applies, and we have both $t_k \triangleright t_j$ and $t_j \succ t_k$ (since $t_j \rightsquigarrow^+ t_k \implies t_j \succ t_k$). This is impossible ($t_k = t_j$ is impossible because $\succ$ is irreflexive, and $t_k \triangleright t_j$ implies that $t_k \succ t_j$ by Proposition 4.3, and with $\succ$ a strict partial order we cannot have both $t_j \succ t_k$ and $t_k \succ t_j$)! $\square$

Since a simplification order only proves that there is no self-embedding derivation, a simplification order *cannot* prove termination of self-embedding and terminating specifications such as $\{f(f(x)) = f(g(f(x)))\}$. Another way to put it is that no simplification order can prove termination of E if $E \cup EMB$ is nonterminating.

To have your own simplification order $\succ_{\text{mine}}$, just make sure that $\succ_{\text{mine}}$ is irreflexive, transitive, monotonic, and that it satisfies the subterm property. If you then can prove $l\sigma \succ_{\text{mine}} r\sigma$ for each equation $l = r$ and each ground substitution σ , you have proved that your specification is terminating. In case you do not want to define your own simplification order, you can use some of the *path orders* introduced next.

4.4.1 The Lexicographic Path Order

The *lexicographic path order* (lpo) [58] is a powerful simplification order which can be applied automatically. lpo requires that you have a strict partial order $\succ$, called a *precedence*, on the function symbols.

Definition 4.7 (Lexicographic path order) *Given a strict partial order $\succ$ on the function symbols, the lexicographic path order $\succ_{lpo}$ is the smallest relation satisfying the following conditions for $m, n \geq 0$:*⁵

lpo-1: If $t_i \succ_{lpo} u$ or $t_i = u$ for some t_i , then

$$f(\dots, t_i, \dots) \succ_{lpo} u.$$

lpo-2: If $f \succ g$ and $f(t_1, \dots, t_n) \succ_{lpo} u_i$ for all $i \leq m$, then

$$f(t_1, \dots, t_n) \succ_{lpo} g(u_1, \dots, u_m).$$

lpo-3: If $(t_1, \dots, t_n) \succ_{lpo}^{lex} (u_1, \dots, u_n)$ for $\succ_{lpo}^{lex}$ the lexicographic extension of $\succ_{lpo}$, and $f(t_1, \dots, t_n) \succ_{lpo} u_i$ for each $2 \leq i \leq n$, then

$$f(t_1, \dots, t_n) \succ_{lpo} f(u_1, \dots, u_n).$$

The lexicographic path order can be extended to terms with variables, where a variable is treated as a constant that is not comparable to anything in the precedence $\succ$, in which case $l \succ_{lpo} r$ implies $l\sigma \succ_{lpo} r\sigma$ for all substitutions σ .

The following result is proved, e.g., in [6]:

Proposition 4.4 $\succ_{lpo}$ is a simplification order for any precedence $\succ$.

Therefore, one way of proving the termination of a finite⁶ specification is to define a precedence $\succ$ on the function symbols (and extend it to variables so that no variable is comparable in $\succ$ with any other symbol) such that $l \succ_{lpo} r$ holds for each equation $l = r$ in the specification.

Functions are often defined using previously defined functions. For example, multiplication ($*$) is defined in terms of addition ($+$), and exponentiation ($**$) is defined in terms of multiplication. In these cases, termination can often be shown by choosing the precedence $\succ$ so that it satisfies $** \succ * \succ + \succ s \succ 0$.

Example 4.13. We prove termination of

$$\left\{ \begin{array}{lll} 0 + x = x, & 0 * x = 0, & x ** 0 = s(0), \\ s(x) + y = s(x + y), & s(x) * y = y + (x * y), & x ** s(y) = x * (x ** y) \end{array} \right\}$$

by showing that each equation is $\succ_{lpo}$ -decreasing when $** \succ * \succ + \succ s \succ 0$:

⁵ This definition also applies to *constants* when $m = 0$ or $n = 0$; for example, $f(c) \succ_{lpo} b$ and $a \succ_{lpo} b$ and $a \succ_{lpo} g(b)$ all hold by lpo-2 if $f \succ b$ and $a \succ b$ and $a \succ g$.

⁶ A finite specification in this case is one with only a finite set of function symbols and/or a finite set of equations. This should be the case for our Maude modules (except for some built-in modules).

- $0 + x \succ_{lpo} x$ holds because of lpo-1.
- $s(x) + y \succ_{lpo} s(x + y)$ follows from lpo-2, since $+ \succ s$, if we can prove $s(x) + y \succ_{lpo} x + y$, which follows from lpo-3, since ‘+’ is the main function symbol in both places, if we can prove $(s(x), y) \succ_{lpo}^{lex} (x, y)$ and $(s(x), y) \succ_{lpo} y$. The latter follows from lpo-1. $(s(x), y) \succ_{lpo}^{lex} (x, y)$ holds because $s(x) \succ_{lpo} x$ by lpo-1.
- $0 * x \succ_{lpo} 0$ follows from lpo-1.
- $s(x) * y \succ_{lpo} y + (x * y)$: Since $* \succ +$ we use lpo-2 and have to prove $s(x) * y \succ_{lpo} y$ and $s(x) * y \succ_{lpo} (x * y)$. The former holds by lpo-1. $s(x) * y \succ_{lpo} (x * y)$ holds by lpo-3 since $(s(x), y) \succ_{lpo}^{lex} (x, y)$ and $s(x) * y \succ_{lpo} y$ both hold as proved above.
- The last two equations: Exercise 62. $\diamond$

The lexicographic path order is fully automatic, since a finite set of function symbols only has a finite number of precedences $\succ$, and checking whether each equation is $\succ_{lpo}$ -decreasing is also a terminating process. A program can then check lpo-termination for each possible precedence $\succ$ (see Exercise 73).

4.4.2 The Multiset Path Order and Other Variations of lpo

In case lpo-3 in the definition of lpo, the immediate subterms $(t_1, \dots, t_n)$ and $(u_1, \dots, u_n)$ are compared *lexicographically*. The *multiset path order* (mpo) is the same as lpo except that $(t_1, \dots, t_n)$ and $(u_1, \dots, u_n)$ are compared as multisets. That is, mpo is defined as lpo, except that the condition lpo-3 is replaced by

mpo-3: If $\{t_1, \dots, t_n\} \succ_{mpo}^{ms} \{u_1, \dots, u_n\}$ (where $\succ_{mpo}^{ms}$ is the “multiset extension” of $\succ_{mpo}$), then $f(t_1, \dots, t_n) \succ_{mpo} f(u_1, \dots, u_n)$.

mpo and lpo are incomparable: only lpo can prove that $\{f(a, b) = f(b, a)\}$ is terminating, and only mpo can prove that $\{g(x, a) = g(b, x)\}$ is terminating.

4.4.2.1 * Combining and Extending lpo and mpo

Instead of comparing the lists of subterms $(t_1, \dots, t_n)$ and $(u_1, \dots, u_n)$ lexicographically or by multiset comparison, they can be compared in different ways for different top symbols. For example, if the top symbol is f we can compare the subterms lexicographically, and if the top symbol is g the subterms can be compared by multiset comparison. It is also possible to compare $(t_1, \dots, t_n)$ and $(u_1, \dots, u_n)$ lexicographically in *any fixed order*, say, by first comparing t_2 and u_2 , and then t_5 and u_5 , etc.

Example 4.14. The specification $\{f(x, s(y), z) = f(x + y + z, y, z + z)\}$ can be proved terminating using lpo when the subterms are compared lexicographically in the order 2. element, 1. element, and 3. element for top symbol f . $\diamond$

Sometimes it is necessary to allow two function symbols f and g to have the *same* precedence in $\succ$; that is, $f \approx g$. The case mpo-3 is changed accordingly to

$$f(t_1, \dots, t_n) \succ_{mpo} g(u_1, \dots, u_m)$$

if $f \approx g$ and $(t_1, \dots, t_n) \succ_{\text{mpo}}^{ms} (u_1, \dots, u_m)$; lpo can be redefined similarly. Terms are considered equivalent if they are equivalent up to $\approx$ -equivalent function symbols.

4.4.3 Comparing Weight Functions and Simplification Orders

As already mentioned, the main difference between “weight functions” and the path orders lpo and mpo is that the former are custom-defined for each specification—requiring ingenuity as well as possibly complex proofs of their suitability for proving termination—whereas the latter are automatic and ready to use.

Intuitively, the path orders seem fairly powerful. They can prove termination of specifications such as $\{f(s(y), x, z) = f(y, x + y + z, z + z)\}$, for which it seems hard to define a “standard” weight function (try!), and the Ackermann function, whose termination cannot be proved by a *polynomial* weight function.⁷

The inherent weakness of simplification orders is that they cannot prove termination of self-embedding systems. The path orders lpo and mpo also cannot prove the termination of a system like

$$\{f(g(h(x))) = f(f(x)), \quad f(g(h(x))) = g(g(x)), \quad f(g(h(x))) = h(h(x))\},$$

whereas their termination can be proved by trivial weight functions such as the size of the term. (If different function symbols can be regarded as the same in the precedence, then the above system *can* be shown terminating using lpo/mpo. However, in that case, the system

$$\begin{aligned} \{f(g(h(x))) = f(f(x)), \quad f(g(h(x))) = g(g(x)), \quad f(g(h(x))) = h(h(x)) \\ f(x) = g(x), \quad g(x) = h(x)\} \end{aligned}$$

cannot be proved using mpo, lpo, or their extensions mentioned above, whereas it can easily be proved terminating using weight functions (Exercise 60).)

If a finite specification can be proved terminating using a simplification order $\succ$, it can also be proved terminating using a weight function into a well-founded domain $(S, >)$: the domain S is the set of ground terms $\mathcal{T}_\Sigma$, the weight function is the identity function, and the comparison operator $>$ is the order $\succ$.

In the other direction, a *monotonic* weight function $\text{weight} : \mathcal{T}_\Sigma \rightarrow S$ (with comparison operator $>_s$) that satisfies the property

$$\text{weight}(f(\dots, t_i, \dots)) >_s \text{weight}(t_i)$$

for each function symbol f and all t_i induces a simplification order $\succ_{\text{weight}}$ on ground terms defined by $t \succ_{\text{weight}} u$ if and only if $\text{weight}(t) >_s \text{weight}(u)$.

⁷ A polynomial weight function is one where $\text{weight}(f(t_1, \dots, t_n))$ is defined as a polynomial in $\text{weight}(t_1), \dots, \text{weight}(t_n)$ for each function symbol f .

Exercise 61 Which of the following specifications are self-embedding? Which are terminating?

1. $\{f(g(x)) = g(f(x))\}$
2. $\{f(a, b, x) = f(x, x, x)\}$
3. (Difficult?) $\{f(g(x, y)) = g(g(f(f(x)), y), y)\}$
4. $\{x * (y + z) = (x * y) + (x * z)\}$ (Distributivity)

Exercise 62 Show that the last two equations in Example 4.13 are $\succ_{lpo}$ -decreasing with the given precedence $\succ$.

Exercise 63 Use lpo to prove termination of the following specifications:

1. $\{f(f(x)) = f(x)\}$
2. $\{f(x) = g(x), g(b) = f(a)\}$
3. $\{f(g(x)) = g(f(x))\}$
4. $\{g(f(x)) = f(g(x))\}$
5. $\{f(s(y), x, z) = f(y, x + (y + z), z + z)\}$
6. $\{x * (y + z) = (x * y) + (x * z)\}$
7. The Ackermann function:

$$\begin{aligned} &\{ack(0, x) = s(x), \\ &ack(s(x), 0) = ack(x, s(0)), \\ &ack(s(x), s(y)) = ack(x, ack(s(x), y))\} \end{aligned}$$

Exercise 64 Can lpo prove that $\{h(f(f(x))) = h(f(g(f(x))))\}$ is terminating?

Exercise 65 Why is the condition $f(t_1, \dots, t_n) \succ_{lpo} u_i$, for each $2 \leq i \leq n$, needed in the case lpo-3 in the definition of lpo? That is, show a nonterminating specification whose equations would be $\succ_{lpo}$ -decreasing without this condition.

Exercise 66 Is there a simplification order which can prove that $\{f(a, b, x) = f(x, x, x)\}$ is terminating?

Exercise 67 Use lpo to prove that the specification of binary trees that you defined in Exercise 13 is terminating.

Exercise 68 Consider the specification $\{f(a) = g(b), g(a) = f(b), f(x) = a\}$.

1. Show that the specification cannot be proved terminating using lpo or mpo if different function symbols cannot have the “same precedence” in $\succ$.
2. Use $\succ_{lpo}$ to prove termination of the specification if two function symbols may have the same precedence in $\succ$.

Exercise 69 Use a combination of mpo and lpo to prove termination of

$$\{f(a, b) = f(b, a), \quad g(x, a) = g(b, x), \quad h(x, a, x) = h(a, b, x)\}.$$

Exercise 70 The order $\succ_o$ extends a total strict partial order $\succ$ on the (finite) set of function symbols, and is defined by $t \succ_o u$ if and only if the list (number of occurrences in t of the $\succ$ -greatest function symbol, ..., number of occurrences in t of the $\succ$ -smallest function symbol) is lexicographically greater than the corresponding list for u . For example, if $f \succ g \succ a$, then $g(f(f(a)), f(f(g(a, a)))) \succ_o f(g(g(f(a), g(a, g(a, f(a))))), a)$, since $(4, 2, 3) \succ^{lex} (3, 4, 5)$.

1. Is $\succ_o$ well-founded?
2. Is $\succ_o$ a simplification order?
3. Is there a specification that can be proved terminating using $\succ_o$, but that cannot be proved terminating using lpo?
4. How can $\succ_o$ deal with variables?

Exercise 71 Consider the specification $\{f(g(x)) = h(x), h(x) = g(f(x)), a = b\}$.

1. Can the system be proved to terminate using weight functions?
2. Can the system be proved to terminate using lpo or mpo?
3. Is there a simplification order that can prove that the system is terminating?

Exercise 72 1. Can the union $E_1 \cup E_2$ of two terminating specifications E_1 and E_2 be nonterminating if E_1 and E_2 do not have any function symbol in common?

2. Assume that E_1 and E_2 can be proved terminating using lpo, and that they do not have any function symbol in common. Can their union $E_1 \cup E_2$ be nonterminating? What if E_1 and E_2 share one function symbol? What if they share two function symbols?

Exercise 73 In this exercise we implement lpo in Maude. We first define a data type for representing equational specifications. A term is represented by a term of sort `Term`. Such a term is either a constant, a variable, or a function symbol applied to a list of terms, so that, e.g., the term $f(a, g(b))$ is represented by `f[a, g[b]]`:

```
sorts FuncSymbol VarSymbol .
ops a ack b c d f g h s 0 + * - v w ... : -> FuncSymbol [ctor] .
ops x x1 x2 x3 x4 x5 y y1 y2 y3 y4 y5 ... : -> VarSymbol [ctor].

sorts Term TermList .
subsorts FuncSymbol VarSymbol < Term < TermList .
op _,_ : TermList TermList -> TermList [ctor assoc prec 120] .
op _[_] : FuncSymbol TermList -> Term [ctor] .
```

A set of equations is represented using the following data type:

```
sorts Equation EquationSet .
subsort Equation < EquationSet .
op none : -> EquationSet [ctor] .
op __ : EquationSet EquationSet -> EquationSet
                                         [ctor assoc comm id: none] .
op eq_=_ : Term Term -> Equation [ctor] .
```

The equations specifying the extremely fast-growing Ackermann function are then represented by the following term of sort `EquationSet`:

```

eq ack[0, x] = s[x] .
eq ack[s[x], 0] = ack[x, s[0]] .
eq ack[s[x], s[y]] = ack[x, ack[s[x], y]] .

```

A precedence is represented by a list of the form $f \gg g \gg h \gg a$:

```

sort Precedence .      subsort FuncSymbol < Precedence .
op emptyPrecedence : -> Precedence [ctor] .
op _>>_ : Precedence Precedence -> Precedence
                                     [ctor assoc id: emptyPrecedence] .

```

1. Define a function

```
op _>>_in_ : FuncSymbol FuncSymbol Precedence -> Bool .
```

such that $f \gg g$ in P equals `true` if and only if f is greater than g in the precedence P . (Hint: it might be useful to extend this function to variables.)

2. Define a function

```
op lpoTerm : EquationSet Precedence -> Bool .
```

that checks whether a given set of equations can be proved terminating using *lpo* with the given precedence. For example,

```

red lpoTerm(eq f[a, b, a] = f[a, b, b] .
             eq f[a, a, b] = f[a, b, a] .
             eq f[b, b, f[a, b, a]] = f[a, b, a] ., f >> a >> b) .

```

should return `true` while

```

red lpoTerm(eq f[a, b, a] = f[a, b, b] .
             eq f[a, a, b] = f[a, b, a] .
             eq f[b, b, f[a, b, a]] = f[a, b, a] ., f >> b >> a) .

```

should return `false`. Test your specification extensively in Maude.

3. Define a function

```
op lpoTerm : EquationSet -> Bool .
```

that returns `true` if there exists a precedence such that the given equations can be proved terminating using *lpo*. Hint: it might be useful to recall Exercise 34.

Exercise 74 Which of the following specifications can be proved terminating using a simplification order?

1. $\{f(h(x), y) = h(g(h(f(y, x))))\}$
2. $\{f(h(x), y) = h(g(h(f(y, x))))\}, \quad h(f(a, b)) = f(h(b), g(a))\}$
3. $\{h(f(x, y)) = f(y, x), \quad f(a, a) = h(h(h(f(a, b))))\}$
4. (Slightly tricky?) $\{f(b, c) = f(g(a, a), h(b, f(c, c)))\}$
5. $\{f(b, c) = f(g(a, b), h(a, f(c, c)))\}$

This chapter explains how to check whether a terminating specification is *confluent*, which ensures that the result of evaluating an expression is independent of the choice of *which* equation is applied to a term, and *where* the selected equation is applied.

Example 5.1. Both equations in $\{f(f(x)) = g(x), a = b\}$ can be applied to the term $f(f(f(a)))$; the first equation can be applied both in position ε and in position 1. $\diamond$

This chapter also considers only unsorted specifications without conditional equations and operator attributes. We first recall the definition of confluence:

Definition 5.1 (Confluence) A specification (Σ, E) is confluent if and only if for all terms t, t_1, t_2 with $t \xrightarrow{*} t_1$ and $t \xrightarrow{*} t_2$, there is a term u such that $t_1 \xrightarrow{*} u$ and $t_2 \xrightarrow{*} u$. The specification is ground confluent if the above property holds for all ground terms t .

Confluence means that if t can be reduced to two different terms t_1 and t_2 (for instance by applying different equations to t), we can always “join” t_1 and t_2 by reducing both to a common term u . This property is shown in Fig. 5.1 (left), where a solid arrow means “for all $\xrightarrow{*}$ ” and a dashed arrow means “there exists $\xrightarrow{*}$ ”.

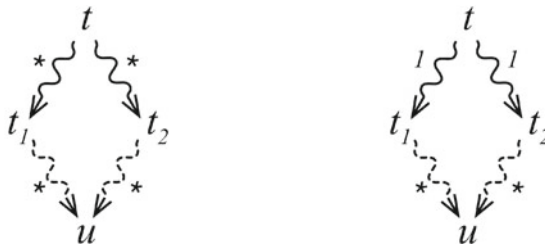


Fig. 5.1 Confluence (left) and *local* confluence (right)

Maude assumes that your specification is ground confluent. Although Chapter 6 shows that it is in general undecidable whether or not a specification is confluent, confluence is decidable if the specification is terminating. This chapter explains how to check whether a terminating specification is confluent.

Example 5.2. The specification in Example 5.1 is not confluent since $f(f(f(x))) \rightsquigarrow f(g(x))$ and $f(f(f(x))) \rightsquigarrow g(f(x))$, and $g(f(x))$ and $f(g(x))$ cannot be reduced to some common element (in fact, they cannot be reduced at all). Adding an equation $f(g(x)) = g(f(x))$ gives a terminating and confluent system which is “logically equivalent” (see Chapter 6) to the original specification, since $f(g(x)) = f(f(f(x))) = g(f(x))$ follows from the equation $f(f(x)) = g(x)$. $\diamond$

Checking “directly” whether a specification is confluent by checking the confluence property for all t, t_1, t_2 with $t \rightsquigarrow^* t_1$ and $t \rightsquigarrow^* t_2$ is not possible, because

1. a large number of terms t_1 and t_2 could be reachable from the term t , and
2. there are usually infinitely many terms t to start with.

We need to reduce the problem to considering (i) only a *limited* number of terms t_1 and t_2 reachable from some t , and (ii) only a *finite* number of terms t to start with.

We first address (i) by showing that for each start term t , only the terms t_1 and t_2 reachable in *one* reduction step from t need to be taken into account.

Definition 5.2 (Local confluence) A specification is locally confluent if and only if for each t and all t_1, t_2 such that $t \rightsquigarrow t_1$ and $t \rightsquigarrow t_2$, there is a term u such that $t_1 \rightsquigarrow^* u$ and $t_2 \rightsquigarrow^* u$.

Local confluence is illustrated in Fig. 5.1 (right), where the $\rightsquigarrow^1$ means “for all (one-step) reductions $\rightsquigarrow$ ”. It is enough to check local confluence instead of confluence:

Theorem 5.1 (Newman’s Lemma) A terminating specification is confluent if it is locally confluent.

Proof. According to Theorem 3.1 it is sufficient to prove that each term has a unique normal form in a locally confluent and terminating specification E . Let mnf_E be the set of terms with *multiple* distinct normal forms. (Notice that if $t \in mnf_E$ and $t' \rightsquigarrow_E^* t$, then also $t' \in mnf_E$.) If mnf_E is non-empty, it has at least one *smallest* element t w.r.t. the relation $\rightsquigarrow_E$ so that if $t \rightsquigarrow_E t'$, then $t' \notin mnf_E$. Since $t \in mnf_E$, it has at least two distinct normal forms t_1 and t_2 . Since t is reducible (otherwise t would be its only normal form), there are terms u_1 and u_2 such that $t \rightsquigarrow_E u_1 \rightsquigarrow_E^* t_1$ and $t \rightsquigarrow_E u_2 \rightsquigarrow_E^* t_2$, where t_1 is the unique normal form of u_1 and t_2 is the unique normal form of u_2 (u_1 and u_2 have unique normal forms, since t is a smallest element in mnf_E). Since E is locally confluent, there is a u with $u_1 \rightsquigarrow_E^* u$ and $u_2 \rightsquigarrow_E^* u$. Let u^* be one normal form of u . Then t_1 and u^* are both normal forms of u_1 ; but since $u_1 \notin mnf_E$, u_1 has a unique normal form. Therefore, $t_1 = u^*$. In the same way, t_2 and u^* are two normal forms of $u_2 \notin mnf_E$; therefore they must be the same. We therefore get $t_1 = u^* = t_2$, which contradicts the assumption that t_1 and t_2 were two different normal forms of t ; therefore, such a smallest element $t \in mnf_E$ cannot exist, which means that mnf_E is empty; which again means that each term has a unique normal form. $\square$

We still have to address issue (ii) above: reducing the check of local confluence to a *finite* number of “start terms” t . For that, we introduce the notion of unification.

Exercise 75 Define a locally confluent specification that is not confluent.

Exercise 76 Define a ground confluent specification that is not confluent.

5.1 Unification

Definition 5.3 (Unifier) A unifier of two terms t and u is a substitution σ such that $t\sigma = u\sigma$.

Example 5.3. $f(x, h(b))$ and $f(h(y), z)$ have a unifier $\sigma = \{x \mapsto h(y), z \mapsto h(b)\}$. Any instance of σ , such as $\sigma' = \{x \mapsto h(f(f(a, a), a)), y \mapsto f(f(a, a), a), z \mapsto h(b)\}$, is also a unifier. On the other hand, $f(g(x))$ and $f(h(z))$ have no unifier (why not?); neither has the pair $f(x)$ and $g(y)$, nor the pair $f(x)$ and $f(g(x))$.

Example 5.3 shows that two terms can have many unifiers. We are interested in finding the *most general unifier* (mgu), which is a unifier ρ such that all other unifiers σ are “instances” of ρ . That is, ρ is an mgu of a pair of terms if for each unifier σ of the pair, there is a substitution π such that $\sigma = \pi \circ \rho$, where $\circ$ denotes function composition, i.e., $(f \circ g)(x) = f(g(x))$.

Example 5.3. (cont.) The substitution σ is an mgu of $f(x, h(b))$ and $f(h(y), z)$. Two other unifiers of these terms are the above σ' and $\sigma'' = \{x \mapsto h(h(h(h(z))))\}$, $z \mapsto h(b), y \mapsto h(h(h(h(z))))\}$. Both σ' and σ'' are instances of σ :

$$\begin{aligned}\sigma' &= \{y \mapsto f(f(a, a), a)\} \circ \sigma \\ \sigma'' &= \{y \mapsto h(h(h(h(z))))\} \circ \sigma\end{aligned}\quad \diamond$$

Proposition 5.1 If two terms have a unifier, then they have a most general unifier. Furthermore, the most general unifier is unique up to renaming of the variables.

A *renaming* changes the names of the variables in a term/equation/... so that the term/equation/... logically is “the same,” just with different variable names.¹ For example, $f(x', y')$, $f(x, y)$, $f(y, x)$, and $f(x, z)$ are all renamed versions of $f(x, y)$, but $f(x, x)$ and $f(z, a)$ are not. A renaming does not change the “logic” of a specification. The specification $\{f(x, y) = g(x), h(x, y) = f(x, y)\}$ is logically the same as $\{f(x, z) = g(x), h(x', y') = f(x', y')\}$.

The following algorithm, due to Martelli and Montanari, can be used to find the mgu of two terms that are unifiable, or to figure out that two terms are not unifiable.

The algorithm maintains a pair (UP, ρ) , where UP is a *set of unification problems* of the form $t \stackrel{?}{=} u$, and ρ is the mgu being constructed. Initially, UP is the

¹ Formally, a renaming is a bijective substitution.

unification problem we want to solve and ρ is the identity (the substitution which “does nothing”). The algorithm proceeds by applying the following steps until it returns `<Not unifiable>` or the desired mgu:

1. Return `<Not unifiable>` if UP contains a unification problem of the form $f(t_1, \dots, t_n) \stackrel{?}{=} g(u_1, \dots, u_m)$, for $m, n \geq 0$, where $f \neq g$. (Obviously there is no unifier for this unification (sub)problem.)
2. If UP has the form²

$$\{f(t_1, \dots, t_n) \stackrel{?}{=} f(u_1, \dots, u_n)\} \uplus UP'$$

then we must find unifiers for $t_1 \stackrel{?}{=} u_1$, and $\dots$, and $t_n \stackrel{?}{=} u_n$. That is,

$$UP := \{t_1 \stackrel{?}{=} u_1, \dots, t_n \stackrel{?}{=} u_n\} \cup UP'.$$

3. If UP contains a unification problem of the form $t \stackrel{?}{=} t$, then just remove this trivial unification problem from UP .
4. If UP contains a unification problem $x \stackrel{?}{=} t$ (or $t \stackrel{?}{=} x$) where x and t are different terms and x occurs in t , then return `<Not unifiable>`. (For example, the terms x and $f(x)$ are not unifiable (why not?).)
5. If UP contains a unification problem of the form $x \stackrel{?}{=} t$ (or $t \stackrel{?}{=} x$) where x and t are (syntactically) different terms and x does not occur in t , then:
 - remove this unification problem from UP ,
 - apply the substitution $\{x \mapsto t\}$ on all remaining unification problems in UP , and
 - apply the substitution $\{x \mapsto t\}$ on ρ (one effect is that $x \mapsto t$ is added to ρ , since ρ does not contain an assignment of x (why not?) and hence has $x \mapsto x$).
6. If UP is empty, then return ρ , which is the desired mgu.

Example 5.4. Let's find the mgu of the pair $f(x, h(x))$ and $f(h(y), z)$ using the algorithm: We start with

$$(\{f(x, h(x)) \stackrel{?}{=} f(h(y), z)\}, Id)$$

where Id is the identity substitution $\{x \mapsto x, y \mapsto y, z \mapsto z\}$. Then

$$\begin{aligned} (\{f(x, h(x)) \stackrel{?}{=} f(h(y), z)\}, Id) &\implies && \text{(by step 2)} \\ (\{x \stackrel{?}{=} h(y), h(x) \stackrel{?}{=} z\}, Id) &\implies && \text{(by step 5)} \\ (\{h(h(y)) \stackrel{?}{=} z\}, \{x \mapsto h(y)\}) &\implies && \text{(by step 5)} \\ (\emptyset, \{x \mapsto h(y), z \mapsto h(h(y))\}) &\implies && \text{(by step 6)} \\ \text{return } \{x \mapsto h(y), z \mapsto h(h(y))\} &&& \diamond \end{aligned}$$

² The symbol $\uplus$ denotes *disjoint union*, which means that $f(t_1, \dots, t_n) \stackrel{?}{=} f(u_1, \dots, u_n)$ does not appear in UP' .

The unification algorithm is correct and terminating (see, e.g., [105] for proof).

Exercise 77 Decide whether the following unification problems have unifiers, and if so, find their mgus:

1. $f(x, y) \stackrel{?}{=} g(a, b)$
2. $f(x, x) \stackrel{?}{=} f(a, b)$
3. $f(x, y) \stackrel{?}{=} f(a, f(a, b))$
4. $f(x, y) \stackrel{?}{=} f(g(x), a)$
5. $f(x, y) \stackrel{?}{=} f(g(y), h(x))$
6. $f(x, x) \stackrel{?}{=} f(g(y), g(h(z)))$
7. $f(a, y) \stackrel{?}{=} f(x, b)$

Exercise 78 Explain why the unification algorithm is terminating. Hint: find a weight function on pairs (UP, ρ) such that each step of the algorithm (except steps 1, 4, and 6, which lead to immediate termination) decreases the weight of the pair. Then explain why the unification algorithm terminates with either $\langle \text{Not unifiable} \rangle$ or with an empty set of remaining unification problems.

5.2 Checking Local Confluence

Newman's Lemma means that it is enough to check the confluence property for all terms $t_1, t_2, \dots$ reachable in *one* step from some term t . However, there can be an infinite number of such start terms t . The next step is therefore to restrict the number of terms t for which to check local confluence.

Let $l_i = r_i$ and $l_j = r_j$ be two equations in our specification (they could be the same equation!), and rename if necessary the variables in $l_j = r_j$ so that l_i and l_j do not have any variables in common. Let p be a position in l_i so that $l_i|_p$ is *not* a variable. If $l_i|_p$ and l_j are unifiable with mgu ρ , then the term $l_i\rho$ may reduce to $r_i\rho$ (by applying $l_i = r_i$ at the top (position ε)). The term $l_i\rho$ may also reduce to $(l_i\rho)[r_j\rho]_p$ (by applying $l_j = r_j$ at position p). That is,

$$l_i\rho \rightsquigarrow r_i\rho \quad \text{and} \quad l_i\rho \rightsquigarrow (l_i\rho)[r_j\rho]_p.$$

To check local confluence we check whether the *critical pair* $(r_i\rho, (l_i\rho)[r_j\rho]_p)$ is *joinable* (that is, whether there is a term u such that $r_i\rho \overset{*}{\rightsquigarrow} u$ and $(l_i\rho)[r_j\rho]_p \overset{*}{\rightsquigarrow} u$). This has to be done for all pairs of equations (including two copies of the same equation), and for all positions, and then we have checked local confluence:

Theorem 5.2 (Critical Pair Lemma [62]) *A specification is locally confluent if and only if all critical pairs are joinable.*

Intuitively, confluence of a terminating specification can be checked as follows:

1. Choose two (not necessarily different) equations from the specification and change the names of the variables in one of them (e.g., by replacing each x with an x') so that the two equations have no variable in common.
2. Check if the *lefthand* sides of the two equations “overlap.” That is, whether the entire term l_j “fits/can be unified with” some *non-variable* subterm of l_i . Perform the two reductions possible from this “overlap term” and check whether the resulting two terms are joinable.³ If the two terms in this critical pair are joinable (or have a common normal form), then repeat step 2 with a different position. Otherwise, the specification is not confluent and the algorithm exits.
3. Repeat step 1 until all pairs of equations have been checked.
4. If all pairs of equations and all overlap-positions within each such pair have been checked successfully, then the specification is confluent.

There is no need to consider the trivial case when the left-hand side l_i of an equation unifies with the renamed left-hand $l_i\sigma$ of the *same equation* at position ε (the top). The resulting trivial critical pair $(r_i\sigma, r_i\sigma)$ is obviously joinable.

Example 5.5. Let us check whether $\{f(f(x)) = g(x)\}$ is confluent. The only pair of equations is $(f(f(x)) = g(x), f(f(x)) = g(x))$. Since they share x , we rename one of them to $f(f(x')) = g(x')$ and check the pair $(f(f(x)) = g(x), f(f(x')) = g(x'))$. Now, l_i is $f(f(x))$ and we need to check all non-variable subterms of $f(f(x))$ for an overlap with $f(f(x'))$. The non-variable subterms of $f(f(x))$ are $f(f(x))$ and $f(x)$, and there is no need to check the trivial overlap with $f(f(x))$.

Therefore, the only potentially interesting case happens if the subterm $f(x)$ (which is the subterm at position 1 of $f(f(x))$) and $f(f(x'))$ are unifiable. Are they? Yes, with mgu $\rho = \{x \mapsto f(x')\}$. The resulting “overlap term” $l_i\rho = f(f(f(x')))$ can be reduced to $g(f(x'))$ by using the first equation at the top, and to $f(g(x'))$ by using the second equation at position 1. We then need to check whether the critical pair $(g(f(x')), f(g(x')))$ is joinable. Since neither $g(f(x'))$ nor $f(g(x'))$ can be further reduced, they are not joinable. Therefore, the specification is *not* confluent, since $f(f(f(x')))) \rightsquigarrow g(f(x'))$ and $f(f(f(x')))) \rightsquigarrow f(g(x'))$, but there is no term u such that both $g(f(x')) \overset{*}{\rightsquigarrow} u$ and $f(g(x')) \overset{*}{\rightsquigarrow} u$. $\diamond$

Example 5.6. The specification $\{0 + x = x, s(x) + y = s(x + y)\}$ is confluent, since there are no non-trivial overlaps between the left-hand sides of the equations.

Equational completion, briefly discussed in Section 6.1.1, is a process that tries to transform an equational specification into a “logically equivalent” specification that is both confluent and terminating, for example by adding an equation $t = u$ when a non-joinable critical pair (t, u) is encountered. In Example 5.2 we oriented the non-joinable critical pair $(g(f(x')), f(g(x')))$ in Example 5.5 and added the equation $f(g(x)) = g(f(x))$ to obtain a confluent and terminating specification. However,

³ Instead of checking joinability directly, one can find some normal forms of the two terms. If they have the same normal forms, they are obviously joinable; if not, the specification is obviously not confluent, since the “overlap term” $l_i\rho$ has two different normal forms.

one must check that the resulting specification is confluent (and terminating), since the new equation could lead to new non-joinable critical pairs.

Exercise 79 A group is a set with a binary function $\circ$, an “identity” element e , and an “inverse” function i satisfying the equations

$$G = \{e \circ x = x, \quad i(x) \circ x = e, \quad (x \circ y) \circ z = x \circ (y \circ z)\}.$$

Is G confluent? Can you prove that G is terminating?

Exercise 80 (From [60])

1. Is $\{(x+y)+z = x+(y+z)\}$ confluent?
2. Show that $\{(x+y)+z = x+(y+z), \quad x+0 = x\}$ is not confluent.

Exercise 81 Prove that the specification in Example 5.1 extended with the equation $f(g(x)) = g(f(x))$ is confluent and terminating. Can you also prove that the specification in Example 5.1 extended with the equation $g(f(x)) = f(g(x))$ (the critical pair in Example 5.5 oriented the other way) is confluent and terminating?

Exercise 82 (From [6])

1. Find terms r_1 and r_2 such that $\{f(g(x)) = r_1, \quad g(h(x)) = r_2\}$ is confluent (and terminating).
2. Is $\{f(g(f(x))) = g(x)\}$ confluent? If it is not confluent, can you add some equation(s) to the specification, so that the resulting specification is confluent, terminating, and “logically the same” as the original specification?
3. Consider the specification

$$\{f(f(x)) = f(x), \quad g(g(x)) = f(x), \quad f(g(x)) = g(x), \quad g(f(x)) = g(x)\}$$

- Prove that the specification is confluent.
- Can you determine the normal form of a term as a function of the number of f ’s and g ’s in the term? Hint: Are there an odd number of g ’s?

Exercise 83 In this exercise we implement the unification algorithm in Section 5.1 in Maude. We use the data types in Exercise 73 to represent terms and equations.

1. Define a sort `Substitution` for representing substitutions, and a supersort `DefSubstitution` with an additional element `fail`.
2. Define a function `applySubst : Term Substitution -> Term` which applies a substitution to a term.
3. Define a function `unifier : Term Term -> DefSubstitution` which returns the mgu of two terms if they are unifiable, and `fail` otherwise.
4. Test your function on the unification problems in Exercise 77.

This chapter explains how we can reason about whether two expressions are “logically equivalent” in a specification E . We consider two different notions of what it means that two terms t and u (which may contain variables) are logically equivalent:

1. $t = u$ follows from the *equations* in E , without considering the signature of E .
2. $t = u$ follows from the equations in E , but taking the signature into account, in the sense that t and u are “equivalent” if and only if t and u are equivalent for each ground instance: $t\sigma = u\sigma$ for each *ground substitution* σ .

Let us start with the first notion. Many mathematical theories, such as the theory of groups, the theory of rings, etc., can be defined by giving a set of equations as the axioms of the theory. Given two terms t and u , a mathematician may be interested in whether the equivalence $t = u$ “follows logically” from the equations. For example, do $x \circ i(x) = e$ and $i(i(x)) = x$ hold in *all groups*? That is, do they follow logically from the *group axioms* $\{e \circ x = x, i(x) \circ x = e, (x \circ y) \circ z = x \circ (y \circ z)\}$?

Chapter 7 defines what “follows logically from a set of equations” means: $t = u$ follows from E if and only if $t = u$ is true in *all possible* mathematical structures/models where the equations E hold. For example, $x \circ i(x) = e$ follows logically from the group axioms if and only if $x \circ i(x) = e$ holds in *all* groups, that is, in all mathematical structures satisfying the group axioms. The problem is of course that it is impossible to explicitly check every structure satisfying E to figure out whether an equality $t = u$ holds in all of them. This chapter therefore introduces *equational logic* as a way to reason about whether an equality $t = u$ “follows logically” from an equational specification E : t and u are logically equivalent if and only if $t = u$ can be deduced from the equations E using the rules of equational logic. The point is that we can use equational logic reasoning instead of checking whether $t = u$ holds in *all* E -structures, since, as shown in Chapter 7, $t = u$ follows from E in equational logic if and only if $t = u$ holds in all structures where the equations E hold.

For *general* theories such as groups, rings, and so on, reasoning about equalities that hold in all structures is exactly what we want. However, quite often we are *not* interested in *all* E -structures, but only in the *intended* structure. When reasoning about NAT-ADD, we are not interested in studying whether something holds in *all*

systems satisfying the two equations $0 + M = M$ and $s(M) + N = s(M + N)$; we are only interested in whether something holds for the *natural numbers*.

For example, addition on natural numbers is commutative, so it should be the case that $m + n = n + m$ holds in NAT-ADD for all “natural numbers” m and n .¹ Likewise, to increase our confidence that we have specified lists and trees correctly, we want to verify that expected properties such as $\text{reverse}(\text{reverse}(bt)) = bt$ and $\text{length}(\text{concat}(l_1, l_2)) = \text{length}(l_1) + \text{length}(l_2)$ are logical consequences of our specifications for *all* binary trees bt and *all* lists l_1 and l_2 .

It turns out that $M + N = N + M$ (for variables M and N) does *not* follow logically from the equations $0 + M = M$ and $s(M) + N = s(M + N)$, since it does *not* hold in *all* structures satisfying the two equations (just add a new constant a to NAT-ADD ; then $a + 0$ cannot be reduced and is therefore different from $0 + a$). However, $M + N = N + M$ holds in the intended NAT-ADD -structure, in the sense that it holds for all instances where M and N are instantiated with the numerals constructed by 0 and s . That is, $m + n = n + m$ holds for all constructor ground terms m and n of sort NAT .

Equalities that only hold in the intended structure, whose data elements are constructor ground terms, are called *inductive theorems*. Chapter 7 formally defines what we mean by the “intended” model of a specification and explains that an inductive theorem holds in this intended structure.

Section 6.1 introduces equational logic and Section 6.2 shows how to prove inductive theorems. We assume in Section 6.1 that (Σ, E) is an unsorted specification without conditional equations, and that Σ contains at least one constant.

6.1 Equational Logic

We write $E \vdash t = u$ for the *sequent* which means that the equality $t = u$ can be *proved* in *equational logic* to follow logically from the equations E .

Definition 6.1 (Equational logic) *For an unsorted equational specification (Σ, E) (without conditional equations), we write $E \vdash t = u$, for terms $t, u \in \mathcal{T}_\Sigma(X)$, if and only if $E \vdash t = u$ can be derived by a finite number of applications of the following axiom schemas and deduction rules of equational logic:*

E_1 (Substitutivity): *The sequent $E \vdash l\sigma = r\sigma$ holds for any equation $l = r$ in E and any substitution σ .*

E_2 (Reflexivity): *$E \vdash t = t$ holds for any term t .*

E_3 (Symmetry): *If $E \vdash t = u$ holds, then $E \vdash u = t$ holds.*

E_4 (Transitivity): *If $E \vdash t_1 = t_2$ and $E \vdash t_2 = t_3$ both hold, then $E \vdash t_1 = t_3$ holds.*

E_5 (Congruence): *If $E \vdash t_1 = u_1, \dots$, and $E \vdash t_n = u_n$ all hold, then*

$$E \vdash f(t_1, \dots, t_n) = f(u_1, \dots, u_n)$$

holds for each function symbol f which takes n arguments.

¹ Terms such as $0, s(0), \dots$ that represent numbers are called *numerals*.

Reasoning with these kinds of logics, or *deduction systems*, may take some time getting used to. The basic facts that we can start each deduction with are that we can deduce $E \vdash t\sigma = t'\sigma$ for each equation $t = t'$ in E and each substitution σ , and that we can deduce $E \vdash t = t$ for each term t . From these basic facts, we can then use the deduction rules of equational logic to deduce new facts, as exemplified below.

Example 6.1. For E the equations $\{f(x) = g(x), a = b, g(c) = c\}$, we can prove $E \vdash b = a$ as follows:

1. By *Substitutivity* we can prove that $E \vdash a = b$, since $a = b$ is an equation in E . The substitution is of course just the empty substitution.
2. Now we have proved $E \vdash a = b$. The deduction rule *Symmetry* says that if $E \vdash a = b$ holds, then so does $E \vdash b = a$. That's all! We have proved the unsurprising fact that $b = a$ follows logically from the above equations E .

Does $f(a) = g(b)$ follow logically from E ? That is, can we prove $E \vdash f(a) = g(b)$?

1. $E \vdash a = b$ holds because of *Substitutivity*, since $a = b$ is an equation in E .
2. Since $E \vdash a = b$, the *Congruence* rule says that then $E \vdash f(a) = f(b)$ also holds.
3. By *Substitutivity* w.r.t. the equation $f(x) = g(x)$ and substitution $\sigma = \{x \mapsto b\}$, we also have $E \vdash f(b) = g(b)$.
4. Since both $E \vdash f(a) = f(b)$ and $E \vdash f(b) = g(b)$ hold, the *Transitivity* rule then says that $E \vdash f(a) = g(b)$ also holds. This is what we wanted to prove. Q.E.D.

The above *proof* of $E \vdash f(a) = g(b)$ can be summarized in the following shorter form (where E_1 denotes *Substitutivity*, E_2 denotes *Reflexivity*, and so on):

1. $E \vdash a = b$ (E_1 ; equation $a = b$)
2. $E \vdash f(a) = f(b)$ (E_5 ; from 1)
3. $E \vdash f(b) = g(b)$ (E_1 ; equation $f(x) = g(x)$)
4. $E \vdash f(a) = g(b)$ (E_4 ; from 2, 3)

Each line in such a deduction/proof must be justified, either by following directly from *Substitutivity* or *Reflexivity*, or by following from claims which have already been justified and one of the deduction rules *Symmetry*, *Transitivity*, or *Congruence*.

A graphical representation of the same proof shows the deductions used, with the assumptions above the line and the conclusion below it. Such proofs must start with instances of *Substitutivity* or *Reflexivity*. The proof of $E \vdash f(a) = g(b)$ can be given as the following *proof tree*:

$$\frac{\frac{\overline{E \vdash a = b} \text{ Substitutivity}}{E \vdash f(a) = f(b)} \text{ Congruence} \quad \frac{\overline{E \vdash f(b) = g(b)} \text{ Substitutivity}}{E \vdash f(a) = g(b)} \text{ Transitivity} \quad \diamond$$

Example 6.2. $\text{NAT-ADD} \vdash s(s(0)) + s(0) = s(0) + s(s(0))$ holds because it can be derived as follows in equational logic²:

² In this chapter, $M \vdash t = t'$ denotes $\text{eqs}(M) \vdash t = t'$ when M is a module name and $\text{eqs}(M)$ are the equations in the module M .

1. $\text{NAT-ADD} \vdash s(s(0)) + s(0) = s(s(0) + s(0))$ (E_1 ; $s(M) + N = s(M + N)$)
2. $\text{NAT-ADD} \vdash s(0) + s(0) = s(0 + s(0))$ (E_1 ; $s(M) + N = s(M + N)$)
3. $\text{NAT-ADD} \vdash 0 + s(0) = s(0)$ (E_1 ; equation $0 + M = M$)
4. $\text{NAT-ADD} \vdash s(0 + s(0)) = s(s(0))$ (E_5 ; from 3)
5. $\text{NAT-ADD} \vdash s(0) + s(0) = s(s(0))$ (E_4 ; from 2,4)
6. $\text{NAT-ADD} \vdash s(s(0) + s(0)) = s(s(s(0)))$ (E_5 ; from 5)
7. $\text{NAT-ADD} \vdash s(s(0)) + s(0) = s(s(s(0)))$ (E_4 ; from 1,6)
8. $\text{NAT-ADD} \vdash s(0) + s(s(0)) = s(0 + s(s(0)))$ (E_1 ; $s(M) + N = s(M + N)$)
9. $\text{NAT-ADD} \vdash 0 + s(s(0)) = s(s(0))$ (E_1 ; equation $0 + M = M$)
10. $\text{NAT-ADD} \vdash s(0 + s(s(0))) = s(s(s(0)))$ (E_5 ; from 9)
11. $\text{NAT-ADD} \vdash s(0) + s(s(0)) = s(s(s(0)))$ (E_4 ; from 8,10)
12. $\text{NAT-ADD} \vdash s(s(s(0))) = s(0) + s(s(0))$ (E_3 ; from 11)
13. $\text{NAT-ADD} \vdash s(s(0)) + s(0) = s(0) + s(s(0))$ (E_4 ; from 7,12) $\diamond$

To prove that an equality follows logically from a set of equations you “just” need to give a sequence of deductions leading to the desired equality. However, it is in principle impossible to say that something, like $E \vdash f(a) = f(c)$, does *not* hold. We can only say something like “I have tried a bunch of deductions and I still could not prove $E \vdash f(a) = f(c)$.” But this could in principle be either because $E \vdash f(a) = f(c)$ does not hold, or because you are not clever enough using the deduction rules. Fortunately, Theorem 6.5 shows that it is easy to prove that “ $E \vdash t = t'$ does not hold,” written $E \not\vdash t = t'$, when the equations E are terminating and confluent.

Another way of proving $E \not\vdash t = u$ is to come up with a mathematical structure satisfying E , but where $t = u$ does not hold. This is because $E \vdash t = u$ holds if and only if $t = u$ holds in *all* “structures” where the equations E hold.

Example 6.3. It seems obvious that $s(0) = 0$ should not follow logically from the equations in NAT-ADD . But how can we prove that? The equations in NAT-ADD all hold for the natural numbers, where 0 is supposed to mean the number 0, $s(n)$ is supposed to mean 1 plus the interpretation of n , and $+$ is supposed to mean addition on natural numbers. Therefore, all equalities that follow from NAT-ADD *must* hold for the natural numbers. $s(0) = 0$ does not hold for the natural numbers since $1 \neq 0$, and we can conclude that $s(0) = 0$ does *not* follow logically from NAT-ADD . $\diamond$

The following theorem may not come as a major surprise after seeing how difficult it is to deduce $\text{NAT-ADD} \vdash s(s(0)) + s(0) = s(0) + s(s(0))$:

Theorem 6.1 *It is undecidable whether*

$$E \vdash t = u$$

holds, even for ground terms t and u .

Proof. This result can be proved in different ways. A well-known proof uses the fact due to Matiyasevich [77] that it is in general undecidable even for ground terms t and u whether $t = u$ follows from the specification³

³ We can omit parentheses because of associativity.

$$\begin{aligned}
& \{x \circ (y \circ z) = (x \circ y) \circ z, \\
& a \circ a \circ a \circ a \circ b \circ b = b \circ b \circ a \circ a \circ b \circ a, \\
& a \circ a \circ b \circ a \circ b \circ b \circ a = b \circ b \circ a \circ a \circ a \circ b \circ a, \\
& a \circ b \circ a \circ a \circ a \circ b \circ b = a \circ b \circ b \circ a \circ b \circ a \circ a, \\
& b \circ b \circ b \circ a \circ a \circ b \circ b \circ a \circ a \circ b \circ a = b \circ b \circ b \circ a \circ a \circ b \circ b \circ a \circ a \circ a \circ a, \\
& a \circ a \circ a \circ a \circ b \circ b \circ a \circ a \circ b \circ a = b \circ b \circ a \circ a \circ a \circ a\}.
\end{aligned}
\quad \square$$

This book has primarily dealt with equational *reduction* (“applying an equation”). The following theorem says that equational *reduction* and equational logic *deduction* can be seen as the same thing:

Theorem 6.2 *For any set E of equations and terms t, u we have*

$$E \vdash t = u \quad \text{if and only if} \quad t \overset{*}{\rightsquigarrow} u.$$

Proof. We prove this theorem by first proving that $E \vdash t = u$ implies $t \overset{*}{\rightsquigarrow} u$, and then we prove the other direction, $t \overset{*}{\rightsquigarrow} u$ implies $E \vdash t = u$.

Since $E \vdash t = u$ by definition means that $E \vdash t = u$ can be derived by a *finite* number of applications of the deduction rules of equational logic, we can prove the “ $E \vdash t = u$ implies $t \overset{*}{\rightsquigarrow} u$ ” part by induction on the number of deduction steps needed to prove $E \vdash t = u$.

- Base case: one application of an axiom of equational logic proves $E \vdash t = u$. This means that either *Substitutivity* or *Reflexivity* was used to prove $E \vdash t = u$.
 - Assume that *Reflexivity* was used to prove $E \vdash t = u$. Then t and u are the same term, which means that we need to prove that $t \overset{*}{\rightsquigarrow} t$, which follows from the definition of $\overset{*}{\rightsquigarrow}$ on page 62.
 - Assume that *Substitutivity* was used to prove $E \vdash t = u$. This means that there is an equation $l = r$ in E and a substitution σ such that $t = l\sigma$ and $u = r\sigma$. We therefore need to prove $l\sigma \overset{*}{\rightsquigarrow} r\sigma$. It follows directly from the definition of a reduction step on page 62 that $l\sigma \rightsquigarrow r\sigma$; this in turns implies that the desired $l\sigma \overset{*}{\rightsquigarrow} r\sigma$ holds by the definition of $\overset{*}{\rightsquigarrow}$.
- Induction step: Assume that $E \vdash t = u$ has been proved using $n + 1$ deduction steps. The induction hypothesis is then that $E \vdash t' = u'$ implies $t' \overset{*}{\rightsquigarrow} u'$ if $E \vdash t' = u'$ can be proved using n deduction steps or less.
 - Assume that the *Transitivity* rule was used in the last step in the proof of $E \vdash t = u$. That is, we have a proof of the form

$$\frac{\begin{array}{c} \vdots \\ E \vdash t = v \end{array} \quad \begin{array}{c} \vdots \\ E \vdash v = u \end{array}}{E \vdash t = u} \text{ Transitivity}$$

Since this proof uses $n + 1$ applications of the rules and axioms of equational logic, both $E \vdash t = v$ and $E \vdash v = u$ can be proved in n steps or less. The

induction hypothesis therefore applies to both $E \vdash t = v$ and $E \vdash v = u$, which means that we can assume that $t \overset{*}{\rightsquigarrow} v$ and $v \overset{*}{\rightsquigarrow} u$. Then we have the desired $t \overset{*}{\rightsquigarrow} u$ since $t \overset{*}{\rightsquigarrow} v \overset{*}{\rightsquigarrow} u$.

- The cases when the last rule used in the proof of $E \vdash t = u$ is either *Symmetry* or *Congruence* are left as Exercise 87.

We now prove that $t \overset{*}{\rightsquigarrow} u$ implies $E \vdash t = u$. By definition, $t \overset{*}{\rightsquigarrow} u$ means that either $t \overset{*}{\rightsquigarrow} u$ is a zero-step derivation (t and u are the same term), or that there is a sequence of $n + 1$ “two-way” reduction steps $t \rightsquigarrow t_1 \rightsquigarrow \dots \rightsquigarrow t_n \rightsquigarrow u$. We prove that $t \overset{*}{\rightsquigarrow} u$ implies $E \vdash t = u$ by induction on the number of reduction steps in $t \overset{*}{\rightsquigarrow} u$.

- $t \overset{*}{\rightsquigarrow} u$ is a derivation with no reduction steps either way. Then t and u are the same term, and we can prove the desired property $E \vdash t = t$ by *Reflexivity*.
- $t \overset{*}{\rightsquigarrow} u$ is derivation $t \rightsquigarrow t' \overset{*}{\rightsquigarrow} u$ of length $n + 1$. Since $t' \overset{*}{\rightsquigarrow} u$ is then a derivation of length n , we can apply the induction hypothesis to $t' \overset{*}{\rightsquigarrow} u$, so that we can assume that $E \vdash t' = u$. The first step $t \rightsquigarrow t'$ in the above derivation is (by definition of $t \rightsquigarrow t'$) either a step $t \rightsquigarrow t'$ or a step $t' \rightsquigarrow t$. We prove the case where $t \rightsquigarrow t'$ and leave the case where $t' \rightsquigarrow t$ as an exercise. If $t \rightsquigarrow t'$ then there is an equation $l = r$ in E , a position p in t , and a substitution σ such that $t|_p = l\sigma$ and $t' = t[r\sigma]_p$. We prove the lemma

$$t \rightsquigarrow t' \quad \text{implies} \quad E \vdash t = t'$$

by induction of the length of the position p :

- If p has length 0, that is, $p = \varepsilon$, then t is $l\sigma$ and t' is $r\sigma$ and the desired $E \vdash t = t'$ follows directly by *Substitutivity*.
- If p has length $n + 1$, that is, it equals $i.p'$, then t is $f(t_1, \dots, t_i, \dots, t_m)$ and t' is $f(t_1, \dots, t_i[r\sigma]_{p'}, \dots, t_m)$, where $t_i|_{p'}$ is $l\sigma$. Since p' is a shorter path than p , and $t_i \rightsquigarrow t_i[r\sigma]_{p'}$, the “local” induction hypothesis for our lemma gives us that $E \vdash t_i = t_i[r\sigma]_{p'}$. Furthermore, *Reflexivity* gives $E \vdash t_j = t_j$ for $j \in \{1, \dots, m\} \setminus \{i\}$, and the *Congruence* rule gives the desired result $E \vdash t = t'$:

$$\frac{E \vdash t_1 = t_1 \quad \dots \quad E \vdash t_i = t_i[r\sigma]_{p'} \quad \dots \quad E \vdash t_m = t_m}{E \vdash f(t_1, \dots, t_i, \dots, t_m) = f(t_1, \dots, t_i[r\sigma]_{p'}, \dots, t_m)} \text{Congruence}$$

We have therefore proved that $t \rightsquigarrow t'$ implies $E \vdash t = t'$. The main induction hypothesis gave us $E \vdash t' = u$, and the desired $E \vdash t = u$ follows by *Transitivity*:

$$\frac{E \vdash t = t' \text{ (lemma above)} \quad E \vdash t' = u \text{ (induction hypothesis)}}{E \vdash t = u} \text{Transitivity}$$

Since we have proved both that $E \vdash t = u$ implies $t \overset{*}{\rightsquigarrow} u$ and that $t \overset{*}{\rightsquigarrow} u$ implies $E \vdash t = u$, we have proved Theorem 6.2. $\square$

It follows that it is undecidable whether $t \overset{*}{\rightsquigarrow} u$ holds, even for ground terms t and u . In addition, it follows trivially that it is also undecidable whether $t \rightsquigarrow u$:

Theorem 6.3 *It is undecidable whether $t \overset{*}{\rightsquigarrow} u$ holds, even for ground terms t and u .*

Proof. Let $\hat{E}$, for any E , contain each equation $l = r$ in E , and its symmetric version $r = l$. Then $t \overset{*}{\rightsquigarrow}_E u$ if and only if $t \overset{*}{\rightsquigarrow}_{\hat{E}} u$. $\square$

We can prove that confluence is undecidable as a corollary to Theorem 6.3, since we can decide $\overset{*}{\rightsquigarrow}$ if we can decide confluence as follows (from [6]): Let $\hat{E}$ again contain each equation in E and its symmetric version. Trivially, $\hat{E}$ is confluent (why?). Then, to decide whether $E \vdash t = u$ holds, we just add a *new* constant a and two equations $a = t$ and $a = u$ to $\hat{E}$. Then it is not too difficult to prove that $\hat{E}$ is confluent if and only if $t \overset{*}{\rightsquigarrow} u$.

In *terminating* and *confluent* specifications it is possible to decide whether $E \vdash t = u$ by, as expected, checking whether their normal forms $t!$ and $u!$ are the same:

Theorem 6.4 *For a terminating and confluent specification E we have*

$$t \overset{*}{\rightsquigarrow} u \quad \text{if and only if} \quad t! = u!$$

for all terms t and u .

Proof. The “if” direction ($t! = u!$ implies $t \overset{*}{\rightsquigarrow} u$) is trivial: $t \overset{*}{\rightsquigarrow} t! = u! \overset{*}{\rightsquigarrow} u$.

The “only if” direction ($t \overset{*}{\rightsquigarrow} u$ implies $t! = u!$) can be proved by induction on the length of the derivation of $t \overset{*}{\rightsquigarrow} u$. If the length is 0, then $t = u$, and $t! = u!$ holds trivially. If the length of the derivation is $n + 1$, then $t \rightsquigarrow t' \overset{*}{\rightsquigarrow} u$ for some t' . The length of $t' \overset{*}{\rightsquigarrow} u$ is n , so the induction hypothesis applies and gives $t'! = u!$. Now, since $t \rightsquigarrow t'$ we have that either $t \rightsquigarrow t'$ or $t' \rightsquigarrow t$. If $t \rightsquigarrow t'$, then $t! = t'! = u!$. If $t' \rightsquigarrow t$, then we have that $t' \rightsquigarrow^* t$ and $t' \rightsquigarrow^* u!$. Because of confluence, there must be a t^* such that $t \rightsquigarrow^* t^*$ and $u! \rightsquigarrow^* t^*$. Since $u!$ cannot be reduced, we have $t^* = u!$ and $t \rightsquigarrow^* u!$, which implies that $t! = u!$ since $t \rightsquigarrow^* u!$ and $u!$ is irreducible. $\square$

We end this section by stating another main result:

Theorem 6.5 *For terminating and confluent E we have*

$$E \vdash t = u \quad \text{if and only if} \quad t! = u!$$

for all terms t and u .

Proof. Follows directly from Theorems 6.2 and 6.4. $\square$

6.1.1 * Knuth-Bendix Completion

We have seen that it is easy to decide whether t and u are logically equivalent in *terminating* and *confluent* specifications. Therefore, if our specification is not terminating and confluent we could try to turn it into an terminating and confluent specification that does not change the meaning of the original specification.

Knuth-Bendix completion [62] is a process which tries to transform a non-confluent and possibly nonterminating specification E into a “logically equivalent” confluent and terminating specification E' so that $E \vdash t = u$ holds if and only if $E' \vdash t = u$ holds.

The main idea is the following: A terminating specification is not confluent if it has a critical pair (t, u) such that t and u have different normal forms $t' \neq u'$. However, if t' is a normal form of t and u' is a normal form of u , and since (t, u) is a critical pair from a term $l\sigma$, we have

$$t' \xrightarrow{*} t \rightsquigarrow l\rho \rightsquigarrow u \xrightarrow{*} u'$$

for some term $l\rho$. That is, $t' \rightsquigarrow u'$, and hence $E \vdash t' = u'$. Therefore, we do not change the equational theory by adding a new equation $t' = u'$ or $u' = t'$ to E . For each critical pair with different normal forms the completion process adds such an equation. The process terminates successfully when all equations are $\succ$ -decreasing, for the selected termination ordering $\succ$, and there are no non-joinable critical pairs.

Example 6.4. We saw in Exercise 79 that the group axioms

$$G = \{e \circ x = x, \quad i(x) \circ x = e, \quad (x \circ y) \circ z = x \circ (y \circ z)\}$$

are not confluent. However, Knuth-Bendix completion of G gives the following equivalent terminating and confluent specification

$$\begin{aligned} \{ & e \circ x = x, & i(x) \circ x = e, & (x \circ y) \circ z = x \circ (y \circ z), \\ & i(x) \circ (x \circ y) = y, & x \circ e = x, & i(e) = e, \\ & i(i(x)) = x, & x \circ i(x) = e, & x \circ (i(x) \circ y) = y, \quad i(x \circ y) = i(y) \circ i(x) \} \end{aligned}$$

that can be used to decide whether $t = u$ holds in all groups [105]. That is, although it is in general *undecidable* whether $E \vdash t = u$, this problem becomes decidable if E can be transformed into an equivalent confluent and terminating specification E' . This example therefore shows that equality in the theory of groups is decidable. $\diamond$

Completion cannot always succeed, since it is in general undecidable whether $E \vdash t = u$. For example, completion cannot succeed for the equations in the proof of Theorem 6.1. The completion process may not terminate because new equations are generated, which lead to new critical pairs, and so on. Or there may be equations which cannot be simplified or oriented so that the system can be proved to terminate.

Exercise 84 Let E be $\{f(x) = g(x), a = b, g(c) = c\}$ and prove:

1. $E \vdash f(b) = f(a)$
2. $E \vdash f(f(a)) = g(f(a))$
3. $E \vdash g(b) = f(a)$
4. $E \vdash f(g(z)) = g(f(z))$
5. $E \vdash f(g(a)) = g(g(b))$

You do not need to re-prove something you have already proved, if you need that fact later. For instance, we have already proved in Example 6.1 that $E \vdash f(a) = g(b)$. If you need this fact, you can just use it.

Exercise 85 Let E' be $\{f(a, x) = f(b, x), c = d\}$.

1. Prove $E' \vdash f(a, c) = f(b, c)$.
2. Can you prove $E' \vdash f(a, c) = f(a, d)$? Explain.
3. Can you prove $E' \vdash f(a, b) = f(b, c)$? Explain.
4. Can you prove $E' \vdash a = b$? Explain.

Exercise 86 Prove that $\text{NAT-ADD} \vdash s(0) + s(0) = s(s(0))$.

Exercise 87 Complete the proof of Theorem 6.2.

Exercise 88 Consider the specification **BOOLEAN** in Section 2.1.2, with constructors *true* and *false*, extended with a function *implies* and having equations

<code>vars X Y : Boolean .</code>	
<code>eq true and X = X .</code>	<code>eq true or X = true .</code>
<code>eq false and X = false .</code>	<code>eq false or X = X .</code>
<code>eq not true = false .</code>	<code>eq not false = true .</code>
<code>eq true implies X = X .</code>	<code>eq false implies X = true .</code>

1. Explain why is it impossible to prove (the desired?) property

$$\text{BOOLEAN} \vdash Y \text{ implies } X = (\text{not } Y) \text{ or } X.$$

2. Prove that

$$\text{BOOLEAN} \vdash t \text{ implies } X = (\text{not } t) \text{ or } X$$

holds for each constructor ground term t of sort Boolean.

Exercise 89 Explain that if $E \vdash t = u$, then it is also the case that $E' \vdash t = u$ for any extension E' of E , that is, for any set E' of equations such that $E \subset E'$.

6.2 Inductive Theorems

As mentioned in the introduction to this chapter, one is often interested in properties of the “intended” model of a specification (defined in Chapter 7), instead of properties that hold in *all* models satisfying the equations. In particular, one only cares about the values defined by the signature. As already explained,

$$(\dagger) \quad \text{NAT-ADD} \vdash M + N = N + M$$

does not hold, for variables M and N , since this equality does not hold in all structures satisfying the equations in **NAT-ADD**. One such structure adds a constant a to **NAT-ADD**; addition is not commutative in this structure since $a + 0 \neq 0 + a$. (Another way to prove $\text{NAT-ADD} \not\vdash M + N = N + M$ is to consider their normal forms; since

NAT-ADD is terminating and confluent, Theorem 6.4 implies that $\text{NAT-ADD} \vdash M+N = N+M$ holds only if $M+N$ and $N+M$ have the same normal form, which is not the case.)

The point is that the property $(\dagger)$ was intended to hold only for the natural numbers; that is, for the *constructor ground terms* $0, s(0), s(s(0)), \dots$. Instead of $(\dagger)$ we want to prove that

$$\text{NAT-ADD} \vdash m+n = n+m$$

holds for *all constructor ground terms* m and n of sort Nat . A property of this kind, which is required to hold for all constructor ground terms, is called an *inductive theorem*.⁴ We write $E \vdash_{\text{ind}} t = u$ when $E \vdash t\sigma = u\sigma$ holds for all substitutions σ which instantiate variables with constructor ground terms.

Example 6.5. Exercise 88 showed that $\text{BOOLEAN} \vdash Y$ implies $X = (\text{not } Y) \text{ or } X$ does not hold, but that $\text{BOOLEAN} \vdash_{\text{ind}} Y$ implies $X = (\text{not } Y) \text{ or } X$ holds. $\diamond$

Equational logic allows us to reason about equalities that hold in all E -structures. Is there a similar proof system for inductive theorems? An *optimal* proof system is one that is:

Sound: Everything that can be proved in the proof system “holds.” That is, one cannot “prove” something that is wrong.

Complete: All properties (equalities in our case) that hold can be proved. This means that the proof system is powerful enough to prove everything that holds.

Algorithmically checkable: It should be algorithmically checkable whether a given sequence of formulas is a correct proof.

As explained in Chapter 7, the proof system for equational logic satisfies these three criteria for proving equalities that hold in all E -structures.

Unfortunately, there is no such optimal proof system for proving inductive theorems. This is a consequence of the (negative) solution to Hilbert’s Tenth Problem (“Is there an algorithm that always can decide whether a given Diophantine equation has a solution?”) developed by Martin Davis, Hilary Putnam, Julia Robinson, and completed by then-22-years-old Yuri Matiyasevich in 1970 [25].⁵

⁴ We assume in this section that our specifications are *sufficiently complete* (see Section 2.3.4); that is, each ground term reduces to some constructor ground term.

⁵ This undecidability result implies that for any sound and finitary proof system PS for the natural numbers with addition and multiplication, there are polynomials p_1 and p_2 over variables $x_1, \dots, x_n$ (and nonnegative coefficients) such that $(\forall x_1, \dots, x_n) p_1(x_1, \dots, x_n) \neq p_2(x_1, \dots, x_n)$ holds for the natural numbers but is not provable in PS . However, this formula is an *inequality*, whereas our inductive theorems are *equalities*. We must introduce another function, such as either equality $= :$ $\text{Nat} \rightarrow \text{Nat}$, defined by $0 = s(x) = 0$, $s(x) = 0 = 0$, $0 = 0 = s(0)$, $s(x) = s(y) = x = y$ (this is our usual function $=$, but to keep within a one-sorted framework it returns 0 instead of *false* and $s(0)$ instead of *true*) or the “monus” function in Exercise 9. The unprovable formula that holds for the natural numbers then becomes $(\forall x_1, \dots, x_n) p_1(x_1, \dots, x_n) = p_2(x_1, \dots, x_n) = 0$ and $s(0) \text{ monus } ((p_1(x_1, \dots, x_n) \text{ monus } p_2(x_1, \dots, x_n)) + (p_2(x_1, \dots, x_n) \text{ monus } p_1(x_1, \dots, x_n))) = 0$, respectively. Since the natural numbers with addition and multiplication are the intended structure for a specification like NAT-MULT, there is no optimal proof system for NAT-MULT extended with *monus* or $=$. Therefore, there is no optimal proof system for inductive theorems in general.

6.2.1 Proving Inductive Theorems for Nat

Consider the simple inductive property

$$\text{NAT-ADD} \vdash_{\text{ind}} x + 0 = x.$$

By the definition of an inductive theorem this means that $\text{NAT-ADD} \vdash t + 0 = t$ holds for all constructor ground terms t of sort Nat . How can we prove this? By induction on the *depth* of t , denoted $\text{depth}(t)$, where the depth of a constant is 1, and $\text{depth}(f(t_1, \dots, t_n)) = 1 + \max(\{\text{depth}(t_1), \dots, \text{depth}(t_n)\})$.

Using induction on the depth of t , we can prove $\text{NAT-ADD} \vdash t + 0 = t$ for all constructor ground terms t as follows (more precisely, we prove by induction on n that “for all n , for all constructor ground terms t with depth n , $\text{NAT-ADD} \vdash t + 0 = t$ ”):

Base case. t has depth 1; i.e., is a constant. Since the only constructor constant in NAT-ADD is 0, we must prove that $\text{NAT-ADD} \vdash 0 + 0 = 0$, which follows directly from *Substitutivity* using the equation $0 + M = M$.

Induction step. Assume that t has depth $n + 1$. Since the only non-constant constructor in NAT-ADD is s , t must have the form $s(t')$. The induction hypothesis is that $\text{NAT-ADD} \vdash u + 0 = u$ for all constructor ground terms u with $\text{depth}(u) \leq n$. Since t' has depth n , the induction hypothesis applies to t' , and we can therefore assume $\text{NAT-ADD} \vdash t' + 0 = t'$, and must prove $\text{NAT-ADD} \vdash s(t') + 0 = s(t')$. Proving the latter is fairly trivial:

$$\frac{\frac{\text{NAT-ADD} \vdash s(t') + 0 = s(t' + 0)}{\text{NAT-ADD} \vdash s(t') + 0 = s(t')} \text{Subst.} \quad \frac{\frac{\text{NAT-ADD} \vdash t' + 0 = t'}{\text{NAT-ADD} \vdash s(t' + 0) = s(t')} \text{Congr.}}{\text{NAT-ADD} \vdash s(t') + 0 = s(t')} \text{Transitivity}$$

An important remark is that, since $E \vdash u = v$ is the same as $u \xrightarrow{*}_E v$ by Theorem 6.2, we can reason in terms of (two-way) reductions instead of equational deductions, which is usually more convenient:

$$s(t') + 0 \rightsquigarrow s(t' + 0) \xleftarrow{*}_{(\text{Ind.hyp.})} s(t').$$

This proves that $\text{NAT-ADD} \vdash t + 0 = t$ holds for all constructor ground terms t and hence that $\text{NAT-ADD} \vdash_{\text{ind}} x + 0 = x$.

We can also formalize the “generic constant” t' and the induction hypothesis in Maude, and use Maude to prove the two steps:

```
fmod NAT-ADD-IND-PROOF is including NAT-ADD .
  op t' : -> Nat .                --- generic constant for induction
  eq t' + 0 = t' .                --- induction hypothesis
endfm
```

```
red 0 + 0 == 0 .                  --- Base case: T is 0.
red s(t') + 0 == s(t') .         --- Induction step: T is s(t').
```

Maude indeed reduces both of these expressions to `true`.

A general induction scheme to prove that some property $P(t)$ holds for all constructor ground terms t of sort Nat is therefore:

Base case: Prove that $P(0)$ holds.

Induction step: Prove that $P(s(t))$ holds, when you can assume that the induction hypothesis $P(t)$ holds. Furthermore, if needed you can assume the stronger induction hypothesis that $P(u)$ holds for all constructor ground terms u with $\text{depth}(u) < \text{depth}(s(t))$.

Example 6.6. Associativity of addition does not follow from the equations in the specification NAT-ADD (Exercise 90). However, we can prove that associativity of addition is an *inductive theorem* in NAT-ADD , that is, $\text{NAT-ADD} \vdash_{\text{ind}} (x + y) + z = x + (y + z)$. In particular, we can prove $\text{NAT-ADD} \vdash (t_1 + t_2) + t_3 = t_1 + (t_2 + t_3)$ for all constructor ground terms t_1, t_2, t_3 of sort Nat by induction on the depth of t_1 :

Base case. t_1 is 0, and we need to prove $\text{NAT-ADD} \vdash (0 + t_2) + t_3 = 0 + (t_2 + t_3)$ for all constructor ground terms t_2 and t_3 . Let t_2 and t_3 be any two constructor ground terms of sort Nat . Then we have $(0 + t_2) + t_3 \rightsquigarrow t_2 + t_3 \rightsquigarrow 0 + (t_2 + t_3)$, using the equation $0 + M = M$ on both sides.

Induction step. Let t_1 be $s(t)$. The induction hypothesis that we can assume is $\text{NAT-ADD} \vdash (t + t_2) + t_3 = t + (t_2 + t_3)$ for all constructor ground terms t_2, t_3 of sort Nat , and we have to prove $\text{NAT-ADD} \vdash (s(t) + t_2) + t_3 = s(t) + (t_2 + t_3)$, which is left to the reader as an easy exercise.

The proof steps can be represented and performed by Maude as follows:

```
fmod NAT-ASSOC-IND-PROOF is including NAT-ADD .
  ops t t2 t3 : -> Nat .
  eq (t + t2) + t3 = t + (t2 + t3) .      --- induction hypothesis
endfm

red (0 + t2) + t3 == 0 + (t2 + t3) .
red (s(t) + t2) + t3 == s(t) + (t2 + t3) .
```

The execution of both commands returns `true`, proving the desired property. $\diamond$

It is not always as easy to prove inductive theorems as in the two examples above. If there are multiple variables, one may need to choose which one to do the induction on. For example, it is much harder to prove that associativity of addition is an inductive theorem in NAT-ADD if you instead choose to try induction on t_2 instead of t_1 (try!). It may even be necessary to do simultaneous induction on the size of *pairs* of constructor ground terms (t_1, t_2) in other cases, and so on.

An important issue is that additional *lemmas* may be needed in such proofs. Typically, if you get “stuck” during a proof, you may need to prove some lemma, that is, a helpful “smaller” inductive theorem, that you can use in the main proof. Indeed, in the following example, we need the following lemmas:

Lemma 1: $\text{NAT-ADD} \vdash t + 0 = t$

Lemma 2: $\text{NAT-ADD} \vdash s(t_1 + t_2) = t_1 + s(t_2)$

for all constructor ground terms t, t_1 , and t_2 of sort Nat .

Example 6.7. We prove that commutativity of addition is an inductive theorem; that is, $\text{NAT-ADD} \vdash t_1 + t_2 = t_2 + t_1$ for all constructor ground terms t_1 and t_2 , by induction on t_1 .

Base case. t_1 is 0, and we need to prove $\text{NAT-ADD} \vdash 0 + t_2 = t_2 + 0$ for all t_2 . Using the equation $0 + M = M$, the left-hand side reduces to t_2 ; and using Lemma 1 above, the right-hand side also reduces to t_2 : $0 + t_2 \rightsquigarrow t_2 \leftarrow_{\text{Lemma 1}} t_2 + 0$.

Induction step. t_1 is $s(t)$. We need to prove $\text{NAT-ADD} \vdash s(t) + t_2 = t_2 + s(t)$, when $\text{NAT-ADD} \vdash t + t_2 = t_2 + t$ follows from the induction hypothesis, for all constructor ground terms t_2 . This can be proved as follows:

$$s(t) + t_2 \rightsquigarrow s(t + t_2) \overset{*}{\rightsquigarrow}_{\text{ind. hyp.}} s(t_2 + t) \overset{*}{\leftarrow}_{\text{Lemma 2}} t_2 + s(t). \quad \diamond$$

6.2.2 Inductive Theorems for Other Data Types

The induction scheme used to prove inductive theorems of NAT-ADD can be generalized to *any* data type. Some property $P(t)$ holds for all constructor ground terms t of sort s if one can prove:

Base case: The depth of t is 1. That is, t is a constant that is a constructor of sort s (or of a subsort of s , since a constructor for a subsort s' of s also constructs terms of sort s). Therefore, we must prove $P(c)$ for all such constructor constants c .

Induction step: The depth of t is $n + 1$. For each non-constant constructor f of sort s , or of a subsort of s , one must prove

$$P(f(t_1, \dots, t_n))$$

for all constructor ground terms $t_1, \dots, t_n$. Since the depth of each t_i is smaller than $n + 1$, we can assume the *induction hypothesis* $P(t_i)$ for each t_i of sort s . More generally, we can assume $P(t')$ for any constructor ground term t' of sort s with $\text{depth}(t') \leq n$.

Note that equational logic extended with a deduction rule corresponding to the above induction scheme in general is not a *complete* proof system. It is beyond the scope of this book to present a proof system for inductive theorems, especially since there are no optimal such proof systems. Instead, a number of examples illustrate reasoning about inductive properties of different data types.

Example 6.8. A property $Q(t)$ holds for all constructor ground terms t of sort s in

```
fmod M is
  sorts s s' .
  ops a b : -> s [ctor] .           ops c d : -> s' [ctor] .
  ops f g : s s' -> s [ctor] .       op h : s' s s' -> s' [ctor] .
  op k : s s' s -> s [ctor] .
  ops l p : s -> s .                 ops d : s -> s' .
  ...   *** variables and equations
endfm
```


if one can prove:

Base case. $Q(a)$ and $Q(b)$.

Induction step.

- $Q(f(t, t'))$ and $Q(g(t, t'))$ for arbitrary constructor ground terms t and t' , in both cases assuming the induction hypothesis $Q(t)$.
- $Q(k(t_1, t_2, t_3))$ for arbitrary constructor ground terms t_1, t_2 , and t_3 . In the proof of this property one may assume *both* $Q(t_1)$ and $Q(t_3)$. $\diamond$

Example 6.9. We prove that the number of elements in a binary tree is the same as the number of elements in the reversed tree. Recall our definition of binary trees:

```
fmod BINTREE-NAT1 is ...
  sort BinTree .
  op empty : -> BinTree [ctor] .
  op bintree : BinTree Nat BinTree -> BinTree [ctor] .
  ops size weight : BinTree -> Nat .
  op reverse : BinTree -> BinTree .

  vars BT BT' : BinTree .      vars N N' : Nat .
  eq size(empty) = 0 .
  eq size(bintree(BT, N, BT')) = s(0) + (size(BT) + size(BT')) .
  eq reverse(empty) = empty .
  eq reverse(bintree(BT, N, BT'))
    = bintree(reverse(BT'), N, reverse(BT)) .
  ...
endfm
```

We prove

$$\text{BINTREE-NAT1} \vdash \text{size}(\text{reverse}(t)) = \text{size}(t)$$

for all constructor ground terms t of sort `BinTree` by proving:

Base case. $\text{BINTREE-NAT1} \vdash \text{size}(\text{reverse}(\text{empty})) = \text{size}(\text{empty})$.

Induction step. Here we must prove

$$\text{BINTREE-NAT1} \vdash \text{size}(\text{reverse}(\text{bintree}(t_1, n, t_2))) = \text{size}(\text{bintree}(t_1, n, t_2)),$$

assuming both

$$\text{BINTREE-NAT1} \vdash \text{size}(\text{reverse}(t_1)) = \text{size}(t_1)$$

and

$$\text{BINTREE-NAT1} \vdash \text{size}(\text{reverse}(t_2)) = \text{size}(t_2).$$

Again, we can use Maude to check both properties:

```

fmod PROVE-BINTREE is including BINTREE-NAT1 .
  ops t1 t2 : -> BinTree .
  op n : -> Nat .
  eq size(reverse(t1)) = size(t1) .      --- Ind. Hyp.
  eq size(reverse(t2)) = size(t2) .      --- Ind. Hyp.
endfm

red size(reverse(empty)) == size(empty) .
red size(reverse(bintree(t1, n, t2))) == size(bintree(t1, n, t2)) .

```

The Maude execution of the first returns `true`; however, the second command gives the result $(\text{size}(t2) + \text{size}(t1)) == (\text{size}(t1) + \text{size}(t2))$. Assuming that our specifications are well-defined (i.e., sufficiently complete), both $\text{size}(t1)$ and $\text{size}(t2)$ are natural numbers, and by the previously proven commutativity property of addition on natural numbers, both sides are the same. $\diamond$

Exercise 90 1. Prove that $\text{NAT-ADD} \not\models (x + y) + z = x + (y + z)$.
 2. Fill in the details of the proof in Example 6.6.

Exercise 91 Prove that $\text{NAT-ADD} \vdash_{\text{ind}} s(x + y) = x + s(y)$.

Exercise 92 Show that commutativity of multiplication is an inductive theorem in NAT-MULT . Remember to reuse the properties of addition that we have proved.

Exercise 93 Let NAT-DOUBLE be NAT-ADD extended with the following function:

```

op double : Nat -> Nat .
var N : Nat .
eq double(0) = 0 .   eq double(s(N)) = s(s(double(N))) .

```

1. Prove $\text{NAT-DOUBLE} \not\models \text{double}(N) = N + N$.
2. Prove $\text{NAT-DOUBLE} \vdash_{\text{ind}} \text{double}(N) = N + N$.

Exercise 94 Consider the proof scheme in Example 6.8.

1. Explain why it is not necessary (or possible) to prove $Q(c)$ and $Q(d)$.
2. Explain why $Q(t')$ cannot be assumed as an induction hypothesis when proving $Q(f(t, t'))$ and $Q(g(t, t'))$.
3. Describe the induction scheme for proving $P(u)$ for all ground constructor terms u of sort s' in the module M .
4. Now assume that s' is a subsort of sort s in the module M . Which induction scheme can be used to prove $Q(t)$ for all constructor ground terms of sort s ?

Exercise 95 Consider the specification LIST-NAT1 of lists in Section 2.4.3.

1. Explain how one can prove a property $P(l)$ for all constructor ground terms l of sort List .

2. To increase our confidence in the correctness of this specification, prove that

$$\text{LIST-NAT1} \vdash \text{length}(\text{concat}(l, l')) = \text{length}(l) + \text{length}(l')$$

for all constructor ground terms l and l' of sort `List`. You can assume that all functions are well-defined (i.e., the specification is sufficiently complete): each ground term reduces to a constructor ground term. You will also need to use lemmas that have already been proved, such as $\text{NAT-ADD} \vdash_{\text{ind}} x + 0 = x$ and $\text{NAT-ADD} \vdash_{\text{ind}} x + y = y + x$. Hint: prove the property by induction on l' .

Exercise 96 Define the function `reverse` on lists in `LIST-NAT1` (or recall your solution from Exercise 10) and prove that you get the original list back if you reverse a list twice: $\text{LIST-NAT1} \vdash_{\text{ind}} \text{reverse}(\text{reverse}(L)) = L$.

Exercise 97 Prove $\text{BINTREE-NAT1} \vdash_{\text{ind}} \text{reverse}(\text{reverse}(T)) = T$.

Exercise 98 Consider the specification `LIST-INT` in Section 2.8.2.

1. Explain what steps are needed to prove a property $P(l)$ for all constructor ground terms l of sort `List`.
2. Define the function `reverse`. (Hint: it might be useful to define one equation for each constructor.)
3. Let furthermore $L1$ and $L2$ be variables of sort `List` and prove the following inductive theorems:

- a. (tricky?) $\text{LIST-INT} \vdash_{\text{ind}} \text{length}(L1 \ L2) = \text{length}(L1) + \text{length}(L2)$
- b. $\text{LIST-INT} \vdash_{\text{ind}} \text{reverse}(\text{reverse}(L1)) = L1$

Exercise 99 Consider your specification S of the Roman numerals in Exercise 24. Let R be a variable of sort `Roman` and let $r \ I$ denote the Roman numeral r with an ‘I’ added “to the right.” You can assume that standard functions such as `if_then_else-fi`, `<`, and `+` satisfy expected properties.

1. Does $S \vdash \text{decimal}(R \ I) = \text{decimal}(R) + 1$ hold?
2. Does $S \vdash_{\text{ind}} \text{decimal}(R \ I) = \text{decimal}(R) + 1$ hold?

The following might be much trickier:

3. Does $S \vdash_{\text{ind}} \text{roman}(\text{decimal}(R)) = R$ hold?

The introduction to this book says that the point of formal modeling is to define a *mathematical model* of a computer system. However, we have not yet seen the mathematical model(s) defined by a Maude functional module. What are they? (The mathematical object defined by a program is called its *denotational semantics*.)

As mentioned in Chapter 6, we are sometimes interested in *all models* of a specification (Σ, E) (such as that of groups), but most often we are only interested in the *intended model* of (Σ, E) . This chapter therefore defines both all mathematical models and the intended model specified by an equational specification (Σ, E) .

Having such models enables us to reason about properties that hold in all (Σ, E) -models and that hold in the intended (Σ, E) -model. For example, does $x \circ y = y \circ x$ hold in *all groups*; that is, in all models of our specification of groups? This chapter proves that equational logic is *sound* and *complete*: $t_1 = t_2$ holds in all models of a specification (Σ, E) if and only if $E \vdash t_1 = t_2$ holds. Furthermore, checking whether $E \vdash t_1 = t_2$ holds is decidable when E is—or can be turned into—a confluent and terminating specification. This is the case for the group axioms (see Example 6.4), so that we can easily check whether an equality holds in *all groups*. The equalities that hold in the intended model of a specification are the inductive theorems introduced in Section 6.2, for which in general there is no sound and complete proof system.

What kind of mathematical models are we looking for? Meseguer and Goguen argue in [85] that a software module/package involves various sorts of data that form *sets*, and defines a number of operations on those data that correspond to *functions* on the corresponding sets. In other words, a software module has the structure of an *algebra*. Specifying a software module therefore means specifying an algebra, which is exactly what a Maude specification (Σ, E) does.

Section 7.1 introduces Σ -algebras for a signature Σ , as well as key notions like Σ -homomorphisms. Section 7.2 defines the class $Alg(\Sigma, E)$ as all Σ -algebras that satisfy the equations E . All such algebras are models of a specification (Σ, E) . Section 7.3 proves the soundness and completeness of equational logic.

Section 7.4 introduces the *intended model* of (Σ, E) as the *initial algebra(s)* in the class $\text{Alg}(\Sigma, E)$ of all (Σ, E) -models. This intended model is not unique, but is a set of algebras that have the exact same properties and only differ in their representation of the elements of the algebra. For example, different textbooks use different representations of the Boolean values: some use *true* and *false*; others use $\top$ and $\perp$, **t** and **f**, or 1 and 0. They all talk about the same structure: the Boolean values and operations, and all of these representations are equally good. This is why we say that a specification (Σ, E) specifies an *abstract* data type: the intended models of `BOOLEAN` do not define a *concrete* representation of the Boolean values.

Section 7.4 also proves that a specification always has intended models, and that these models have the same structure (are *isomorphic*). In particular, any specification (Σ, E) has an initial model $\mathcal{T}_{\Sigma, E}$ whose elements can be concretely represented as equivalence classes of ground terms.

Is such an intended model really the mathematical object we wanted to specify? For example, the constructor ground terms should represent all the “values” we want. Therefore, the elements in the intended model should not have “junk” elements that are not represented by some ground term. Likewise, only elements that are represented by E -equivalent ground terms should have the same value in the intended model. Section 7.4 proves that our intended models satisfy these properties.

To keep the exposition simple, most of this chapter assumes that specifications are unsorted and have no conditional equations or functional attributes. The extension to the many-sorted case is straightforward as long as all the sorts have ground terms. Section 7.5 briefly discusses equational logic with many-sorted specifications that may have empty sorts.

This chapter, which roughly follows [85, Sections 1-4] and [60], barely scratches the surface of algebras and algebraic specifications. For a more thorough introduction, the reader is referred to classics on the topics such as [35, 85, 108].

7.1 Many-Sorted Σ -Algebras

An algebra is a mathematical structure which defines an *interpretation* (or *model*) of a signature Σ . This section defines Σ -algebras for many-sorted signatures Σ .

Definition 7.1 For a many-sorted signature $\Sigma = (S, \Sigma)$, a Σ -algebra A consists of:

- a set A_s for each sort $s \in S$; and
- a function $f_A : A_{s_1} \times \cdots \times A_{s_n} \rightarrow A_s$ for each function symbol $f \in \Sigma_{s_1, \dots, s_n, s}$.

The set A_s is the interpretation of the sort s , and the function $f_A : A_{s_1} \times \cdots \times A_{s_n} \rightarrow A_s$ is the interpretation of the function symbol $f : s_1, \dots, s_n \rightarrow s$ in Σ . The family of sets A_s for $s \in S$ is called an S -indexed set. If A and B are two S -indexed sets, then a mapping $f : A \rightarrow B$ is an S -indexed family of functions $\{f_s : A_s \rightarrow B_s \mid s \in S\}$.

Example 7.1. An algebra `NB` interpreting (the signature of) the module `NAT<` in Section 2.1.3 has:

- Domains $\text{NB}_{\text{Nat}} = \mathbb{N}$ (the natural numbers) and $\text{NB}_{\text{Boolean}} = \{\mathbf{t}, \mathbf{f}\}$.

- The interpretation $0_{\mathbb{NB}}$ of 0 in $\mathbb{NB}$ is 0 ; the function $s_{\mathbb{NB}}$ interpreting s in $\mathbb{NB}$ is defined $s_{\mathbb{NB}}(n) = n + 1$; and the interpretation $+_{\mathbb{NB}}$ of $+$ is the addition function on natural numbers. The interpretations $\text{true}_{\mathbb{NB}}$ and $\text{false}_{\mathbb{NB}}$ of true and false in $\mathbb{NB}$ are **t** and **f**, respectively. The other Boolean operations in $\text{NAT}<$ are interpreted in the expected way. Finally, the function $<_{\mathbb{NB}}$ interpreting $<$ in $\mathbb{NB}$ is the function $<$ that maps $m < n$ to **t** if m is less than n , and to **f** otherwise. $\diamond$

Example 7.2. A less standard model $\mathbb{S}$ of the signature of $\text{NAT}<$ has:

- The domain $\mathbb{S}_{\text{Nat}}$ is the set $\{a, b, c\}^*$ of all strings on letters a , b , and c , and $\mathbb{S}_{\text{Boolean}}$ is the set $\{\mathbf{t}, \mathbf{f}\}$.
- The interpretation $0_{\mathbb{S}}$ of 0 in $\mathbb{S}$ is the empty string ϵ , the interpretation $s_{\mathbb{S}}$ of s is the function that appends an “a” to a string, and the interpretation $+_{\mathbb{S}}$ of $+$ is string concatenation. The Booleans are interpreted in the standard way. Finally, the interpretation $<_{\mathbb{S}}$ is the (proper) substring function. $\diamond$

Example 7.3. Some algebras interpreting the signature of NAT-ADD are:

1. The algebra $\mathbb{N}$, whose domain $\mathbb{N}_{\text{Nat}}$ is the natural numbers $\{0, 1, 2, 3, 4, \dots\}$, and where the interpretations $0_{\mathbb{N}}$, $s_{\mathbb{N}}$, and $+_{\mathbb{N}}$ of the function symbols 0 , s , and $+$ in NAT-ADD are defined as expected:
 - $0_{\mathbb{N}}$ is the number 0 ;
 - $s_{\mathbb{N}}$ is the “successor function” s on natural numbers: $s(n) = n + 1$; and
 - $+_{\mathbb{N}}$ is the standard addition function $+$ on natural numbers.
2. The algebra $\mathbb{N}_{\perp}$, which adds an element $\perp$ (for “undefined”) to $\mathbb{N}$. The above functions are extended to $\perp$ as follows: $s(\perp) = \perp$ and $\perp + x = x + \perp = \perp$.
3. The algebra $\mathbb{N}_x$ whose domain $\mathbb{N}_{x\text{Nat}}$ is $\mathbb{N}$, and where 0 , s , and $+$ are interpreted by, respectively, 84 , “ $+4$ ”, and the function returning the first element of a pair.
4. $\mathcal{E}$, whose domain $\mathcal{E}_{\text{Nat}}$ is the set $\{0, 2, 4, 6, \dots\}$ of even numbers, and where the interpretations of the function symbols 0 , s , and $+$ in the algebra $\mathcal{E}$ are as expected: $0_{\mathcal{E}} = 0$, $s_{\mathcal{E}}(n) = n + 2$, and $n +_{\mathcal{E}} m = n + m$.
5. The integers $\mathbb{Z}$, whose domain $\mathbb{Z}_{\text{Nat}}$ is the integers $\{\dots, -2, -1, 0, 1, 2, \dots\}$ and where 0 , s , and $+$ are interpreted in the standard way.
6. The algebra *squares* with
 - $\text{squares}_{\text{Nat}} = \{0, 1, 4, 9, 16, \dots\}$
 - $0_{\text{squares}} = 0$
 - $s_{\text{squares}}(n) = n + 2\sqrt{n} + 1$
 - $m +_{\text{squares}} n = (\sqrt{m} + \sqrt{n})^2$
7. The algebra *bits* of bit sequences without leading zeros. The domain bits_{Nat} is $\{0\} \cup \{\text{sequences of bits not starting with } 0\}$, and 0 , s , and $+$ are interpreted in *bits* by, respectively, 0 , the function which gives the string resulting from adding one to a given bit string, and the standard addition function on bit sequences.
8. The algebra *bits0*, which is defined as *bits* except that its domain $\text{bits0}_{\text{Nat}}$ is all bit strings, including those with leading zeros.

9. The algebra $*$ with a single element $*$. That is, $*_{\text{Nat}} = \{*\}$, and the functions are interpreted in the only possible way (which way?).
10. The algebra $+2$ with domain $+2_{\text{Nat}} = \{0, 1, 2, 3, 4, 5, \dots\}$, but where s is interpreted as “plus 2”; that is, 0 , s , and $+$ are interpreted by 0 , $\lambda m. m + 2$, and $\lambda m, n. m + n$, respectively.
11. The algebra $\mathbb{Q}_{\geq 0}$ of non-negative rational numbers, with the obvious interpretations of 0 (the number 0), s (“plus one”), and $+$ (standard addition on rationals).
12. For any number $k > 1$, the algebra $\mathbb{N}_k$ with domain $\mathbb{N}_{k\text{Nat}} = \{0, 1, \dots, k-1\}$, with $0_{\mathbb{N}_k} = 0$, $s_{\mathbb{N}_k}$ the function $\lambda n. n + 1 \bmod k$, and $+_{\mathbb{N}_k}$ the function $\lambda m, n. (m + n) \bmod k$, where $n \bmod k$ is the remainder when n is divided by k .
13. The algebra AB with $AB_{\text{Nat}} = \{*, a, b\}$, $0_{AB} = *$, s_{AB} the identity function, and $+_{AB}$ the function $+$ defined by $* + X = X$, $a + X = a$, and $b + X = b$ for all X . $\diamond$

Example 7.4. The signature of *groups* (see Exercise 79) has one sort s , a constant e (the identity element), a unary function symbol i (inverse), and a binary function symbol $\circ$. Three algebras for this signature are:

1. $\mathbb{Z}$, with domain $\mathbb{Z}_s$ the integers $\{\dots, -2, -1, 0, 1, 2, \dots\}$, and with functions 0 , (unary) $-$, and $+$ interpreting e , i , and $\circ$, respectively.
2. $\mathbb{R}_{>0}$, with domain all positive rational numbers, and interpretations 1 , $\lambda x. 1/x$, $*$ (multiplication) of e , i , and $\circ$.
3. The algebra $\text{funcs}(\{a, b, c\})$, whose domain is the set of all *bijective* functions from $\{a, b, c\}$ to $\{a, b, c\}$. The group operations are interpreted as expected:
 - e is interpreted by the identity function $\lambda x. x$ on $\{a, b, c\}$;
 - i is interpreted by the *inverse* function $\lambda f. f^{-1}$; and
 - $\circ$ is interpreted as standard function composition: $(f \circ g)(x) = f(g(x))$. $\diamond$

In this introductory book I refer to, e.g., [50] for the treatment of *order-sorted* algebras, and just mention that in such an algebra A , the domain A_s must be a subset of the domain $A_{s'}$ whenever s is a subsort of s' in the signature. For example, the (sub)sorts $\text{NzNat} < \text{Nat} < \text{Int}$ could be interpreted by the corresponding three sets $\{1, 2, 3, \dots\} \subseteq \{0, 1, 2, 3, \dots\} \subseteq \{\dots, -2, -1, 0, 1, 2, \dots\}$, which satisfy the subset requirement.

7.1.1 Homomorphisms and Isomorphisms

A homomorphism between two Σ -algebras A and B is an S -indexed family of functions from (the domains of) A to B that “preserve” the operations in Σ :

Definition 7.2 A Σ -homomorphism ϕ between two many-sorted (S, Σ) -algebras A and B is an S -indexed set of functions $\{\phi_s : A_s \rightarrow B_s \mid s \in S\}$ such that for each function symbol $f : s_1, \dots, s_n \rightarrow s$ (for $n \geq 0$) in Σ , the following holds:

$$\phi_s(f_A(a_1, \dots, a_n)) = f_B(\phi_{s_1}(a_1), \dots, \phi_{s_n}(a_n)).$$

A Σ -homomorphism is often called just a homomorphism when Σ can be understood from the context.

If $SP = (\Sigma, E)$, then $\text{sign}(SP)$ denotes Σ and $\text{eqs}(SP)$ denotes E .

Example 7.5. A $\text{sign}(\text{NAT}<)$ -homomorphism $\phi : \mathbb{N}\mathbb{B} \rightarrow \mathbb{S}$ between the $\text{sign}(\text{NAT}<)$ -algebras in Examples 7.1 and 7.2 consists of:

- A function $\phi_{\text{Nat}} : \mathbb{N} \rightarrow \text{“strings”}$ that maps the number n to the string $\underbrace{a a \cdots a}_n$.
- A function $\phi_{\text{Boolean}} : \{\mathbf{t}, \mathbf{f}\} \rightarrow \{\mathbf{t}, \mathbf{f}\}$ which is the identity function.

To prove that ϕ is a homomorphism, we must prove the homomorphism condition for each operator $0, s, +, \text{true}, \text{false}, \text{and}, \dots$, and $<$ in **BOOLEAN**:

- For 0 , we must prove $\phi_{\text{Nat}}(0_{\text{NB}}) = 0_{\mathbb{S}}$. This holds since $\phi_{\text{Nat}}(0_{\text{NB}})$ is the string with zero a 's, which is the empty string (the interpretation $0_{\mathbb{S}}$ of 0 in $\mathbb{S}$).
- For s , we must prove $\phi_{\text{Nat}}(s_{\text{NB}}(n)) = s_{\mathbb{S}}(\phi_{\text{Nat}}(n))$, which holds, since both sides of the equality evaluate to the string $\underbrace{a a \cdots a}_{n+1}$.
- For $+$, we must prove $\phi_{\text{Nat}}(m +_{\text{NB}} n) = \phi_{\text{Nat}}(m) +_{\mathbb{S}} \phi_{\text{Nat}}(n)$, which holds, since both sides evaluate to $\underbrace{a a \cdots a}_{m+n}$.
- There is nothing interesting to prove for the Boolean operators, since both algebras interpret them in the same way, and ϕ_{Boolean} is the identity function.
- For $<$, we must prove $\phi_{\text{Boolean}}(m <_{\text{NB}} n) = \phi_{\text{Nat}}(m) <_{\mathbb{S}} \phi_{\text{Nat}}(n)$, which holds, since $\underbrace{a a \cdots a}_m$ is a proper substring of $\underbrace{a a \cdots a}_n$ if and only if $m < n$. $\diamond$

To avoid cluttering the exposition with details, the rest of this chapter considers *unsorted* specifications. The extension to the many-sorted case is straightforward when all sorts are non-empty. In the unsorted case, a homomorphism is just a single function ϕ from the domain A_s of the algebra A to the domain B_s of the algebra B . The homomorphism condition can then be written $\phi(f_A(a_1, \dots, a_n)) = f_B(\phi(a_1), \dots, \phi(a_n))$. I also write A for the domain A_s of an algebra A , and hope that using the same name for both an algebra and its domain will not lead to confusion.

Example 7.6. (From [60].) The function $\phi : \mathbb{Z} \rightarrow \mathbb{R}_{>0}$ defined by $\phi(x) = 2^x$ is a homomorphism between the algebras $\mathbb{Z}$ and $\mathbb{R}_{>0}$ in Example 7.4, since:

- $\phi(e_{\mathbb{Z}}) = 2^0 = 1 = e_{\mathbb{R}_{>0}}$,
- $\phi(i_{\mathbb{Z}}(x)) = \phi(-x) = 2^{-x} = 1/2^x = i_{\mathbb{R}_{>0}}(2^x) = i_{\mathbb{R}_{>0}}(\phi(x))$, and
- $\phi(m \circ_{\mathbb{Z}} n) = \phi(m + n) = 2^{m+n} = 2^m * 2^n = \phi(m) \circ_{\mathbb{R}_{>0}} \phi(n)$. $\diamond$

Example 7.7. The function $\text{double} : \mathbb{N} \rightarrow \mathcal{E}$ defined by $\text{double}(n) = 2n$ is a homomorphism from between the algebras $\mathbb{N}$ and $\mathcal{E}$ in Example 7.3. The identity function is a homomorphism from $\mathbb{N}$ to $\mathbb{N}_{\perp}$, and the function $\lambda n . n \bmod k$ is a homomorphism from $\mathbb{N}$ to $\mathbb{N}_k$. $\diamond$

There is sometimes no Σ -homomorphism from a Σ -algebra A to a Σ -algebra B :

Example 7.8. There is no $\text{sign}(\text{NAT-ADD})$ -homomorphism from the algebra $*$ to the algebra $\mathbb{N}$ in Example 7.3. Proof: Assume that a function $\phi : \{*\} \rightarrow \mathbb{N}$ is a homomorphism from $*$ to $\mathbb{N}$, which means that ϕ satisfies $\phi(0_*) = 0_{\mathbb{N}} = 0$ and $\phi(s_*(*)) = s_{\mathbb{N}}(\phi(*))$. Consider the expression $\phi(s_*(0_*))$. By the above equations, $\phi(s_*(0_*)) = s_{\mathbb{N}}(0_{\mathbb{N}}) = 1$. On the other hand, $s_*(*) = *$, so that $\phi(s_*(0_*)) = \phi(*) = \phi(0_*) = 0$, which means that $\phi(s_*(0_*))$ equals both 1 and 0, which is impossible. $\diamond$

Sometimes there can be more than one homomorphism from A to B :

Example 7.9. Consider a signature Σ with a single constant a and no other function symbol. Let A and B be two Σ -algebras A and B with domains $A = \{1, 2\}$ and $B = \{1, 3\}$ and with $a_A = 1$ and $a_B = 1$. The homomorphism condition requires that $\phi(a_A) = a_B$ for any homomorphism from A to B . The functions $\phi_1 = \{1 \mapsto 1, 2 \mapsto 3\}$ and $\phi_2 = \{1 \mapsto 1, 2 \mapsto 1\}$ are both homomorphisms from A to B . $\diamond$

Definition 7.3 A Σ -homomorphism $\phi : A \rightarrow B$ is a Σ -isomorphism if ϕ is surjective and injective.

If there exists a Σ -isomorphism ϕ between two Σ -algebras A and B then those two algebras are *isomorphic* algebras. Furthermore, the inverse ϕ^{-1} of ϕ is an isomorphism from B to A .

Example 7.10. The function $\phi : \mathbb{N} \rightarrow \mathcal{E}$ defined by $\phi(n) = 2n$ is an isomorphism between the algebra $\mathbb{N}$ of natural numbers and the algebra $\mathcal{E}$ of even numbers. That ϕ is a $\text{sign}(\text{NAT-ADD})$ -homomorphism is immediate:

- $\phi(0_{\mathbb{N}}) = 0 = 0_{\mathcal{E}}$
- $\phi(s_{\mathbb{N}}(n)) = \phi(n+1) = 2n+2 = s_{\mathcal{E}}(2n) = s_{\mathcal{E}}(\phi(n))$
- $\phi(m +_{\mathbb{N}} n) = 2(m+n) = 2m+2n = \phi(m) +_{\mathcal{E}} \phi(n)$

It is also easy to see that ϕ is surjective and injective:

- ϕ is surjective: for *each* even number $n \in \mathcal{E}$ there is a natural number $m \in \mathbb{N}$ (namely, $n/2$) such that $\phi(m) = n$.
- ϕ is injective: if $m \neq n$ then $2m \neq 2n$ which means $\phi(m) \neq \phi(n)$. $\diamond$

Example 7.11. The algebras $\mathbb{N}$ and *bits* are isomorphic $\text{sign}(\text{NAT-ADD})$ -algebras (see Exercise 101). $\diamond$

Isomorphic algebras have the same structure; they only differ in the representation of the elements. Isomorphic algebras are therefore often said to be “abstractly the same algebra.” For example, it does not really matter whether we represent the natural numbers by $0, 1, 2, 3, \dots$, by $0, 2, 4, 6, \dots$, or by $0, 1, 10, 11, \dots$, as long as the functions behave in the same way in these algebras.

Example 7.12. The algebras $\mathbb{N}$ and $\mathbb{N}_3$ are not isomorphic: there is no injective function from $\mathbb{N}$ to $\{0, 1, 2\}$ (and no surjective function from $\{0, 1, 2\}$ to $\mathbb{N}$). $\diamond$

Example 7.13. The algebras $\mathbb{N}$ and $\mathbb{N}_\perp$ are *not* $\text{sign}(\text{NAT-ADD})$ -isomorphic.¹ Proof: assume that $\phi : \mathbb{N} \rightarrow \mathbb{N}_\perp$ is a $\text{sign}(\text{NAT-ADD})$ -homomorphism. The homomorphism requirements

$$\phi(0_{\mathbb{N}}) = 0_{\mathbb{N}_\perp} \quad \text{and} \quad \phi(s_{\mathbb{N}}(n)) = s_{\mathbb{N}_\perp}(\phi(n))$$

force $\phi(0)$ to be 0, and force $\phi(n+1) = \phi(s_{\mathbb{N}}(n)) = s_{\mathbb{N}_\perp}(\phi(n)) = 1 + \phi(n)$. The only function ϕ satisfying both $\phi(0) = 0$ and $\phi(n+1) = \phi(n) + 1$ is the identity function, which is not surjective, since $\perp$ cannot be reached. $\diamond$

7.1.2 Term Algebras

Given a signature Σ and a disjoint set X of “variables,” there is a Σ -algebra, called the *term algebra* and denoted $\mathcal{T}_\Sigma(X)$, whose elements are the Σ -terms with variables in X . A function symbol f in Σ is interpreted in the algebra $\mathcal{T}_\Sigma(X)$ as the function $f_{\mathcal{T}_\Sigma(X)}$ which takes as arguments terms $t_1, t_2, \dots, t_n$ and returns the term $f(t_1, t_2, \dots, t_n)$. That is, $f_{\mathcal{T}_\Sigma(X)}(t_1, t_2, \dots, t_n) = f(t_1, t_2, \dots, t_n)$. When X is empty, we write $\mathcal{T}_\Sigma$ instead of $\mathcal{T}_\Sigma(\emptyset)$, and call the term algebra the *ground term algebra*.

Example 7.14. There is a $\text{sign}(\text{NAT-ADD})$ -homomorphism ϕ from $\mathcal{T}_{\text{sign}(\text{NAT-ADD})}$ to $\mathbb{N}$ defined by $\phi(t) =$ “the number encoded by t ,” since ϕ satisfies the conditions:

$$\phi(0) = 0 \quad \phi(s(t)) = \phi(t) + 1 \quad \phi(t_1 + t_2) = \phi(t_1) + \phi(t_2).$$

There is no homomorphism from $\mathbb{N}$ to $\mathcal{T}_{\text{sign}(\text{NAT-ADD})}$. Such a homomorphism φ would require $\varphi(0) = 0$ and $\varphi(m+n) = \varphi(m) + \varphi(n)$, which imply $0+0 = \varphi(0+0) = \varphi(0) = 0$; this is impossible, since 0 and $0+0$ are *different* terms. $\diamond$

Example 7.15. Let $\Sigma_{0,s}$ be the signature $\text{op } 0 : \rightarrow \text{Nat} \text{ . op } s : \text{Nat} \rightarrow \text{Nat} \text{ .}$ Then, the $\Sigma_{0,s}$ -algebras $\mathcal{T}_{\Sigma_{0,s}}$ and $\mathbb{N}$ (when seen as an $\Sigma_{0,s}$ -algebra by just forgetting about having to interpret $+$) are isomorphic (see Exercise 104). $\diamond$

Exercise 100 To have a $\text{sign}(\text{NAT-})$ -homomorphism also from $\mathbb{S}$ to $\mathbb{NB}$ it would be tempting to use the length function from strings to natural numbers (and the identity function on the Booleans). Explain why this is not a homomorphism from $\mathbb{S}$ to $\mathbb{NB}$. Can you find a homomorphism from $\mathbb{S}$ to $\mathbb{NB}$?

Exercise 101 Show that the algebras $\mathbb{N}$ and bits are isomorphic.

Exercise 102 Assume that for two Σ -algebras A and B there is a Σ -homomorphism $\phi_1 : A \rightarrow B$ and a Σ -homomorphism $\phi_2 : B \rightarrow A$. Are A and B Σ -isomorphic?

¹The sets $\mathbb{N}$ and $\mathbb{N} \cup \{\perp\}$ are isomorphic in the sense that there exists a bijective function f between them (e.g., $f(0) = \perp$, $f(1) = 0$, $f(2) = 1, \dots$). However, this function is not a $\text{sign}(\text{NAT-ADD})$ -homomorphism.

Exercise 103 Show that, for any Σ -algebra A , a variable assignment $\sigma : X \rightarrow A$ can be uniquely extended to a Σ -homomorphism from $\mathcal{T}_\Sigma(X)$ to A .

Exercise 104 Prove that $\mathcal{T}_{\Sigma_{0,s}}$ and $\mathbb{N}$ are isomorphic $\Sigma_{0,s}$ -algebras.

Exercise 105 Prove that there is at most one $\Sigma_{0,s}$ -homomorphism from $\mathcal{T}_{\Sigma_{0,s}}$ to any of the algebras in Example 7.3 (when they are seen as $\Sigma_{0,s}$ -algebras). That is, if ϕ_1 and ϕ_2 are two such homomorphisms, they must be the same function.

Exercise 106 For each pair A and B of algebras in Example 7.3:

1. Define a $\text{sign}(\text{NAT-ADD})$ -homomorphism from A to B , or show that no such homomorphism exists. Is there more than one such homomorphism?
2. Do the same in the opposite direction (i.e., from B to A).
3. Prove that A and B are isomorphic algebras, or that they are not isomorphic.

7.2 (Σ, E) -Models: (Σ, E) -Algebras

A (Σ, E) -algebra—a mathematical model for (Σ, E) —is a Σ -algebra that satisfies each equation in E . This section treats the unsorted (and unconditional) case.

Let X be a set of mathematical variables and A a Σ -algebra. A variable assignment $\sigma : X \rightarrow A$ can be uniquely extended to a Σ -homomorphism $\sigma^* : \mathcal{T}_\Sigma(X) \rightarrow A$ (see Exercise 103). The algebra A satisfies an equation $(\forall X) t = t'$ if and only if $\sigma^*(t) = \sigma^*(t')$ for each variable assignment σ . The algebra A is a (Σ, E) -algebra if it satisfies all equations in E :

Definition 7.4 A Σ -algebra A is a (Σ, E) -algebra if and only if $\sigma^*(t) = \sigma^*(t')$ for each equation $(\forall X) t = t'$ in E , and each assignment $\sigma : X \rightarrow A$, where $\sigma^* : \mathcal{T}_\Sigma(X) \rightarrow A$ denotes the homomorphic extension of σ .

We write $A \models (\forall X) t_1 = t_2$, or just $A \models t_1 = t_2$, if A satisfies $(\forall X) t_1 = t_2$, and write $A \models E$ if A satisfies all equations in a set E of equations. The set of all Σ -algebras satisfying a set of equations E is denoted $\text{Alg}(\Sigma, E)$. We write $\text{Alg}(\Sigma, E) \models (\forall X) t_1 = t_2$ when all algebras in $\text{Alg}(\Sigma, E)$ satisfy the equality $(\forall X) t_1 = t_2$.

Example 7.16. $\mathbb{N}$ is a NAT-ADD -algebra, since it satisfies both equations in NAT-ADD :

- For the equation $0 + M = M$ we must show $\sigma^*(0 + M) = \sigma^*(M)$ for any variable assignment $\sigma : \{M\} \rightarrow \mathbb{N}$. Since σ^* is a $\text{sign}(\text{NAT-ADD})$ -homomorphism, $\sigma^*(0 + M)$ equals $\sigma^*(0) + \sigma^*(M)$, and $\sigma^*(0)$ equals 0. Hence, $\sigma^*(0 + M) = \sigma^*(0) + \sigma^*(M) = 0 + \sigma^*(M) = \sigma^*(M)$.
- For the equation $s(M) + N = s(M + N)$ in NAT-ADD , we show $\sigma^*(s(M) + N) = \sigma^*(s(M + N))$ for any variable assignment $\sigma : \{M, N\} \rightarrow \mathbb{N}$ as follows:

$$\begin{aligned}
 \sigma^*(s(M) + N) &= \sigma^*(s(M)) + \sigma^*(N) && (\sigma^* \text{ is a homomorphism}) \\
 &= (\sigma^*(M) + 1) + \sigma^*(N) && (\sigma^* \text{ is a homomorphism}) \\
 &= 1 + (\sigma^*(M) + \sigma^*(N)) && (\text{math.}) \\
 &= \sigma^*(s(M + N)) && (\sigma^* \text{ is a homomorphism}) \quad \diamond
 \end{aligned}$$

Example 7.17. The algebra $\mathbb{N}_x$ in Example 7.3 is not a NAT-ADD -algebra. It does not satisfy the first (or the second) equation: $\sigma^*(0 + M) = \sigma^*(0) +_{\mathbb{N}_x} \sigma^*(M) = \sigma^*(0) = 84 \neq \sigma^*(M) = \sigma(M)$ for variable assignments $\sigma : \{M\} \rightarrow \mathbb{N}$ with $\sigma(M) \neq 84$. $\diamond$

7.2.1 Quotient Algebras

If A is a Σ -algebra, and $\approx$ is a congruence (see Appendix A) on the domain of A for all functions in A that interpret function symbols in Σ , then $\approx$ is a Σ -congruence on A . The *quotient algebra of A over $\approx$* , denoted $A/\approx$, is then defined as follows:

- The domain $A/\approx$ is the set $\{[a]_{\approx} \mid a \in A\}$ of all $\approx$ -equivalence classes of A .
- A function symbol f in Σ is interpreted in $A/\approx$ by the function $f_{A/\approx}$ defined by $f_{A/\approx}([a_1]_{\approx}, \dots, [a_n]_{\approx}) = [f_A(a_1, \dots, a_n)]_{\approx}$. Since $\approx$ is a Σ -congruence, the function $f_{A/\approx}$ is independent of the choice of representative a_i from the equivalence class $[a_i]_{\approx}$, and hence is well-defined.

Example 7.18. The quotient algebra of the $\text{sign}(\text{NAT-ADD})$ -algebra $\mathbb{N}$ over the congruence $\equiv_3$ (equality modulo 3) is the algebra $\mathbb{N}_3$. $\diamond$

7.2.2 The Algebra $\mathcal{T}_{\Sigma, E}$

An important algebra is the quotient of the ground term algebra $\mathcal{T}_{\Sigma}$ induced by a set of equations E . Let $=_E$ be the equivalence relation on ground terms defined by

$$t =_E t' \quad \text{if and only if} \quad \vdash Et = t'.$$

$=_E$ is a Σ -congruence because of the congruence rule of equational logic. The quotient ground term algebra induced by E is the quotient algebra $\mathcal{T}_{\Sigma}/=_E$, which is often written $\mathcal{T}_{\Sigma, E}$. The elements of $\mathcal{T}_{\Sigma, E}$ are therefore equivalence classes $[t]_{=_E}$ of ground terms $t \in \mathcal{T}_{\Sigma}$, where two ground terms t_1 and t_2 belong to the same equivalence class if and only if they can be proved equal in equational logic: $E \vdash t_1 = t_2$.

Example 7.19. The elements in $\mathcal{T}_{\text{sign}(\text{NAT-ADD}), \text{eqs}(\text{NAT-ADD})}$ are

$$\begin{aligned} [0]_{=_E} &= \{0, 0+0, 0+(0+0), \dots\} \\ [s(0)]_{=_E} &= \{s(0), 0+s(0), s(0)+0, 0+(s(0)+0), \dots\} \\ [s(s(0))]_{=_E} &= \{s(s(0)), s(0)+s(0), s(s(0+0)), \dots\} \\ &\vdots \end{aligned} \quad \diamond$$

It is intuitively fairly obvious that an algebra where the interpretation of two ground terms t_1 and t_2 are the same element if $E \vdash t_1 = t_2$ satisfies all the equations in E (see, e.g., [85, proof of Theorem 11]):

Theorem 7.1 $\mathcal{T}_{\Sigma,E}$ is a (Σ, E) -algebra.

7.2.3 The Normal Form Algebra

If the equations E are terminating and ground confluent, then the algebra closest to what we compute is the *normal form algebra*, also called the *canonical term algebra*, $\mathcal{C}_{\Sigma,E}$, whose elements are the E -normal forms of the ground terms: $\{t!_E \mid t \in \mathcal{T}_{\Sigma}\}$, and where the interpretation $f_{\mathcal{C}_{\Sigma,E}}$ of a function symbol f in Σ takes $t_1, \dots, t_n$ to the normal form of $f(t_1, \dots, t_n)$; that is, $f_{\mathcal{C}_{\Sigma,E}}(t_1, \dots, t_n) = (f(t_1, \dots, t_n))!_E$.

Example 7.20. The equations in NAT-ADD are terminating and confluent. The elements of the algebra $\mathcal{C}_{\text{NAT-ADD}}$ are the normal forms $\{0, s(0), s(s(0)), \dots\}$, and the interpretation of the function $+$ in $\mathcal{C}_{\text{NAT-ADD}}$ is the function $+_{\mathcal{C}_{\text{NAT-ADD}}}$ where

$$\underbrace{s(\dots s(0) \dots)}_m +_{\mathcal{C}_{\text{NAT-ADD}}} \underbrace{s(\dots s(0) \dots)}_n = \underbrace{s(\dots s(0) \dots)}_{m+n}. \quad \diamond$$

Exercise 107 Is $\mathbb{S}$ a NAT- algebra?

Exercise 108 Which of the algebras in Example 7.3 are NAT-ADD -algebras?

Exercise 109 For each of the NAT-ADD -algebras in Example 7.3, can you extend the algebra to a NAT-MULT -algebra by adding a suitable interpretation of $*$? Show that the resulting algebras indeed are NAT-MULT -algebras.

Exercise 110 Show that each algebra in Example 7.4 satisfies the group axioms $\{e \circ x = x, i(x) \circ x = e, (x \circ y) \circ z = x \circ (y \circ z)\}$.

Exercise 111 (Tricky?) Is the set of (Σ, E) -algebras closed under isomorphism? That is, if A and B are Σ -isomorphic Σ -algebras, is it the case that A satisfies E if and only if B does so?

7.3 Soundness and Completeness of Equational Logic

The key point about equational logic is that it allows us to reason about equalities that hold in *all* (Σ, E) -models. What we want is therefore the equivalence

$$\text{Alg}(\Sigma, E) \models (\forall X) t = t' \quad \text{if and only if} \quad E \vdash t = t'.$$

This means that equational logic is:

- *sound*: it is impossible to prove something that does not hold in all E -models; and
- *complete*: every equality $t = t'$ that holds in all E -models can be proved using equational logic.

How can we prove soundness and completeness of equational logic? Soundness seems quite obvious: the axioms and deduction rules of equational logic look fairly harmless and intuitively should not allow us to deduce wrong things. Completeness, on the other hand, seems much more tricky: how can we prove that *all* equalities that hold in all (Σ, E) -models actually can be proved using equational logic? Especially when we remember that equational logic is not strong enough to prove all equalities that hold in the *intended* (Σ, E) -models defined in Section 7.4.

To prove completeness we use one of the classic tricks in the book: define a structure S in which (the interpretation of) t is equal to t' in S if and only if $E \vdash t = t'$. Hopefully, this structure S is a (Σ, E) -algebra. If so, then completeness of equational logic, $\text{Alg}(\Sigma, E) \models (\forall X) t = t' \implies E \vdash t = t'$, follows from a contra-positive argument: Assume that $E \not\vdash t = t'$; then by definition of S , $S \not\models t = t'$, and if S indeed is a (Σ, E) -algebra, then $t = t'$ does not hold in *all* (Σ, E) -algebras, and hence $\text{Alg}(\Sigma, E) \not\models (\forall X) t = t'$.

What is this structure S ? Perhaps not unexpectedly, it is the quotient $\mathcal{T}_\Sigma(X)/=E$. It must be shown that $=E$ is a Σ -congruence, and that $\mathcal{T}_\Sigma(X)/=E$ is indeed a (Σ, E) -algebra, both of which can be proved fairly easily (see, e.g., [60, Chapter 9]).

This is the key idea to prove what is called Birkhoff's Completeness Theorem:

Theorem 7.2 (Birkhoff's Completeness Theorem) *Given a set E of equations over terms in $\mathcal{T}_\Sigma(X)$, then for any terms $t_1, t_2 \in \mathcal{T}_\Sigma(Y)$,*

$$\text{Alg}(\Sigma, E) \models (\forall Y) t_1 = t_2 \quad \text{if and only if} \quad E \vdash t_1 = t_2.$$

Proof. We assume for simplicity and without loss of generality that $Y \subseteq X$.

Completeness: The desired $\text{Alg}(\Sigma, E) \models (\forall Y) t_1 = t_2 \implies E \vdash t_1 = t_2$ is equivalent to $E \not\vdash t_1 = t_2 \implies \text{Alg}(\Sigma, E) \not\models (\forall Y) t_1 = t_2$, which can be proved as follows: Assume $E \not\vdash t_1 = t_2$. Then, by definition, $[t_1]_{=E}$ and $[t_2]_{=E}$ are different elements in $\mathcal{T}_\Sigma(X)/=E$. Therefore, the variable assignment $\sigma : Y \rightarrow \mathcal{T}_\Sigma(X)/=E$, with $\sigma(x) = [x]_{=E}$ for each $x \in Y \subseteq X$, is such that $\sigma^*(t_1) = [t_1]_{=E} \neq [t_2]_{=E} = \sigma^*(t_2)$. This means that $\mathcal{T}_\Sigma(X)/=E \not\models t_1 = t_2$. Since Theorem 7.1 says that $\mathcal{T}_\Sigma(X)/=E$ is a (Σ, E) -algebra, then not all (Σ, E) -algebras satisfy $t_1 = t_2$, hence the desired result $\text{Alg}(\Sigma, E) \not\models (\forall Y) t_1 = t_2$.

Soundness: $E \vdash t_1 = t_2 \implies \text{Alg}(\Sigma, E) \models (\forall Y) t_1 = t_2$. Assume that A is a (Σ, E) -algebra, and σ a variable substitution from Y to A . We prove that $E \vdash t_1 = t_2$ implies $\sigma^*(t_1) = \sigma^*(t_2)$ by induction on the size of the proof of $E \vdash t_1 = t_2$.

The base case, when the length of the proof is 0, means that we used either *Substitutivity* or *Reflexivity* to prove $E \vdash t_1 = t_2$:

- If *Reflexivity* was used to prove $E \vdash t_1 = t_2$, then t_1 and t_2 are the same term, and hence $\sigma^*(t_1) = \sigma^*(t_2)$.
- If *Substitutivity* was used to prove $E \vdash t_1 = t_2$, it means that there is an equation $t = t'$ in E , and a substitution $\rho : Y \rightarrow \mathcal{T}_\Sigma(X)$ such that $\rho^*(t) = t_1$ and $\rho^*(t') = t_2$. Since A is a (Σ, E) -algebra, it follows from Exercise 113 that $\theta(t) = \theta(t')$ for all

homomorphisms $\theta : \mathcal{T}_\Sigma(X) \rightarrow A$. In particular, it is easy to see that $(\sigma^* \circ \rho^*)$ is such a homomorphism. This implies the desired equality as follows:

$$\begin{aligned}
 \sigma^*(t_1) &= \sigma^*(\rho^*(t)) & (t_1 = \rho^*(t)) \\
 &= (\sigma^* \circ \rho^*)(t) \\
 &= (\sigma^* \circ \rho^*)(t') & (\text{since } A \text{ is an } E\text{-model}) \\
 &= \sigma^*(\rho^*(t')) \\
 &= \sigma^*(t_2) & (t_2 = \rho^*(t')).
 \end{aligned}$$

If $n + 1$ proof steps were needed to prove $E \vdash t_1 = t_2$, we can assume as induction hypothesis that $\sigma^*(t_3) = \sigma^*(t_4)$ for each $E \vdash t_3 = t_4$ that was proved in fewer steps:

- *Symmetry*: Assume that the last step in the proof of $E \vdash t_1 = t_2$ used *Symmetry*, from $E \vdash t_2 = t_1$. Since the proof of $E \vdash t_2 = t_1$ is smaller than the proof of $E \vdash t_1 = t_2$, the induction hypothesis applies and gives $\sigma^*(t_2) = \sigma^*(t_1)$, which of course means that the desired $\sigma^*(t_1) = \sigma^*(t_2)$ holds.
- *Transitivity and Congruence*: equally easy, and left as Exercise 114. $\square$

Using Birkhoff's Completeness Theorem we can show that $\text{NAT-ADD} \not\models M + N = N + M$: there is a NAT-ADD -model, namely AB in Example 7.3, where the $\sigma^*(M + N) = \sigma^*(N + M)$ does not hold for the homomorphism induced by the variable assignment $\sigma = \{M \mapsto a, N \mapsto b\}$, since $a + b = a \neq b = b + a$.

Exercise 112 Show that $\text{funcs}(\{a, b, c\})$ in Example 7.4 does not satisfy $x \circ y = y \circ x$. (This is probably the simplest example of a non-Abelian group.)

Exercise 113 Show that if A is a (Σ, E) -algebra and $t = t'$ is an equation in E , then $\phi(t) = \phi(t')$ holds for all homomorphisms $\phi : \mathcal{T}_\Sigma(X) \rightarrow A$.

Exercise 114 Fill in the missing parts in the proof of Theorem 7.2.

7.4 Intended Models: Initial Algebras

We have seen a number of NAT-ADD -algebras. But which of them is the one that we wanted to specify? Is it the algebra $\mathbb{N}$, the algebra *bits* of binary numbers, the algebra $\mathcal{T}_{\text{NAT-ADD}}$, or the normal form algebra $\mathcal{C}_{\text{NAT-ADD}}$? Or could it be the integers $\mathbb{Z}$, the algebra *squares*, the even numbers $\mathcal{E}$, the algebra $+2$, or even AB ? The answer by ‘‘ADJ’’ (Goguen, Thatcher, Wagner, and Wright) in 1975 was that the intended model of an equational specification (Σ, E) is the *initial algebra(s)* in the class $\text{Alg}(\Sigma, E)$ of all (Σ, E) -algebras [49].

Definition 7.5 A Σ -algebra A is *initial* in a class $\mathbb{A}$ of Σ -algebras if and only if for each algebra $B \in \mathbb{A}$, there is exactly one Σ -homomorphism from A to B .

The initial algebra in $\text{Alg}(\Sigma, E)$ is a fairly abstract definition of the intended (Σ, E) -model, which does not define the elements of the intended model. Is there one intended model, or could there be many initial (Σ, E) -algebras? Can we be even sure that a specification (Σ, E) has an intended model at all?

It turns out that a class of Σ -algebras may have many initial algebras, but they are all essentially the same algebra: they are all isomorphic.

Theorem 7.3 *If A and B are two initial Σ -algebras in a class $\mathbb{A}$ of Σ -algebras, then A and B are isomorphic.*

Proof. Since both A and B are initial in $\mathbb{A}$, there are Σ -homomorphisms $\phi_1 : A \rightarrow B$ and $\phi_2 : B \rightarrow A$. Since, as proved in Exercise 115, the composition of two homomorphisms is a homomorphism, there are Σ -homomorphisms $\phi_2 \circ \phi_1 : A \rightarrow A$ and $\phi_1 \circ \phi_2 : B \rightarrow B$. The identity functions $\text{id}_A : A \rightarrow A$, defined by $\text{id}_A(a) = a$, and $\text{id}_B : B \rightarrow B$ are also Σ -homomorphisms. Since A and B are initial in $\mathbb{A}$, there is exactly *one* homomorphism from A to A , and exactly one from B to B . This means that $\phi_2 \circ \phi_1 = \text{id}_A$ and $\phi_1 \circ \phi_2 = \text{id}_B$, which imply that ϕ_1 and ϕ_2 are surjective and injective Σ -homomorphisms (see Exercise 116), and hence A and B are isomorphic.

Can we be sure that a specification (Σ, E) has initial models? After all, Exercise 117 shows that some specification formalisms fail to guarantee initial models. It turns out that there is always an initial (Σ, E) -algebra, namely, the algebra $\mathcal{T}_{\Sigma, E}$:

Theorem 7.4 *The algebra $\mathcal{T}_{\Sigma, E}$ is an initial algebra in the class $\text{Alg}(\Sigma, E)$.*

Proof. (The proof roughly follows a similar proof in [60].) We must prove that there is one, and only one, Σ -homomorphism $\hat{\phi} : \mathcal{T}_{\Sigma, E} \rightarrow A$ for each (Σ, E) -algebra A .

Since A is a Σ -algebra, it has interpretations c_A and f_A for each constant c and each non-constant f in Σ . We define a Σ -homomorphism $\phi : \mathcal{T}_{\Sigma} \rightarrow A$ as follows:

$$\begin{aligned} \phi(c) &= c_A && \text{for each constant } c \text{ in } \Sigma \\ \phi(f(t_1, \dots, t_n)) &= f_A(\phi(t_1), \dots, \phi(t_n)) && \text{for each } f \text{ in } \Sigma. \end{aligned}$$

We define the function $\hat{\phi} : \mathcal{T}_{\Sigma, E} \rightarrow A$ by $\hat{\phi}([t]_E) = \phi(t)$, and must prove that:

1. The function $\hat{\phi}$ is well-defined.
2. The function $\hat{\phi}$ is a Σ -homomorphism from $\mathcal{T}_{\Sigma, E}$ to A .
3. $\hat{\phi}$ is the only such homomorphism.

These three properties are proved as follows:

1. $\hat{\phi}$ is well-defined function. This means that it returns the same element for the same argument. That is, if $[t_1]_E = [t_2]_E$, then it must also be the case that $\hat{\phi}([t_1]_E) = \hat{\phi}([t_2]_E)$. If $[t_1]_E = [t_2]_E$, then, by definition of $\mathcal{T}_{\Sigma, E}$, we have $E \vdash t_1 = t_2$, and hence $\phi(t_1) = \phi(t_2)$, since A is an E -algebra and equational logic is sound. But then we have the desired $\hat{\phi}([t_1]_E) = \phi(t_1) = \phi(t_2) = \hat{\phi}([t_2]_E)$.

2. To prove that the function $\hat{\phi}$ is a Σ -homomorphism we must prove:

$$\begin{aligned}\hat{\phi}([c]_E) &= c_A && \text{for each constant } c \text{ in } \Sigma \\ \hat{\phi}([f(t_1, \dots, t_n)]_E) &= f_A(\hat{\phi}([t_1]_E), \dots, \hat{\phi}([t_n]_E)) && \text{for each } f \text{ in } \Sigma.\end{aligned}$$

For constants, we have $\hat{\phi}([c]_E) = \phi(c) = c_A$, as desired. For the non-constant case, we have $\hat{\phi}([f(t_1, \dots, t_n)]_E) = \phi(f(t_1, \dots, t_n))$ by definition of $\hat{\phi}$. The latter by definition of ϕ equals $f_A(\phi(t_1), \dots, \phi(t_n))$, which by definition of $\hat{\phi}$ (backwards: $\phi(t) = \hat{\phi}([t]_E)$ for all t) equals the desired $f_A(\hat{\phi}([t_1]_E), \dots, \hat{\phi}([t_n]_E))$.

3. $\hat{\phi}$ is the *only* such homomorphism: If ρ is a Σ -homomorphism from $\mathcal{T}_{\Sigma, E}$ to A , then ρ and $\hat{\phi}$ are the same function; i.e., $\hat{\phi}([t]_E) = \rho([t]_E)$ for all $[t]_E \in \mathcal{T}_{\Sigma, E}$, which we prove by induction on the depth of the term t :

- Base case, t is a constant c , and we must show $\rho([c]_E) = \hat{\phi}([c]_E)$. Since ρ is a homomorphism, $\rho([c]_E) = c_A$, and in item (2) above we proved that $\hat{\phi}([c]_E) = c_A$, giving the desired $\rho([c]_E) = c_A = \hat{\phi}([c]_E)$.
- Assume that t is a term of depth $k + 1$, that is, it has the form $f(t_1, \dots, t_n)$. We can then apply the induction hypothesis $\hat{\phi}([t_i]_E) = \rho([t_i]_E)$ on all subterms t_i to prove $\rho([f(t_1, \dots, t_n)]_E) = \hat{\phi}([f(t_1, \dots, t_n)]_E)$ as follows:

$$\begin{aligned}\rho([f(t_1, \dots, t_n)]_E) &= f_A(\rho([t_1]_E), \dots, \rho([t_n]_E)) && (\rho \text{ is a homomorphism}) \\ &= f_A(\hat{\phi}([t_1]_E), \dots, \hat{\phi}([t_n]_E)) && (\text{induction hypothesis}) \\ &= \hat{\phi}([f(t_1, \dots, t_n)]_E) && (\hat{\phi} \text{ is a homomorphism})\end{aligned}$$

□

So the algebra $\mathcal{T}_{\text{NAT-ADD}}$ is an intended model of the specification NAT-ADD . The algebra $\mathbb{N}$ of natural numbers is another intended model of NAT-ADD .

Example 7.21. $\mathbb{N}$ is an initial algebra in the class $\text{Alg}(\text{NAT-ADD})$, since it is isomorphic to the initial algebra $\mathcal{T}_{\text{NAT-ADD}}$. This can be proved as follows. We have previously shown that NAT-ADD is terminating, confluent, and sufficiently complete: all ground terms reduce to constructor ground terms, which, cannot be reduced further. All this implies that the elements in $\mathcal{T}_{\text{NAT-ADD}}$ are $\{[0], [s(0)], [s(s(0))], \dots\}$. Let the function $\phi : \mathcal{T}_{\text{NAT-ADD}} \rightarrow \mathbb{N}$ be defined by $\phi(\underbrace{[s(\dots s(0) \dots)]}_n) = n$.

ϕ is a $\text{sign}(\text{NAT-ADD})$ -homomorphism, since $\phi([0]) = 0 = 0_{\mathbb{N}}$, and $\phi([s(t)]) = 1 + \phi(t) = s_{\mathbb{N}}(\phi(t))$, and $\phi([t_1 + t_2]) = \phi(t_1) + \phi(t_2)$, which can be proved easily.

ϕ is an isomorphism, since ϕ is injective ($[t_1] \neq [t_2]$ implies $\phi([t_1]) \neq \phi([t_2])$, which should be fairly obvious), and ϕ is surjective (for any $n \in \mathbb{N}$ there is a t such that $\phi([t]) = n$, which also holds, for t equal $\underbrace{s(\dots s(0) \dots)}_n$). ◇

Any algebra that is isomorphic to $\mathcal{T}_{\text{NAT-ADD}}$ or $\mathbb{N}$ is therefore the intended model of NAT-ADD .

The canonical term algebra $\mathcal{C}_{\Sigma, E}$ is also the initial algebra for a confluent and terminating specification E , which means that the denotational semantics (the initial model) is the same as the computational model (the canonical term algebra).

Theorem 7.5 *If the equations E are confluent and terminating, the normal form algebra $\mathcal{C}_{\Sigma,E}$ is isomorphic to $\mathcal{T}_{\Sigma,E}$ and therefore an initial algebra in $\text{Alg}(\Sigma, E)$.*

Proof. The homomorphism $\phi : \mathcal{T}_{\Sigma,E} \rightarrow \mathcal{C}_{\Sigma,E}$ is defined in the obvious way, namely: $\phi([t]_E) = t!_E$. It must then be proved that ϕ actually is a Σ -homomorphism and that it is an injective and surjective function, which is left as Exercise 118. $\square$

As mentioned above, the definition of the intended model as the initial algebra is somewhat abstract. How can we easily see what is an initial algebra? Two key properties characterizing any initial (Σ, E) -algebra A are:

- *No junk:* A does not contain any “junk” element that is not an interpretation of some ground term.
- *No confusion:* If two ground terms t_1 and t_2 are interpreted by the same element in A , then $E \vdash t_1 = t_2$. That is, A does not identify elements that are not E -equal.

Intuitively, if an algebra A has “confusion,” i.e., identifies (the interpretation) of two ground terms t_1 and t_2 that are not E -equivalent, then there is no homomorphism from A to $\mathcal{T}_{\Sigma,E}$. Such a homomorphism should map t_{1A} (the interpretation of t_1 in A) to $[t_1]_E$ and should map t_{2A} to $[t_2]_E \neq [t_1]_E$, which is impossible if $t_{1A} = t_{2A}$.

Example 7.22. In the algebra $\mathbb{N}_3$ both $s(s(s(0)))$ and 0 are interpreted as the number 0 , even though $\text{NAT-ADD} \not\vdash s(s(s(0))) = 0$. There is no homomorphism from $\mathbb{N}_3$ to the $\mathcal{T}_{\text{NAT-ADD}}$, since a homomorphism $\phi : \mathbb{N}_3 \rightarrow \mathcal{T}_{\text{NAT-ADD}}$ would have

$$[s(s(s(0)))] = \phi(s_{\mathbb{N}_3}(s_{\mathbb{N}_3}(s_{\mathbb{N}_3}(0_{\mathbb{N}_3})))) = \phi(0_{\mathbb{N}_3}) = [0],$$

which is impossible, since $[s(s(s(0)))] \neq [0]$. $\diamond$

That A contains no junk means that the (unique) Σ -homomorphism $\phi : \mathcal{T}_{\Sigma,E} \rightarrow A$ is surjective. No confusion means that this ϕ is injective. Therefore, if A satisfies both no junk and no confusion, then ϕ is both surjective and injective, and hence ϕ is a Σ -isomorphism. This again means that A is an initial (Σ, E) -algebra, since any algebra that is isomorphic to the initial algebra $\mathcal{T}_{\Sigma,E}$ is an initial (Σ, E) -algebra.

That the intended models contain no junk also implies that the reasoning about inductive theorems; i.e., equalities that hold in the initial models, in Section 6.2, is sound, as long as the specification is sufficiently complete (each ground term is equivalent to some constructor ground term).

Exercise 115 *Show that if $\phi_1 : A \rightarrow B$ and $\phi_2 : B \rightarrow C$ are two Σ -homomorphisms, then their composition $\phi_2 \circ \phi_1$ defined by $(\phi_2 \circ \phi_1)(a) = \phi_2(\phi_1(a))$ is also a Σ -homomorphism.*

Exercise 116 *Show that $\phi_2 \circ \phi_1 = id_A$ and $\phi_1 \circ \phi_2 = id_B$ imply that ϕ_1 and ϕ_2 are both injective and surjective.*

Exercise 117 *Assume that we have a specification formalism that supports disjunctions (or) of equations, with the obvious meaning. Then show that the class of algebras satisfying the specification $a = b$ or $a = c$ does not have an initial algebra.*

Exercise 118 Show that the function ϕ in the proof of Theorem 7.5 is surjective, injective, and a Σ -homomorphism.

Exercise 119 Which of the NAT-ADD -algebras in Example 7.3 satisfy the “no junk” property? Which of them satisfy the “no confusion” property? Which ones are initial algebras in $\text{Alg}(\text{NAT-ADD})$?

7.5 Empty Sorts and Many-Sorted Equational Logic

Going from the unsorted case to the many-sorted case is straightforward, as long as all sorts have ground terms. If some sorts do not have any ground terms, which may be useful, for example, when reasoning about parametric modules, the obvious extension of unsorted equational logic to the many-sorted setting is unsound.

Example 7.23. The sort `Empty` has no ground terms in the following specification:

```
fmod EMPTY-SORT is including BOOL .
  sort Empty .
  op f : Empty -> Bool .
  var X : Empty .
  eq f(X) = true .
  eq f(X) = false .
endfm
```

A model A of this specification has elements $A_{\text{Empty}} = \emptyset$ and $A_{\text{Bool}} = \{\mathbf{t}, \mathbf{f}\}$, and the function $\mathbf{f}$ could be interpreted in A by the function f defined by $(\forall e \in \emptyset) f(e) = \mathbf{t}$. This algebra A is a model of the specification `EMPTY-SORT`, with the interpretations $\mathbf{t}$ and $\mathbf{f}$ of, respectively, `true` and `false`, being different elements. A satisfies both equations: $(\forall x \in \emptyset) f(x) = \text{false}$ holds in A ; if it were not to hold, there should be some element e such that $f(e) \neq \mathbf{f}$. Obviously, there is no such e in the empty set.

However, $\text{EMPTY-SORT} \vdash \text{true} = \text{false}$ could be proved in a straightforward extension of unsorted equational logic. Since this equality does *not* hold in A , which is a model of `EMPTY-SORT`, the logic would be unsound. $\diamond$

Meseguer and Goguen have defined a sound and complete many-sorted equational logic that treats variables carefully [85]. In their logic we can prove $\text{EMPTY-SORT} \vdash (\forall x : \text{Empty}) \text{true} = \text{false}$, but not the undesired $\text{EMPTY-SORT} \vdash \text{true} = \text{false}$.

Part II
Specification and Analysis of Distributed
Systems in Maude

This chapter introduces *rewriting logic*, which can be used to model dynamic systems and to reason about concurrent change in a distributed system. This chapter may be read together with Chapter 9, which explains how rewriting logic models can be executed in Maude.

8.1 Dynamic Systems

Part I of this book deals with specifying data types by defining what terms are *equivalent*. There is (mathematically) no *dynamic* behavior in an equational specification. A term represents an *expression*, and two expressions are either equivalent or there is no relation between them. Due to *symmetry* of the equivalence relation, both $\text{length}(2\ 5\ 7) = 3$ and $3 = \text{length}(2\ 5\ 7)$ hold. Likewise, $2+1$ is always 3, and 3 is always $2+1$.

The rest of this book deals with modeling and analyzing *dynamic* (or *changing* or *evolving*) systems, where the *state*—which is also represented as a term—of the system changes over time. Consider modeling the life of a person. A state in such a model can be represented by a term `person("Peter", 46, married)`, denoting a person named Peter, who is currently 46 years old and married. This state can *change* to `person("Peter", 47, married)` or to `person("Peter", 46, divorced)`.

Change is quite often *irreversible*: a human being cannot go from being 46 years old to being 45 years old; a football game with the score 39–38 cannot change the score back to 14–38; a bad chess move cannot be reversed; an unfortunate email sent into the network cannot be called back; and so on.

A component in a dynamic computer system is often a *reactive system*, which interacts with its *environment* by *reacting* to input from the environment by changing its state and/or by providing some output. The prototypical example of a reactive

system/program is an operating system. Such a system reacts to input, say commands such as `ls` and `rm` from the terminal, by providing output (such as the list of all files) and/or by changing the state of the system (e.g., by removing some files). Reactive systems (together with their environment) are normally nonterminating and nondeterministic, and their properties of interest are “the current state” of the system and their response to stimuli from the environment.

A *distributed system* consists of a number of multiple distributed “components” (e.g., computing devices) that communicate with each other, e.g., by reading and writing the same “shared variable” or by sending messages to each other.

8.1.1 Properties of Dynamic and Distributed Systems

Sequential computer programs and functional Maude modules are typically *deterministic*: starting the system from the same state/expression will always lead to the same result, no matter how often we execute the system. Sorting a given list will always give the same result, and $3+2$ is always 5. Furthermore, functional modules and sequential programs should typically be *terminating*.

In contrast, many dynamic systems are *nondeterministic*: they may have many different behaviors—possibly resulting in different final outcomes (if any)—from the *same* starting state. For example, a state `person("Peter", 46, married)` could change in one “step” to either the state `person("Peter", 47, married)`, the state `person("Peter", 46, deceased)`, or the state `person("Peter", 46, divorced)` in a model of a person. Likewise, consider a game of chess: from the initial configuration where all the pieces are in their starting positions, the state of the game could evolve in many different ways, resulting in different final states.

Networked systems are intrinsically nondeterministic. Consider two persons vying to buy the last super cheap plane ticket from Oslo to San Francisco at the same time. The person whose message is routed the fastest from his/her computer to the ticket reservation server will get the desired plane ticket. However, the winner depends on a number of factors: network load, routing, whether sites along the route in the network are down, and so on. The next time the same persons want to buy a plane ticket at the same time, the outcome may be different.

While some of these systems (such as the life of a person) are *terminating*, many distributed systems are *not* supposed to terminate: your operating system, airplane ticket reservation services, and most other web services should always be up.

8.1.2 Behaviors of Distributed Systems

Consider a system with three components c_1 , c_2 , and c_3 , where each component wants to execute the four operations op_1 , op_2 , op_3 , and op_4 sequentially. What are the behaviors of the entire system?

When an *interleaving semantics* is used, a behavior of the system is considered to be a list of single events, ordered according to when the events took place. That is,

only one event can take place in the system at the same time. We typically assume that *any* component can execute its next operation at any time, and must consider *all possible* sequences of “interleaved” operations of the three components. Two such behaviors of the above system are:

1. $c_1:op_1 \rightarrow c_2:op_1 \rightarrow c_3:op_1 \rightarrow c_2:op_2 \rightarrow c_3:op_2 \rightarrow c_1:op_2 \rightarrow c_2:op_3 \rightarrow c_2:op_4 \rightarrow c_1:op_3 \rightarrow c_3:op_3 \rightarrow c_3:op_4 \rightarrow c_1:op_4$
2. $c_3:op_1 \rightarrow c_3:op_2 \rightarrow c_3:op_3 \rightarrow c_1:op_1 \rightarrow c_1:op_2 \rightarrow c_2:op_1 \rightarrow c_3:op_4 \rightarrow c_1:op_3 \rightarrow c_2:op_2 \rightarrow c_2:op_3 \rightarrow c_2:op_4 \rightarrow c_1:op_4$

Understanding and analyzing distributed systems is hard. Even this trivial example has a whopping 34,650 different behaviors!

The interleaving model is particularly suitable for systems where only one component can execute at the same time, for example, because they are all sharing some resource (like a server executing the operations, or they are accessing the same shared data). However, in a distributed system, two or more components could execute at the *same time*, or *concurrently*. For example, all three components could perform their first operation at the same time, and c_1 and c_3 could perform their second operation at the same time, as long as they execute on separate computers and do not access shared resources:

$$\{c_1:op_1, c_2:op_1, c_3:op_1\} \rightarrow c_2:op_2 \rightarrow \{c_1:op_2, c_3:op_2\} \rightarrow \dots$$

Exercise 120 Three persons share a bank account x , which initially contains \$100, and each person wants to add \$20 to this account by executing the three (atomic) operations

$$y := read(x); \quad z := y + 20; \quad write(x, z);$$

in the given order. Assume that the three persons all execute the same program more or less at the same time in an interleaved fashion.

1. What are the possible outcomes (i.e., balance of the shared bank account x) if y and z are local variables?
 2. What are the possible outcomes if also y and z are shared variables?
-

8.2 Modeling Dynamic Systems in Rewriting Logic

How can we model dynamic systems? Equational specifications seem to be the wrong formalism, because: (i) change is “reversible” in equational logic; (ii) equational specifications are supposed to be confluent, whereas many dynamic systems are nondeterministic and therefore often not confluent; and (iii) many dynamic systems should not terminate, whereas equational specifications should be terminating.

For example, trying to model the aging of a person using an equation like

$$\text{person}(X, N, S) = \text{person}(X, N + 1, S)$$

would allow us to deduce properties like $\text{person}(\text{"Peter"}, 46, \text{married}) = \text{person}(\text{"Peter"}, 20, \text{married})$, which unfortunately does not hold. We instead use *rewriting logic* [16, 80] to model dynamic systems.

8.2.1 Rewrite Rules

In rewriting logic, dynamic behavior is modeled by *rewrite rules* that define the local transitions of a system. The “birthday” action in our simple example could be modeled by a rewrite rule

$$\text{person}(X, N, S) \longrightarrow \text{person}(X, N + 1, S)$$

for appropriate variables X , N , and S .

Given the local atomic transitions in the form of rewrite rules, the *deduction rules of rewriting logic* (see pages 139–140) define how the state of a system *may* evolve. A rewriting logic *sequent* (or *formula*)

$$t \longrightarrow t'$$

means that the *state* t *can* change to (or *evolve* to, or *reach*) the *state* t' using the rewrite rules zero or more times. (I will use “term” and “state” interchangeably since the state of a system is represented by a term.) Therefore, the sequent $\text{person}(\text{"Peter"}, 46, \text{married}) \longrightarrow \text{person}(\text{"Peter"}, 56, \text{married})$ holds, but $\text{person}(\text{"Peter"}, 46, \text{married}) \longrightarrow \text{person}(\text{"Peter"}, 20, \text{married})$ does *not* hold.

A rewrite rule can be equipped with a *label* which names the *action* or *event* that causes the state change. In our example, the labeled rule could be

$$\text{birthday} : \text{person}(X, N, S) \longrightarrow \text{person}(X, N + 1, S).$$

The label does not influence Maude computations.

The data of a dynamic system are defined as data types with equations as usual. In the *birthday* rule above, ‘+’ is a function defined equationally (or is built in). In this specification $\text{person}(\text{"Roland"}, 7, \text{single})$ can change to $\text{person}(\text{"Roland"}, 8, \text{single})$ in *one* step, since this state is equivalent to the state $\text{person}(\text{"Roland"}, 7 + 1, \text{single})$. Therefore, rewriting is *modulo* the equations E in the specification: if $t \longrightarrow t'$ and $E \vdash t = u$ and $E \vdash t' = u'$, then $u \longrightarrow u'$.

8.2.2 Rewriting Logic Specifications

A *rewriting logic specification* is an equational logic specification extended with labeled rewrite rules which describe the local transitions (state changes) in a system.¹

Definition 8.1 (Rewriting logic specification [80]) A rewriting logic specification (also called a *rewrite theory*) is a tuple $\mathcal{R} = (\Sigma, E, L, R)$ where Σ is an algebraic signature, E is a set of equations and membership axioms, L is a set of labels, and R is a set of unconditional labeled rewrite rules

$$l : t \longrightarrow t'$$

and conditional labeled rewrite rules of the form

$$l : t \longrightarrow t' \text{ if } \text{cond}$$

where $l \in L$, t and t' are terms in $\mathcal{T}_\Sigma(X)$, and cond is a conjunction of rewrite conditions of the form $u \longrightarrow u'$, equational conditions of the form $v = v'$ and membership conditions $w : s$, for u, u', v, v', w terms in $\mathcal{T}_\Sigma(X)$ and s a sort in Σ . The terms t and t' must be terms of the same kind.²

A rewrite theory is specified in Maude as a *system module*, which is declared using the keywords `mod` and `endm` instead of `fmod` and `endfm`, to indicate that we are no longer in a *functional* world. Rewrite rules are written

`rl [l] : t => t' .`

and

`crl [l] : t => t' if cond .`

in the conditional case. A conjunct in the condition *cond* may be a term of sort `Bool`, an equality, a membership test, or a *rewrite condition*, which is written $u \Rightarrow u'$. Such a rewrite condition is satisfied if an instance of u' can be reached in *zero or more* rewrite steps from the instance of u obtained when the rule is instantiated.

As explained above, many dynamic systems are nondeterministic, due to race conditions and other factors. Nondeterministic behaviors cannot be modeled in equational logic, but can be easily specified in rewriting logic, by having different rewrite rules that may apply to the same state.

Example 8.1. (Borrowed from [80]) A nondeterministic choice operator `__?__`, which nondeterministically returns one of its arguments, can be specified as follows:

¹ Although we use membership equational logic as the underlying equational logic in the definition below, rewriting logic is actually *parametric* in the underlying equational logic, which could be unsorted, order-sorted, membership, or some other kind of equational logic.

² In order-sorted specifications, the sorts of t and t' must be in the same connected component of $(S, \leq)$.

```

mod CHOICE-INT is including INT .
  op _?_ : Int Int -> Int [ctor] .
  vars I J : Int .
  rl [choose_first] : I ? J => I .
  rl [choose_second] : I ? J => J .
endm

```

A term $3 ? 5$ can change into either 3 or 5. ◇

Nondeterministic behavior could mean that the set of rewrite rules is not confluent. Since many distributed systems are nonterminating, the set of rewrite rules may well be both non-confluent and nonterminating.

8.2.3 Examples

This section presents some specifications which are executed in Chapter 9.

Simulating a Football Game.

The following rewriting logic specification is supposed to simulate an (“American”) football game. For European readers it may also serve the purpose of a *specification* explaining *how* the score of a game changes as the result of various actions such as a “touchdown” or a “safety.” A *state* of a game is a term of the form

"Steelers" vs "Patriots" 35 : 0

where the first string ("Steelers") denotes the home team, the second string ("Patriots") the visiting team, and the rest is the current score. The possible “behaviors” of a football game can then be specified by the following Maude module:³

```

mod ONE-FOOTBALL-GAME is protecting NAT + STRING .
  sort Game .
  op _vs_ _:_ : String String Nat Nat -> Game [ctor] .
  vars HOME AWAY : String .    vars M N : Nat .

  *** The following rules model the home team scoring:
  rl [touchdown-home] :
    HOME vs AWAY M : N => HOME vs AWAY (M + 6) : N .
  rl [field-goal-home] :
    HOME vs AWAY M : N => HOME vs AWAY (M + 3) : N .
  rl [extra-point-kick-home] :
    HOME vs AWAY M : N => HOME vs AWAY (M + 1) : N .
  rl [two-point-conversion-home] :
    HOME vs AWAY M : N => HOME vs AWAY (M + 2) : N .
  rl [safety-home] :
    HOME vs AWAY M : N => HOME vs AWAY (M + 2) : N .

```

³ The module expression $module_1 + module_2$ gives the union of the two modules.

```

*** Scoring possibilities for the visiting team:
rl [touchdown-away] :
  HOME vs AWAY M : N => HOME vs AWAY M : (N + 6) .
rl [field-goal-away] :
  HOME vs AWAY M : N => HOME vs AWAY M : (N + 3) .
rl [extra-point-kick-away] :
  HOME vs AWAY M : N => HOME vs AWAY M : (N + 1) .
rl [two-point-conversion-away] :
  HOME vs AWAY M : N => HOME vs AWAY M : (N + 2) .
rl [safety-away] :
  HOME vs AWAY M : N => HOME vs AWAY M : (N + 2) .
endm

```

Modeling the Life of a Person.

The following Maude module⁴ models the possible lives of a person, where a state is represented by a term `person(name, age, status)`, with *status* the “civil status” and *age* the age of the person:

```

mod ONE-PERSON is protecting NAT + STRING .
  sorts Person Status .
  op person : String Nat Status -> Person [ctor] .
  ops single engaged married separated divorced deceased
    widow widower : -> Status [ctor] .

  var X : String . var N : Nat . var S : Status .

  crl [birthday] : person(X, N, S) => person(X, N + 1, S)
    if N <= 1000 /\ S /= deceased .

  crl [successful-proposal] :
    person(X, N, S) => person(X, N, engaged)
    if N >= 15 /\ (S == single or S == divorced) .

  rl [marriage] : person(X, N, engaged) => person(X, N, married) .
  ...
endm

```

A Coffee Bean Game.

The *coffee bean game* (which I found in [60]) is a one-person game in which one is given a *sequence* of coffee beans, where a coffee bean may be either white or black. The rules of the game are simple: Two black beans next to each other may be replaced by one white bean, while a white bean next to a black bean may be removed. The goal of the game is to end up with the fewest number of beans.

Exercise 121 Complete the module `ONE-PERSON` with rules for, e.g., broken engagement, separation, divorce, death, death of a spouse, and other possible events.

⁴ Parts of the specification are omitted and are replaced by ‘...’.

Exercise 122 Consider the coffee game described above.

1. Specify the coffee bean game in Maude.
2. Explain why you used rewrite rules instead of equations to describe this game.
3. Explain why the game is terminating.
4. Show that the game is not confluent (e.g., by using the techniques in Chapter 5).
5. Show that from a starting state $\circ \circ \bullet \bullet \circ \bullet$ one may reach a final state with one white bean, and another final state with five white beans.
6. Use techniques from Chapter 5 to make the specification confluent by adding one rule to the specification. Prove that the resulting specification is confluent.

Exercise 123 In the whiteboard game there are a bunch of non-zero natural numbers on a whiteboard. Specify the following versions of this exciting game in Maude:

1. Any two numbers m and n on the whiteboard can be replaced by the number $(m+n) \text{ quo } 2$.
2. As above; in addition, if there are two occurrences of the number m on the whiteboard, then one of them may be replaced by the numbers $m-1, m-2, \dots, 2, 1$.
3. Any two numbers m and n on the whiteboard can be replaced by $m+n+(m \cdot n)$.

Exercise 124 The “Tower of Hanoi” is a classic “puzzle” with m rods and n disks of different sizes. The puzzle starts with all the disks, ordered by size, on rod 1, with the smallest on top. The objective is to move all disks onto the “last” rod m , by repeatedly moving the upper-most disk from some rod onto another rod, so that a disk is never placed on top of a smaller disk. A rod can be represented in Maude as a term `rod i stack disks`, with i the number of the rod and disks a list of natural numbers between 1 and n . The state of the system is a multiset of m such rods.

1. Define an initial state `init(m,n)` with m rods and n disks.
2. Define all possible legal moves of this “puzzle” in Maude.

Exercise 125 Recall the Traveling Salesman (TS) problem: Given a set of cities and a cost for traveling between each pair of cities, can a salesman start in his home city and visit every other city exactly once before returning to his home city, for a total cost of the journey less than equal to some limit K ?

Assume that `cost(c1, c2)` gives the cost of traveling between c_1 and c_2 , and that cities gives the set of cities to visit. There are at least three cities to visit. For example, some cities and the cost between them could be given in Maude as follows:

```

sorts City Cities .      subsort City < Cities .
op none : -> Cities [ctor] .
op _;_ : Cities Cities -> Cities [ctor assoc comm id: none] .
ops PhnomPenh SiemReap Sisophon Battambang KompongSom : -> City [ctor] .

op cities : -> Cities .
eq cities = PhnomPenh ; SiemReap ; Sisophon ; Battambang ; KompongSom .

eq cost(PhnomPenh, SiemReap) = 2 .      eq cost(PhnomPenh, KompongSom) = 4 .
eq cost(PhnomPenh, Sisophon) = 9 .      eq cost(PhnomPenh, Battambang) = 6 .
eq cost(SiemReap, Sisophon) = 3 .        eq cost(SiemReap, Battambang) = 1 .
eq cost(SiemReap, KompongSom) = 3 .      eq cost(Sisophon, Battambang) = 3 .
eq cost(Sisophon, KompongSom) = 7 .      eq cost(Battambang, KompongSom) = 9 .

```

A complete TS trip could be the author's 1993 journey: PhnomPenh $\rightarrow$ SiemReap $\rightarrow$ Sisophon $\rightarrow$ Battambang $\rightarrow$ KompongSom $\rightarrow$ PhnomPenh, with total cost 21.

We use the following data type to define a (possibly incomplete) journey:

```
sort Trip .      subsort City < Trip .
op _-->_ : Trip Trip -> Trip [ctor assoc] .
```

1. Define a function $ts : \text{NzNat} \rightarrow \text{Bool}$ so that $ts(K)$ returns *true* if and only if there is a TS trip with total cost less than or equal to K .
2. One thing is knowing that it is possible to travel for less than $\$K$; another thing is knowing which route to take. Explain why we cannot have a “well-defined” function $okTrip : \text{NzNat} \rightarrow \text{Trip}$ which returns a trip with total cost $\leq K$.
3. Define a sort *State* for the states in your system, and define a suitable initial state. Each state must contain the journey undertaken so far.
4. Specify all possible behaviors of a traveling salesman in Maude.
5. It may sometimes be cheaper to go via a third city instead of traveling directly between two cities. For example, if you are in *Sisophon* and must head home to *PhnomPenh*, you can save money by going through *SiemReap*. Specify all possible behaviors of the salesperson when (s)he can visit a city multiple times.

Exercise 126 Define a simulator for Turing machines in Maude with states the form *machine*: TM *state*: q *tapeLeft*: tape_1 *head*: symbol *tapeRight*: tape_2 , where TM is a Maude representation of the (transitions of the) Turing machine, q is the current “state” of the machine, tape_1 and tape_2 represent, respectively, the tape to the left and to the right of the current “head,” and symbol is the symbol on the square the head is pointing at.

1. Assume sorts *Symbol* and *State*. Define the data types *TuringMachine*, representing a Turing machine, and *Tape*, representing tapes of a Turing machine.
2. Define the rewrite rules for simulating the steps of a given Turing machine.

8.3 Concurrency

Different actions may take place *concurrently*, i.e., at the same time, in a distributed system. Rewriting logic is a logic of change in which the statements have the form

“state t may evolve to a state t' .”

In addition, rewriting logic is a logic for *reasoning* about possible *concurrent* computation steps which allows us to reason about properties of the form

“the system in state t may perform actions concurrently to reach a state t' in *one concurrent step*.”

One way to think about “possible concurrent computation steps” is: assume that we have as many processors as we want and a way of delegating jobs to different processors. What actions could under this scenario be performed at the same time?

8.3.1 Sideways Concurrency

Assume that from state t_1 a system may evolve in *one* step to state u_1 . (If it helps your intuition, imagine that each action takes, say, 10 minutes to perform.) Assume furthermore that a state t_2 could evolve to a state u_2 in one step (which may also take 10 minutes). It then seems reasonable that a state $f(t_1, t_2)$ could evolve to the state $f(u_1, u_2)$ in one *concurrent* step, in which the steps $t_1 \rightarrow u_1$ and $t_2 \rightarrow u_2$ have been computed in parallel. (I emphasize that this is abstract reasoning about *possible* concurrent computations. A concrete implementation on a distributed architecture would have to take care of the task of distributing the two computation tasks to two processors, of synchronizing the results, and so on.)

Example 8.2. The computation of an expression

```
squareroot(9762385199087) + findPrime(13852379)
```

could obviously be distributed so that one processor could spend, say, 15 minutes on computing `squareroot(9762385199087)`, and another processor could be assigned to compute `findPrime(13852379)` in the same time. That is, if `squareroot(9762385199087)  $\rightarrow m$` and `findPrime(13852379)  $\rightarrow n$` , for some numbers m and n , can be computed in one step each, then

```
squareroot(9762385199087) + findPrime(13852379)
```

could evolve to a state $m+n$ in one concurrent step. ◇

This observation can be generalized. If t_1 can be computed to u_1 in one step, t_2 can be computed to u_2 in one step, $\dots$, and t_n can be computed to u_n in one step, then there should be a concurrent step taking a state $f(t_1, \dots, t_n)$ to the state $f(u_1, \dots, u_n)$. José Meseguer calls this kind of concurrency “sideways concurrency.”

We can think of this as getting a term $f(t_1, \dots, t_n)$, and then having the *possibility* of letting one processor “compute” t_1 , another processor t_2 , etc., and that they then report back their respective values $u_1, \dots, u_n$. The processor getting the task of dealing with t_i may itself use other processors to compute subparts of t_i concurrently. That is, the step $t_i \rightarrow u_i$ may itself be a concurrent step.

Example 8.3. Consider the following specification:

```
mod CONC-1 is
  sort s .
  ops a a' b b' c c' d d' e e' f f' : -> s [ctor] .
  op g : s s -> s [ctor] .
  op h : s s s -> s [ctor] .
  rl [11] : a => a' .           rl [14] : d => d' .
  rl [12] : b => b' .           rl [15] : e => e' .
  rl [13] : c => c' .           rl [16] : f => f' .
endm
```

A concurrent step takes $g(a, b)$ to $g(a', b')$ (just let one processor compute $a \rightarrow a'$ and another processor compute $b \rightarrow b'$). Similarly, there is a concurrent step $g(c, d)$ to $g(c', d')$ and another concurrent step $g(e, f)$ to $g(e', f')$. Furthermore, there is a concurrent step

$$h(g(a, b), g(c, d), g(e, f)) \rightarrow h(g(a', b'), g(c', d'), g(e', f'))$$

in which six actions are performed concurrently. $\diamond$

Example 8.4. Our specification `ONE-PERSON` simulates only one person. In this example we consider an entire *population*, which is modeled as a multiset of persons:

```
mod POPULATION is protecting NAT + STRING .
  sorts Person Population Status .
  subsort Person < Population .
  op empty : -> Population [ctor] .
  op __ : Population Population -> Population
                                     [ctor assoc comm id: empty] .
  op person : String Nat Status -> Person [ctor] .
  ops single divorced : -> Status [ctor] .
  ops engaged separated married : String -> Status [ctor] .

  vars X X' : String .   vars M N : Nat .   vars S S' : Status .

  crl [birthday] :
    person(X, N, S)  =>  person(X, N + 1, S)  if N <= 1000 .

  crl [engagement] :
    person(X, N, S)          person(X', M, S')
  =>
    person(X, N, engaged(X')) person(X', M, engaged(X))
    if (S == single or S == divorced) /\ N >= 16
       /\ (S' == single or S' == divorced) /\ M >= 16 .

  rl [wedding] :
    person(X, N, engaged(X')) person(X', M, engaged(X))
  =>
    person(X, N, married(X')) person(X', M, married(X)) .
  ...
endm
```

An example of a population is

```
person("Claudius", 60, married("Gertrude"))
person("Gertrude", 50, married("Claudius"))
person("Hamlet", 28, single)
person("Ophelia", 19, single)
person("Old Norway", 67, married("Ingrid"))
person("Fortinbras", 40, single)
person("Laertes", 22, single).
```

There is a concurrent step from

```
person("Hamlet", 28, single)  person("Ophelia", 19, single)
```

to a state

```
person("Hamlet", 29, single)  person("Ophelia", 20, single)
```

in which *two* birthday steps have been performed at the same time. (The above state has the “form” $f(a, b)$, where $a \longrightarrow a'$ and $b \longrightarrow b'$ can be seen as the two birthday steps and f as the multiset union operator $_{_}$.)

From a state

```
person("Hamlet", 28, single)      person("Ophelia", 19, single)
person("Rosencrantz", 38, single)  person("Juliet", 16, single)
```

it should be possible to arrange two engagements concurrently, e.g., to the state

```
person("Hamlet", 28, engaged("Rosencrantz"))
person("Ophelia", 19, engaged("Juliet"))
person("Rosencrantz", 38, engaged("Hamlet"))
person("Juliet", 16, engaged("Ophelia")).
```

However, it should not be possible for one person (say, “Ophelia”) to be involved in two engagements at the same time (which reception should she attend?). It is of course also possible to go from a state

```
person("Hamlet", 28, single)      person("Ophelia", 19, single)
person("Rosencrantz", 38, single)  person("Juliet", 16, single)
```

to a state

```
person("Hamlet", 28, engaged("Ophelia"))
person("Ophelia", 19, engaged("Hamlet"))
person("Rosencrantz", 38, single)  person("Juliet", 16, single).
```

That is, not everyone has to engage in festivities in a rewrite step. $\diamond$

8.3.2 Nested Concurrency

Let $l : f(x) \longrightarrow g(x)$ be a rewrite rule. Then, an action takes “ f ” to “ g ” no matter what x is. Therefore, it should be possible to let a processor “work on” x while another processor takes f to g . For example, if $l' : a \longrightarrow b$ is another rewrite rule, then a term $f(a)$ should be able to rewrite to $g(b)$ in one concurrent step. We can think of this as a processor seeing $f(\dots)$, and knowing that it can take f to g in one step. It can “delegate” to another processor the task of concurrently working on the “interior.” This kind of concurrency is sometimes called *nested concurrency*.

Exercise 127 What concurrent steps are possible from $g(g(a, a), g(b, c))$ and $h(a, b', g(c, d))$ in the specification in Example 8.3?

Exercise 128 Complete the specification *POPULATION* in Example 8.4 by giving rewrite rules for separation, divorce, and death. (Don't worry about marriage being something between a male and a female.) As there is no status *deceased*, death should result in removal from the population.

Exercise 129

1. What is the largest number of “actions” that can be performed concurrently in one step starting from the state

```

person("Claudius", 60, married("Gertrude"))
person("Gertrude", 50, married("Claudius"))
person("Hamlet", 28, single)
person("Ophelia", 19, single)
person("Old Norway", 67, married("Ingrid"))
person("Fortinbras", 40, single)
person("Laertes", 22, single)

```

2. What is the largest number of concurrent actions possible in one step from the above state if we do not count birthdays and deaths?
3. Is it possible to reach a state in which "Ophelia" is older than "Hamlet" from the above state?

Exercise 130 How many actions (rule applications) can be performed in one step from a state $f(f(f(a)))$ in the specification $\{l_1 : f(x) \longrightarrow g(x), l_2 : a \longrightarrow b\}$?

8.4 Deduction in Rewriting Logic

This section formally defines the rewrite relation and the notion of concurrent rewrite steps. For simplicity of exposition, we consider *one-sorted specifications* without *conditional* rewrite rules.

Given a rewriting logic specification $\mathcal{R} = (\Sigma, E, L, R)$ the *sequents* (“logical formulas”) of rewriting logic have the form

$$t \longrightarrow u$$

for t and u terms in $\mathcal{T}_\Sigma(X)$ belonging to sorts of the same connected component. This sequent intuitively means that it is *possible* to reach the state u from the state t using the rules in $\mathcal{R}$ (zero or more times).

Notation. I sometimes write $t(x_1, \dots, x_n)$ for a term t to emphasize that all the variables in t are in the list $x_1, \dots, x_n$. I write $t(u_1/x_1, \dots, u_n/x_n)$ for the term t where each occurrence of x_i has been replaced by the term u_i . For example, if t is $f(g(x), h(a, y))$, then $t(g(y)/x, a/y)$ denotes the term $f(g(g(y)), h(a, a))$.

Definition 8.2 (Deduction rules of rewriting logic) *The sequent*

$$t \longrightarrow u$$

holds in a rewriting logic specification $\mathcal{R} = (\Sigma, E, L, R)$ (which we for simplicity assume is one-sorted and has no conditional rules), also written $\mathcal{R} \vdash t \longrightarrow u$, if and only if $t \longrightarrow u$ can be obtained by finite application of the following deduction rules:

Reflexivity: $t \longrightarrow t$ holds for each term t in $\mathcal{T}_\Sigma(X)$.

Equality: If $t \longrightarrow t'$ and $E \vdash t = u$ and $E \vdash t' = u'$ hold, then $u \longrightarrow u'$ also holds.

Congruence: For each function symbol f in Σ , if $t_1 \longrightarrow u_1, \dots$, and $t_n \longrightarrow u_n$ all hold, then $f(t_1, \dots, t_n) \longrightarrow f(u_1, \dots, u_n)$ also holds.

Replacement: For each rewrite rule $l: t(x_1, \dots, x_n) \longrightarrow u(x_1, \dots, x_n)$ in R , if $t_1 \longrightarrow u_1, \dots$, and $t_n \longrightarrow u_n$ all hold, then

$$t(t_1/x_1, \dots, t_n/x_n) \longrightarrow u(u_1/x_1, \dots, u_n/x_n)$$

also holds.

Transitivity: If $t_1 \longrightarrow t_2$ and $t_2 \longrightarrow t_3$ both hold, then $t_1 \longrightarrow t_3$ also holds.

These deduction rules look very similar to the deduction rules of equational logic (with *Replacement* corresponding to *Substitutivity*). Indeed, only the *Symmetry* property of equational logic is missing.

The rewrite relation $\longrightarrow$ corresponds to applying rewrite rules from left to right zero or more times, and to equational reduction in the following sense:

Proposition 8.1. *Given an equational specification (Σ, E) , it is easy to see (Exercise 131) that*

$$t \overset{*}{\longrightarrow}_E u \quad \text{if and only if} \quad (\Sigma, \emptyset, \{l\}, \text{rules}(E)) \vdash t \longrightarrow u$$

(where $\text{rules}(E)$ transforms each equation $t_1 = t_2$ into a rewrite rule $l: t_1 \longrightarrow t_2$).

The following fact follows trivially from Theorem 6.3 and Proposition 8.1:

Corollary 8.1. *It is in general undecidable whether a given term t rewrites to a given term u in a given rewriting logic specification $\mathcal{R}$.*

8.4.1 Concurrent Steps

The *Congruence* rule corresponds to sideways concurrency: if $a \longrightarrow b$ and $c \longrightarrow d$ can be performed in “one step,” then $f(a, c) \longrightarrow f(b, d)$ holds and can be performed in one sideways concurrent step. The *Replacement* rule models nested concurrency, where an “outer” step applies a rule, and an “inner” step performs actions on the variables of the rule. If $l: f(x, y) \longrightarrow g(x, y)$ is a rule, and $a \longrightarrow b$ and $c \longrightarrow d$ can be performed in “one step,” then it is reasonable to assume that $f(a, c) \longrightarrow g(b, d)$ can be performed in one nested concurrent step. These observations motivate the formal definition of concurrent steps:

Definition 8.3 (Concurrent Rewrite Steps)

- A sequent $t \longrightarrow u$ is called a one-step concurrent rewrite if it can be obtained using only the rules *Reflexivity*, *Equality*, *Congruence*, and *Replacement* of rewriting logic. (That is, *Transitivity* cannot be used in a one-step concurrent rewrite.)
- A one-step concurrent rewrite is called a (one-step) sequential rewrite if the *Replacement* rule (which is where a rule is “applied”) is used once in the deduction.

In a one-step *sequential* rewrite, some rule is applied once, which means that a one-step sequential rewrite corresponds to one-step equational simplification:

$t \rightsquigarrow_E u$ if and only if $(\Sigma, \emptyset, \{l\}, \text{rules}(E)) \vdash t \longrightarrow u$ is a one-step sequential rewrite.

Example 8.5. The rewriting logic specification⁵ $\{l_1 : f(x) \longrightarrow g(x), l_2 : a \longrightarrow b\}$ has a one-step concurrent rewrite $f(f(f(a))) \longrightarrow g(g(g(b)))$ where rule l_1 has been applied three times and rule l_2 once, since the proof

$$\frac{\frac{\frac{\frac{\overline{a \longrightarrow b}}{\text{Replacement}}}{f(a) \longrightarrow g(b)}{\text{Replacement}}}{f(f(a)) \longrightarrow g(g(b))}{\text{Replacement}}}{f(f(f(a))) \longrightarrow g(g(g(b)))}{\text{Replacement}}$$

does not use the *Transitivity* rule. ◇

Example 8.6. There is a one-step concurrent rewrite

$$h(g(a, b), g(c, d), g(e, f)) \longrightarrow h(g(a', b'), g(c', d'), g(e', f'))$$

in the module `CONC-1` in Example 8.3, since the proof

$$\frac{\frac{\frac{\overline{a \longrightarrow a'}}{g(a, b) \longrightarrow g(a', b')}}{\frac{\overline{b \longrightarrow b'}}{g(c, d) \longrightarrow g(c', d')}}}{\frac{\frac{\overline{c \longrightarrow c'}}{g(e, f) \longrightarrow g(e', f')}}{h(g(a, b), g(c, d), g(e, f)) \longrightarrow h(g(a', b'), g(c', d'), g(e', f'))}}$$

only uses *Replacement* and *Congruence*. It is also possible to prove that

$$h(g(a, b), g(c, d), g(e, f)) \longrightarrow h(g(a', b), g(c, d), g(e, f))$$

is a one-step sequential rewrite, since $b \longrightarrow b$, $c \longrightarrow c$, etc., hold by *Reflexivity*. ◇

The example above and Exercise 132 indicate that a concurrent step may be decomposed into a number of sequential steps:

Proposition 8.2 (Sequentializability [80]). *For each concurrent rewrite $t \longrightarrow t'$, either $E \vdash t = t'$ or there is a chain of one-step sequential rewrites*

$$t \longrightarrow t_1 \longrightarrow \cdots \longrightarrow t_n \longrightarrow t'.$$

⁵ We follow the notational conventions for one-sorted equational specifications when writing one-sorted rewriting logic specifications.

A (finite or infinite) sequence of one-step rewrites is called a *behavior* or *run* of a system.

Example 8.7. Consider the following specification of a “swap” operation on lists.

```
mod SORT is protecting INT .
  sort List .
  subsort Int < List .
  op nil : -> List [ctor] .
  op __ : List List -> List [assoc id: nil ctor] .

  vars I J : Int .      var L : List .
  crl [swap] : I L J => J L I if J < I .
endm
```

Repeated use of the `swap` rule (for example by using Maude’s `rew` command explained in Chapter 9) will swap integers until the list is sorted:

```
Maude> rew 8 4 0 -3 76 54 21 0 -9 3 23 .
```

```
result List: -9 -3 0 0 3 4 8 21 23 54 76
```

The list `0 3 2 1` can be sorted in *one* concurrent step, since $0\ 3\ 2\ 1 \longrightarrow 0\ 1\ 2\ 3$ can be proved as follows:

1. $0 \longrightarrow 0$, $3 \longrightarrow 3$, $2 \longrightarrow 2$, and $1 \longrightarrow 1$ are all deducible in rewriting logic because of *Reflexivity*.
2. The *Replacement* deduction rule used on the rule `swap` with $3 \longrightarrow 3$ “for” `I`, $2 \longrightarrow 2$ for `L`, and $1 \longrightarrow 1$ for `J` gives that $3\ 2\ 1 \longrightarrow 1\ 2\ 3$ is deducible.
3. *Congruence* (w.r.t. the function symbol `__`) together with the proven assumptions $0 \longrightarrow 0$ and $3\ 2\ 1 \longrightarrow 1\ 2\ 3$ gives $0\ (3\ 2\ 1) \longrightarrow 0\ (1\ 2\ 3)$, which due to the `assoc` attribute of `__` can be written $0\ 3\ 2\ 1 \longrightarrow 0\ 1\ 2\ 3$. $\diamond$

8.4.2 Termination and Confluence

The notions of *termination* and *confluence* carry over to rewrite systems as expected: a rewriting logic specification is terminating if (the underlying equational specification is terminating and) there is no infinite sequence of one-step rewrites. Likewise, a rewriting logic specification is confluent if and only if $t \longrightarrow t_1$ and $t \longrightarrow t_2$ imply that there is a term u such that both $t_1 \longrightarrow u$ and $t_2 \longrightarrow u$ hold.

Exercise 131 Prove Proposition 8.1.

Exercise 132 Show a deduction of $f(f(f(a))) \longrightarrow g(g(g(b)))$ in the specification of Example 8.5 which involves only sequential rewrite steps (and Transitivity).

Exercise 133 Recall the coffee bean game described in Section 8.2.3 and your Maude specification of it which solved Exercise 122.

1. Prove formally that the state (representing) $\circ \bullet \bullet \circ$ rewrites in one sequential step to the state (representing) $\circ \circ \circ$.
2. What is the highest number of “rule applications” that can be performed in one concurrent step from the state (representing) $\circ \circ \bullet \bullet \circ \bullet \circ \bullet$. What is the resulting state? Does it make sense? That is, do you think that it is natural to delegate the job to different processors in this way (you can again think that every step takes 10 minutes to perform!)?

Exercise 134 Consider the specifications *CHOICE-INT*, *ONE-FOOTBALL-GAME*, your extensions of *ONE-PERSON* and *POPULATION*, the three versions of the whiteboard game in Exercise 123, and your “Tower of Hanoi” and traveling salesman specifications. Which of them have terminating, respectively confluent, rewrite rules?

Exercise 135 Which/how many “actions” can be performed concurrently in: (i) the different versions of the whiteboard game with seven numbers on the whiteboard; and (ii) your “Tower of Hanoi” specification with five rods and seven disks?

Exercise 136 Consider the specification *SORT* in Example 8.7.

1. Prove formally using the deduction rules of rewriting logic that

$$4 \ 8 \ 5 \ 0 \ 1 \longrightarrow 0 \ 1 \ 4 \ 5 \ 8.$$

2. Prove that *SORT* is terminating.
3. Is it possible to sort the list $8 \ 4 \ 0 \ -3 \ 76 \ 54 \ 21$ in one concurrent step? How about the list $1 \ 3 \ 2 \ 0$?
4. An even shorter “sorting program” replaces the *swap* rule with the rule

`cr1 [swap] : I J => J I if J < I .`

- a. What is the smallest number of concurrent rewrite steps required to sort the lists $8 \ 4 \ 0 \ -3 \ 76 \ 54 \ 21$ and $1 \ 3 \ 2 \ 0$ in the modified specification?
- b. Is there any list which can be sorted by fewer steps in the modified specification than in the original one?
5. What would be the undesired consequence of adding an equationally-defined function `op sorted : List -> Bool` to the module *SORT*? (The expression `sorted(l)` reduces to `true` if *l* is a sorted list, and to `false` otherwise.) Hint: think about the Congruence rule of rewriting logic. See also Section 8.5.

8.5 * Frozen Operators

Consider extending the module *SORT* in Example 8.7 with a function

`op first : List -> Int .`

which returns the first element in a list. This function should be defined equationally. In the (extended) module `SORT` there is a rewrite $5\ 2 \longrightarrow 2\ 5$. Using the *Congruence* rule of rewriting logic it follows that $\text{first}(5\ 2) \longrightarrow \text{first}(2\ 5)$, which by the *Equality* rule of rewriting logic gives $5 \longrightarrow 2$, which seems undesirable. To avoid such undesired rewrites caused by functions mapping a “dynamic” domain onto a “static” domain, one can declare such functions to be *frozen*:

```
op first : List -> Int [frozen] .
```

Semantically, this frozenness means that a rewrite $t \longrightarrow t'$ does not allow us to deduce $\text{first}(t) \longrightarrow \text{first}(t')$; that is, the *Congruence* rule of rewriting logic does not apply to frozen operators. It is also possible to specify that only *some* of the argument places (e.g., the second argument) of a function are frozen.

8.6 * Denotational Semantics

The intended model of an equational specification is the *initial algebra* of the specification. In an algebra, (the interpretation of) two expressions either denote the *same* element in the domain of the algebra, or *different* elements with no relationship between them. What is the intended model of a rewrite theory? The (interpretation of) two terms t and t' may denote different values, but could still be related by rewriting: $t \longrightarrow t'$. Therefore, algebraic models, whose domains are sets with no relationship between different elements in the set, may not be the best models. Instead, the models of rewrite theories are *categories*, which are sets with *arrows* between elements.

Definition 8.4 (Category) A category $\mathcal{A}$ is a pair $(\mathbf{A}, \mathbf{M})$, where $\mathbf{A}$ is a set (of objects) and $\mathbf{M}$ is set of morphisms (or arrows) $f : A \rightarrow B$, for $A, B \in \mathbf{A}$, such that:

1. If $f : A \rightarrow B$ and $g : B \rightarrow C$ are two morphisms in $\mathbf{M}$, then there is a designated composite morphism $f;g : A \rightarrow C$ in $\mathbf{M}$;
2. each object A has an identity morphism $\text{id}_A : A \rightarrow A$ such that $\text{id}_A;f = f$ and $g;\text{id}_A = g$ for any morphisms $f : A \rightarrow B$ and $g : B \rightarrow A$; and
3. morphism composition is associative: $(f;g);h = f;(g;h)$ for all morphisms $f : A \rightarrow B$, $g : B \rightarrow C$, and $h : C \rightarrow D$.

That is, there must be an arrow from an object to itself, and arrows compose. As the reader may have guessed, the initial model $\mathcal{T}_{\mathcal{R}}$ of a rewrite theory is a category, whose objects are the elements of the underlying initial algebra $\mathcal{T}_{\Sigma,E}$, and where there is a morphism $p : t \rightarrow t'$ if and only if $t \longrightarrow t'$ (the p is the “proof term” representing the proof of $t \longrightarrow t'$). In particular, because of *Reflexivity* of rewriting logic, there is an arrow from each t to itself, and because of *Transitivity*, arrows compose.

It is beyond the scope of this book to further discuss the models of rewrite theories; a thorough exposition is given in [80]. A non-categorical model theory for rewriting logic with frozen operators is defined in [16].

This chapter introduces some ways in which a rewriting logic model of a dynamic system can be analyzed by execution in Maude.

Since an *equational* specification is assumed to be terminating and confluent, and the main goal is to compute the normal form of an expression, such specifications can be executed by applying equations until no equation can be applied, without worrying about *which* equation to apply or *where* to apply it. *Rewriting logic* (or just *rewrite*) specifications, on the other hand, model *all* possible behaviors of a dynamic system, and might not be terminating or confluent. The above execution approach may therefore not make much sense for rewrite specifications.

This chapter discusses the following ways of executing a rewrite specification in Maude. Chapter 16 explains how Maude can be used to analyze whether each behavior of a system satisfies a *temporal logic* formula.

1. The Maude commands `rew` (or `rewrite`) and `frew` (“fair rewrite”) “simulate” one of the many possible system behaviors from a given initial state of the system. This is done by applying rewrite rules to the state, starting with the initial state. Since this process may not terminate, the user can give an upper bound on the number of rewrite steps to perform.
2. Maude’s `search` command uses a breadth-first search strategy to check whether a given state pattern *can* be reached from the initial system state.

9.1 Executing One Sequential Rewrite Step

Although rewriting logic allows reasoning about *concurrent* rewrites, the Maude system only executes one-step *sequential* rewrites, i.e., applying a rewrite rule once in each step. No rewrites are lost by this approach, since, by Proposition 8.2, a concurrent rewrite can be decomposed into a sequence of *one-step sequential* rewrites.

It is fairly easy to see that applying a rewrite rule when there are no equations in the specification is the same as applying an equation in the “corresponding equational specification.” The problem of executing a rewrite rule boils down to dealing with the *Equality* rule of rewriting logic. Even checking whether a rule $l \longrightarrow r$ applies to the *root* (or *top*) position of a term t requires checking whether there is a substitution σ such that $E \vdash l\sigma = t$, for E the equational part of the rewriting logic specification, which is in general undecidable.

Example 9.1. Given a Maude specification

```
mod NON-COHERENT is
  sort s .      ops a b d : -> s .      ops c f : -> s [ctor] .
  eq a = b .
  eq b = c .
  eq d = f .
  rl [1] : b => d .
endm
```

Both a and c should rewrite to d , since $a \longrightarrow d$ and $c \longrightarrow d$ (because a, b , and c are equivalent according to the equations). However, Maude cannot immediately “see” that rule 1 applies to a and c , but would have to check whether b is equal to a (resp. c) to decide whether the rule can be applied to a (resp. c). $\diamond$

While the search for E -equivalent forms in the above small example does not seem disastrous, this check whether a rule can be applied to a term is in general undecidable. Maude therefore assumes that the equational part E of a rewriting logic specification is confluent and terminating, and *first* reduces a term t to its E -normal form $t!$ using the equations in the specification, and *then* checks whether a rewrite rule can be applied to $t!$. If so, the rewrite rule is applied, and the resulting term t' is normalized to $t'!$. To avoid “losing” rewrites when applying rewrite rules in this way, the *left-hand side of a rewrite rule should be a constructor term*.

Example 9.2. The left-hand side of the rule in the specification NON-COHERENT is *not* a constructor term, and Maude will first reduce a state a to c , and will then check whether a rule applies. Therefore, Maude “misses” the rewrite $a \longrightarrow d$. The rule $\text{rl [1] : } b \Rightarrow d$ should therefore be replaced by the rule $\text{rl [1] : } c \Rightarrow d$. $\diamond$

Example 9.3. Applying rule `birthday` on `person("Edward", 21+11, single)` is done by reducing the state to its normal form `person("Edward", 32, single)`, then applying the rule, giving `person("Edward", 32+1, single)`, which is normalized to `person("Edward", 33, single)` using the (built-in) equations for $+$. $\diamond$

Example 9.4. A rule

```
rl [gettingYounger] : person(X, N + 1, S) => person(X, N, S) .
```

would never be applied, since a term would first be normalized to a form such as `person("Gilgamesh", 50, married)`. The problem is that the left-hand side of the rule is not a constructor term since it contains the defined symbol $+$. A better rule is

```
rl [gettingYounger] : person(X, s N, S) => person(X, N, S) .  $\diamond$ 
```


A conjunct in the condition of a rewrite rule (or an equation) may also have the form

$$x := t$$

where x is a variable which does *not* appear in the left-hand side of the rule. Logically, this is just an equational condition with the same logical meaning as $x = t$. Operationally, it instantiates the variable x to the value to which the corresponding instance of t is evaluated by the equations E . While it does not make much sense in our simple example, our birthday rule could have been written

```
var X : String .   vars M N : Nat .
crl [birthday] : person(X, N) => person(X, M)  if M := N + 1 .
```

Conjunctions of conditions are evaluated from left to right, and while one can have more than one instantiation of new variables, in each conjunct of the condition, all the variables (except those being instantiated in the conjunct) must have appeared in the left-hand side of the rule or must have been instantiated in earlier conjuncts.

The left-hand side of such a matching equation in a condition does not need to be just a variable, but could have the form

$$t(x_1, \dots, x_n) := t'$$

for any *constructor* term t with variables $x_1, \dots, x_n$. The conjunct holds if there are terms $t_1, \dots, t_n$ such that $t(t_1/x_1, \dots, t_n/x_n)$ equals the normal form of t' ; if that is the case, then each x_j gets assigned the value t_j .

Finally, the truth of a rewrite condition

```
... if ... /\ u => u' /\ ...
```

cannot be determined by just computing “normal forms.” Maude must search (in a breadth-first way) all computation paths from (the instance of) u to check if (the corresponding instance of) u' can be reached. If u' cannot be reached from u , Maude might search forever just to determine whether the rule can be applied!

9.2 Simulating Single Behaviors

Maude’s `rew` and `frew` commands are used to execute *a single* behavior of a system. These commands apply rewrite rules to perform one-step sequential rewrites until no rule can be applied, or until a user-given bound on the number of rewrites has been reached. The execution could go on forever if the specification is nonterminating and the user does not provide an upper bound on the number of rewrites.

Since each term is reduced to its E -normal form before a rewrite rule is applied, a finite Maude execution with the `rew` or `frew` command has the form

$$t \xrightarrow{*} t! \longrightarrow t_1 \xrightarrow{*} t_1! \longrightarrow t_2 \xrightarrow{*} t_2! \longrightarrow \dots \longrightarrow t_n \xrightarrow{*} t_n!$$

and returns the term $t_n!$. Such an execution is often referred to as “simulating one behavior” of the system. Giving the Maude command

```
set trace on .
```

before running the `rew` command shows all the intermediate steps in the execution.

The syntax for the `rew` command is `rew  $t$  .` or `rew [ $n$ ]  $t$  .`, where n is the maximum number of rewrite steps to execute, and t is the term to rewrite. The `frew` command has similar syntax. In case the execution should take place in a module different from the “current” module, one can specify in which module the rewrite should take place:

```
Maude> rew [100] in ONE-PERSON : person("Peter", 46, married) .
result Person: person("Peter", 146, married)
```

Since the specification is not (necessarily) confluent, the choice of *which* rule to apply in each step, and *where* in the term to apply it, is important, as different choices give different results. Both the `rew` and the `frew` commands try to apply the rules in a “round-robin” format. However, the highest priority of `rew` is to apply rules as close to the “top” of the term as possible, and thereafter to apply the rules to the *leftmost* subterms. The `frew` command is more “fair” w.r.t. *where* in the term to apply the rules. Both `rew` and `frew` are deterministic in the sense that two `frew` executions starting with the same initial term will give the same result.

Example 9.5. The following examples compare the rewrite commands `rew` and `frew`. Counters of the form `rule2( $n$ )` indicate that `rule2` was applied n times. Both `rew` and `frew` choose rules in a “fair” way when all rewrites happen at the top:

```
mod TEST-REW1 is protecting NAT .
  sort Counter .
  ops rule1 rule2 rule3 : Nat -> Counter [ctor] .
  op f : Counter Counter Counter -> Counter [ctor] .
  vars N M K : Nat .
  rl [rule1] : f(rule1(N), rule2(M), rule3(K))
              => f(rule1(s N), rule2(M), rule3(K)) .
  rl [rule2] : f(rule1(N), rule2(M), rule3(K))
              => f(rule1(N), rule2(s M), rule3(K)) .
  rl [rule3] : f(rule1(N), rule2(M), rule3(K))
              => f(rule1(N), rule2(M), rule3(s K)) .
endm
```

```
Maude> rew [100] f(rule1(0), rule2(0), rule3(0)) .
result Counter: f(rule1(34), rule2(33), rule3(33))
```

```
Maude> frew [100] f(rule1(0), rule2(0), rule3(0)) .
result Counter: f(rule1(34), rule2(33), rule3(33))
```

In both cases, `rule1` was applied 34 times and the other rules 33 times each. The application of the rules seems less fair when the rewrites happen in a subterm, since `rew` applies rules in a leftmost-outermost way, while `frew` is fair also w.r.t. giving each subterm a chance to rewrite:

```

mod TEST-REW2 is protecting NAT .
  sort Counter .
  ops rule1 rule2 rule3 : Nat -> Counter [ctor] .
  op f : Counter Counter Counter -> Counter [ctor] .
  var N : Nat .
  rl [rule1] : rule1(N) => rule1(s N) .
  rl [rule2] : rule2(N) => rule2(s N) .
  rl [rule3] : rule3(N) => rule3(s N) .
endm

Maude> rew [100] f(rule1(0), rule2(0), rule3(0)) .
result Counter: f(rule1(100), rule2(0), rule3(0))

Maude> rew [100] f(rule3(0), rule2(0), rule1(0)) .
result Counter: f(rule3(100), rule2(0), rule1(0))

Maude> frew [100] f(rule1(0), rule2(0), rule3(0)) .
result (sort not calculated): f(rule1(34), rule2(33), rule3(33))

Maude> frew [100] f(rule1(0), rule1(0), rule1(0)) .
result (sort not calculated): f(rule1(34), rule1(33), rule1(33))

```

Since `rew` first looks at the leftmost subterm, it always rewrites the rules that are applicable there, while `frew` tries to apply rules in all subterms. $\diamond$

Exercise 137 Declare an associative (*assoc*) and commutative (*comm*) choice operator `_?_` and use only one rewrite rule so that e.g. the term `1 ? 2 ? 3 ? 4` can change to either `1`, `2`, `3`, or `4`. Use Maude's `rew` and `frew` commands to test which element is chosen from the terms `1 ? 2 ? 3 ? 4` and `6 ? 2 ? 3`.

Exercise 138 The module `ONE-FOOTBALL-GAME` is a nonterminating specification where games are never stopped.

1. Simulate a football game with 15 scoring actions in Maude.
2. Add a rule which gives the possibility of “stopping” the game at any time and displaying the final score as a term of the form

"49ers" vs "Giants" FinalScore: 39 : 38

3. Explain why the resulting specification is not terminating.

Exercise 139 Another version of the coffee bean game has the following rules:

$$\begin{array}{ll}
 \bullet \bullet \longrightarrow \circ \circ \circ \circ & \bullet \circ \longrightarrow \circ \circ \circ \bullet \\
 \circ \bullet \longrightarrow \bullet & \circ \circ \longrightarrow \circ
 \end{array}$$

1. Specify this game in Maude and play it in Maude. Does it always terminate?
2. Prove that the game is nonterminating or prove that it is terminating.
3. Is the game confluent?
4. If the game is confluent and terminating, what is the result of playing the game?

Exercise 140 Execute your specifications of all the whiteboard games in Exercise 123 with both `rew` and `frew` on an initial state with the numbers 2, 11, 21, 27, 77, and 85. Who ends up with the smallest number: you or Maude?

Exercise 141 Simulate your “Tower of Hanoi” specification with four rods and five disks for at most 1000 rewrite steps. Does Maude find the right solution?

Exercise 142 Execute your Traveling Salesman specifications from Exercise 125 with `rew` and `frew`. Does Maude select a trip with cost less than 21?

Exercise 143 Execute your Turing machine simulator from Exercise 126, for example on the Turing machines solving Exercises 51 and 52.

9.3 Search

While using the `rew` and `frew` commands to execute one out of possibly many different behaviors can be very useful for a first prototyping of a specification, such executions may not be sufficient to deeply understand a specification. For example, no matter how many times we execute the module `ONE-FOOTBALL-GAME`, the home team *never* loses. After many such tests one could therefore be tempted to conclude that “the visiting team cannot win a football game,” which is clearly wrong. We therefore need to be able to analyze specifications further.

Maude provides a *search* command which searches through all behaviors from a given initial state and returns all—or a user-given number of—states which can be reached from the initial state and which satisfy the given search condition. The search may be restricted to analyze all behaviors up to n rewrite steps.

Maude’s search command searches in *breadth-first* way through all behaviors from the initial state. That is, Maude first visits all terms reachable in *one* (sequential) rewrite step from the initial state, then it visits all states reachable in *two* steps from the initial state, and so on. Maude stores the visited states and ignores states which have been visited earlier during the search. This kind of search may not terminate if an infinite number of states are reachable from the initial state.

The basic forms of the search command are

```
search  $t_0$  arrow pattern .
```

and

```
search  $t_0$  arrow pattern such that cond .
```

The term t_0 is the initial state, *pattern* is a *constructor* term which can contain variables, and *cond* is a condition which has the same form as a condition of an equation. A term t satisfies the search condition if *pattern* matches t and *cond* holds for the matching substitution. The *arrow* is either `=>1`, `=>*`, `=>+`, or `=>!` and indicates in how many (sequential) rewrite steps the desired terms are to be found:

- =>1: states which can be reached in *exactly one* step from the initial state t_0 ;
- =>*: states reachable in *zero or more* steps from t_0 ;
- =>+: states reachable in *one or more* steps from t_0 ; and
- =>!: states that cannot be further rewritten.

Example 9.6. The command

```
Maude> search person("Babko", 84, widow) =>1 P:Person .
```

searches for all states reachable in *one* step from `person("Babko", 84, widow)` that match the variable `P:Person`. (Remember that variables of the form *name : sort* can be used without being explicitly declared. A search pattern can use both such undeclared variables and variables declared in the module being analyzed.) The variable `P:Person` matches all terms of sort `Person`, so the command searches for all states reachable in one step from `person("Babko", 84, widow)`. The output from a search is all the matching substitutions:

```
Solution 1 (state 1)
P:Person --> person("Babko", 84, deceased)
```

```
Solution 2 (state 2)
P:Person --> person("Babko", 85, widow)
```

No more solutions.

(It seems that my specification does not allow widows to remarry.)

To find out what could happen to "Edward" when he is 35 years old one may give the command

```
Maude> search person("Edward", 32, single) =>* person("Edward", 35, S) .
```

which gives eight matching substitutions (two of which are shown):

```
Solution 1 (state 9)
S --> single
...
Solution 8 (state 46)
S --> divorced
```

No more solutions.

We can also check whether a person can become younger:

```
Maude> search person("Edward", 32, single) =>* person("Edward", N, S)
      such that N < 32 .
```

No solution.

Finally, one may be interested in how it may end; that is, what are the possible final states from which nothing more will happen?

```
Maude> search person("Peter", 46, married) =>! P:Person .
```

◇

The command

```
show path n .
```

outputs the shortest rewrite sequence from the initial state to state number *n* in the previous search, and the command

```
show path labels n .
```

outputs the sequence of rules (represented by their labels) applied in that sequence.

Example 9.7. In Example 9.6 we search for all states where the age of "Edward" is 35. The solution in which this person was `divorced` had number 46. The command `show path 46 .` will then let Maude show the path leading to the `divorced` state:

```
Maude> show path 46 .
```

```
state 0, Person: person("Edward", 32, single)
===[ crl person(X, N, S) => person(X, N+1, S) if N <= 1000 = true
      /\ S /= deceased = true [label birth-day] . ]===>
state 2, Person: person("Edward", 33, single)
===[ crl person(X, N, S) => person(X, N+1, S) if N <= 1000 = true
      /\ S /= deceased = true [label birth-day] . ]===>
...
===[ rl person(X, N, separated) => person(X, N, divorced)
      [label divorce] . ]===>
state 46, Person: person("Edward", 35, divorced)
```

```
Maude> show path labels 46 .
```

```
birth-day
birth-day
birth-day
successful-proposal
marriage
separation
divorce
```



A search (with an arrow different from $\Rightarrow$) will not terminate if there are infinitely many states reachable from the initial state. This is because the search command looks for *all* results. One may therefore put an upper bound on the number of solutions, using the syntax `search [n] ...`, and/or put an upper bound *d* on the number of rewrite steps in the behaviors, using the syntax `search [n, d] ...` and `search [, d] ...`.

Example 9.8. Consider the specification `ONE-FOOTBALL-GAME` on page 132 in which the visiting team could not lead in `rew` and `frew` simulations. To settle the issue of whether the visiting team can lead we could try the command

```
Maude> search [1]
      "Packers" vs "49ers" 0:0 =>* "Packers" vs "49ers" M:N
      such that M < 7 /\ N > 41 .
```

```
Solution 1 (state 691)
M --> 0
N --> 42
```



The execution of a search command may fail to terminate even when we restrict the number of desired solutions. A search for *one* solution will fail if there is no solution and the set of reachable states is infinite.

Example 9.9. The execution of the search command

```
Maude> search [1]
         "49ers" vs "Giants" 39:41 =>* "49ers" vs "Giants" 39:38.
```

will fail to terminate in ONE-FOOTBALL-GAME. Why?



To summarize, because search is performed in a breadth-first way, n desired solutions will be found if there are at least n reachable states satisfying the search condition. Furthermore, Maude will find the n states reachable in the smallest number of rewrite steps (why?). If there are not at least n solutions, then the search will first output the existing solutions, and will then either terminate if only a *finite* number of *distinct* states are reachable from the initial state, or will loop forever (searching for the remaining non-existing solutions) otherwise. Of course, if a bound on the number of rewrites in the behaviors is added, a search command will terminate.

It is worth remarking that—as illustrated in the following chapters—the reachable state space explored by the search command can grow quickly. As a rough estimate, if the system may perform k different actions from any state, then there are more than k^d (not necessarily different) states reachable from the initial state in d rewrite steps. Since Maude stores all the states it has encountered during a search, a Maude search may take long time and could run out of memory.

Exercise 144

1. Assume that Maude instead would search the rewrite paths from the initial state in a “depth-first” way. Could we still guarantee that searching for n solutions would always be successful if there exist at least n solutions?
2. Can you use Maude’s search command to prove that it is impossible to go from the state `person("Gilgamesh", 50, married)` to a state in which the noble man’s age is less than 50, provided the `birthday` rule has no age limit?
3. Explain why it is impossible to implement a search command which always terminates and which can be used to find whether there exists (at least) one reachable state from the initial state that is matched by the search pattern.

Exercise 145 In this exercise you should use Maude’s search command to analyze the coffee bean game described in Section 8.2.3.

1. What are the possible results of the game when starting with the bean sequence

○ ● ○ ○ ○ ● ● ○ ○ ● ● ● ○ ○

Ask Maude to display the run which resulted in the fewest remaining beans.

2. Is it the case that each state reachable from an initial state with an even number of black beans will contain an even number of black beans? Test this on the initial states $\circ \bullet \circ \circ \circ \bullet \bullet \circ \circ \bullet \bullet \circ \circ$ and $\circ \circ \bullet \bullet \circ \bullet \circ \bullet$.
3. Search for all the results of playing the game when the initial state contains an odd number of black beans. Try this for a couple of initial states, such as for example $\bullet \circ \bullet \circ \circ \circ \bullet \bullet \circ \circ \bullet \bullet \circ \circ$ and $\bullet \circ \circ \bullet \bullet \circ \bullet \circ \bullet$. Do you see any pattern in the answers?
4. Check some more examples and suggest whether it is always possible to end up with one coffee bean no matter what the initial state is.

Exercise 146 Use Maude to prove that each nonextensible rewrite sequence in the module `SORT` on page 142 starting with the list $8\ 4\ 0\ -3\ 76\ 54\ 21$ ends with the sorted list $-3\ 0\ 4\ 8\ 21\ 54\ 76$.

Exercise 147 Analyze your solutions of the whiteboard game to see if it possible to end up with a number smaller than 13 or greater than 65, starting from the initial state in Exercise 140.

Exercise 148 There is supposedly no efficient solution of the “Tower of Hanoi” puzzle with four or more rods. Use Maude search to find the behavior which solves the puzzle with the smallest number of moves, for four rods and five disks. Also check whether it is possible to reach a state where a larger disk is on top of a smaller one.

Exercise 149 Use search to analyze your specifications of the Traveling Salesman problem in Exercise 125.

1. Is it possible, in your specification of the standard version of the problem, to reach a non-final state where the salesperson visits a city for the second time?
2. For each of the specifications: is it possible to find a trip with cost less than 17?

Exercise 150 Use search to check whether all executions of your Turing machines in Exercise 143 end with the expected tape/state values.

Exercise 151 Storing all visited states can be a bottleneck in a Maude search.

1. Would not storing all visited states lead to (significantly?) less memory usage in a breadth-first search?
2. What would be the disadvantages of not storing all visited states?

A distributed system can be naturally modeled as an object system by modeling each component of the system as an object. The components may communicate with each other by sending and receiving messages. The state of a system can therefore be represented as a *multiset* of objects and messages traveling between the objects.

This chapter starts by explaining how concurrent objects can be modeled directly in rewriting logic. Section 10.2 introduces *Full Maude*, a Maude interface which supports object-oriented specification by adding “syntactic sugar” for defining classes, messages, objects, and rewrite rules in a more object-oriented style. Section 10.3 shows how the classical *dining philosophers problem* can be specified in Maude in an object-oriented style, and Section 10.4 shows how randomized simulations can be used to simulate different strategies for playing blackjack.

The theoretical foundations of how different object-oriented concepts can be represented in rewriting logic are thoroughly discussed in [81].

10.1 Modeling Concurrent Objects in Maude

One way of modeling an object in Maude is to let a term

$$\langle o : C \mid att_1 : val_1, \dots, att_n : val_n \rangle$$

denote an *object* of class C which has the *name* (or *identifier*) o and *attributes* att_1 to att_n , whose current values are val_1 to val_n , respectively. Continuing our example from Chapter 8, a *Person* object in a certain state could be represented by a term

$$\langle \text{"Edward"} : \text{Person} \mid \text{age} : 32, \text{status} : \text{single} \rangle.$$

Letting a sort *Object* denote objects, a class C can be declared using a constructor

$$\text{op } \langle _ : C \mid att_1 : _, \dots, att_n : _ \rangle : \text{Oid } s_1 \dots s_n \rightarrow \text{Object } [\text{ctor}] .$$

where `Oid` is some sort denoting object identifiers, and s_1 to s_n are the sorts of the attributes att_1 to att_n . A class `Person` may therefore be declared

```
sorts Object Oid .
subsort String < Oid .
op <_: Person | age:_, status:_> : Oid Nat Status -> Object [ctor] .
```

The objects of the above form are then terms of sort `Object`.

A system may also contain messages, which are terms of a sort `Msg`. A distributed system can then be seen as a multiset of objects and of messages traveling between objects. A sort `Configuration` denoting such multisets can be defined as expected:

```
sorts Object Msg Configuration .
subsorts Object Msg < Configuration .
op none : -> Configuration [ctor] .
op __ : Configuration Configuration -> Configuration
                                         [ctor assoc comm id: none] .
```

A term of sort `Configuration` could be for example

```
< "Edward" : Person | age: 32, status: single >
< "Mette" : Person | age: 47, status: married("Rich") >
< "Chrissie" : Person | age: 25, status: single >.
```

10.1.1 Rewrite Rules for Objects

Rewrite rules define the behavior of objects, including their treatment of messages. The left-hand side is a multiset of objects and messages, and so is the right-hand side. A rule may involve zero, one, or many objects, and zero or more messages. The objects need not be the same on both sides: objects may be created and/or deleted by a rule, and so may messages.

The following rewrite rule has the same object on both sides:

```
vars X X' : String . vars N N' : Nat . vars S S' : Status .

crl [birthday] :
  < X : Person | age: N, status: S >
=>
  < X : Person | age: N + 1, status: S > if N < 999 .
```

The rule defines the local state change for an object. With this rule we have

```
< "Mette" : Person | age: 21, status: single > C ->
< "Mette" : Person | age: 22, status: single > C
```

for any configuration C because of the *Congruence* rule in rewriting logic, since

```
< "Mette" : Person | age: 21, status: single > ->
< "Mette" : Person | age: 22, status: single >
```

holds by *Replacement* and *Equality*, and $C \longrightarrow C$ holds by *Reflexivity*, and by *Congruence* with respect to the operator `__` we get the above sequent.

Objects can perform independent actions concurrently. If $o_1 \longrightarrow o'_1, \dots$, and $o_n \longrightarrow o'_n$ are one-step rewrites of objects $o_1, \dots, o_n$, then there is a concurrent step

$$o_1 \dots o_n \longrightarrow o'_1 \dots o'_n,$$

since the *Congruence* rule of rewriting logic with respect to the operator `__` gives a concurrent step $o_1 o_2 \longrightarrow o'_1 o'_2$, another application of *Congruence* then gives $(o_1 o_2) o_3 \longrightarrow (o'_1 o'_2) o'_3$, and so on. For example, in the concurrent one-step rewrite

```
< "Peter" : Person | age: 20, status: single >
< "Mette" : Person | age: 21, status: single >
< "Ingrid" : Person | age: 17, status: single >
  →
< "Peter" : Person | age: 21, status: single >
< "Mette" : Person | age: 22, status: single >
< "Ingrid" : Person | age: 18, status: single >
```

three birthdays are celebrated concurrently in one step.

More than one object may be involved in a rewrite rule:

```
cr1 [engagement] :
  < X : Person | age: N, status: single >
  < X' : Person | age: N', status: single >
=>
  < X : Person | age: N, status: engaged(X') >
  < X' : Person | age: N', status: engaged(X) >
  if N > 15 /\ N' > 15 .
```

Such a rule models *synchronous communication* where two (or more) objects meet and perform an action together. Any two objects may meet in this way, due to commutativity and associativity of the constructor `__`. For example there is a rewrite

```
< "Bence" : Person | age: 35, status: single >
< "Peter" : Person | age: 36, status: single >
< "Daniele" : Person | age: 29, status: single >
  →
< "Bence" : Person | age: 35, status: engaged("Daniele") >
< "Peter" : Person | age: 36, status: single >
< "Daniele" : Person | age: 29, status: engaged("Bence") >
```

since the state

```
< "Bence" : Person | age: 35, status: single >
< "Peter" : Person | age: 36, status: single >
< "Daniele" : Person | age: 29, status: single >
```

is the same as

```
< "Bence" : Person | age: 35, status: single >
< "Daniele" : Person | age: 29, status: single >
< "Peter" : Person | age: 36, status: single >
```

because `__` is declared `assoc` and `comm`.

10.1.1.1 Creation and Deletion of Objects

It is not necessary that the *same* objects occur in both sides of a rule. Objects may be “removed” in the right-hand side, as in the rule for the death of a single `Person`:

```
rl [death1] : < X : Person | age: N, status: single > => none .
```

An application of this rewrite rule takes the state

```
< "Hamlet" : Person | age: 28, status: single >
< "Old Norway" : Person | age: 67, status: married("Ingrid") >
< "Fortinbras" : Person | age: 40, status: single >
```

to the state

```
< "Hamlet" : Person | age: 28, status: single >
< "Old Norway" : Person | age: 67, status: married("Ingrid") >.
```

The right-hand side of a rule may contain objects not present in the left-hand side, in which case these additional objects are “created” by the rule and added to the state. For example, to model the birth of a new person, we include an extra object, containing a list of attractive names, in the state. Using the module

```
fmod STRING-LIST is protecting STRING .
sort StringList .
subsort String < StringList .
op nil : -> StringList [ctor] .
op __ : StringList StringList -> StringList [ctor assoc id: nil] .
endfm
```

which defines lists of strings, a class containing attractive names could be declared

```
op <_ : Names | OKnames:_ : Oid StringList -> Object [ctor] .
```

The following rule then models the birth of a person, where the name of the newborn is chosen nondeterministically among the favored names:

```
vars L L' : StringList .
crl [birth] :
  < X : Person | age: N, status: married(X') >
  < X'' : Names | OKnames: L X''' L' >
=>
  < X : Person | age: N, status: married(X') >
  < X'' : Names | OKnames: L X''' L' >
  < X''' : Person | age: 0, status: single > if N < 60 .
```

Then there is a rewrite step in which “Zeus” is born:

```
< "PossibleNames" : Names | OKnames: "Zeus" "Poseidon" "Hades" >
< "Kronos" : Person | age: 800, status: married("Rhea") >
< "Rhea" : Person | age: 21, status: married("Kronos") >
→
< "PossibleNames" : Names | OKnames: "Zeus" "Poseidon" "Hades" >
< "Kronos" : Person | age: 800, status: married("Rhea") >
< "Rhea" : Person | age: 21, status: married("Kronos") >
< "Zeus" : Person | age: 0, status: single >.
```

10.1.1.2 Communication Through Message Passing

These days, a separation usually starts with a letter, an e-mail, or a message on the answering machine. Therefore we declare a message type

```
op separate : Oid -> Msg [ctor] .
```

where `separate(X)` is a message *to* `X` intended to mean that `X`'s spouse wants to separate. In the rule

```
rl [separationInit] :
  < X : Person | age: N, status: married(X') >
=>
  < X : Person | age: N, status: separated(X') >
  separate(X') .
```

where `X` initiates a separation, the message `separate(X')` is created. In the rule

```
rl [acceptSeparation] :
  separate(X)
  < X : Person | age: N, status: married(X') >
=>
  < X : Person | age: N, status: separated(X') > .
```

the message `separate(X)` is read and consumed by the unsuspecting spouse.¹

The message passing is modeled abstractly in that the “traveling” of the message is due to the fact that `__` is associative and commutative. That is, the whole system can be seen as a “soup,” where objects and messages are “swimming around” and sometimes “meet.” For example, from the state

```
< "Zeus" : Person | age: 700, status: married("Dione") >
< "Hera" : Person | age: 19, status: single >
< "Dione" : Person | age: 21, status: married("Zeus") >
```

“Zeus” may want a separation (and later a divorce) so that he can marry his sister “Hera”. In one application of the rule `separationInit` the above state rewrites to

```
< "Zeus" : Person | age: 700, status: separated("Dione") >
separate("Dione")
< "Hera" : Person | age: 19, status: single >
< "Dione" : Person | age: 21, status: married("Zeus") >
```

which, due to associativity and commutativity of `__`, is the same as

```
< "Zeus" : Person | age: 700, status: separated("Dione") >
< "Hera" : Person | age: 19, status: single >
separate("Dione")
< "Dione" : Person | age: 21, status: married("Zeus") >
```

which rewrites by the use of rule `acceptSeparation` to

¹ Unfortunately, this straightforward way of separating by message passing may destroy future marriages, as explained in Section 11.2.1.1.

```

< "Zeus" : Person | age: 700, status: separated("Dione") >
< "Hera" : Person | age: 19, status: single >
< "Dione" : Person | age: 21, status: separated("Zeus") >.

```

We often call communication by message passing *asynchronous communication* because the objects do not synchronize in performing the action. Quite a lot of time (e.g., some birthday events) may elapse between the `separationInit` and the corresponding `acceptSeparation` event.

10.1.1.3 The Specification

The executable Maude specification—with some parts omitted—is given as follows:

```

mod OO-POPULATION is protecting NAT + STRING-LIST .

  *** Objects, messages, object names, and configurations:
  sorts Oid Object Msg Configuration .
  subsorts Object Msg < Configuration .
  op none : -> Configuration [ctor] .
  op __ : Configuration Configuration -> Configuration
      [ctor assoc comm id: none] .

  subsort String < Oid .          *** Object names are Strings

  *** Classes:
  op <_ : Names | OKnames:_ : Oid StringList -> Object [ctor] .
  op <_ : Person | age:_ , status:_ : Oid Nat Status -> Object
      [ctor] .

  *** Message for separating from spouse:
  op separate : Oid -> Msg [ctor] .

  sort Status .
  op single : -> Status [ctor] .
  ops engaged married separated : Oid -> Status [ctor] .

  vars X X' X'' X''' : String .    vars N N' : Nat .
  var S : Status .                vars L L' : StringList .

  crl [birthday] :
    < X : Person | age: N, status: S >
  =>
    < X : Person | age: N + 1, status: S > if N < 999 .

  crl [engagement] :
    < X : Person | age: N, status: single >
    < X' : Person | age: N', status: single >
  =>
    < X : Person | age: N, status: engaged(X') >
    < X' : Person | age: N', status: engaged(X) >
    if N > 15 /\ N' > 15 .

  crl [birth] :

```

```

    < X : Person | age: N, status: married(X') >
    < X'' : Names | OKnames: L X''' L' >
=>
    < X : Person | age: N, status: married(X') >
    < X'' : Names | OKnames: L X''' L' >
    < X''' : Person | age: 0, status: single > if N < 60 .

rl [separationInit] :
  < X : Person | age: N, status: married(X') >
=>
  < X : Person | age: N, status: separated(X') >
  separate(X') .

rl [acceptSeparation] :
  separate(X)
  < X : Person | age: N, status: married(X') >
=>
  < X : Person | age: N, status: separated(X') > .

*** Some rules are exercises and are therefore omitted

op greeks : -> Configuration .
eq greeks =
  < "PossibleNames" : Names | OKnames: "Hera" "Zeus" "Hades" >
  < "Gaia" : Person | age: 999, status: married("Uranus") >
  < "Uranus" : Person | age: 900, status: married("Gaia") >
  < "Kronos" : Person | age: 800, status: married("Rhea") >
  < "Rhea" : Person | age: 21, status: married("Kronos") > .
endm

```

To avoid typing large states each time you execute your specification it can be useful to define “abbreviations” for initial states such as the constant `greeks` above, so that we can execute the specification as follows:

```
Maude> frew [10] greeks .
```

```

result (sort not calculated):
  separate("Gaia")   separate("Uranus")
  < "PossibleNames" : Names | OKnames: "Hera" "Zeus" "Hades" >
  < "Gaia" : Person | age: 999, status: separated("Uranus") >
  < "Kronos" : Person | age: 803, status: separated("Rhea") >
  < "Rhea" : Person | age: 23, status: separated("Kronos") >
  < "Uranus" : Person | age: 901, status: separated("Gaia") >

```

Exercise 152

1. Is there a one-step concurrent rewrite

```

  < "Zeus" : Person | age: 700, status: single >
  < "Hera" : Person | age: 19, status: single >
  < "Dione" : Person | age: 21, status: single >
  →

```

```
< "Zeus" : Person | age: 700, status: engaged("Dione") >
< "Hera" : Person | age: 20, status: single >
< "Dione" : Person | age: 21, status: engaged("Zeus") >
```

in which "Hera" celebrates her birthday while the others are getting engaged?

2. *Is there a one-step concurrent rewrite*

```
< "Zeus" : Person | age: 700, status: single >
< "Dione" : Person | age: 21, status: single >
      ↓
< "Zeus" : Person | age: 700, status: engaged("Dione") >
< "Dione" : Person | age: 22, status: engaged("Zeus") >
```

where "Dione" celebrates her birthday and her engagement at the same time?

3. *Define the rule for marriage.*
4. *Use Maude's search command to prove that there is a behavior from greeks to a state in which the age of "Kronos" is 807. Try to avoid mentioning the other objects explicitly in the search pattern. Repeat the search for ages 810 and 811.*
5. *Search for a state in which both "Zeus" and "Hades" have been born.*
6. *Define a rule twinBirth for the birth of twins in one step. You may assume that the list of names contains at least two distinct names.*
7. *Can more than one person be born at the same time using rule birth?*
8. *Define the rules for separation, divorce, and the death of a non-single person.*
9. *Use the command `frew` to execute your specification in Maude.*

10.2 Concurrent Objects in Full Maude

Although concurrent objects can be specified naturally in Maude, it would be more elegant to define classes, messages, etc., as such.

Full Maude [20,21] provides support for specifying object-oriented systems in *object-oriented modules*, which give syntactic support for declaring *classes*, *subclasses*, and *messages*, and which allow us to write shorter rewrite rules by omitting attributes that do not affect, and are not affected by, the application of the rule. Full Maude also extends Maude's search command to the object-oriented case by taking subclasses into account in searches, and by allowing us to only mention relevant attributes in the search pattern. Full Maude internally transforms an object-oriented module into an ordinary Maude module, which can then be executed by Maude.

Full Maude is a Maude specification/program written by Francisco Durán and is given in the file `full-maude.maude` in the Maude distribution.

10.2.1 Using Full Maude

Full Maude is a Maude specification given in the file `full-maude.maude` and is started as an ordinary Maude module, that is, by starting Maude with


```
linux> maude full-maude.maudef
```

or by giving the Maude command

```
Maude> load full-maude.maudef
```

Input is given to Full Maude by enclosing it between a pair of parentheses. Full Maude accepts the modules and commands of Maude with some exceptions:

```
Maude> (fmod NAT-ADD is
>       sort Nat .
>       op 0 : -> Nat .
>       op s : Nat -> Nat [ctor] .
>       op _+_ : Nat Nat -> Nat .
>       vars M N : Nat .
>       eq 0 + M = M .
>       eq s(M) + N = s(M + N) .
>       endfm)
Introduced module NAT-ADD
```

```
Maude> (red s(s(0)) + s(0) .)
reduce in NAT-ADD :
  s(s(0)) + s(0)
result Nat :
  s(s(s(0)))
```

One Full Maude command worth mentioning here is `(show all .)`, which displays the Maude module which results from Full Maude's translation into Maude. The Maude command `trace exclude FULL-MAUDE .` should be given (without parentheses) after the command `set trace on .` to trace a Full Maude execution.

10.2.2 Object-Oriented Modules in Full Maude

Object-oriented modules are declared with syntax

```
(omod M is ... endom)
```

The sorts `Oid`, `Object`, `Msg`, and `Configuration` with the constructors described above are defined in the following module `CONFIGURATION` (given in the file `prelude.maudef`) which is automatically imported in any object-oriented module.²

```
mod CONFIGURATION is
  sorts Attribute AttributeSet .
  subsort Attribute < AttributeSet .
  op none : -> AttributeSet [ctor] .
  op _,_ : AttributeSet AttributeSet -> AttributeSet
          [format (o m so o) ctor assoc comm id: none] .
```

² The module below has been slightly changed by the author to get better formatted output; the same formatting should be added to Full Maude's `CONFIGURATION` module.

```

sorts Oid Cid Object Msg Portal Configuration .
subsort Object Msg Portal < Configuration .
op <_:_|_> : Oid Cid AttributeSet -> Object
                                     [ctor object format (b r b g b o b o)] .
op none : -> Configuration [ctor] .
op __ : Configuration Configuration -> Configuration
      [format (o n o) ctor config assoc comm id: none] .
op <> : -> Portal [ctor] .
endm

```

The sort `Cid` denotes *class identifiers*, and the sort `AttributeSet` denotes *multisets* of attribute-value pairs, so that the order in which the attributes are given does not matter. Classes are declared with syntax (note the blank also *before* the colon)

```
class C | att1 : s1, ..., attn : sn .
```

In our running example we could therefore write

```
class Person | age : Nat, status : Status .
```

No *values* are predefined in the sort `Oid`, so we could declare

```
subsort String < Oid .
```

if object identifiers are strings. Objects are written as before, with the difference that a colon is preceded by a blank, and that the order of attributes does not matter:

```
< "Edward" : Person | status : single, age : 32 > .
```

An object with no attributes is written with syntax

```
< o : EmptyClass | > .
```

Only a few of the attributes of an object may affect, or be affected by, the application of a rewrite rule. Only attributes whose values are changed need to be present in the right-hand side of a rule, and only those attributes whose values affect the applicability of a rule, the new values of the attributes changed by the rule, or the messages need to be present in the left-hand side of a rule.

For example, since the status of a person is not changed in the `birthday` rule, and the status does not affect the “next” age of a person, the `status` attribute may be omitted from the `birthday` rule:

```

crl [birthday] :
  < X : Person | age : N >
=>
  < X : Person | age : N+1 > if N < 999 .

```

The age of a person influences whether the person can be engaged, but is not itself changed by the engagement, so the `age` may be omitted from the right-hand side:

```

crl [engagement] :
  < X : Person | age : N, status : single >
  < X' : Person | age : N', status : single >
=>
  < X : Person | status : engaged(X') >
  < X' : Person | status : engaged(X) > if N > 15 and N' > 15 .

```

The partial specification can then be given as follows (note the parentheses):

```
load full-maude

(omod POPULATION is protecting NAT + STRING .
  sort Status .
  op single : -> Status [ctor] .
  ops engaged married separated : Oid -> Status [ctor] .

  subsort String < Oid .
  class Person | age : Nat, status : Status .
  vars N N' : Nat .   vars X X' : String .

  crl [birthday] :
    < X : Person | age : N >
  =>
    < X : Person | age : N+1 >   if N < 999 .

  crl [engagement] :
    < X : Person | age : N, status : single >
    < X' : Person | age : N', status : single >
  =>
    < X : Person | status : engaged(X') >
    < X' : Person | status : engaged(X) >   if N > 15 and N' > 15 .

  op greeks : -> Configuration .
  eq greeks =
    < "Gaia" : Person | age : 999, status : married("Uranus") >
    < "Uranus" : Person | age : 900, status : married("Gaia") > .
endom)
```

This model can be simulated as expected:

```
Maude> (frew [10] greeks .)
```

10.2.3 Subclasses

Full Maude supports *class inheritance* using *subclasses*. If C is a class declared

```
class C | att1 : s1, ..., attn : sn .
```

and B is a class declared

```
class B | att'1 : s'1, ..., att'k : s'k .
```

we can declare that B is a subclass of C as follows:

```
subclass B < C .
```

Just as for subsorts, where `subsort Ape < Animal` means that every `Ape` is also an `Animal` and therefore “inherits” all properties and functionalities of an animal, so `subclass B < C` means that every B -object is also a C -object which inherits all the attributes and all functionality of the class C , so that a rule

```
rl [l] : < o : C | ... > => < o : C | ... >
```

also applies to *B*-objects, whose set of attributes are $att_1, \dots, att_n, att'_1, \dots, att'_k$.

Full Maude supports *multiple inheritance*, where a class may be a subclass of a number of classes:

```
subclass C < C1 ... Cn .
```

In this case, the set of attributes of a *C*-object is the union of the sets of attributes of C_1 to C_n and those declared in *C*. The class *C* also inherits all rewrite rules of its superclasses. A superclass C_i may itself be a subclass of some other class.

Example 10.1. We extend our example to model the fact that some people are Christian, some are Muslim, and some are neither. Important events for a Christian are *baptism* and *confirmation*, and an important event for a Muslim is the *hajj* (the pilgrimage to Mecca). Both Christians and Muslims are persons: they celebrate birthdays, engagements, marriages, and they separate, divorce, and die like all persons.

There are at least two different ways of extending the module `POPULATION` with the important religious events:

1. The religion of a person is given at birth.
2. A person is born without religion, but can become Christian or Muslim by being baptized or by being read the call-for-prayer (or publicly pronounce the declaration of faith), respectively.

We first model that one's religion is given at birth. Since Christians and Muslims are persons, we define the classes `Christian` and `Muslim` as subclasses of `Person`:

```
sort ChristianStatus .
ops notBapt baptized confirmed : -> ChristianStatus [ctor] .

class Christian | chrStatus : ChristianStatus .
class Muslim | hajji : Bool .
subclass Christian Muslim < Person .
```

(The attribute `hajji` is `true` iff a Muslim has done a *hajj*.) The rules for baptism and confirmation are straightforward:

```
rl [baptism] :
  < X : Christian | chrStatus : notBapt >
=>
  < X : Christian | chrStatus : baptized > .

rl [confirmation] :
  < X : Christian | chrStatus : baptized >
=>
  < X : Christian | chrStatus : confirmed > .
```

The rule for *hajj* shows that a Muslim can do the pilgrimage more than once:

```
rl [hajj] : < X : Muslim | > => < X : Muslim | hajji : true > .
```

Since the rule `birth` also applies to Muslims and Christians, a religious couple may get a non-religious offspring. The following rule models the *possibility* that a newborn child is a Muslim if one of his/her parents is Muslim:

```
crl [birthMuslim] :
  < X : Names | OKnames : L X' L' >
  < X'' : Muslim | age : N, status : married(X'') >
  < X''' : Person | age : N', status : married(X'') >
=>
  < X : Names | OKnames : L L' >
  < X'' : Muslim | >
  < X''' : Person | >
  < X' : Muslim | age : 0, status : single, hajji : false >
  if N < 60 or N' < 60 .
```

A typical initial state of this system is

```
< "Possible names" : Names | OKnames : "Aaron" "Isaac" >
< "Imtiaz" : Muslim | age : 30, status : married("Maiken"),
    hajji : false >
< "Maiken" : Christian | age : 29, status : married("Imtiaz"),
    chrStatus : confirmed >
< "Panchen Lama" : Person | age : 28, status : single >.
```

In the second version, only a `Person` is born, and can later become a `Christian` or a `Muslim`. We therefore need a rule for baptism, so that both non-Christians and Muslims can be baptized, while Christians are already baptized and cannot be baptized again. The rule

```
rl [baptism] :
  < X : Person | age : N, status : S >
=>
  < X : Christian | age : N, status : S, chrStatus : baptized >.
```

cannot be used since it would allow a `Christian` to be baptized again. How can we modify the rule so that only non-believers and Muslims can be baptized? The easiest way is to define two sorts `ChrObject` and `MuslimObject` using memberships:

```
sorts ChrObject MuslimObject .
subsorts ChrObject MuslimObject < Object .
mb (< X : Christian | >) : ChrObject .
mb (< X : Muslim | >) : MuslimObject .
```

and then define the baptism rule as follows:

```
crl [baptism] :
  < X : Person | age : N, status : S >
=>
  < X : Christian | age : N, status : S, chrStatus : baptized >
  if not (< X : Person | >) :: ChrObject .
```

Likewise you cannot convert to Islam if you are already a Muslim:

```
cr1 [convertIslam] :
  < X : Person | age : N, status : S >
=>
  < X : Muslim | age : N, status : S, hajji : false >
  if not (< X : Person | >) :: MuslimObject .
```

The change of class corresponds to the deletion of an object and the creation of another object with the same name but a different class. *All* the attributes of the new object must therefore be provided in the right-hand sides of class-changing rules. ◇

10.2.4 Search in Full Maude

A search pattern $\langle o : C \mid att : pattern \rangle$ in an object-oriented system will match any object of class C or of a subclass of C whose attribute att is matched by $pattern$. Therefore, there is no need to worry about subclasses or mentioning all the attributes in the search pattern. This can be seen from the “echo” of the search command:

```
Maude> (search [1] greeks =>*
      C:Configuration < "Uranus" : Person | age : 902 > .)
```

```
search [1] in POPULATION : greeks =>* C:Configuration
< "Uranus" : V#0:Person | age : 902, V#1:AttributeSet > .
```

```
Solution 1
C:Configuration -->
  < "Gaia" : Person | age : 999, status : married("Uranus") > ;
V#0:Person --> Person ;
V#1:AttributeSet --> status : married("Gaia")
```

The command echo shows that Full Maude replaces the class names with variables (V#0 above) that can be used to capture objects belonging to subclasses of the class C (Person in the above example). Likewise, the “remaining” attributes of each object are captured by variables (V#1 above) of the sort `AttributeSet`. The search result shows that the (least) class of the object is `Person`.

Warning: When you search for a pattern that contains an object whose attribute values you are *not* interested in, you must use `none` for the attribute set in the search pattern instead of just leaving the “place for the attributes” empty.

Variables in search commands *with* such that-conditions need to be written in their “explicit” form `var:sort`:

```
Maude> (search [1] greeks =>*
      C:Configuration < "Uranus" : Person | age : N:Nat >
      such that N:Nat > 902 .)
```

10.2.4.1 Obtaining the Search Path

The `show path` commands are not provided by Full Maude. To obtain the path leading to a state found during a search, we must transform the Full Maude specification into an equivalent (core) Maude specification using the Full Maude command

```
(show all .)
```

which outputs the (core) Maude version of the current module. One can then cut-and-paste the output from this command into (core) Maude and perform the search in (core) Maude:

```
Maude> search [1] greeks =>*
      C:Configuration
      < "Uranus" : Person | age : 903, status : S:Status > .
```

```
Solution 1 (state 3)
C:Configuration -->
  < "Gaia" : Person | age : 999, status : married("Uranus") >
S:Status --> married("Gaia")
```

```
Maude> show path labels 3 .
birthday
birthday
birthday
```

Instead of cutting-and-pasting, you can specify your Full Maude module in a file, say `file.maude`, which ends with the lines

```
(show all .)
q
```

The linux command

```
linux> maude file.maude > core-maude-file.maude
```

will then write the equivalent Maude module to the file `core-maude-file.maude`. Remove the welcome and farewell greetings from this file and enter it into Maude. The specification can then be analyzed using all of Maude's features.

10.2.5 Using Full Maude: Repetition

Some things to remember when using Full Maude:

- When Full Maude is active, input to Full Maude is enclosed by a pair of parentheses. Input that is not enclosed in such a way is input to (core) Maude.
- Each module given to Full Maude must be enclosed by a pair of parentheses. However, since Full Maude also can import core Maude modules, a good idea is to first introduce all your non-OO modules into core Maude, and then start Full Maude and import those modules in your object-oriented Full Maude modules.

- Commands such as `red`, `rew`, and `search` should likewise be enclosed by a pair of parentheses.
- The commands `in` and `load` should be treated by (core) Maude and should *not* be enclosed by parentheses.
- Many Maude commands and features—such as the debugger and the `show path` command—are not available in Full Maude. See the Maude manual for details.
- Load the file `full-maude.maude` to activate Full Maude.

Exercise 153 Complete the Full Maude module `POPULATION` with rules for birth, marriage, separation, divorce, and death. Avoid superfluous attributes. Execute your specification in Full Maude.

Exercise 154 The second version of our example allows a *Christian* to convert to *Islam*, and vice versa. How would you modify the specification to disallow that?

10.3 Example: The Dining Philosophers

The *dining philosophers problem* [29] is a classic example due to Dijkstra. It is used to illustrate some concepts in distributed systems whose components need to access shared resources such as printers or shared memory.

10.3.1 Problem Description

Five philosophers sit around a round table with an enormous bowl filled with delicious dumplings in the middle of the table. Each philosopher spends her life alternating between thinking, being hungry, eating, then thinking again, and so on, in a never-ending cycle. However, even this seemingly idyllic setting is not perfect. By a cruel quirk of fate there are only *five* chopsticks on the table: one chopstick between each neighboring pair of philosophers, as seen in Fig. 10.1. We all know that dumplings are delicious but hot and slick, so a philosopher needs *both* her left chopstick and her right chopstick to eat.

A hungry philosopher will first grab a (left or right) chopstick if one is available, and will then hold on to this stick until she grabs the other chopstick and starts eating. No philosopher can eat forever, so after a finite time of eating, an eating philosopher must put back both chopsticks, and start thinking.

There are some intriguing questions about this world. Is it possible that all philosophers will starve to death due to lack of available chopsticks? Is it possible that *one* philosopher will starve to death while the others are feasting?

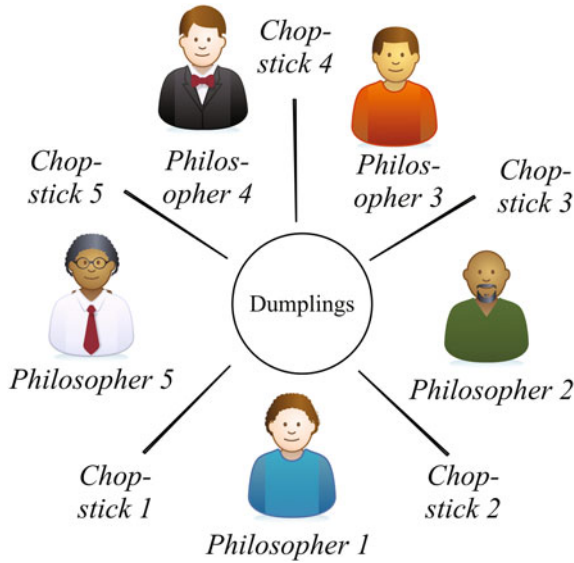


Fig. 10.1 The table setting for the dining philosophers

10.3.2 Modeling the Dining Philosophers

This section presents an object-oriented model which specifies all possible behaviors of the philosophers system.

I choose to model an available chopstick as a message, so that a “message” `chopstick (i)` means that chopstick i is available, and can be seen as a message which can be read and consumed by a philosopher, who then “has” the chopstick. When the philosopher stops eating she sends two `chopstick` messages into the configuration, making the chopsticks available again. Chopsticks are defined as follows:

```
msg chopstick : Nat -> Msg .
```

Each philosopher is modeled as an object with an attribute denoting the current state (thinking? hungry? eating?) of the philosopher and an attribute storing the number of chopsticks currently in the philosopher’s hands. I also add for analysis purposes a counter `#eats` that records how many times the philosopher has eaten. A philosopher object is therefore a term

```
< i : Philosopher | state : s, #sticks : j, #eats : k >
```

where i denotes the number of the philosopher. The philosopher class is declared

```
class Philosopher | state : State, #sticks : Nat, #eats : Nat .

subsort Nat < Oid .    --- object names are numbers

sort State .
ops thinking hungry eating : -> State [ctor] .
```

Each philosopher starts in a thinking state without a chopstick in hand. The rule `hungry` models the philosopher becoming hungry:

```
vars I J K : Nat .

rl [hungry] :
  < I : Philosopher | state : thinking >
=>
  < I : Philosopher | state : hungry > .
```

The rule `grabFirst` models the philosopher grabbing her first chopstick, which could be either her left or her right chopstick:

```
crl [grabFirst] :
  chopstick(J)
  < I : Philosopher | state : hungry, #sticks : 0 >
=>
  < I : Philosopher | state : hungry, #sticks : 1 >
  if I can use stick J .

op right : Nat -> Nat .    --- index of chopstick to the right
eq right(I) = if I == 5 then 1 else I + 1 fi .
op _can`use`stick_ : Nat Nat -> Bool .
eq I can use stick J = (I == J) or (J == right(I)) .
```

A philosopher can start eating when she grabs her second chopstick:

```
crl [grabSecond] :
  chopstick(J)
  < I : Philosopher | #sticks : 1, #eats : K >
=>
  < I : Philosopher | state : eating, #sticks : 2, #eats : K+1 >
  if I can use stick J .
```

The last rule stops the eating and puts the chopsticks back on the table:

```
rl [stopEating] :
  < I : Philosopher | state : eating >
=>
  < I : Philosopher | state : thinking, #sticks : 0 >
  chopstick(I) chopstick(right(I)) .
```

The initial state is declared as follows:

```
op initState : -> Configuration .
eq initState
= chopstick(1) chopstick(2) chopstick(3)
  chopstick(4) chopstick(5)
  < 1 : Philosopher | state : thinking, #sticks : 0, #eats : 0 >
  < 2 : Philosopher | state : thinking, #sticks : 0, #eats : 0 >
  < 3 : Philosopher | state : thinking, #sticks : 0, #eats : 0 >
  < 4 : Philosopher | state : thinking, #sticks : 0, #eats : 0 >
  < 5 : Philosopher | state : thinking, #sticks : 0, #eats : 0 > .
```

10.3.3 Deadlock and Livelock

A distributed system where processes need exclusive access to shared resources may *deadlock*. This means that the system is stuck and nothing can happen in the system because no process can proceed until it gets a shared resource which is controlled by another process (which is also stuck, since it may need, e.g., some resource controlled by the first process). A deadlock here could be a state where each philosopher has one chopstick, and cannot do anything because there are no chopsticks available. (Exercise 155 uses Full Maude to analyze whether the system may deadlock.)

Livelock (also known as *starvation*) is a trickier property which means that *one* philosopher could starve to death because she can never get hold of both chopsticks, while at the same time the other philosophers could feast merrily.

10.3.4 Fairness Issues

The *fairness* assumptions about this problem are: an eating philosopher *eventually* stops eating; a thinking philosopher eventually becomes hungry; and a philosopher will eventually pick up a needed chopstick if it is available infinitely often. These assumptions are *not* captured by our specification (why not?). However, each *finite* behavior (simulated with `frew [n]`) is “correct,” since it is a *prefix* of a behavior in which no philosopher eats continuously. Therefore, this deficiency does not affect the reasoning about deadlocks. However, a livelock (starvation) is an infinite scenario, so that certain livelock behaviors allowed by a specification may not satisfy the fairness constraints.

Fairness criteria which say that *eventually* (i.e., “some time in the future”) something must happen cannot be “implemented” in full generality, since there is no bound on *when* that “something” must happen. Instead, as explained in Chapter 16, we can analyze properties of the form “property X holds in all ‘fair’ computations.”

10.3.5 Version 2: A Deadlock-Free Solution

A solution which has been proposed to avoid deadlocks is to let each philosopher grab both chopsticks at the same time (and not allow them to grab only one).

10.3.6 Version 3: A Deadlock-Free and Livelock-Free Solution

The philosophers could get stuck in a deadlock situation where each philosopher proudly holds, say, her right chopstick and waits for the other chopstick, which will never become available. The solution where each philosopher grabs both chopsticks removes the possibility of deadlock, but not the possibility of livelock.

The following solution has been proposed to avoid also livelocks: Philosophers should not contemplate the deep questions of existence in the dining room, but in the adjacent library! Furthermore, there is now a doorman (or a sophisticated turnstile system) allowing at most *four* philosophers to be in the dining room at any time.

A state in this new setting can be an object of the form

```
< GlobalSystem : DinPhilHouse | diningRoom : philsAndSticks,
                                #inDinRoom : n,
                                library : philosophers >
```

where *philsAndSticks* is a *Configuration* consisting of all available chopsticks and those philosophers who are currently in the dining room, the number *n* denotes the number of philosophers currently in the dining room, and *philosophers* is a *Configuration* consisting of the philosophers in the library.

In this system, we have configurations, that is, object-oriented systems, inside an object. The class *DinPhilHouse* is declared as follows:

```
class DinPhilHouse | diningRoom : Configuration, #inDinRoom : Nat,
                    library : Configuration .
```

```
op GlobalSystem : -> Oid [ctor] .
```

Philosophers and chopsticks are modeled as before, and so is the rule *hungry* which lets a thinking philosopher become hungry (although this transformation now takes place in the library). A new rule lets a hungry philosopher enter the dining room if there are less than four philosophers in the dining room:

```
var O : Oid .  vars C C' : Configuration .

crl [enterDinRoom] :
  < O : DinPhilHouse | diningRoom : C, #inDinRoom : K,
                      library :
                        (< I : Philosopher | state : hungry > C') >
=>
  < O : DinPhilHouse | diningRoom : (< I : Philosopher | > C),
                      #inDinRoom : K + 1, library : C' >
  if K < 4 .
```

The variable *C* matches the configuration consisting of the philosophers and chopsticks already in the dining room, and *C'* matches the philosophers left in the library.

The rules *grabFirst* and *grabSecond* apply as before. We could be harsh and require that a philosopher leaves the dining room at the moment she stops eating. A gentler version keeps the rule *stopEating* and adds the rule

```
rl [enterLibrary] :
  < O : DinPhilHouse | diningRoom :
                        (< I : Philosopher | state : thinking > C),
                        #inDinRoom : s K, library : C' >
=>
  < O : DinPhilHouse | diningRoom : C, #inDinRoom : K,
                      library : (< I : Philosopher | > C') > .
```

in which a philosopher who has started thinking leaves the dining room.

In the initial state all philosophers are in the library thinking, while the delicious dumplings and the chopsticks are in the dining room:

```
< GlobalSystem : DinPhilHouse |
    diningRoom : chopstick(1) chopstick(2) chopstick(3)
                chopstick(4) chopstick(5),
    #inDinRoom : 0,
    library :
    (< 1 : Philosopher | state : thinking, #sticks : 0, #eats : 0 >
    < 2 : Philosopher | state : thinking, #sticks : 0, #eats : 0 >
    < 3 : Philosopher | state : thinking, #sticks : 0, #eats : 0 >
    < 4 : Philosopher | state : thinking, #sticks : 0, #eats : 0 >
    < 5 : Philosopher | state : thinking, #sticks : 0, #eats : 0 >)
>
```

I have used Maude to verify in a fully automatic way that this version of the dining philosopher's problem indeed is livelock-free (see Exercise 237).

Exercise 155 Consider the original specification of the dining philosophers.

1. Execute the dining philosophers system using Full Maude's `rew` and `frew` commands. Do all philosophers get to eat sufficiently often?
2. Use Full Maude's search command to show that the system could deadlock.
3. Show a scenario (a "run") which results in a deadlock.
4. Use search to check whether there is a reachable state in which each philosopher has eaten at least twice.
5. Does the system allow starvation? That is, use Full Maude's search command to check whether there is a behavior in which one philosopher has yet to eat, while at least three other philosophers have eaten at least twice each.
6. What is the maximum number of events (i.e., rule applications) that could happen in a concurrent rewrite step?

Exercise 156 Consider Version 2 of the dining philosophers in Section 10.3.5.

1. Specify and execute this version of the dining philosophers in Full Maude.
2. Use Full Maude's search command to search for a deadlock in the new specification. Explain the outcome of the search.
3. Explain why there cannot be a deadlock in the specification. That is, explain why some rule can always be applied.
4. Show that the specification is not livelock-free. That is, show that there is an infinite behavior of the system (in which all fairness criteria are satisfied) in which there is some philosopher who never has the possibility of grabbing chopsticks.

Exercise 157 Consider the version of the dining philosophers in Section 10.3.6.

1. Specify this version of the dining philosophers and execute your specification.
2. Explain why there cannot be a deadlock in this specification.

3. *Explain why there cannot be a livelock in the specification, when we assume the additional fairness constraint that each eating philosopher will eventually leave the dining room. That is, explain that there is no scenario in which a hungry philosopher can never grab both chopsticks in the future.*
4. *Can two philosophers exit the dining room at the same time?*
5. *Can a philosopher enter the dining room while another is exiting it?*
6. *Can two philosophers stop eating at the same time?*
7. *Can two philosophers each grab a chopstick, another one become hungry, and yet another philosopher leave the dining room, all in one concurrent step?*

10.4 Randomized Simulations: Winning in Vegas

The enticing casinos in Las Vegas offer the possibility of striking it rich quickly. Instead of experimenting with different strategies on the casino floor or perform complex error-prone statistical calculations to come up with a winning strategy, we use Maude to simulate the outcome of gambling with different strategies.

Blackjack (“21”) is a popular card game in which each *player* plays against the casino (called the *dealer*). The goal of a player is to amass cards with total value closer to 21 than the dealer, but without going over 21 (“busting”). A player faces many choices during a game: should he ask for another card? should he “double down,” “split,” or “surrender”? should he play at a table marked “dealer must stand on all 17’s” or at one marked “dealer must hit soft 17’s”? and so on.

Our approach to striking gold in Vegas is to simulate many rounds of the game with the desired strategy and see how much money we are left with. We use Maude’s built-in pseudo-random number generator `random` to perform *randomized simulations*: the next card is drawn “randomly” from the remaining cards in the deck.

10.4.1 Blackjack

In blackjack, a face card counts as 10, and an ace counts as either 1 or 11. A player/dealer has *blackjack* if he has *two* cards with total value 21.

A round of blackjack goes as follows. The player places his bet and gets one card; the dealer then gets a card that can be seen by the player; and the player gets his second card. The player then considers the situation and ask for new cards (“hit”), one by one, until the player is satisfied or goes bust. The dealer *must* follow a fixed pre-defined strategy, and gets his remaining cards when the player is done.

The player loses his bet if either:

- the sum of (the values of) his cards is greater than 21 even when his aces count as 1 and even if the dealer also busts;
- the dealer has blackjack and the player has not; or

- the sum of the dealer's cards is closer to 21, without going over 21, than the sum of the player's cards.

The player keeps his bet if either:

- both the player and the dealer have blackjack; or
- neither has blackjack and the best sum of their cards have the same value $v \leq 21$.

The player wins 1.5 times his bet if he has blackjack and the dealer has not. In all other cases, the player wins an amount equal to his bet.

After getting his first two cards, the player may perform any of the following actions (typically at most once, although rules may vary):

Double down: Double his bet and get exactly one more card.

Split: If the two cards have the same value, the player may split them into two separate hands and play on with two separate hands.

Surrender: Give up, and keep half his bet.

There are many different strategies to consider for the blackjack player, including:

- The dealer *must* hit (get new cards) until he gets 17; in some casinos, the dealer must hit on “soft 17” (i.e., an ace and other cards with total sum 6) and in other casinos the dealer must stand on “soft 17.” In which casino should you play?
- In general, how should you play based on the dealer's visible card?
- Should you play with one deck of cards or with multiple decks?
- When should you double down, split, or surrender?

10.4.2 Modeling Blackjack Rounds

For simplicity of exposition, this section models the play of a nervous first-time visitor to Las Vegas who adopts the following very simple strategy: stand if the *least* value of your hand is ≥ 15 or if its *best* value is ≥ 18 . In Exercise 163 you can modify this strategy to your strategy of choice.

We use `::`-separated *lists* of cards (since we will randomly draw the n -th remaining card) such as `< diamonds , A >` (ace of diamonds). A *deck* is a special list:

```
fmod CARD is
  sorts Suit Value Card .
  ops 2 3 4 5 6 7 8 9 10 J Q K A : -> Value [ctor] .
  ops spades hearts clubs diamonds : -> Suit [ctor] .
  op <_,_> : Suit Value -> Card [ctor] .

  sort Cards .   subsort Card < Cards .
  op nil : -> Cards [ctor] .
  op _::_ : Cards Cards -> Cards [assoc id: nil ctor] .

  op deck : -> Cards .
  eq deck = generate(spades) :: generate(hearts) ::
             generate(diamonds) :: generate(clubs) .
```

```

op generate : Suit -> Cards . --- generate all cards of a suit
var S : Suit .
eq generate(S) = < S, 2 > :: < S, 3 > :: < S, 4 > :: < S, 5 > ::
               < S, 6 > :: < S, 7 > :: < S, 8 > :: < S, 9 > ::
               < S, 10 > :: < S, J > :: < S, Q > :: < S, K > ::
               < S, A > .
endfm

```

Since an ace can count as either 1 or 11, we define here and in Exercise 158 different sums of the cards in a hand: `leastValue`, `largestValue`, and `bestValue`.

```

fmod RESULT is protecting CARD + NAT .
ops leastValue largestValue bestValue : Cards -> Nat .

var S : Suit .   var V : Value .   var BET : Nat .
vars HAND PLAYER DEALER : Cards .  var CARD : Card .

eq leastValue(< S, 2 >) = 2 .      eq leastValue(< S, 3 >) = 3 .
...
eq leastValue(< S, K >) = 10 .    eq leastValue(< S, A >) = 1 .
eq leastValue(nil) = 0 .
ceq leastValue(CARD :: HAND)
  = leastValue(CARD) + leastValue(HAND) if HAND /= nil .

```

The expression `result(player, dealer, bet)` defines the payment (including the original bet) to a player after a game in which he bet $\$bet$ and ended with hand *player*, while the dealer finished with the hand *dealer*:

```

op result : Cards Cards Nat -> Nat .
eq result(PLAYER, DEALER, BET)
  = if blackjack(PLAYER) and (not blackjack(DEALER))
    then (5 * BET) quo 2 --- blackjack!
    else (if bestValue(PLAYER) <= 21 and
            (bestValue(PLAYER) > bestValue(DEALER)
             or leastValue(DEALER) > 21)
          then (BET + BET) --- player wins
          else (if (blackjack(PLAYER) and blackjack(DEALER))
                  or
                  ((not blackjack(DEALER))
                   and
                   (bestValue(PLAYER) <= 21)
                   and (bestValue(PLAYER) == bestValue(DEALER)))
                then BET --- push
                else 0 fi) fi) fi . --- player loses
endfm

```

where `blackjack` checks whether a hand is a blackjack (see Exercise 158).

The expression `getRandomCard(cards, k)` uses the k -th number generated by the function `random` on page 40 to pick a card pseudo-randomly from *cards*:

```

fmod RANDOM-CARD is protecting RANDOM + CARD .
var N : Nat .   var CARD : Card .   var CARDS : Cards .

```



```

op size : Cards -> Nat . --- no of cards in a hand
eq size(nil) = 0 .    eq size(CARD :: CARDS) = s(size(CARDS)) .

op getNthCard : Nat Cards ~> Card . --- get card N+1
eq getNthCard(0, CARD :: CARDS) = CARD .
eq getNthCard(s N, CARD :: CARDS) = getNthCard(N, CARDS) .

op getRandomCard : Cards Nat ~> Card .
eq getRandomCard(CARDS, N)
  = getNthCard(random(N) rem size(CARDS), CARDS) .
endfm

```

We next model the game in an object-oriented style, where the state consists of three classes of objects: dealer, players, and a `Table` objects which contains the remaining cards (attribute `shoe`), the index for the `random` function (`rndIndex`), and information about whose turn is next. Since I tend to be alone with the dealer on my one-and-done forays to the high-roller table, I assume for simplicity that there is only one player at the table (extending this is trivial; see Exercise 160).

Since we deal with objects, we start using Full Maude:

```

load full-maude

(omod PLAY-BJ is protecting RESULT + RANDOM-CARD .

class Table | shoe : Cards, rndIndex : Nat, turn : Oid .
class Player | hand : Cards, bet : Nat .
class Dealer | hand : Cards .

```

The following rewrite rules model the start of the game: first the player gets his first card (`startGame`), then the dealer gets his first card (`dealerFirstCard`), followed by the player getting his second card (`playerSecond`). The index for the `random` function must increase each time a card is taken:

```

vars CARD CARD2 : Card .    vars CARDS CARDS2 : Cards .
var N : Nat .    var NZN : NzNat .    vars T P D : Oid .

rl [startGame] :
  < T : Table | shoe : CARDS, rndIndex : N >
  < P : Player | hand : nil, bet : NZN >
=>
  < T : Table | shoe : remove(getRandomCard(CARDS, N), CARDS),
    rndIndex : s N >
  < P : Player | hand : getRandomCard(CARDS, N) > .

rl [dealerFirstCard] :
  < T : Table | shoe : CARDS, rndIndex : N >
  < D : Dealer | hand : nil >
  < P : Player | hand : CARD >
=>
  < T : Table | shoe : remove(getRandomCard(CARDS, N), CARDS),
    rndIndex : s N >
  < D : Dealer | hand : getRandomCard(CARDS, N) >
  < P : Player | > .

```

```

rl [playerSecond] :
  < T : Table | shoe : CARDS, rndIndex : N >
  < P : Player | hand : CARD >
  < D : Dealer | hand : CARD2 >
=>
  < T : Table | shoe : remove(getRandomCard(CARDS, N), CARDS),
                    rndIndex : s N, turn : P >
  < P : Player | hand : CARD :: getRandomCard(CARDS, N) >
  < D : Dealer | > .

```

Next, the player hits or stands according to the simple strategy described above:

```

cr1 [playerHit] :
  < T : Table | shoe : CARDS, rndIndex : N, turn : P >
  < P : Player | hand : CARDS2 >
=>
  < T : Table | shoe : remove(getRandomCard(CARDS, N), CARDS),
                    rndIndex : s N >
  < P : Player | hand : CARDS2 :: getRandomCard(CARDS, N) >
  if not (leastValue(CARDS2) >= 15 or bestValue(CARDS2) >= 18) .

cr1 [playerStand] :
  < T : Table | turn : P >
  < P : Player | hand : CARDS2 >
  < D : Dealer | >
=>
  < T : Table | turn : D >
  < P : Player | >
  < D : Dealer | >
  if leastValue(CARDS2) >= 15 or bestValue(CARDS2) >= 18 .

```

The final rule models a dealer that “stands on all 17’s”:

```

cr1 [dealerTakesMore] :
  < T : Table | turn : D, shoe : CARDS, rndIndex : N >
  < D : Dealer | hand : CARD :: CARDS2 >
=>
  < T : Table | shoe : remove(getRandomCard(CARDS, N), CARDS),
                    rndIndex : s N >
  < D : Dealer | hand : CARD :: CARDS2 :: getRandomCard(CARDS, N) >
  if bestValue(CARD :: CARDS2) < 17 .

```

Finally, we define three object identifiers:

```

ops caesarsPalace peter t : -> Oid [ctor] .
endom)

```

We can then simulate one game of blackjack, starting with random number 7:

```

Maude> (rew < t : Table | shoe : deck, rndIndex : 7, turn : t >
        < caesarsPalace : Dealer | hand : nil >
        < peter : Player | hand : nil, bet : 100 > .)

```

```

result Configuration :

```

```
< caesarsPalace : Dealer | hand : <clubs, 3> :: <hearts, 5> ::
                                     <spades, 4> :: <spades, K> >
< peter : Player | bet : 100, hand : <clubs, A> :: <diamonds, 9> >
< t : Table | rndIndex : 13, ... >
```

So far, so good. Simulating single rounds is, however, not very efficient. We therefore model a player who spends an entire day (or as long as money lasts) at the blackjack table as an object of the subclass `MultiPlayer`, which adds attributes `gamesLeft` (number of games left to play), `money` (total amount of player money), and `eachBet` (bet in each round) to the class `Player`, and add two rules: `reset` cleans up the table after the previous round, and `restart` starts a new round if the player has sufficient funds:

```
(omod PLAY-MANY-ROUNDS is protecting PLAY-BJ .

class MultiPlayer | gamesLeft : Nat, money : Nat, eachBet : NzNat .
subclass MultiPlayer < Player .

vars T D P : Oid .          var NZN : NzNat .
vars CARDS1 CARDS2 : Cards . vars N N2 : Nat .

crl [reset] :
  < T : Table | rndIndex : N, turn : D >
  < D : Dealer | hand : CARDS1 >
  < P : MultiPlayer | hand : CARDS2, bet : NZN, money : N2 >
=>
  < T : Table | shoe : deck, rndIndex : s N, turn : T >
  < D : Dealer | hand : nil >
  < P : MultiPlayer | hand : nil, bet : 0,
                      money : N2 + result(CARDS2, CARDS1, NZN) >
  if bestValue(CARDS1) >= 17 .

crl [restart] :
  < P : MultiPlayer | gamesLeft : s N, bet : 0,
                      money : N2, eachBet : NZN >
=>
  < P : MultiPlayer | gamesLeft : N, bet : NZN,
                      money : sd(N2, NZN) >   if NZN <= N2 .
endom)
```

If we start the day with \$1000, how much money do we have left after playing 100 rounds of blackjack with our trivial strategy?

```
Maude> (rew < t : Table | shoe : deck, rndIndex : 1, turn : t >
        < caesarsPalace : Dealer | hand : nil >
        < peter : MultiPlayer | hand : nil, bet : 0,
          gamesLeft : 100, money : 1000, eachBet : 100 > .)

result Configuration :
< peter : MultiPlayer | gamesLeft : 0, money : 800, ... > ...
```

This is surprisingly good; the player only lost \$200 after 100 rounds of \$100-games.

10.4.3 Further Guarantees

Even if the results of a few simulations of your blackjack strategy look good, you want stronger guarantees before quitting your day job. Our specification can be seen as a *probabilistic* rewrite theory (see Chapter 17) where each card is drawn from the deck with the same probability. The following analysis methods, discussed in Chapter 17, provide stronger guarantees than single executions:

Probabilistic model checking: One could prove properties such as “the likelihood of ending up with more \$1200 after a day’s work is more than 60%.”

Statistical model checking: Unfortunately, probabilistic model checking can be very inefficient. *Statistical model checking* [102, 109] trades certainty for efficiency by simulating single runs until the desired confidence level is reached, and allows you to ascertain properties like “with confidence level 0.9, the likelihood of ending up with more \$1200 after a day’s work is more than 60%.”

Value estimation: To better plan your economy as a professional blackjack player, you may be more interested in *estimating the amount of money you have at the end of the day* than the likelihood of making more than \$200.

Exercise 158 Define the functions `largestValue`, `bestValue`, and `blackjack`.

Exercise 159 How would you simulate a game played with multiple decks of cards?

Exercise 160 Define a model which allows up to seven players at the table.

Exercise 161 Simulate games at a “dealer must hit soft 17’s” table.

Exercise 162 For more extensive analysis: extend the specification to simulate a player who plays like a `MultiPlayer` every day for x days.

Exercise 163 Our player was a nervous first-timer. In this exercise we analyze more sophisticated blackjack strategies.

1. Define a player who also takes the dealer’s visible card into consideration. For example, one is advised to stand on 13-21 when the dealers face-up card shows 2.
2. You can refer to Wikipedia’s Blackjack entry for a recommended strategy.
2. Define a player who also “doubles down” when appropriate. For example, Wikipedia says that you should double down when your cards total 11 or 10 and the dealer’s visible card shows 2 to 9.
3. Extend the player to also “split” at appropriate times when the first two cards show the same value. For example, you should always split if you get two aces.
4. Extend the player to also surrender at appropriate times.
5. With all these capabilities, you have defined an expert player. Perform extensive simulations and check if this is a good way to make a living in the long run.

Exercise 164 Define a function `shuffle : Cards Nat -> Cards` that shuffles a list of cards (for a given random number index), and modify our specification so that the cards are shuffled before each game instead of being drawn randomly from the shoe each time a card is needed.

Chapter 10 explained how a concurrent system can be represented as a multiset of concurrent objects. This chapter shows how different forms of communication between such objects can be modeled in rewriting logic.

We need to model different forms of communication, because:

1. different kinds of devices have different communication capabilities;
2. we need to be able to model systems at different *levels of abstraction*, so that unnecessary details can be omitted;
3. of generality: a protocol may be applicable not only to one kind of system, but to all systems that satisfy certain assumptions about their communication.

Examples of different communicating devices include a computer communicating using TCP/IP, a satellite broadcasting TV signals, and a node in a wireless sensor network. These three devices have very different communication capabilities.

To make modeling and analyzing distributed systems feasible, it is imperative to omit as much detail as possible (but not more!). Details such as how messages are divided into packets (or “frames”), what the “header” fields in a packet are, or how a packet is routed from source to destination, can often be ignored when analyzing distributed systems designs. A model should therefore *abstract* from such details. This chapter shows how communication can be modeled at a high level of abstraction, and abstracts from details about how communication is actually achieved.

Communication may be *synchronous*: the objects *synchronize* in the communication event, such as when two people talk to each other. Communication may also be *asynchronous*: the parties do *not* synchronize to communicate. Examples of asynchronous communication include sending and receiving letters using the postal service, sending/receiving email, leaving messages on the voice mail, and writing on and reading a shared message board.

Asynchronous communication may be *ordered* (the recipient of a set of messages from the same sender reads the messages in the order in which they were sent)

or *unordered* (the recipient may read messages in a different order). Examples of ordered delivery include messages left on an answering machine and communication by messages sent along the same cable or link. Examples of unordered delivery include email sent on the Internet or letters sent in the mail.

A communication event may involve *two* parties, such as a person writing a letter to a loved one (*unicast*), or may involve *many* parties, such as a party sending junk emails to thousands of computer users (*multicast*), or a satellite sending the same pictures to all households with an appropriate satellite dish (*broadcast*).

Communication may be *unreliable*, such as when email or letters are lost or misplaced, or when the content of a data packet is corrupted. Other forms of communication are *reliable*, such as (hopefully) the communication inside an airplane.

In contrast to many modeling formalisms for distributed systems, rewriting logic does *not* provide any fixed communication primitive. Many forms of communication can instead be easily modeled directly in rewriting logic. This gives the modeler the flexibility to define the desired form of communication without having to encode it using some fixed communication primitive.

This chapter shows how some forms and features of communication can be modeled in rewriting logic. Section 11.1 treats *synchronous* communication; the rest deals with *asynchronous* communication. Section 11.2 presents a model of unordered unicast message transmission, and shows how the seemingly trivial task of getting a *separation* from your spouse is complicated when done by message exchange. This example is supposed to give some flavor of the intrinsic difficulty in understanding asynchronously communicating systems—underscoring the necessity of being able to analyze them. Our basic unicast model is then extended to *multicast* and *broadcast*, and to *unreliable* communication. To illustrate the ease with which new forms of communication can be modeled, Section 11.2.4 shows how wireless broadcast can be modeled. Section 11.3 explains how *ordered* asynchronous communication can be modeled using explicit *link* objects, through which messages between pairs of nodes are transmitted. This model is also extended to model unreliable communication, and to model links with limited capacity. Finally, Section 11.4 proposes a way of modeling asynchronous communication using *shared variables*, and illustrates by exercises some difficulties with concurrent transactions on shared data, such as shared bank accounts or an attractive airplane seat.

11.1 Synchronous Communication

Synchronous (“handshake”) communication, in which objects *synchronize* (“meet”) to perform a communication event together, is modeled in Maude by having all the objects involved in the communication event in the rewrite rule, such as in the rule

```

crl [engagement] :
  < X : Person | age : N, status : single >
  < X' : Person | age : N', status : single >
=>
  < X : Person | status : engaged(X') >
  < X' : Person | status : engaged(X) >  if N > 15 and N' > 15 .

```

in which two parties communicate their mutual desire to marry. Objects can be seen as “swimming” in the “soup” which makes up the state, and can meet to perform the rule. More than two objects may of course participate in a communication event.

11.2 Unordered Asynchronous Communication by Message Passing

This section describes different ways of modeling *unordered asynchronous communication* between objects by message passing. We consider reliable and unreliable communication, object-to-object communication (*unicast*) as well as *multicast* (one-to-many communication), *broadcast* (one-to-all communication), and wireless broadcast (broadcast to all nodes within the sender’s transmission range).

11.2.1 Unordered Unicast

Unordered delivery is a natural model of many forms of asynchronous communication, including email transmission and communication by sending letters in the mail. An email sent from *A* to *B* may well arrive after another email from *A* to *B* which was sent later. One difficulty in designing distributed systems is that one has to allow for *any* possible order of message delivery, and that one may not know whether a message is significantly delayed, lost, or never was sent.

Unordered message passing communication is modeled so that a message *m* is sent by adding it to the global state; i.e., the message *m* appears only in the *right-hand* side of a rewrite rule. A rewrite rule in which a message *m* occurs only in the *left-hand* side of the rule models the *consumption* of the message *m*.

11.2.1.1 Example: Separating using Messages

To illustrate asynchronous communication—and its difficulties—consider the seemingly trivial task of arranging a *separation* using message passing: a separation is initiated by a letter (from the lawyer?). To make the example as simple as possible, there is no possibility of reconciliation, and there are no message losses.

A message `separate(x)` is a message to *x* that *x*’s spouse wants to separate. Section 10.1.1.2 presents the following model of message-based separation:

```
msg separate : Oid -> Msg .

rl [initiateSeparation] :
  < X : Person | status : married(X') >
=>
  < X : Person | status : separated(X') >
  separate(X') .
```

```

rl [acceptSeparation] :
  separate(X)
  < X : Person | status : married(X') >
=>
  < X : Person | status : separated(X') > .

```

and shows that these rules could lead to a successful separation.

Unfortunately, an *old* separation may destroy a *new* marriage: Assume that *both* "JR" *and* "Sue Ellen" want to separate more or less at the same time:

```

< "JR" : Person | age : 50, status : married("Sue Ellen") >
< "Cally" : Person | age : 25, status : single >
< "Sue Ellen" : Person | age : 45, status : married("JR") >
< "Cliff" : Person | age : 46, status : single >
      ────────────>
< "JR" : Person | age : 50, status : married("Sue Ellen") >
< "Cally" : Person | age : 25, status : single >
< "Sue Ellen" : Person | age : 45, status : separated("JR") >
separate("JR")
< "Cliff" : Person | age : 46, status : single >
      ────────────>
< "JR" : Person | age : 50, status : separated("Sue Ellen") >
separate("Sue Ellen")
< "Cally" : Person | age : 25, status : single >
< "Sue Ellen" : Person | age : 45, status : separated("JR") >
separate("JR")
< "Cliff" : Person | age : 46, status : single >

```

Both "JR" and "Sue Ellen" are now separated, and can divorce (using a straightforward synchronous divorce rule¹), leading to the state

```

< "JR" : Person | age : 50, status : single >
separate("Sue Ellen")
< "Cally" : Person | age : 25, status : single >
< "Sue Ellen" : Person | age : 45, status : single >
separate("JR")
< "Cliff" : Person | age : 46, status : single >

```

"JR" is again single and starts courting "Cally", and eventually marries her. Likewise, "Sue Ellen" goes on and marries "Cliff", leading us to the state

```

< "JR" : Person | age : 50, status : married("Cally") >
separate("Sue Ellen")
< "Cally" : Person | age : 25, status : married("JR") >
< "Sue Ellen" : Person | age : 45, status : married("Cliff") >
separate("JR")
< "Cliff" : Person | age : 46, status : married("Sue Ellen") >

```

¹ There is no contradiction in using a synchronous divorce rule, since the parties do not talk to each other and hence do not know that *both* of them have initiated a separation when they meet in court.

And disaster strikes! "JR" reads the `separate("JR")` message (sent by "Sue Ellen") which has been lying around, and thinks that "Cally" wants a separation:

```
< "JR" : Person | age : 50, status : separated("Cally") >
separate("Sue Ellen")
< "Cally" : Person | age : 25, status : married("JR") >
< "Sue Ellen" : Person | age : 45, status : married("Cliff") >
< "Cliff" : Person | age : 46, status : married("Sue Ellen") >
```

In the same way, "Sue Ellen"—now happily married to "Cliff"—could read the old separation message from "JR". Two happy marriages have been broken up by old `separate` messages! The problems are that

1. a `separate` message is not read if you are in a state `separated` (you don't look for `separate` messages if you think that you have separated), and
2. an old `separate` message can arrive a couple of years later, destroying a new and happy marriage.

The first of these problems could be fixed by adding a rule

```
rl [sep2] :
  separate(X)
  < X : Person | status : separated(X') >
=>
  < X : Person | > .
```

Adding this rule does not solve the second problem, since the unfortunate behavior above is still possible. (Adding the sender to the `separate` message does *not* solve our problems, since "JR" and "Sue Ellen" might remarry after their first divorce, and the *old* `separate` message would destroy their *new* and happy marriage.)

Fortunately, it is possible to separate safely as follows: A new status `waitSep(p)` denotes that a separation from *p* has been initiated and that the person is waiting for the answer. The following rules specify this way of separating:

```
rl [initSep] :
  < X : Person | status : married(X') >
=>
  < X : Person | status : waitSep(X') >
  separate(X') .

rl [acceptSep] :
  separate(X)
  < X : Person | status : married(X') >
=>
  < X : Person | status : separated(X') >
  separate(X') .

rl [acceptSep2] :
  separate(X)
  < X : Person | status : waitSep(X') >
=>
  < X : Person | status : separated(X') > .
```

This specification describes a *protocol* for how each spouse should behave to successfully separate. “Programs” for distributed systems are often protocols which define how the distributed components should interact. Correctness of the separation protocol follows from the fact that *each* party must send exactly one `separate` message, and must consume one `separate` message, in the separation process.

This example illustrates the difficulty with asynchronously communicating systems. It seems almost impossible to find a simpler example: only one communication event (a separation) should take place, and there is no loss or corruption of messages, yet the problem has a fairly unintuitive solution. Furthermore, if messages can get lost, then the problem becomes really hard (or unsolvable).

11.2.1.2 Message Wrappers

A letter sent in the mail typically consists of an *envelope*, with the sender and receiver addresses, inside which there is some *message content*. In the rest of this chapter, we use a *message wrapper* (“envelope”), so that a unicast message in the global configuration is a term of the form

`msg content from sender to receiver`

where *content* is the *message content*. This wrapper is defined as follows:

```
(mod MESSAGE-CONTENT is
  sort MsgContent .      --- message content, application-specific
endm)

(omod MESSAGE-WRAPPER is including MESSAGE-CONTENT .
  op msg_from_to_ : MsgContent Oid Oid -> Msg [ctor] .
endom)
```

11.2.2 Multicast

Multicast means that a sender sends a message to a group of recipients at once, for example sending stock quotes or conference announcements to groups of recipients who subscribe to such notifications.

A group of receivers can be modeled as a set of object identifiers:

```
class Sender | multicast-group : OidSet, ...

sort OidSet .      subsort Oid < OidSet .
op none : -> OidSet [ctor] .
op _;/_ : OidSet OidSet -> OidSet [ctor assoc comm id: none] .
```

The idea is to introduce a “multicast” message wrapper, so that a multicast message

`multicast content from sender to rcv1 ; rcv2 ; ... ; rcvn`

to a multicast group is equivalent to a *separate unicast message to each recipient* in the group. The multicast message above reduces to the multiset of messages

```

(msg content from sender to rcv1)
(msg content from sender to rcv2)
...
(msg content from sender to rcvn)

```

using the following equations:

```

(omod MULTICAST is including OID-SET + MESSAGE-WRAPPER .
  op multicast_from_to_ : MsgContent Oid OidSet -> Msg [ctor] .

  var MC : MsgContent .   vars SENDER ARECEIVER : Oid .
  var OTHER-RECEIVERS : OidSet .

  eq multicast MC from SENDER to none = none .
  eq multicast MC from SENDER to ARECEIVER ; OTHER-RECEIVERS =
    (msg MC from SENDER to ARECEIVER)
    (multicast MC from SENDER to OTHER-RECEIVERS) .
endom)

```

Multicasting a message content *content* to a multicast group can then be modeled by rewrite rules having the form

```

rl [multicast] :
  < a : Sender | multicast-group : receivers, ... >
=>
  < a : Sender | ... >
  multicast content from a to receivers .

```

11.2.3 Broadcast

Broadcast means that a node sends a message to all the (other) nodes in the system. An example is a television satellite system that broadcasts TV signals to all households in the world that have certain kinds of reception equipment. Unlike for multicast, a broadcasting node does not know the group of receivers. The idea is to transform a broadcast message into a multicast message to all the other nodes in the system. To have “control” over all the nodes in the system, we introduce an operator

```

sort GlobalSystem .
op {_} : Configuration -> GlobalSystem [ctor] .

```

and require that the *whole state* has the form $\{conf\}$, for some configuration *conf*. A broadcast message wrapper can be declared

```

op broadcast_from_ : MsgContent Oid -> Configuration .

```

Assuming that the nodes in the system are objects of a class *Node*,² and knowing that all the objects in systems are enclosed within the curly braces, the following equations define a *broadcast* message to be a multicast message to all other nodes in the system:

² This is not a significant restriction, since any class can be a subclass of the class *Node*.

```

var REST : Configuration .    vars O O' : Oid .
var MSG : Msg .              MC : MsgContent .
eq {< O : Node | > (broadcast MC from O) REST}
  = {< O : Node | > (multicast MC from O to objectIds(REST)) REST} .

op objectIds : Configuration -> OidSet [frozen (1)] .
eq objectIds(< O : Node | > REST) = O ; objectIds(REST) .
eq objectIds(MSG REST) = objectIds(REST) .
eq objectIds((broadcast MC from O) REST) = objectIds(REST) .
eq objectIds(none) = none .

```

The function `objectIds` gives the set of object identifiers in a configuration. Broadcasting a message is done by a rule of the form

```

rl [broadcast] :
  < o : ... > => < o : ... > broadcast content from o .

```

11.2.4 Wireless Broadcast

Wireless transmission can be seen as broadcast to nodes within the *transmission range* of the sender. To model such communication, each node must have a location:

```

sort Location .
class Node | location : Location, ... .

```

Given a function `withinTransRangeOf`, where l `withinTransRangeOf` l' holds if and only if a node in location l' can be reached (with sufficient signal strength) by a wireless broadcast from a node in location l , wireless broadcast can be modeled by a *wireless broadcast message* `wl-broadcast content from sender` defined as follows:

```

op wl-broadcast_from_ : MsgContent Oid -> Configuration .
eq {< O : Node | location : L > (wl-broadcast MC from O) REST}
  = {< O : Node | > (multicast MC from O to nodesInRange(L, REST))
    REST} .
op nodesInRange : Location Configuration -> OidSet [frozen (2)] .
eq nodesInRange(L, < O : Node | location : L' > REST)
  = (if L withinTransRangeOf L' then O else none)
    ; nodesInRange(L, REST) .
eq nodesInRange(L, MSG REST) = nodesInRange(L, REST) .
eq nodesInRange(L, (wl-broadcast MC from O) REST)
  = nodesInRange(L, REST) .
eq nodesInRange(L, none) = none .

```

11.2.5 Modeling Unreliable Communication

Messages can get lost or corrupted during transmission. Corruption can typically be detected by the communication infrastructure and is usually modeled as a message

loss. In many systems, a sender *resends* a message after a certain amount of time if it has not heard from a receiver in the meantime. To avoid having to deal with time and timeouts (see Section 17.1 for the treatment of time in Maude), such retransmission is sometimes modeled abstractly as the *duplication* of a message in the system.

Since the message wrapper `msg_from_to_` is used for all messages in transmission, message loss and duplication can be modeled by the following modules:

```
(omod MESSAGE-LOSS is including MESSAGE-WRAPPER .
  var MC : MsgContent .    vars O O' : Oid .

  rl [lose-msg] :  msg MC from O to O'  =>  none .
endom)

(omod MESSAGE-DUPLICATION is including MESSAGE-WRAPPER .
  var MC : MsgContent .    vars O O' : Oid .

  rl [duplicate-msg] :
    msg MC from O to O'
  =>
    (msg MC from O to O') (msg MC from O to O') .
endom)

(omod MESSAGE-LOSS-DUPLICATION is
  including MESSAGE-LOSS + MESSAGE-DUPLICATION .
endom)
```

Another solution is to have a “shark” object that swims in the configuration and devours and duplicates messages:

```
class Shark .

rl [devour-msg] :
  (msg MC from O to O') < O'' : Shark | > => < O'' : Shark | > .

rl [duplicate-msg] :
  (msg MC from O to O') < O'' : Shark | >
=>
  < O'' : Shark | > (msg MC from O to O') (msg MC from O to O') .
```

The advantage of the last solution is that it can easily be modified to model a setting where, say, at most 20 messages are lost or duplicated in a single execution. Its disadvantage is the lack of concurrency, and that it defines a less elegant model.

Exercise 165 Extend your specification of a population with the “standard” rules for asynchronous separation (including the rule *sep2*), so that it includes (synchronous rules) for divorce, engagement, marriage, etc.

1. Use Full Maude’s search capabilities to show that a married couple can turn into a couple in which one of them is married to the same spouse, while the other spouse is separated, and there is no pending (unread) *separate* message in the system. (This case corresponds to the case of “JR” and “Sue Ellen” remarrying and then one of them discovers the old *separation* message.)

2. Use Maude's search capabilities to show that, starting from a normal state in which "JR" and "Sue Ellen" are married and "Cally" is single, it is possible to reach a state in which "JR" is separated from "Cally", "Cally" is married to "JR", and there is no message pending.

Exercise 166 Consider the correct solution to the separation problem.

1. Repeat the searches for the bad states described in Exercise 165 in the new protocol. (Hint: You may want to set a lower maximal age to speed up the search.) Can you state that the protocol is correct based on these executions?
2. Set the age limit in the `birthDay` rule to 25, and find all states without messages that are reachable from the above initial state. Do they all look OK?

Exercise 167 A node wants to distribute an important message to all other nodes (that are reachable from the sender) in a network where each node knows its neighbors. There is only one message to transmit. The following protocol achieves this:

- The sender multicasts the very important message to its neighbors.
 - When a node reads an important message for the first time, it stores the content of the message, and multicasts the message to its neighbors except the node from which it just received the message.
 - When a node receives an important message but has already received some important message (hopefully the same message), it just ignores the message.
1. Specify the protocol in Full Maude.
 2. Define an initial state `initState` corresponding to the case when node *b* wants to distribute a very important message, and where
 - node *a* has neighbors *b* and *e*,
 - node *b* has neighbors *a* and *d*,
 - node *c* has neighbors *d*,
 - node *d* has neighbors *b*, *c*, and *e*, and
 - node *e* has neighbors *a* and *d*.
 3. Execute the protocol using Full Maude's `frew` command.
 4. Use Full Maude's search command to check that each final state reachable from `initState` is as expected.

Exercise 168 Assume that there are three classes `Satellite`, `HouseWithAntenna`, and `HouseWithoutAntenna`. Modify the definition of `broadcast` so that a broadcast message only reaches objects of class `HouseWithAntenna`. Test your specification.

Exercise 169 Define the function `withinTransRangeOf` for wireless broadcast, when locations are points (x,y) in the plane, and r is the transmission range.

Exercise 170 To save battery power, wireless devices may send wireless signals with different signal strength. Define a model of wireless broadcast where the sender broadcasts messages of the form `wl-broadcast content from o withRange r`, where r is the transmission distance.

Exercise 171 Define a class *LimitedShark*, whose objects can cause at most 10 message losses and 10 message duplications during an execution.

Exercise 172 (Somewhat tricky?) Atomic multicast [54] is an important primitive used to order events (such as conflicting distributed transactions) in distributed systems. Any node can atomically multicast a message to a set of receivers. Messages that are atomically multicast (possibly by different nodes) must be read in pairwise the same order: if nodes n_3 and n_4 both receive the atomically multicast messages m_1 and m_2 , they must receive (or “be served”) m_1 and m_2 in the same order. (Note that m_2 can be received before m_1 even if m_2 was atomically multicast after m_1 .)

Such atomic multicast does not necessarily impose a global order on all events. If each of the messages m_1 , m_2 , and m_3 is atomically multicast to two of the receivers A, B, and C, then A can read m_1 before m_2 , B can read m_2 before m_3 , and C can read m_3 before m_1 . These reads satisfy the pairwise same order requirement, since there is no conflict between any pair of receivers. Nevertheless, atomic multicast has failed to globally order the messages m_1 , m_2 , and m_3 . If atomic multicast is used to impose a global order, it should also satisfy the following uniform acyclic order property: the relation $<$ on (atomic-multicast) messages is acyclic, where $m < m'$ holds if there exists a node that reads m before m' .

1. Define atomic multicast communication which satisfies the pairwise same order requirement in Maude, so that a sender can atomically multicast a message by sending a “message” atomic-multicast mc from sender to receivers. How should the rewrite rules for receiving a message look like? Hint: The state may need to maintain a global “table” of received (and not-yet-received?) messages.
2. Define an atomic multicast primitive which also satisfies the uniform acyclic order property in Maude.
3. Analyze your models of both forms of atomic multicast, using search, to ensure that: (i) all messages have been received in a consistent order; (ii) the messages can be received in any order that satisfies the corresponding consistency requirement; and (iii) there are no deadlocks; that is, all atomically multicast messages can be read.

11.3 Ordered Asynchronous Communication using Links

Ordered message delivery typically means that a sequence of messages sent from a node a to a node b are received by b in the order in which they were sent by a .

An infrastructure that provides ordered communication can be seen as a *link* (or a *channel* or a *buffer*) between two components, and can therefore be abstractly modeled using *link* objects. A *one-directional* link from a node (with identifier) a to a node b can be represented by an object

$$\langle a \text{ to } b : \text{Link} \mid \text{content} : mc_1 :: mc_2 :: \dots :: mc_k \rangle$$

where $mc_1 :: mc_2 :: \dots :: mc_k$ is a list of message (contents) traveling from a to b , with mc_1 the “first” message. Since the sender and receiver are given by the “name” of the link object, it is enough to store just the *message contents* in the links:

```
(omod LINK is including MESSAGE-CONTENT .
  sorts MsgContentList .
  subsort MsgContent < MsgContentList .
  op nil : -> MsgContentList [ctor] .
  op _::_ : MsgContentList MsgContentList -> MsgContentList
                                     [ctor assoc id: nil] .

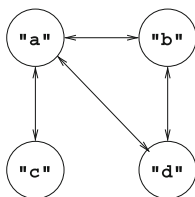
  op _to_ : Oid Oid -> Oid [ctor] .    --- link names

  class Link | content : MsgContentList .
endom)
```

A *bidirectional* communication link can be modeled by two one-directional links:

```
< a to b : Link | content : mc1 :: mc2 >
< b to a : Link | content : mc3 >.
```

The global state should contain one `Link` object (two for two-ways communication channels) between each pair of nodes that are connected. The network



can be represented by the state

```
< "a" : Node | ... >          < "b" : Node | ... >
< "c" : Node | ... >          < "d" : Node | ... >
< "a" to "b" : Link | ... >   < "b" to "a" : Link | ... >
< "a" to "c" : Link | ... >   < "c" to "a" : Link | ... >
< "a" to "d" : Link | ... >   < "d" to "a" : Link | ... >
< "b" to "d" : Link | ... >   < "d" to "b" : Link | ... >.
```

A message is sent by inserting its content at the *back* of the link, so that the sending of a message with content mc from an object a to an object b is modeled by a rule of the form

```
var MCL : MsgContentList .

rl [send-mc] :
  < a : ... | ... >
  < a to b : Link | content : MCL >
=>
  < a : ... | ... >
  < a to b : Link | content : MCL :: mc > .
```


An object *b* reads the “next” message (content) from an object *a* by removing the *first* element in the link from *a* to *b*:

```
rl [read-mc] :
  < b : ... | ... >
  < a to b : Link | content : mc :: MCL >
=>
  < b : ... | ... >
  < a to b : Link | content : MCL > .
```

11.3.1 Unreliable Links

A *lossy* link, i.e., a link in which messages in transit can be lost, can be modeled as an object of the following subclass `LossyLink`, where the rule `lose-msg` models the loss of any message (content):

```
class LossyLink .
subclass LossyLink < Link .

vars MCL MCL' : MsgContentList .   var MC : MsgContent .
vars SOURCE DEST : Oid .

rl [lose-msg] :
  < SOURCE to DEST : LossyLink | content : MCL :: MC :: MCL' >
=>
  < SOURCE to DEST : LossyLink | content : MCL :: MCL' > .
```

A link which allows for a message in transmission to be duplicated can be modeled as an object of the following class `DuplLink`:

```
class DuplLink .
subclass DuplLink < Link .

rl [duplMsg] :
  < SOURCE to DEST : DuplLink | content : MCL :: MC :: MCL' >
=>
  < SOURCE to DEST : DuplLink | content : MCL :: MC :: MCL' :: MC > .
```

Finally, the following class `UnrelLink` specifies links where messages can get lost as well as getting duplicated during transmission:

```
class UnrelLink .
subclass UnrelLink < LossyLink DuplLink .
```

For full generality, rewrite rules involving sending and receiving messages should mention links of the superclass `Link`, so that they apply to all kinds of links. The initial states should then specify exactly what kind of links are used in each case. In this way, we can easily model systems with different kinds of links: some links may be reliable while other links can be lossy and/or duplicating.

11.3.2 Links with Limited Capacity

In many cases messages are dropped because the link is full. A link which can transport at most N messages can be modeled by the following class `BoundedLink`:

```
class BoundedLink | content : MsgContentList, capacity : NzNat,
                    currentSize : Nat .
```

where `currentSize` is the size of the list in the `content` attribute.³ The sending of a message m through such a link should be modeled by rules of the forms

```
cr1 [send-OK] :
  < a : ... >
  < a to b : BoundedLink | content : MCL, capacity : NZ,
                        currentSize : N >
  =>
  < a : ... >
  < a to b : BoundedLink | content : MCL :: m, currentSize : sN >
  if N < NZ .

rl [send-full] :
  < a : ... >
  < a to b : BoundedLink | capacity : NZ, currentSize : NZ >
  =>
  < a : ... >
  < a to b : BoundedLink | > .
```

Exercise 173 Sometimes, for example in fiber-optic cables, it should be possible to insert an element into a link and read an element from the link at the same time.

1. Explain why an object cannot insert an element into a link while another object reads an element from the same link.
2. Define a link model that allows such concurrency by representing a single link using two objects

```
< a to b : LinkFront | front : mcl >
< a to b : LinkBack | back : mcl' >
```

and by using an equation to move messages from the back of the link to the front.

3. Prove that in the resulting specification

```
< "o1" : Node | ... >      < "o2" : Node | ... >
< "o1" to "o2" : LinkFront | front : m3 :: m2 >
< "o1" to "o2" : LinkBack | back : nil >
```

rewrites in one concurrent step (in which "o1" sends $m1$ and "o2" reads $m3$) to

```
< "o1" : Node | ... >      < "o2" : Node | ... >
< "o1" to "o2" : LinkFront | front : m2 :: m1 >
< "o1" to "o2" : LinkBack | back : nil >.
```

³ The `currentSize` attribute is not needed, since its value can be computed given the `content` value; however, it is usually more efficient to have such an attribute.

11.4 Asynchronous Communication Using Shared Variables

Instead of sending messages, different components may communicate by writing to and reading “shared variables.” An easy way to model such communication is to let a shared variable x be represented by an object

```
< x : SharedVar | value : v >
```

where v is the current value of x . If the shared variable ranges over elements of sort s , such objects are instances of the class

```
class SharedVar | value : s .
```

If the system contains shared variables of different sorts $s_1, \dots, s_n$, one could either

1. declare a class `SharedVars $s_i$` for each s_i ,
2. let each s_i be a subsort of a supersort `Data` and let `Data` be the sort of the `value` attribute, or
3. define a sort `Data` and an operator `[_]` : $s_i \rightarrow \text{Data}$ for each sort s_i , so that a variable is represented by an object `< x : SharedVar | value : [ v ] >`.

Exercise 174 In Exercise 120 we analyzed “by hand” what could happen if three persons deposit \$20 each to a shared bank account x at the same time (but in different branches of the bank). In particular, a bank clerk:

- first checks the current balance of the account x , and stores this result in a local variable y (e.g., a post-it note on his desk);
- receives \$20 from the depositor and computes the new balance of the bank account in a new post-it note/local variable ($z := y + 20$); and finally
- writes the value of z as the new balance of the account x .

That is, each bank clerk performs the program

$$y := \text{read}(x); \quad z := y + 20; \quad \text{write}(x, z);$$

where x is a shared variable and y and z are local variables. Each statement is atomic (can be executed in one step), but since the three bankers perform these operations more or less at the same time, the execution of the statements can be interleaved: other clerks may execute statements also between the execution of two statements by any given clerk.

1. Model this system in Full Maude, with each clerk represented by an object.
2. Use Full Maude search to find all the possible balances of the account x after the three persons have deposited \$20 each, when the original balance was \$100.
3. What are the possible outcomes if also y and z are shared variables?

(In databases, the three instances of the program above are three transaction (requests), and any database management system is expected to ensure atomicity (either all operations or no operations in a transaction are applied to the database))

and serializability of concurrent transactions: the result of executing the three transactions in parallel must be the same as some execution without interleaving. Such transaction support would ensure that no deposit was lost in our example.)

Exercise 175 Multiple agents (Orbitz, Expedia, Priceline, etc.) access a global database for flight tickets, searching for a certain trip, for which there is only one seat left. Each agent *a* performs the following transaction:

```
x := read(seat);
/* If seat is free, wait until the customer makes up her mind.
   Then ask for name and credit card details. Takes time */
if x == free then {
  y := getCreditCardDetails();
  if ok(y) then {write(seat, sold(a)); chargeCustomer(y);}
}
```

1. Model a system with multiple agents and one desired plane ticket, also recording which agents charged its customer. Assume that there are two customers, and either (a) both customers want to buy the plane ticket (modeled by `ok(y)` being `true`), and (b) only one customer wants to buy the ticket.
2. Use search and show that something can go wrong in case (a).
3. Add a test whether the ticket is still available just before selling the ticket (this is one atomic action: a database query). Can something still go wrong? What went wrong is called a “lost update” in the database community.
4. Variants of the following solution are often used in practice:

```
x := read(seat);
if x == free then {
  write(seat, sold(a)); /* Hold ticket for up to 15 minutes */
  y := getCreditCardDetails(); /* Takes some time */
  if ok(y) then chargeCustomer(y) else write(seat, free);
}
```

What is the disadvantage of this solution?

It seems hard to solve this problem without either blocking the data item too long (leading to unsold tickets) or selling the same ticket twice. Chapter 13 discusses some approaches to make the best of the situation.

This chapter illustrates how (Full) Maude can be used to model and analyze a series of protocols for achieving reliable ordered communication on top of an underlying unreliable transmission medium. For example, the IP protocol does not guarantee reliable delivery of a single message, and may also reorder a sequence of messages between two nodes as they cross the Internet. The TCP protocol in the transport layer of the Internet protocol stack then provides reliable ordered communication of a sequence of messages between two nodes on top of IP.

Section 12.1 specifies a simple protocol that uses *sequence numbers* and *acknowledgments* to achieve reliable and ordered delivery of a sequence of messages when the underlying infrastructure is unreliable and does not guarantee ordered delivery. If the infrastructure provides ordered, but unreliable, message delivery (lossy links), we can use the same protocol, but then we only need *two* sequence numbers. This yields the well-known *alternating bit protocol* discussed in Section 12.2.

These protocols are not very efficient: the sender must know that the receiver has seen a message before it transmits the next message. In the *sliding window protocol*, the sender may send *multiple* different messages before getting acknowledgments from the receiver. Sliding window may be the best known algorithm in computer networking, and the TCP protocol is essentially just the sliding window protocol on top of IP [96]. Section 12.3 describes the sliding window protocol for both unordered and ordered communication infrastructures, but leaves the actual Maude modeling and analysis as an exercise/course project.

12.1 Reliable Communication Using Sequence Numbers

You want to send a sequence of important messages, and want to be absolutely certain that the receiver gets *all* the messages and in the intended order. Unfortunately,

the underlying communication infrastructure (such as IP or the postal service) may lose messages, or may deliver messages out of order.

The following protocol for achieving such reliable and ordered communication of a sequence of messages between two nodes is based on adding a *sequence number* to each message. The sender part of the protocol is as follows:

1. Send the first message, together with the sequence number 1.
2. Wait for an *acknowledgment* from the receiver that it has received the message with sequence number 1. If the sender does not get such an acknowledgment within a certain time: **goto** 1.
3. Send the second message to be transmitted, with sequence number 2.
4. Wait for an *acknowledgment* for sequence number 2. If the sender does not get such an acknowledgment within a certain time: **goto** 3.
- ... And so on, for all the other messages to be transmitted.

The receiver side of this protocol is straightforward:

- Each time the receiver sees a *new* message, store it (or deliver it to its “application”), and reply with an acknowledgment for the sequence number.
- Ignore any message with a sequence number that the receiver has already seen.
- Resend an acknowledgment, with the largest sequence number it has received, from time to time.

This description includes waiting for *a certain amount of time* and then resending a message or an acknowledgment. To make our specification more abstract and to avoid dealing with real-time issues (see Section 17.1 for a way of modeling real-time systems in Maude), we ignore the actual timing features. Instead, the sender can send a message *at any time*, and the receiver can send an acknowledgment at any time. The sender protocol in this more abstract setting is therefore:

1. Send the *first* message, with sequence number 1, every once in a while, until the sender receives an acknowledgment for sequence number 1.
2. Repeatedly send the *second* message, with sequence number 2, until the sender receives an acknowledgment for this message.
- ... And so on, for all the other messages to be transmitted.

In this timeless world, the receiver

- repeatedly sends an acknowledgment for the greatest sequence number it has seen, and
- each time it sees a *new* message, it stores (or delivers to the application that calls the protocol) this new message.

12.1.1 Maude Modeling

Since we assume that message delivery may be lossy and out of order, we use the “standard” model of communication in Section 11.2. In particular, we use the message wrapper (“envelope”) there, so that each message has the form

msg content from *sender* to *receiver*.

We assume that the sender wants to transmit a sequence of strings, such as "Sequence" ++ "numbers" ++ "are" ++ "great" ++ "fun", to the receiver. The *content* of the messages from sender to receiver is then a string together with a sequence number; for example "great" withSeqNo 4. Likewise, acknowledgments should also have sequence numbers: ack withSeqNo 3. Message contents and lists of strings can be defined as expected in (Full) Maude:

```
(omod SEQNO-UNORDERED is
  including MESSAGE-LOSS . --- msg wrapper and loss
  protecting STRING + NAT .

  sort Content . --- message content without sequence numbers
  subsort String < Content . --- "messages" are just strings
  op ack : -> Content [ctor] . --- acknowledgment message

  --- sequence number wrapper:
  op _withSeqNo_ : Content Nat -> MsgContent [ctor] .

  --- lists of strings:
  sort StringList .
  subsort String < StringList .
  op nil : -> StringList [ctor] .
  op _+_ : StringList StringList -> StringList
                                     [ctor assoc id: nil] .
```

The sender is modeled as an object instance of the following class

```
class Sender | msgsToSend : StringList,
               currentMsg : StringList,
               currentSeqNo : Nat,
               receiver : Oid .
```

where

msgsToSend contains the list of strings that have not yet been sent;
currentMsg denotes the “current” string to send;
currentSeqNo denotes the sequence number of the “current” message; and
receiver denotes the (identifier of the) receiver object.

(The attribute *currentMsg* has sort *StringList* instead of *String* so that it may also contain the empty list *nil* when there are no more messages to send.)

The sender protocol is simple. First, the sender gets ready by setting the first string to be transmitted into *currentMsg* and set *currentSeqNo* to 1:

```
vars N N' : Nat .      var NZ : NzNat .      vars O O' : Oid .
var S : String .      var SL : StringList .

rl [start] :
  < O : Sender | msgsToSend : S ++ SL, currentMsg : nil >
=>
  < O : Sender | msgsToSend : SL, currentMsg : S,
               currentSeqNo : 1 > .
```

The sender repeatedly sends the current string with the current sequence number (the rule cannot be applied when `currentMsg` is `nil`, since `S` is a variable of sort `String`):

```
rl [sendCurrentMsg] :
  < O : Sender | currentMsg : S, currentSeqNo : N,
    receiver : O' >
=>
  < O : Sender | >
  msg (S withSeqNo N) from O to O' .
```

If the sender gets an acknowledgment for the *current* sequence number, it prepares for the sending of the next message. If the current string was the last to be sent, `currentMsg` is set to `nil`, and the sender will not send more messages:

```
rl [receiveCurrentAckNotLast] :
  (msg (ack withSeqNo N) from O' to O)
  < O : Sender | currentSeqNo : N, msgsToSend : S ++ SL >
=>
  < O : Sender | currentSeqNo : N + 1, currentMsg : S,
    msgsToSend : SL > .

rl [receiveAckLast] :
  (msg (ack withSeqNo N) from O' to O)
  < O : Sender | currentSeqNo : N, msgsToSend : nil >
=>
  < O : Sender | currentSeqNo : N + 1, currentMsg : nil > .
```

A sender just ignores acknowledgments of older messages:

```
cr1 [rcvTooOldAck] :
  (msg (ack withSeqNo N) from O' to O)
  < O : Sender | currentSeqNo : N' >
=>
  < O : Sender | > if N < N' .
```

The receiver protocol is simple: repeatedly acknowledge the greatest sequence number seen. For analysis purposes, we also store the sequence of received strings:

```
class Receiver | greatestSeqNoRcvd : Nat,
  sender : Oid,
  msgsRcvd : StringList .
```

The receiver repeatedly sends an acknowledgment for the greatest sequence number it has seen:

```
rl [sendAck] :
  < O : Receiver | greatestSeqNoRcvd : NZ, sender : O' >
=>
  < O : Receiver | >
  msg (ack withSeqNo NZ) from O to O' .
```

When the receiver receives a *new* message, it stores the content of the new message and updates its `greatestSeqNoRcvd` attribute:


```

rl [rcvNewPacket] :
  (msg (S withSeqNo s N) from O' to O)
  < O : Receiver | greatestSeqNoRcvd : N, msgsRcvd : SL >
=>
  < O : Receiver | greatestSeqNoRcvd : s N,
                    msgsRcvd : SL ++ S > .

```

Finally, the receiver ignores messages which it has already seen:

```

crl [rcvOldPacket] :
  (msg (S withSeqNo N) from O' to O)
  < O : Receiver | greatestSeqNoRcvd : N' >
=>
  < O : Receiver | >    if N <= N' .
endom)

```

12.1.2 Formal Analysis

To analyze our protocol, we define an initial state *init*, in which "Alice" wants to use the protocol to transmit the sequence "Sequence" ++ "numbers" ++ "are" ++ "great" ++ "fun" of strings to "Bob":

```

(omod TEST-SEQNO-UNORDERED is including SEQNO-UNORDERED .
  subsort String < Oid .
  op init : -> Configuration .    --- initial state

  eq init
    = < "Alice" : Sender | msgsToSend : "Sequence" ++ "numbers" ++
                                         "are" ++ "great" ++ "fun",
      currentMsg : nil,
      currentSeqNo : 0,
      receiver : "Bob" >
      < "Bob" : Receiver | greatestSeqNoRcvd : 0, msgsRcvd : nil,
      sender : "Alice" > .
  endom)

```

We can then use Maude rewriting to quickly test our protocol:

```
Maude> (frew [200] init .)
```

```

result Configuration :
  < "Bob" : Receiver | greatestSeqNoRcvd : 5, sender : "Alice",
    msgsRcvd :
      ("Sequence" ++ "numbers" ++ "are" ++ "great" ++ "fun") >
  < "Alice" : Sender | currentMsg : nil, currentSeqNo : 6,
    msgsToSend : nil, receiver : "Bob" >
  msg ack withSeqNo 5 from "Bob" to "Alice"

```

Although this looks good, we have just analyzed *one* out of the many possible behaviors. We use search to analyze all possible behaviors. The following command searches for a *bad* state in which the receiver has received sequence number 5, but where its stored sequence of strings is different from the desired one:

```

Maude> (search [1] init =>+
      C:Configuration
      < "Bob" : Receiver | msgsRcvd : SL:StringList,
        greatestSeqNoRcvd : 5 >
      such that SL:StringList =/=
        "Sequence" ++ "numbers" ++ "are" ++ "great" ++ "fun" .)

```

The execution of this search command does not terminate (before the operating systems kills it), since (i) the reachable state space is infinite, and (ii) such a bad state should not be reachable, and hence Maude searches forever for the unreachable bad state. A *bounded* search (`search [1,25] ...`), which checks whether the bad state can be reached in 25 rewrite steps or less, will terminate.

Although the fact that the search command does not find bad states increases our confidence in the correctness of the protocol, it does *not* allow us to conclude that the protocol is correct. It could happen that bad states could be found if we searched for a few more hours/days/years. Furthermore, we have only analyzed the protocol for *one* initial state. Maybe the protocol behaves incorrectly for *other* initial states?

Exercise 176 Consider the protocol defined above.

1. Explain why the set of states reachable from *init* is infinite.
 2. The receiver part of the protocol is not terminating: the receiver can never stop sending messages. Why not?
 3. Can you modify the protocol so that the receiver side is terminating, whereas the sender side may become nonterminating?
 4. Can you modify the protocol so that both sides are terminating and know that the receiver has received all messages?
 5. Use Maude to check whether a state in which the receiver has stored "great" right after "numbers" is reachable from *init* within 25 rewrite steps.
-

12.2 The Alternating Bit Protocol

Assume now that the underlying communication infrastructure provides *ordered* but *lossy* message transmission. That is, the communication can be seen to take place using lossy links, as explained in Section 11.3.1. The above protocol can of course be used to achieve reliable communication also in such an infrastructure, with the only difference that we use link objects for the communication (Exercise 177).

However, this solution is not optimal when a large number of messages are transmitted, since the sequence numbers can become very large. The point is that all those sequence numbers are no longer needed when communication is through lossy (but not duplicating) links. It is enough to consider the sequence numbers 0 and 1. Each sequence number n in the original protocol is just replaced by its *parity* $n \bmod 2$: the first packet to be transmitted gets sequence number 1, the second packet gets sequence number 0, the third packet gets sequence number 1, the fourth gets the number 0, and so on. The reason we can do this optimization is that

if the largest sequence number in the current state of system is n , then each message/acknowledgment in the links has sequence number n or $n - 1$ (Exercise 177).

This optimized protocol is the well-known *alternating bit protocol*, which can be summarized as follows:

1. Use the protocol from Section 12.1, but with messages traveling in lossy links.
2. Each sequence number n in that protocol is replaced by its parity bit $n \bmod 2$.

The following Maude specification of the alternating bit protocol is a straightforward modification of our specification in Section 12.1:

```
(fmod BIT is sort Bit .          --- data type for bits
  ops 0 1 : -> Bit [ctor] .
  op not : Bit -> Bit .
  eq not(0) = 1 .
  eq not(1) = 0 .
endfm)

(omod MESSAGES is              --- same as before, except with bits
  protecting STRING + BIT .    including MESSAGE-CONTENT .
  sort Content .
  subsort String < Content .
  op ack : -> Content [ctor] .
  op _withBit_ : Content Bit -> MsgContent [ctor] .
endom)

(omod ALTERNATING-BIT-PROTOCOL is
  including STRING-LIST + MESSAGES .
  including LOSSY-LINK .      --- Links and the rule lose-msg

  --- Sender protocol:
  class Sender | msgsToSend : StringList,
                  currentMsg : StringList,
                  currentBit : Bit,
                  receiver : Oid .

  vars B B' : Bit .          vars O O' : Oid .          var S : String .
  var SL : StringList . var MCL : MsgContentList .

  rl [start] :
    < O : Sender | msgsToSend : S ++ SL, currentMsg : nil >
  =>
    < O : Sender | msgsToSend : SL, currentMsg : S,
                  currentBit : 1 > .

  rl [sendCurrentMsg] :
    < O : Sender | currentMsg : S, currentBit : B,
                  receiver : O' >
    < O to O' : Link | content : MCL >
  =>
    < O : Sender | >
    < O to O' : Link | content : MCL :: (S withBit B) > .
```

```

rl [receiveCurrentAckNotLast] :
  < O : Sender | currentBit : B, msgsToSend : S ++ SL >
  < O' to O : Link | content : (ack withBit B) :: MCL >
=>
  < O : Sender | currentBit : not(B), currentMsg : S,
    msgsToSend : SL >
  < O' to O : Link | content : MCL > .

...
endom)

```

Exercise 177 In this exercise we consider our sequence number protocol in Section 12.1, but where communication is through lossy links.

1. Model the protocol in Section 12.1 where communication is through links.
2. Define a suitable initial state with lossy links.
3. Perform the same Maude analysis as in Section 12.1 on your specification.
4. Explain why the sequence numbers, in the messages in the links and in the `greatestSeqNoRcvd` attribute, are either n or $n - 1$ if the current value of `currentSeqNo` is n .
5. Use Maude search to analyze the property above: Search for a state where a message/acknowledgment in a link, or the `greatestSeqNoRcvd` attribute, has a sequence number that is two less than the sender's `currentSeqNo` attribute.

Exercise 178 In this exercise we model and analyze the alternating bit protocol.

1. Complete the above specification of the alternating bit protocol.
 2. Define an appropriate initial state with lossy links.
 3. Perform the “usual” Maude analysis:
 - a. test your specification using rewriting;
 - b. search for a bad state in which the receiver has received at least as many strings as the sender wanted to transmit, but where the sequence is different from the one the sender wanted to send; and
 - c. search for a state in which the receiver has received the desired messages.
 4. Explain why the alternating bit protocol does not work if the links also may duplicate messages according to the link model in Section 11.3.1.
 5. Use an initial state with lossy and duplicating links, and use Maude search to show that the alternating bit protocol does not work in this setting.
-

12.3 The Sliding Window Protocol

In the above protocols, the sender waits for an acknowledgment of a message before sending the next message. The two versions of the *sliding window protocol*

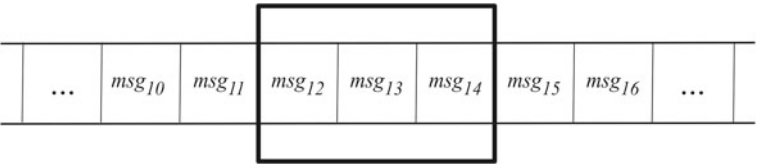


Fig. 12.1 A window of size 3 at the sender.

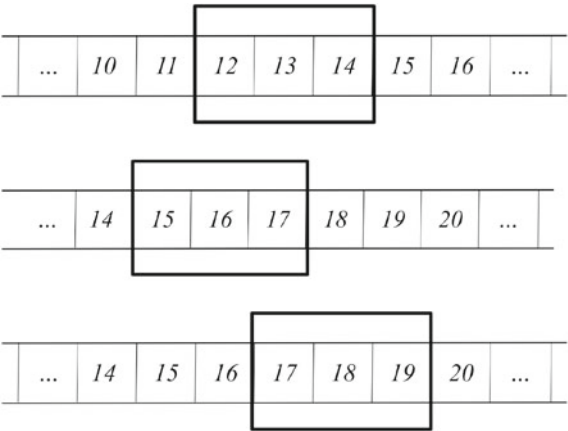


Fig. 12.2 The sliding window of the sender after receiving acknowledgments of, respectively, message 11 (top), message 14 (center), and message 16 (bottom).

presented in this section generalize our previous two protocols so that the sender can send *multiple* different messages *before* getting an acknowledgment.

Both the sender and the receiver have a *window* (or “buffer”) of a certain size, and the sender can send *any* of the messages in its sending window. For example, Figure 12.1 shows the sending window, of size 3, which currently contains the messages with sequence numbers 12, 13, and 14. The sender should continuously send these messages until it receives an acknowledgment of one of the messages. For example, if the sender receives an acknowledgment of message 14, it “slides” the window and starts sending the messages 15, 16, and 17, as illustrated in Figure 12.2. If the sender then receives an acknowledgment of message 16, it again slides the window and starts sending messages 17, 18, and 19.

The receiver keeps track of the greatest sequence number (*currentAck*) for which it has seen *all* messages with sequence number $\leq$ *currentAck*. In Figure 12.3 (top), *currentAck* is 11: the receiver has seen all messages with sequence number 1, 2, ..., 11, and has delivered them to its application. Since the receiver can receive either message 12, 13, or 14 next, it must have a buffer (“window”) in which it stores the messages that cannot be sent to the application yet. For example, if it receives message 14 next, it cannot send this message to its application, since it has

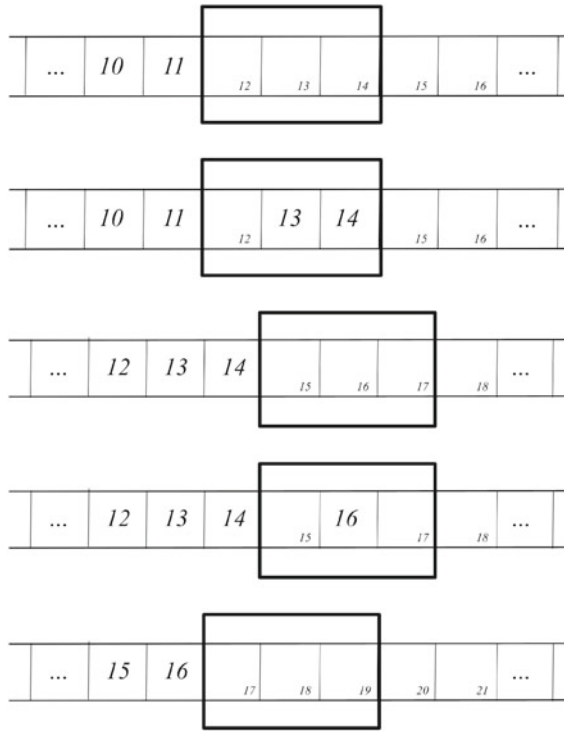


Fig. 12.3 The window of the receiver after having received all messages up to sequence number 11 (top); then after also receiving messages 13 and 14 (second row); then after also receiving message 12 (third row); then after also receiving message 16 (fourth row); and, finally, after also receiving message 15 (bottom).

not yet received messages 12 and 13. Therefore, it stores message 14 in its receiving buffer/window. If it then receives 13 and thereafter message 12, the receiver has seen the first 14 messages, and (i) transfers messages 12, 13, and 14 to its application, (ii) updates *currentAck* to 14, and (iii) “slides” its receiving window/buffer to make space for the messages 15, 16, and 17. If the receiver *instead* receives message 12 *before* message 13, it acknowledges message 12 and moves its window to make room for messages 13, 14, and 15 (note that message 14 is already buffered).

More precisely, the sender protocol goes as follows, where k is the window size:

- Initially: put the messages $1, \dots, k$ into the sending window.
- Repeatedly send *any* of the messages in the sending window.
- If the sender receives an acknowledgment (with the sequence number) for a message that is not in its sending window: just ignore the acknowledgment.
- If the sender receives an acknowledgment for a sequence number n that *is* in the sending window, put the packets with sequence numbers $n + 1, \dots, n + k$ in the sending window (unless there are no more messages to be sent).

The receiver protocol is as follows:

- Maintain a state variable *currentAck*, denoting the greatest sequence number q such that the receiver has received all messages $1, \dots, q$.
- Maintain a receiving window/buffer of length k .¹
- Repeatedly send an acknowledgment for message number *currentAck*.
- Ignore received messages with sequence number $\leq \text{currentAck}$.
- If a message with sequence number $i > \text{currentAck}$ is received:
 - If all messages with sequence numbers $\text{currentAck} + 1, \dots, i - 1$ are stored in the receiving window:
 - Let $j \geq i$ be the largest sequence number such that all the messages with sequence numbers $\text{currentAck} + 1, \dots, i - 1, i + 1, \dots, j$ are stored in the receiver's window, and such that message $j + 1$ is not.
 - Transfer messages $\text{currentAck} + 1, \dots, j$ to the application (and remove them from the receiver's window).
 - Set *currentAck* to j , “sliding” the receiver's window to $j + 1, \dots, j + k$.
 - Otherwise, store the new message in the receiver's window (if it is not there already).

This sliding window protocol—which is modeled and analyzed in Exercise 179—generalizes the protocol in Section 12.1, which can be seen as the special case of sliding window where the window size k is 1.

12.3.1 Sliding Window with Links

If communication is through lossy *links*, we can optimize the sliding window protocol, just as we did for the alternating bit protocol. It turns out that it is sufficient to use only $2k$ sequence numbers; the alternating bit protocol can then be seen as the special case of this version of sliding window when the window size $k = 1$. For example, if the window size is 3, the sequence numbers used could be $0, \dots, 5$; and the packet that comes after packet 5 has sequence number 0. We model and analyze this version of the sliding window protocol in Exercise 180.

Exercise 179 *This exercise models and analyzes the sliding window protocol in Maude when the underlying communication infrastructure provides lossy and unordered message delivery. The setting is the same as in Section 12.1: a sender wants to use the sliding window protocol to transfer a sequence of strings to a receiver.*

1. *Model the sliding window protocol (with lossy and unordered communication) in (Full) Maude by generalizing the Maude specification of the protocol in Section 12.1. Make sure that the sender can nondeterministically select to send any message in the sending window.*

¹A receiving window of size $k - 1$ is sufficient (why?).

2. Define an initial state in which the sender wants to transfer the sequence "Sliding" ++ "window" ++ "is" ++ "an" ++ "amazing" ++ "protocol". Make the window size a parameter of the initial state, so that `init(k)` denotes the initial state with window size k .
3. Use the `rew` command to test your protocol.
4. Use the `search` command to search for a state reachable from `init(4)` where the receiver has stored the entire sequence "Sliding" ++ "window" ++ "is" ++ "an" ++ "amazing" ++ "protocol" in its `msgsRcvd` attribute.
5. Repeat the same search, but from initial state `init(2)`.
6. Define a function `_prefixOf_ : StringList StringList -> Bool` which checks whether a list is a prefix of another list. Test your function.
7. Use Maude to analyze whether it is possible to reach, in less than 19 rewrite steps, a state in which the receiver's `msgsRcvd` attribute is not a prefix of "Sliding" ++ "window" ++ "is" ++ "an" ++ "amazing" ++ "protocol".

Exercise 180 In this exercise we model the version of the sliding window protocol where communication takes place through lossy (but not duplicating) links, and where we use the sequence numbers $0, 1, \dots, 2k - 1$, with k the size of the sender's window. You should analyze the protocol with both reliable links and lossy links.

1. What could go wrong if we use less than $2k$ sequence numbers? Show a bad behavior when k is 3, and only the sequence numbers $0, \dots, 4$ are used.
2. Model this version of the sliding window protocol in Maude.
3. Define initial states corresponding to those in Exercise 179. Define one parametric initial state with reliable links and one with lossy links.
4. Perform all the analyses in Exercise 179, for both lossy and reliable links. (If needed, use a smaller window size (2 or 3) and/or fewer strings stored (4 or 5).)
5. Make a rough estimate of the number of states encountered during a search:
 - a. What is the smallest number of rewrite steps needed to go from an initial state with window size 4 to a state in which the receiver has stored all 6 messages in its `msgsRcvd` attribute?
 - b. How many different rewrite steps can be performed from a state in which the sending window is full and each lossy link contains two messages?
 - c. Based on the answers to the above questions, give a very rough estimate of the size of the "search tree" from the initial state until a good final state is reached. (This search tree will contain multiple copies of the same state, but nevertheless gives you an impression of the state space encountered during a Maude search.)
6. Modify your specification so that the sequence numbers are $0, \dots, 2k - 2$ and use Maude analysis to show that the protocol then does not work correctly; use window size $k = 3$, or, if that analysis does not terminate within a reasonable amount of time, use $k = 2$.

This chapter shows how Maude can be used to formally model and analyze a number of textbook distributed algorithms; that is, algorithms (or protocols) in which a number of nodes use message passing communication to achieve a common goal.

Section 13.1 explains in detail how Maude can model and analyze the *two-phase commit* protocol for transactions on distributed databases, and includes a discussion on general techniques for modeling node failures and recoveries. Sections 13.2–13.4 treat, respectively, distributed mutual exclusion algorithms, distributed leader election algorithms, and distributed consensus algorithms. The algorithms discussed are cornerstones of state-of-the-art cloud computing and wireless systems. For example, the two-phase commit protocol, distributed leader election, and the Paxos consensus algorithm mentioned in Section 13.4 are all key building blocks in Google’s Mega-store cloud computing infrastructure used for Gmail, Google+, and AppEngine [9].

13.1 Atomicity of Distributed Transactions: Two-Phase Commit

A *transaction* is a sequence of operations on databases that should logically be seen as a *single* operation. In particular, either *all* operations in a transaction are *committed* (actually applied to the databases) or *no* operation is committed.

A transaction may write to *multiple* databases or to *replicated* databases. It is then often necessary to ensure that either *all* participating sites commit the transaction, or that *no* site commits the transaction. The *two-phase commit protocol* [66] is a well-known protocol that tries to achieve this.

Distributed Transactions.

An upscale travel agent may issue the following transaction for a person *X* who wants to visit Paris, stay at the Ritz, and have dinner at *Chez M*:

```
reserve(X, OSL-CDG, KLM, Dec 6 to 15);  
reserve(X, Ritz, Imperial Suite, Dec 6 to 15);  
reserve(X, Chez M, dinner, Dec 9);  
pay(X, 6000, MasterCard, 1234567891234567, 11/17, ...);
```

This is a *distributed transaction*, as it involves operations on airline reservation sites, dinner reservation sites, hotel reservation sites, and a payment processing site. The transaction must be treated as an *atomic* transaction: either the entire transaction “goes through” or no operation is committed. For example, if the Imperial Suite at Ritz is not available, or all tables at Chez M are reserved, or the payment does not go through, then the *entire* transaction must be cancelled, or *aborted*.

Replicated Databases.

Databases may be *replicated* (or “copied”) for various reasons. First of all, it would be highly imprudent of your local bank, national tax authority, university, etc., to have only a single database with all critical data. If that database fails ...

Another reason is that many web services, such as search engines, social media sites, electronic payment processing (credit/debit cards), online auction sites, airplane reservation sites, and so on, should be available *anywhere* and *all the time*, even when servers fail or are being upgraded, and even under peak load, network congestion, etc. If your favorite search engine, social network, or airline reservation system is often slow or unavailable, you will start using another service. To achieve this availability, the databases used by such services must be widely replicated.

A replicated database should preferably be *consistent*: all replicas should have the same values. Unfortunately, it is impossible in general to have both very high availability, tolerance w.r.t. network and site failures, and consistency (see Exercise 181). Many widely-replicated sites, such as search engines and social media sites, can live with inconsistent data in different replicas. For other kinds of replicated databases, such as your local bank, it is quite important that data are consistent in the different replicas: If you deposit \$1000 to your bank account, you want this transaction to be committed in *all* replicas; if not, it is better to abort the transaction and try another day. Likewise, the replicas of a world-wide online auction service must be consistent, so that a single item is not sold to different bidders.

Therefore, for both distributed transactions and transactions on replicated data, it is often necessary to ensure that a transaction either goes through in all nodes, or that it is aborted in all nodes.

13.1.1 The Two-Phase Commit Protocol

The *two-phase commit* (2PC) protocol [66] tries to achieve atomicity of transactions on multiple sites: either *all* distributed components *commit* to physically update the databases, or *no* component does so. Furthermore, if some participant votes to abort

the transaction, then no updates are performed, and if all nodes can commit, then all components should be updated. The databases are not physically updated *during* the database transaction. Instead, the database is physically changed only at the end of the transaction if everything went well in *each* database (replica).

The 2PC protocol starts by selecting some component to be the *coordinator*. The two phases of 2PC are then given as follows in the textbook [40, Chapter 23]:¹

Phase 1. When all participating databases signal the coordinator that the part of the multi-database transaction involving each has concluded, the coordinator sends a message *prepare for commit* to each participant to get ready for committing the transaction. Each participating database receiving that message will force-write all log records and needed information for local recovery and then send a *ready to commit* or *OK* signal to the coordinator. If the force-writing to disk fails or the local transaction cannot commit for some reason, the participating database sends a *cannot commit* or *not OK* signal to the coordinator. If the coordinator does not receive a reply from the database within a certain amount of time, it assumes a *not OK* response.

Phase 2. If *all* participating databases reply *OK*, and the coordinator's vote is also *OK*, the transaction is successful, and the coordinator sends a *commit* signal for the transaction to the participating databases. [...] Each participating database completes transaction commit by writing a *commit* entry for the transaction in the log and permanently updating the database if needed. On the other hand, if one or more of the participating databases or the coordinator have a *not OK* response, the transaction has failed, and the coordinator sends a message to *roll back* or *UNDO* the local effect of the transaction to each participating database. This is done by undoing the transaction operations.

Notice that 2PC can solve the problem in the world-wide online auction site where two bidders, one in Norway and one on Tanna, Vanuatu, both (try to) bid in the dying seconds of the auction: Before the bid from Norway is committed, all replicas must accept it; however, the replica closest to Tanna could veto the conflicting bid. The result would be that no bid is committed, and no one gets the item.

13.1.2 Abstraction

When analyzing 2PC we are interested in whether the different databases are updated or not; we are not interested in their actual *content*, which therefore can be abstracted away. The description of 2PC says that “if the coordinator does not receive a reply from the database within a certain amount of time, it assumes a *not OK* response.” We could use timers to capture this, which would give us a more precise description of 2PC, but at the cost of having to deal with *time*. Instead, we abstract from time and the details of how the underlying timeout mechanism detects the loss of a message, and assume that a *prepare for commit* message always gets a reply, where the timeout scenario above corresponds to receiving a *not OK* message. Other aspects of a database system, such as reading and writing from/to the database, do not appear in the description of the 2PC protocol and do not need to be modeled.

¹Elmasri, Ramez; Navathe, Shamkant B., FUNDAMENTALS OF DATABASE SYSTEMS, 6th Ed., ©2011. Reprinted by permission of Pearson Education, Inc., New York, New York.

13.1.3 Assumptions

The above description of 2PC leaves many assumptions implicit. Is communication ordered or unordered? Is communication reliable? Does a node know the other nodes? Reading the textbook carefully and/or having experience in database theory would answer these questions: communication can be assumed to be unordered; each node that will ever be a coordinator knows all the other nodes; and communication may be unreliable. Furthermore, nodes may crash and then recover. The point is that an executable formal specification makes all such assumptions explicit.

13.1.4 Specification and Analysis of 2PC in Maude

This section shows how 2PC can be formally specified and analyzed using Maude. We first specify and analyze 2PC *without* communication and site failures. Section 13.1.4.3 then analyzes 2PC in the presence of message losses. Section 13.1.4.4 presents some general techniques for modeling site failures and recoveries in Maude that allow us to analyze 2PC also in the presence of site failures.

13.1.4.1 Maude Specification of 2PC Without Failures

Each component of the database is modeled as an object of the class 2PCDB:

```
class 2PCDB | updated : Bool, state : CommitState, veto : Bool,
              otherNodes : OidSet, coordState : CoordState .

sort CoordState .
op notCoord : -> CoordState [ctor] .      --- not coordinator
op waitFor : OidSet -> CoordState [ctor] . --- wait for replies

sort CommitState .
ops initial ready abort : -> CommitState [ctor] .
```

The attribute `updated` is `true` if and only if the database has performed the update. `state` is the internal state of the node (`initial` in the beginning; and then the node decides whether *it* is `ready` to commit or must `abort`). `otherNodes` denotes the other nodes, `coordState` is `notCoord` for nodes that are not currently coordinators, and is `waitFor(os)` when a coordinator is waiting for replies from the nodes `os`, and, finally, `veto` is `true` if the coordinator has received a veto.

The messages are declared as follows, where a “message” `startCommit` starts a run of the protocol:

```
ops prepare OK notOK abort commit : -> MsgContent [ctor] .
msg startCommit : Oid -> Msg .
```

2PC starts with the coordinator (the node receiving the `startCommit` message) sending a `prepare` message to all the other nodes, and going into waiting mode:

```

vars O O' : Oid .   var OS : OidSet .

rl [prepareReq] :
  startCommit(O)
  < O : 2PCDB | state : initial, otherNodes : OS >
=>
  < O : 2PCDB | coordState : waitFor(OS) >
  multicast prepare from O to OS .

```

When a node gets a prepare message, it replies OK or notOK:

```

rl [ok] :
  (msg prepare from O to O')
  < O' : 2PCDB | state : initial >
=>
  < O' : 2PCDB | state : ready >
  (msg OK from O' to O) .

rl [notOK] :
  (msg prepare from O to O')
  < O' : 2PCDB | state : initial >
=>
  < O' : 2PCDB | state : abort >
  (msg notOK from O' to O) .

```

The coordinator itself should also vote (see also Exercise 182):

```

rl [coordNotOk] :
  < O : 2PCDB | state : initial, coordState : waitFor(OS) >
=>
  < O : 2PCDB | state : abort, veto : true > .

rl [coordOk] :
  < O : 2PCDB | state : initial, coordState : waitFor(OS) >
=>
  < O : 2PCDB | state : ready > .

```

In the second phase, the coordinator reads the responses and decides whether or not to order a global abort or a global commit. First, it reads the responses, and sets veto to true if some node cannot commit:

```

rl [recOK] :
  (msg OK from O' to O)
  < O : 2PCDB | coordState : waitFor(O' ; OS) >
=>
  < O : 2PCDB | coordState : waitFor(OS) > .

rl [recNotOk] :
  (msg notOK from O' to O)
  < O : 2PCDB | coordState : waitFor(O' ; OS) >
=>
  < O : 2PCDB | coordState : waitFor(OS), veto : true > .

```

Next, the coordinator sends its decision and stops being a coordinator (and updates its own database if needed):

```

rl [commitAll] :
  < O : 2PCDB | coordState : waitFor(none),
                    otherNodes : OS, veto : false >
=>
  < O : 2PCDB | coordState : notCoord, updated : true >
  (multicast commit from O to OS) .

rl [abortAll] :
  < O : 2PCDB | coordState : waitFor(none),
                    otherNodes : OS, veto : true >
=>
  < O : 2PCDB | coordState : notCoord, updated : false >
  (multicast abort from O to OS) .

```

Finally, the other nodes receive the coordinator's decision and decide whether to physically update the database:

```

rl [recAbort] :
  (msg abort from O to O')
  < O' : 2PCDB | >
=>
  < O' : 2PCDB | updated : false > .

rl [recCommit] :
  (msg commit from O to O')
  < O' : 2PCDB | >
=>
  < O' : 2PCDB | updated : true > .

```

13.1.4.2 Analyzing 2PC Without Message Loss

Our specification does not include rules for message loss, so we first analyze our protocol in a reliable setting. The following module, where some parts are replaced by '...', defines an initial state with five databases (or database replicas):

```

(omod TEST-2PC is including TWO-PHASE-COMMIT . protecting STRING .
  subsort String < Oid .
  op init : -> Configuration .
  eq init
  = startCommit("a")
    < "a" : 2PCDB | updated : false, state : initial,
                    otherNodes : "b" ; "c" ; "d" ; "e",
                    coordState : notCoord, veto : false >
    < "b" : 2PCDB | updated : false, state : initial,
                    otherNodes : "a" ; "c" ; "d" ; "e",
                    coordState : notCoord, veto : false >
    < "c" : 2PCDB | updated : false, state : initial,
                    otherNodes : "b" ; "a" ; "d" ; "e",
                    coordState : notCoord, veto : false >
    < "d" : 2PCDB | ... >
    < "e" : 2PCDB | ... > .
endom)

```

We start by rewriting to get some quick first feedback:

```
Maude> (frew init .)
```

```
result Configuration :
  < "a" : 2PCDB | state : ready, updated : false, ... >
  < "b" : 2PCDB | state : abort, updated : false, ... >
  < "c" : 2PCDB | state : ready, updated : false, ... >
  < "d" : 2PCDB | state : abort, updated : false, ... >
  < "e" : 2PCDB | state : ready, updated : false, ... >
```

This is promising: the databases "b" and "d" could not commit the transaction, and no database was updated. Since the rewrite command only analyzes one possible behavior, we check for consistency of the distributed databases *at the end of a run of 2PC* by searching for a “bad” *final* state in which one component has updated its database while another component has not done so:

```
Maude> (search [1] init =>! < O:Oid : 2PCDB | updated : false >
      < O':Oid : 2PCDB | updated : true >
      C:Configuration .)
```

No solution.

The result shows that it is not possible to reach an inconsistent final state from *init*. However, the correctness requirement of 2PC also says that: (i) if one database decides to abort, then no database should update; and (ii) if all databases are ready to update, then they should indeed all update. Again, we analyze these properties by searching for *final* states in which the properties do *not* hold:

```
Maude> (search [1] init =>! < O:Oid : 2PCDB | state : abort >
      < O':Oid : 2PCDB | updated : true >
      C:Configuration .)
```

No solution.

```
Maude> (search [1] init =>!
      < O1:Oid : 2PCDB | state : ready, updated : false >
      < O2:Oid : 2PCDB | state : ready >
      < O3:Oid : 2PCDB | state : ready >
      < O4:Oid : 2PCDB | state : ready >
      < O5:Oid : 2PCDB | state : ready > MSGS:Configuration.)
```

No solution.

Although everything looks good, we have *not* proved 2PC correct, only that it works well from state *init*. Maybe inconsistent states can be reached from other initial states? Nevertheless, this analysis has increased our confidence that 2PC is correct.

13.1.4.3 Analyzing 2PC with Unreliable Communication

We next analyze 2PC when messages may be lost during transmission. As mentioned in Section 13.1.2, we assume that a *prepare* request always gets a reply, so

the loss of `prepare`, `OK`, and `notOK` messages does not need to be modeled. The following module extends our model of 2PC with a rewrite rule modeling the loss of an `abort` or a `commit` message:

```
(omod TWO-PHASE-COMMIT-WITH-MESSAGE-LOSS is including TEST-2PC .
  vars O O' : Oid .    var MC : MsgContent .

  crl [lose-abortCommit] :  msg MC from O to O'  =>  none
    if MC == abort or MC == commit .
endom)
```

A first test of the new model looks very promising:

```
Maude> (frew init .)
```

```
result Configuration :
  < "a" : 2PCDB | state : ready, updated : false, ... >
  < "b" : 2PCDB | state : abort, updated : false, ... >
  < "c" : 2PCDB | state : ready, updated : false, ... >
  < "d" : 2PCDB | state : abort, updated : false, ... >
  < "e" : 2PCDB | state : ready, updated : false, ... >
```

Let us now check whether it is possible to reach an inconsistent final state:

```
Maude> (search [1] init =>! < O:Oid : 2PCDB | updated : false >
      < O':Oid : 2PCDB | updated : true >
      C:Configuration .)
```

```
Solution 1
... ; O':Oid --> "a" ; O:Oid --> "e"
```

The result shows that it is possible to reach an inconsistent final state. It is necessary to *exhibit* a behavior leading to the inconsistent state, for the following reasons:

- To ensure that the faulty behavior really corresponds to a flaw in 2PC, and is not just an error in our model of 2PC.
- To learn about the flaw in the protocol.

Since Full Maude cannot exhibit the path to a state found during a search, we use the method described in Section 10.2.4.1 to transform a Full Maude module into a (core) Maude module, repeat the search in (core) Maude, and obtain the path to the inconsistent state. The path shows that all nodes could commit; however, the `commit` message from "a" to "e" was lost, so that node "e" never updates its database.

13.1.4.4 Modeling Process Failure and Recovery

A process (server, database, etc.) can “fail” in a number of ways for various reasons. A common source of unavailability is scheduled upgrades of software or hardware. A failed process can behave in different ways, from being unresponsive (*omission failures*) to producing completely arbitrary values/messages (*Byzantine failures*).

Byzantine failures happen for example when an airplane sensor is broken and reports bogus values, or when the process is (taken over by) an *attacker* sending bogus messages. Chapter 14 defines such a Byzantine attacker on a security protocol.

This section focuses on omission failures, such as *crash* failures, where a failed process becomes unresponsive. We also model the recovery of a failed process.

Representing Failed Processes.

There are many ways of representing failed processes in Maude. One option is to add a new Boolean attribute `failed` in all classes whose objects may fail. The disadvantage is that each rewrite rule (in normal operation) needs to include the attribute/value pair `failed : false` in its left-hand side.

Another approach is to define new classes for failed nodes, and transform an object into an instance of such a class when it fails. The new classes for failed nodes must contain the attributes that are needed when the node has crashed (mostly for a possible recovery or for analysis purposes). In the case of the 2PC protocol, the textbook [23, p. 521] states: “To deal with the possibility of crashing, each server saves information relating to the two-phase commit protocol in permanent storage. This information can be retrieved by a new process that is started to replace a crashed server.” The failed process therefore saves all the information of the non-failed node, and can be represented as an object instance of the following class:

```
class Failed2PCDB | updated : Bool, state : CommitState,
                      veto : Bool, otherNodes : OidSet,
                      coordState : CoordState .
```

Modeling Failure and Recovery.

The following rewrite rule models the fact that any node could fail at any time:

```
rl [nodeFailure1] :
  < O : 2PCDB | updated : B, state : S, otherNodes : OS,
    coordState : CS, veto : B2 >
=>
  < O : Failed2PCDB | updated : B, state : S, otherNodes : OS,
    coordState : CS, veto : B2 > .
```

This rule creates a *new* object of the class `Failed2PCDB`, with the old object identifier, and deletes the old object. Since a new object is created, *all* the attributes of the new object must be present in the right-hand side.

The recovery of a failed process can be modeled by the following rewrite rule:

```
rl [nodeRecovery1] :
  < O : Failed2PCDB | updated : B, state : S, otherNodes : OS,
    coordState : CS, veto : B2 >
=>
  < O : 2PCDB | updated : B, state : S, otherNodes : OS,
    coordState : CS, veto : B2 > .
```

This model could lead to too many failures and quickly makes search unfeasible. It is often more practical and common to explicitly *inject faults* by using messages `fail` and `recover`, so that a node fails when it reads a `fail` message and recovers when it reads a `recover` message. If the message does not specify *which* node should fail, then *any* node could fail at any time. Including n such `fail` messages in the initial state would allow us to analyze the protocol with any combination of n failures, including the possibility that the same node fails multiple times. This approach is used next to analyze the 2PC protocol with process failures and recoveries.

2PC with Process Failure and Recovery.

The Maude model of 2PC with failures and recoveries, using a special failure class and fault injection as explained above, is given as follows:

```
(omod 2PC-WITH-NODE-FAILURES is
  including TWO-PHASE-COMMIT-WITH-MESSAGE-LOSS .

  class Failed2PCDB | updated : Bool,   state : CommitState,
                                veto : Bool,   otherNodes : OidSet,
                                coordState : CoordState .

  msgs fail recover : -> Msg .
```

The following rewrite rules model a node failing and recovering from failure:

```
vars O O' : Oid .      var S : CommitState . var OS : OidSet .
var CS : CoordState .  vars B B2 : Bool .   var MC : MsgContent .

rl [nodeFailure] :
  fail
  < O : 2PCDB | updated : B, state : S, otherNodes : OS,
                    coordState : CS, veto : B2 >
=>
  < O : Failed2PCDB | updated : B, state : S, otherNodes : OS,
                    coordState : CS, veto : B2 > .

rl [nodeRecovery] :
  recover
  < O : Failed2PCDB | updated : B, state : S, otherNodes : OS,
                    coordState : CS, veto : B2 >
=>
  < O : 2PCDB | updated : B, state : S, otherNodes : OS,
                    coordState : CS, veto : B2 > .
```

A failed node ignores any received messages (except `recover` messages):

```
rl [ignoreMsgs] :
  (msg MC from O to O')
  < O' : Failed2PCDB | >
=>
  < O' : Failed2PCDB | > .
```

Finally, the term `initWithFailures` defines an initial state with two arbitrary failures and only one recovery, by adding two `fail` messages and one `recover` message to the previous initial state `init`:

```
op initWithFailures : -> Configuration .
eq initWithFailures = fail fail recover init .
endom)
```

Exercise 181 Explain informally why it is impossible (in the context of replicated data stores) to have both very high availability, tolerance for network failures, and consistency. (This impossibility result is called the CAP Theorem [15].)

Exercise 182 Modify the above specification of 2PC so that the coordinator itself must choose whether it is ready to commit or wants to abort.

Exercise 183 One problem with 2PC is that the system could deadlock when the coordinator fails. Use Maude search to show that it is possible to reach a deadlocked state where no node has received a `prepare` message.

13.2 Distributed Mutual Exclusion

Multiple computers/processes may need to access *shared resources*, such as a shared printer, a shared file, or shared data. However, Exercises 120 and 174 show what can go wrong if different processes running at the same time access the same shared resource (the bank account x). To avoid undesired behaviors in which multiple processes update the same resource at the same time, a process must have *exclusive access* to the shared resource while it is using the resource. This property is called *mutual exclusion*: if one process has access to the shared resource, all the other processes are (temporarily) excluded from accessing that resource. Mutual exclusion is also needed for wireless communication: two nodes should not transmit at the same time to avoid that their signals interfere with each other.

A process that uses a shared resource is said to be in a *critical section*. For example, in Exercise 174, the “program fragment”

$$y := \text{read}(x); \quad z := y + 20; \quad \text{write}(x, z);$$

where x is a shared variable and y and z are local variables, is a critical section: other processes should *not* access/update x while the process is executing this program part. A *distributed mutual exclusion algorithm* is intended to achieve mutual exclusion between processes using message communication, and should ensure that:

1. At most one process executes in its critical section at any time.
2. If a process wants to enter its critical section, it will eventually succeed.

A much stronger fairness condition is the following:

3. The processes enter their critical sections in the order in which they wanted to enter it.

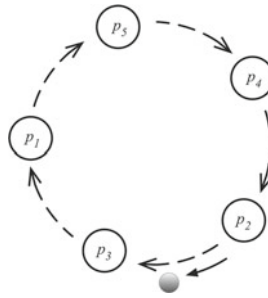


Fig. 13.1 Token ring, where the token is being sent from process p_2 to process p_3

Each process accessing a shared resource executes the following “program scheme”:

```
<execute outside critical section>;
<request to enter critical section>; // wait until access granted
<execute in critical section>;      // access shared resources
<release critical section>;
<execute outside critical section>;
```

We consider three well-known distributed mutual exclusion algorithms:

1. An algorithm where a central server gives nodes access to the critical section.
2. The “*token ring*” algorithm avoids the extra server. Instead, the nodes are logically seen as forming a “ring” as shown in Figure 13.1. Each node passes a “token” to the next node, and only the node that “has the token” can enter its critical section. This algorithm also has some disadvantages:
 - a. a node may wait for a long time to enter its critical section;
 - b. nodes communicate even if no node wants to enter its critical section; and
 - c. nodes may not enter their critical sections in the order in which they want.
3. *Maekawa’s voting algorithm* does not need an extra server, and will not send messages when no node wants to enter its critical section. Instead, a node i that wants to enter its critical section sends a message to each node in its *voting set* V_i . The node i can then only enter its critical section when all nodes in its voting set V_i allow it. This algorithm will only work if for each pair of nodes (i, j) , their respective voting sets V_i and V_j have at least one element in common. The main disadvantages of this algorithm are that more messages are sent than in the central server algorithm and that it may lead to deadlock.

The algorithms in this section assume reliable (but unordered) communication and that nodes do not fail.

We show how the central server algorithm can be modeled and analyzed in Maude, and leave the other algorithms as exercises.

13.2.1 Modeling the Central Server Algorithm

A node/process p can be modeled as an object $\langle p : \text{Node} \mid \text{state} : s \rangle$, where s is either *beforeCS* (the node is executing before accessing the shared resource),

`waitForCS` (the node is waiting to enter its critical section), `insideCS` (the node is executing in its critical section), and `afterCS` (the node has left the critical section):

```
(omod MUTEX-WITH-CENTRAL-SERVER is including MESSAGE-WRAPPER .

class Node | state : MutexState .

sort MutexState .
ops beforeCS waitForCS insideCS afterCS : -> MutexState [ctor] .
```

The central server can be modeled as an object

```
< server : MutexServer | nodeInCS : b, waiting : waiting nodes >
```

where *b* is true when a node is in its critical section, and *waiting nodes* is the list of processes that are waiting to enter their critical sections:

```
class MutexServer | nodeInCS : Bool, waiting : OidList .
op server : -> Oid [ctor] .      --- name of server object

sort OidList .      subsort Oid < OidList .
op nil : -> OidList [ctor] .
op _::_ : OidList OidList -> OidList [ctor assoc id: nil] .
```

When a node wants to enter its critical section, it sends a `requestCS` message to the server. If the server's `nodeInCS` value is false, it grants the node access to the critical section by sending an `accessGranted` message; otherwise, the requesting node is added to the server's waiting list and remains in the `waitForCS` state:

```
ops requestCS accessGranted releaseCS : -> MsgContent [ctor] .

vars O O' : Oid .      var OL : OidList .

rl [requestAccessToCS] :
  < O : Node | state : beforeCS >
=>
  < O : Node | state : waitForCS >
  (msg requestCS from O to server) .

rl [grantAccess] :
  (msg requestCS from O to server)
  < server : MutexServer | nodeInCS : false >
=>
  < server : MutexServer | nodeInCS : true >
  (msg accessGranted from server to O) .

rl [putInWaitQueue] :
  (msg requestCS from O to server)
  < server : MutexServer | nodeInCS : true, waiting : OL >
=>
  < server : MutexServer | waiting : OL :: O > .
```

```

rl [startExecutingInCS] :
  (msg accessGranted from server to O)
  < O : Node | state : waitForCS >
=>
  < O : Node | state : insideCS > .

```

When a process has finished executing its critical section, it sends a `releaseCS` message to the server. If nodes are waiting, the longest-waiting node is given access:

```

rl [exitCS] :
  < O : Node | state : insideCS >
=>
  < O : Node | state : afterCS >
  (msg releaseCS from O to server) .

rl [nooneWaiting] :
  (msg releaseCS from O to server)
  < server : MutexServer | waiting : nil >
=>
  < server : MutexServer | nodeInCS : false > .

rl [grantAccessToFirstWaiting] :
  (msg releaseCS from O to server)
  < server : MutexServer | waiting : O' :: OL >
=>
  < server : MutexServer | waiting : OL >
  (msg accessGranted from server to O') .
endom)

```

13.2.2 Analyzing the Central Server Algorithm

The term `init(n)` defines an initial state with n nodes and one server:

```

(omod MUTEX-WITH-CENTRAL-SERVER-INITIAL-STATE is
  including MUTEX-WITH-CENTRAL-SERVER . protecting NAT .
  op node : NzNat -> Oid [ctor] . --- names node(1), node(2), ...

  var N : Nat .    var NZN : NzNat .

  op init : NzNat -> Configuration . --- initial states
  eq init(NZN)
    = < server : MutexServer | nodeInCS : false, waiting : nil >
      generateNodes(NZN) .

  op generateNodes : Nat -> Configuration .
  eq generateNodes(s N)
    = < node(s N) : Node | state : beforeCS > generateNodes(N) .
  eq generateNodes(0) = none .
endom)

```

The mutual exclusion property can be analyzed in Maude by searching for a “bad” state in which the desired property does *not* hold, namely, a state where (at least) two processes are inside the critical section:

```
Maude> (search [1] init(4) =>*
      REST:Configuration < O1:Oid : Node | state : insideCS >
                        < O2:Oid : Node | state : insideCS >.)
```

No solution

It is easy to see that requests to enter the critical section eventually will succeed (why?). However, this algorithm does not ensure that processes access their respective critical sections in the order in which they wanted to access it (why not?). Section 16.3.5 explains how Maude can be used to analyze these two properties.

Exercise 184 Modify the central server mutual exclusion algorithm so that each process executes forever, alternating between executing outside and inside the critical section. Use Maude to analyze whether the mutual exclusion property is satisfied. Will the search command terminate? Is it still the case that each process will eventually be able to enter its critical section? Could the system deadlock?

Exercise 185 In the “token ring” mutual exclusion algorithm, the nodes logically form a “ring” structure, as shown in Figure 13.1 where a node only knows the next node in this ring. The algorithm works as follows: there is one “token,” and only the node that holds the token may enter its critical section. The node then holds on to the token during its execution in the critical section, and passes the token to the next node in the ring when it exits its critical section. If a node that is not waiting to enter its critical section receives the token, it just passes the token to the next node.

1. Model the token ring algorithm in Maude.
2. Use Maude to analyze whether this algorithm guarantees mutual exclusion.
3. Does the algorithm guarantee that nodes enter the critical section in the order in which they want to enter the critical section?
4. Explain why the algorithm cannot terminate, even after all nodes have finished executing their critical sections.
5. Can you modify/extend the algorithm so that it terminates?
6. Modify your model so that each node executes forever, again alternating between executing outside and inside the critical section.
7. In this new version, is it possible that a node that wants to enter its critical section never gets to do so?

Exercise 186 In Maekawa’s voting algorithm, each node i has a voting set V_i , so that any pair (V_i, V_j) of voting sets has at least one element in common: $V_i \cap V_j \neq \emptyset$.

A node that wants to enter its critical section multicasts a request message to all nodes in its voting set. The node then enters its critical section when it has received a go-ahead message from each node in its voting set. When the node exits its critical section, it multicasts a release message to the nodes in its voting set.

A node that receives a request message replies with a go-ahead message if: (i) it is not in the critical section itself, and (ii) it has not already voted (i.e., has not sent a go-ahead message) for someone without receiving a release message from that node. Otherwise, the node just queues the request.

When a node receives a release message, it sends a go-ahead message to the first node in its request queue (if any).

1. Model this algorithm in Maude.
2. Define a number of suitable initial states.
3. Use Maude to analyze whether this algorithm guarantees mutual exclusion.
4. Use Maude to analyze whether the system may deadlock.

Exercise 187 Although these algorithms were not designed to tolerate message losses and node crashes, we can nevertheless analyze what kinds of failures, if any, each algorithm can withstand. Therefore, for each of the three algorithms:

1. What messages can be lost without affecting the operation of the system?
2. What nodes (and in which circumstances) can crash (and not recover) without affecting the rest of the system?

13.3 Distributed Leader Election

A distributed system often needs to select one of the nodes to be the *leader*. For example, the two-phase commit protocol assumes that there is a leader, called the coordinator. Likewise, airplanes typically have multiple “copies” of each computer/cabinet, in case one fails; which computer is currently running the airplane?

A *leader election algorithm* should elect one of the nodes to be the leader, and all nodes should agree on the leader. If the leader crashes, then another leader must be elected. Since multiple nodes may discover that the leader is down, more than one node may initiate a leader election process. This section considers two leader election algorithms: a *ring-based* algorithm and a *spanning-tree-based* algorithm. The goal of these algorithms is to elect the node with the best value of some parameter (e.g., processor capacity, remaining amount of energy, number of Facebook friends, etc.) as the leader. These algorithms do not tolerate node or communication failures.

The *bully algorithm* [46] is a well-known leader election algorithm that can deal with node failures (and recoveries) but requires real-time features such as timeouts and time-bounded communication, since it is impossible to detect a node failure in an *untimed* asynchronous distributed system (why?).

13.3.1 A Ring-based Leader Election Algorithm

In the ring-based leader election algorithm by Chang and Roberts [18], the nodes are arranged in a logical ring and each node knows the next node in the ring.

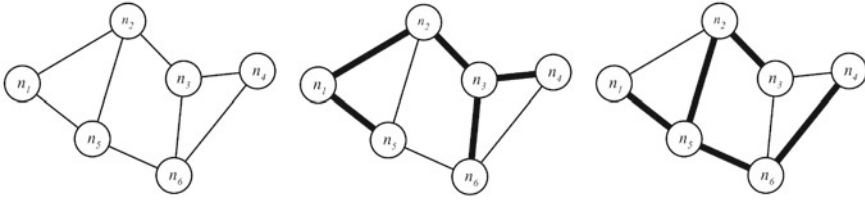


Fig. 13.2 A graph (left) and two of its spanning trees (the “thick” edges)

A node that starts a new round of the leader election algorithm, for example upon discovering that the current leader has failed, sends an *election* message, containing its own value and identity, to the next node in the ring. When a node receives an *election* message, it compares the received value with its own value: If the received value is better, the node forwards the *election* message to the next node in the ring; if the received value is worse, then the node sends an *election* message with its own value and id to the next node; and, finally, if a node receives an *election* message with its own identity,² then the node knows that it is the new leader (why?), and sends a *leader* message with its own identity to the next node in the ring. A node that receives a *leader* message, stores the identity of the new leader; furthermore, if the receiver is *not* the new leader, it forwards the *leader* message to the next node in the ring. Exercise 188 deals with modeling and analyzing this algorithm in Maude.

13.3.2 A Spanning-Tree-based Algorithm for Wireless Networks

In wireless networks, and in many other networks, a node has a number of *neighbors* that it can reach in “one hop.” It is desirable to use one-hop communication as much as possible. The ring-based algorithm is not well suited for such networks since it assumes that the nodes are arranged in a ring structure. However, finding a ring of one-hop links—if it exists—is an NP-hard problem (the “Hamiltonian Circuit” problem), and therefore quite costly. Furthermore, this must be done quite often since the topology in a wireless network may change frequently.

The following spanning-tree-based leader election algorithm assumes that each node knows its neighbors, and that the network topology is a connected undirected graph. The algorithm has three “phases”:

1. Build a “tree” of all the nodes in the graph. Such a tree is called a *spanning tree*. (Figure 13.2 shows a graph and two of its spanning trees.) The starting node sends an *election* message to its neighbors. A node that sees an *election* message for the *first* time, remembers the sender as its *parent* in the tree, and sends the *election* message to its other neighbors.

²Two nodes cannot have the same value.

2. When the spanning tree has been built, each node sends the best value in its “subtree” to its parent, starting with the leaf nodes and going towards the root. The root/starting node will receive the best value in each of its subtrees, and can determine the best-valued node in the entire system.
3. The root node then sends a *leader* message, with the new leader, to all its neighbors, who then propagate this information to their neighbors, and so on.

Phase 1 can be described in more detail as follows:

- The node starting the leader election sends an *election* message to its neighbors.
- A node that receives an *election* message for the first time, sets its *parent* to be the sender of this message. It then sends an *election* message to all other neighbors.
- A node that receives an *election* message, but not for the first time, simply replies with an *ack(0)* message.

Each node maintains a value *max* that stores the best node value that the node has seen; initially the value of *max* is the node’s own value. Phase 2 of the algorithm can then be described as follows:

- When a node has received an *ack* message from all neighbors, except its parent, it sends a message *ack(max)* to its parent (unless it is the root node).
- When a node receives a message *ack(n)*, it updates *max* to *n* if *n* is better than the node’s current *max* value.

When all this is done, the root node knows the best node in the entire system and can start propagating the identity of the leader *l* by sending a *leader(l)* message to its neighbors, who then send the message to their neighbors, and so on (Phase 3).

The Maude Model.

We assume that each node’s identifier (*Oid*) is a number > 0 which gives the node’s value, and also assume that the highest value is the “best” value.

```
(omod ST-LEADER-ELECTION is including MULTICAST . protecting NAT .
  subsort Nat < Oid .      --- object names/values are numbers

  class Node | parent : Oid,  max : Oid,  state : STstate,
                leader : Oid,  neighbors : OidSet .

  sort STstate .
  ops idle waitForLeader : -> STstate [ctor] .
  op waitForAck : OidSet -> STstate [ctor] .
```

The *max* attribute denotes the best value in the node’s subtree and is initially set to the node’s value; *parent* and *leader* are initially 0. The *state* is *idle* before the node starts the election, is *waitForAck(nodes)* when the node awaits *ack* messages from *nodes*, and is *waitForLeader* after the node has sent an *ack* to its parent.

A message *electLeader(n)* starts the algorithm with *n* as the starting node. This node sets itself as its parent and multicasts an *election* message to its neighbors:

```

msg electLeader : Oid -> Msg .      --- kick off leader election
op election : -> MsgContent [ctor] .
ops ack leader : Oid -> MsgContent [ctor] .

vars MAX O O1 LEADER MAX : NzNat .      var N : Nat .
var OS : OidSet .                        var S : STstate .

rl [startLeaderElection] :
  electLeader(O)
  < O : Node | neighbors : OS >
=>
  < O : Node | state : waitForAck(OS), parent : O >
  (multicast election from O to OS) .

```

When a node receives an election message for the first time (the node is *idle*), it remembers its parent, sets its state to wait for acknowledgments from its other neighbors, and propagates the election message to those neighbors:

```

rl [rcvElection1] :
  (msg election from O1 to O)
  < O : Node | state : idle, neighbors : O1 ; OS >
=>
  < O : Node | parent : O1, state : waitForAck(OS) >
  (multicast election from O to OS) .

```

A node that is already in an election (the state is different from *idle*) just replies with an *ack(0)* message when it receives another election message:

```

crl [rcvElection2] :
  (msg election from O1 to O)
  < O : Node | state : S >
=>
  < O : Node | >
  (msg ack(0) from O to O1)    if S /= idle .

```

When a node receives an *ack* message, from a “child” or a “sibling” in the spanning tree, it removes the sender from the set of nodes from which it awaits an *ack* message, and updates its *max* attribute if it received a better *max* value:

```

rl [rcvAck] :
  (msg ack(N) from O1 to O)
  < O : Node | state : waitForAck(O1 ; OS), max : MAX >
=>
  < O : Node | state : waitForAck(OS), max : max(MAX, N) > .

```

When a node has received all the *acks* it is waiting for, it sends an *ack* message to its parent with the best-value node in its subtree:

```

crl [ackParent] :
  < O : Node | state : waitForAck(none), max : MAX, parent : O1 >
=>
  < O : Node | state : waitForLeader >
  (msg ack(MAX) from O to O1)    if O1 /= O .

```

When the root node (whose `parent` points to itself) has received all the `acks` it is waiting for, its `max` attribute denotes the best node in the entire tree. The root node then starts Phase 3 of the protocol by propagating the new leader downstream:

```
rl [sendLeader] :
  < O : Node | state : waitForAck(none), neighbors : OS,
    max : MAX, parent : O >
=>
  < O : Node | state : idle, leader : MAX >
  (multicast leader(MAX) from O to OS) .
```

A node that sees the `leader` message for the first time stores the new leader and propagates the `leader` message further downstream:

```
rl [rcvLeader1] :
  (msg leader(LEADER) from O1 to O)
  < O : Node | state : waitForLeader, neighbors : O1 ; OS >
=>
  < O : Node | state : idle, leader : LEADER >
  (multicast leader(LEADER) from O to OS) .
```

Finally, a node that has already seen the `leader` message just ignores it:

```
rl [rcvLeader2] :
  (msg leader(LEADER) from O1 to O)
  < O : Node | state : idle >
=>
  < O : Node | > .
endom)
```

This model only allows one round of the leader election algorithm; I leave it to the reader to come up with an extension supporting multiple elections.

Maude Analysis.

The following module defines an initial state `init1` with three nodes:

```
(omod ST-LEADER-STATES is protecting ST-LEADER-ELECTION .
  op init1 : -> Configuration .
  eq init1
    = electLeader(1)
    < 1 : Node | state : idle, max : 1, parent : 0, leader : 0,
      neighbors : 2 ; 3 >
    < 2 : Node | state : idle, max : 2, parent : 0, leader : 0,
      neighbors : 1 ; 3 >
    < 3 : Node | state : idle, max : 3, parent : 0, leader : 0,
      neighbors : 1 ; 2 > .
endom)
```

The algorithm should terminate with node 3 as the leader. To analyze whether this is the case, we search for a final state where some node has a different leader:

```
Maude> (search init1 =>!
      C:Configuration < O:Oid : Node | leader : N:Nat >
      such that N:Nat /= 3 .)
```

- Exercise 188** 1. Model the ring-based leader election algorithm in Maude.
 2. Use Maude to analyze whether all final states have the correct leader.
 3. If your model does not support multiple elections, extend it to multiple elections and analyze your model in Maude.

Exercise 189 Assume that a node may fail, but that the failed node is so kind as to let its predecessor know about both the failure and its next neighbor in the ring. Show that the “obvious extension” of the ring-based algorithm, that just bypasses the failed node, may fail to terminate.

Exercise 190 Extend the spanning-tree-based algorithm to deal with multiple elections at the same time. You may assume that a node never initiates more than one election. Hint: Maybe it is useful to label each round of the algorithm with its initiator, and just “vacate” the leader election process initiated by a lower-valued node? Model and analyze your algorithm in Maude.

13.4 Consensus Algorithms

As already mentioned, the replication needed for availability-critical services could lead to situations where one replicating site in the online auction system sells the unique item X to bidder A , whereas another site sells the same item to bidder B .

The two-phase commit protocol can be used to avoid the untenable situation that two persons are sold the same item, by:

- not *committing* (“finalizing”) transactions until the two-phase commit protocol has been executed; and
- allowing the site selling item X to bidder B to veto another site’s attempt to commit the sale of item X to bidder A .

This solution has some disadvantages:

1. Instead of aborting conflicting transactions, so that the item is not sold to anybody, it would be better if the replicas could *agree* on the buyer of item X .
2. The two-phase commit protocol requires that *all* replicating sites can commit before the transaction is committed. However, in systems with a large number of replicating sites, most of the time some replica would be down or unreachable. This would make it impossible for most transactions to go through, even if just one replicating site is down. It is often better to allow transactions to go through, and then let the failed sites “catch up” when they recover.

To solve the first issue, all non-failed sites should *reach consensus* on a certain value, such as the buyer of item X . However, reaching consensus in an asynchronous system when nodes may fail is in general impossible [44]. A solution to the second issue is to agree on a value, or agree to commit, if a *majority* of the sites agree.

The goal of a distributed *consensus algorithm* is to have all nodes agree on a meaningful value (such as the buyer of item X). Since this is in general impossible, a consensus algorithm should: (i) *ensure* that two different nodes never “agree” on different values, and (ii) make it *possible* for all non-failed nodes to agree on a value. Notice that both reaching consensus about whether to commit or abort a transaction and electing a leader can be seen as special cases of reaching consensus.

The following is a simple algorithm for trying to reach consensus on a value in a distributed system where messages may be lost and/or nodes can fail and recover:

1. *Elect a leader.* A node proposes itself as the leader. If a majority of the nodes agree on a leader, a leader is elected.
2. *Propose a value.* The leader proposes a value to all other nodes.
3. *Count replies.* If a majority of the nodes reply “ok,” the value is selected.
4. *Send out the value.* The leader sends out the agreed-upon value to all nodes.

This algorithm does not assume bounds on the communication times, and it allows communication and replicating sites to fail. Furthermore, as shown in Exercise 191, the algorithm ensures that different nodes do not “agree” on different values, and that all (non-failed) nodes will agree on a value if we are lucky.

This algorithm also has some disadvantages, including:

- Multiple nodes may propose themselves as leaders, and neither gets a majority.
- The leader may fail.

One idea to improve the situation is to run this algorithm again if it fails to achieve consensus the first time. This is not entirely trivial, but is the main idea of one of the most celebrated algorithms in distributed systems: the *Paxos Consensus Algorithm* by Leslie Lamport [64,65]. This somewhat hard-to-understand protocol is a key part of many cloud computing systems. It is unfortunately beyond the scope of this introductory book to describe the Paxos algorithm, but modeling Paxos in Maude should be an interesting exercise.

Exercise 191 *Model the consensus algorithm described above in Maude. Include the possibility of message losses and that a site may fail (to ensure termination, it might be useful not to model node recovery). Define a number of suitable initial states, and use Maude to analyze the following properties:*

1. *It is impossible to reach a state in which two nodes “agree” on different values.*
2. *It is possible to reach a final state in which all nodes agree on a value.*
3. *It is possible to reach a final state in which no node has agreed on a value.*
4. *It is possible to reach a final state in which no node has been elected leader.*

Exercise 192 *Explain how nodes easily can reach consensus if they have access to an atomic multicast primitive (see Exercise 172).*

Exercise 193 *(Slightly tricky?) Model and analyze the Paxos algorithm in Maude. The paper [65] gives a fairly precise and brief description of Paxos.*

Web services such as email, photo, social networks, internet commerce, and online banking require that entities *authenticate* themselves. Scrooge McDuck *must* be sure that he (it?) is communicating with the bank, and not with some bad guy with a look-a-like web page. Likewise, when the bank gets the request “transfer 5 gazillions from my account to the Beagle Boys” from “Scrooge,” the bank must be sure that it is communicating with Scrooge and not with the Beagle Boys.

Back in the 20th century, such *mutual authentication* was trivial: you knew that you were entering your bank by its imposing building, and the bank authenticated you by asking you to show some photo identification. But how can we achieve authentication online? Messages can be faked, communication can be overheard and/or intercepted, and genuine-looking web sites can easily be set up.

Authentication protocols are used to achieve the desired authentication. In this chapter we model and analyze one of the most well known and influential mutual authentication protocols: the *Needham-Schroeder public-key authentication protocol* (NSPK) [88] from 1978. Is this protocol secure, or can the Beagle Boys fool the bank into thinking that it is has a trusted connection with Scrooge?

Instead of thinking hard and trying to break this well-known and well-studied protocol (see, e.g., [17, 79]) by finding some really clever attacks, we will do a “brute force” analysis of the protocol by adding an *intruder* to the system, and by modeling *all possible* behaviors of an intruder. If the protocol is safe with such intruders, then the protocol *is* safe.

14.1 Public-Key Cryptography

In *public-key cryptography* [28, 98] each *agent A* has a *public key*, denoted PK_A , and a *private key*, denoted $PrvK_A$. All agents know the public key of each

agent,¹ but the private key of an agent A is *only* known by A . An agent which knows key K can *encrypt* the *plaintext* data m with K . The data m encrypted with key K is written $\{m\}_K$. Data which have been encrypted with a public key PK_A can only² be *decrypted* with the private key $PrvK_A$ i.e., only by A . Likewise, data encrypted with $PrvK_A$ can only be decrypted with PK_A .³

The amazing thing about public-key cryptography is that two parties Alice and Bob can communicate secretly *without* having a *shared* secret key! (This is obviously very useful when you want to communicate securely, for example sending credit card numbers, with a web service that you have not interacted with before.) If Alice wants to send a secret message m that only Bob can understand, she just sends the message encrypted with Bob's public key ($\{m\}_{PK_{Bob}}$). Only Bob can decrypt this message; no other agent who sees this message can decrypt it. This does not solve the authentication problem, however: Bob cannot be sure that Alice sent the message; *everybody* knows Bob's public key and can send the message.

Public-key cryptography is based on finding public/private key pairs and encryption/decryption algorithms so that it is computationally infeasible (meaning that it should take large networks of computers many years) to:

1. figure out the private key of an agent, and
2. decrypt an encrypted message without knowing the decryption key.

The RSA algorithm is the main framework for public-key cryptography. It is based on selecting two very large (1024 bits or so) prime numbers p and q ; their product $n = p \cdot q$ is part of the public key. RSA cryptography relies on the fact that it is impossible to factor n into its two constituents p and q within reasonable time.⁴ (If there were a *quick* way to factor a very large number, for example by quantum computing [104], then RSA-based public-key cryptography would no longer work.)

Encryption and decryption in RSA is done by modular exponentiation: the encrypted version of the plaintext message m is $\{m\}_{(n,e)} = m^e \bmod n$, and decryption also uses modular exponentiation: $\text{decrypt}(\{m\}_{(n,e)})_{(n,d)} = (\{m\}_{(n,e)})^d \bmod n = m$, where (n, e) and (n, d) is a public key/private key pair, with the secret d easily obtained from e and the prime factors p and q of $n = p \cdot q$.

14.1.1 Digital Signatures

In real life a person signs a document to prove that (s)he wrote/saw the document and to ensure that the document cannot be forged. Public-key cryptography can be used to “sign” digital contracts. If Peter wants to sign a contract m (such as “Peter owes the bank \$1000”) with the bank so that:

¹An agent can get the public key of another agent from a trusted *key server*, but we abstract from such details. Such a key server setup is, however, itself a nontrivial issue.

²This is called the *perfect cryptography assumption*. In reality, keys could be weak enough to be broken, depending on available technology.

³In some cryptosystems, the public key cannot be used to decrypt encrypted data.

⁴In 2009, a 768-bit RSA number n was factored by a state-of-the-art distributed implementation using around two thousand CPU years in total [61].

1. the bank knows—and can prove—that Peter has agreed to the contract, and
2. neither Peter nor the bank can later fake the contract (to either “Peter owes the bank \$1” or to “Peter owes the bank \$1,000,000”)

then Peter just encrypts the contract m with his *private key* and sends the encrypted message $\{m\}_{PrvK_{\text{Peter}}}$ to the bank.

The bank can now decrypt the received message with *Peter’s public key*: if the result is as expected, then the bank knows that Peter signed the document (nobody else could send the message $\{m\}_{PrvK_{\text{Peter}}}$). Furthermore, Peter cannot later on claim that the contract has been altered (since the bank can just present $\{m\}_{PrvK_{\text{Peter}}}$ and decrypt it), and the bank cannot fake the contract, since it cannot produce the encrypted version $\{m'\}_{PrvK_{\text{Peter}}}$ of the *faked* message m' .

As explained below, public-key cryptography is somewhat inefficient. The *entire* message m is therefore usually not encrypted. Instead, a hash function h “shortens” the message to $h(m)$, and the pair $(m, \{h(m)\}_{PrvK_{\text{Peter}}})$ is sent to the bank.

14.1.2 Symmetric-Key Cryptography

One problem with public-key cryptography is that encryption/decryption is computationally costly. It takes time to perform the modular exponentiation at the heart of RSA. Public-key cryptography is therefore not well suited to encrypt large data.

In *symmetric-key* cryptosystems, the two parties that want to exchange information securely therefore share (the same) *secret key*, and use this shared secret key to communicate secretly. The point is that DES or AES encryption/decryption used in symmetric-key cryptography is typically between hundred and many thousand times faster than RSA encryption/decryption.

The problem with symmetric-key cryptography is that the two parties first must authenticate themselves and then must agree on a shared secret key. Public-key cryptography can be used to establish a secure channel between two agents and then be used to agree on a shared secret key between the two agents. The ensuing secret communication then takes place using the much faster secret keys. This is what happens for example in the TLS protocol.

14.2 The Needham-Schroeder Public-Key (NSPK) Protocol

The Needham-Schroeder public-key authentication protocol (NSPK) [88] uses *nonces*, which are “freshly” generated random numbers to be used in a single run of the protocol. It is assumed that these numbers cannot be guessed by other agents. A nonce generated by an agent A is denoted N_a below.

The NSPK protocol is described as follows in [73, 79]:

Message 1.	$A \rightarrow B :$	$A . B . \{N_a . A\}_{PK_B}$
Message 2.	$B \rightarrow A :$	$B . A . \{N_a . N_b\}_{PK_A}$
Message 3.	$A \rightarrow B :$	$A . B . \{N_b\}_{PK_B}$

The agent A is the *initiator* who wants to establish a communication session with the *responder* B .

In the first step, A generates the nonce N_a , adds her identity A , encrypts this concatenation $N_a . A$ with the public key of B , and sends this encrypted message, together with her own and B 's name (unencrypted) to B . When B receives this first message, he decrypts the encrypted part using his private key $PrvK_B$ to obtain the nonce N_a . Only A and B know the value of N_a at this stage, even if there are eavesdroppers "listening" to the messages being transmitted in the network. (Why?)

The responder B then generates his own nonce N_b , and returns the nonce N_a along with the new nonce N_b , encrypted with the public key of A . In addition, B adds the names B and A (unencrypted) and sends this Message 2 back to A . When A receives this Message 2 she decrypts it with her private key to read both N_a and N_b . It seems that at this stage of the protocol run A should be assured that she is talking to B while B cannot be sure that he is talking to A .

To convince B that he is talking to A , the initiator A encrypts the received nonce N_b with B 's public key, and sends the message back to B (together with the receiver and sender names). Since only A could decrypt Message 2, only A and B know N_b , and when B receives $\{N_b\}_{PK_B}$ he is convinced that only A could have sent this message. At the end of a protocol run A is convinced that she is talking to B , and B is convinced that he is talking to A .

Exercise 194 Assume that we have intruders who can send fake messages but cannot guess private keys and nonces. After A has received Message 2,

1. why would it seem that A should be assured that she is talking to B ?, and
2. why cannot B be sure he is talking to A ?

Exercise 195 Does it seem necessary to encrypt N_b in Message 3? What do you think is the reason for this encryption?

Exercise 196 Can you indicate how the NSPK protocol can be extended/used to establish a secret key between two (mutually authenticated) agents?

14.3 Modeling NSPK in Maude

This section shows how the NSPK protocol can be modeled in Maude. Although the informal specification of NSPK only describes a single run of the protocol, our

model allows more than two agents in the system and also allows multiple concurrent runs, or *sessions*, of the protocol. An agent can be either an *initiator*, a *responder*, or both initiator and responder (in different runs of the protocol). For simplicity I assume that an agent A can initiate at most one run of the protocol with the same responder. Two agents may however simultaneously initiate contact with each other.

For reasons explained above we assume that: (i) no agent can successfully guess the value of a nonce or a private key whose value it does not know, (ii) no agent can decrypt a ciphertext (encrypted plaintext) whose decryption key it does not know, and (iii) no agent can encrypt plaintext with a key whose value it does not know.

Modeling Nonces and Keys. We abstract from the numerical value of a nonce, and represent the i -th nonce generated by agent A by the term $\text{nonce}(A, i)$:

```
(omod NSPK is protecting NAT . including MESSAGE-WRAPPER .
  sort Nonce .
  op nonce : Oid Nat -> Nonce [ctor] .
```

The public key of A is modeled by a term $\text{pubKey}(A)$:

```
sort Key .
op pubKey : Oid -> Key [ctor] .
```

It is not necessary to model the private keys since we assume that *only* the agent A can decrypt a ciphertext which was encrypted with the public key of A .

Modeling the Messages. The three messages in the protocol all have the form $O_1 . O_2 . \{message\ content\}_K$ where *message content* is either a nonce and an agent identifier, two nonces, or just a single nonce. This part of the message content is modeled by the following sort `PlainTextMsgContent`:

```
sorts PlainTextMsgContent EncrMsgContent .
op _;_ : Nonce Oid -> PlainTextMsgContent [ctor] . --- Message 1
op _;_ : Nonce Nonce -> PlainTextMsgContent [ctor] . --- Message 2
subsort Nonce < PlainTextMsgContent . --- Message 3
```

(where we use ‘;’ instead of ‘.’ as the concatenation operator).

The specification uses the following syntax for *encrypted* message contents:

```
op encrypt_with_ : PlainTextMsgContent Key -> EncrMsgContent [ctor] .
```

Finally, a message is equipped with the (presumed!) sender and receiver identities; they are included in the usual message wrapper, which means that an encrypted message content is the content of a message sent around the network:

```
subsort EncrMsgContent < MsgContent .
```

For example, a particular Message 1 could be represented by the term

```
msg (encrypt (nonce(A, 3) ; A) with pubKey(B)) from A to B.
```

Modeling Initiators. An agent which can *initiate* a run of the protocol is modeled as an object of the following class `Initiator`:

```
class Initiator | initSessions : InitSessions, nonceCtr : Nat .
```

The initiator must remember the nonce it sent in Message 1, so that it can check whether this is the same nonce that it receives in Message 2. Since an initiator may be simultaneously involved in many runs of the protocol, it must remember the nonces in all these sessions. The attribute `initSessions` of an initiator A stores such information in a multiset of elements of the following kinds:

- `notInitiated( $B$ )` indicates that A wants to initiate contact with B but has not yet done so;
- `initiated( $B, N$ )` indicates that A has sent Message 1 to B with nonce N and is waiting for Message 2 from B ; and
- `trustedConnection( $B$ )` indicates that A has established (what she thinks is) an authenticated connection with B .

The data type representing this kind of information is defined as follows:

```

sorts Sessions InitSessions .
subsort Sessions < InitSessions .
op emptySession : -> Sessions [ctor] .
op ___ : InitSessions InitSessions -> InitSessions
      [ctor assoc comm id: emptySession] .
op ___ : Sessions Sessions -> Sessions
      [ctor assoc comm id: emptySession] .
op notInitiated : Oid -> InitSessions [ctor] .
op initiated : Oid Nonce -> InitSessions [ctor] .
op trustedConnection : Oid -> Sessions [ctor] .

```

The attribute `nonceCtr` denotes the index of the next nonce generated by the object.

The following variables are used in the definition of the initiator:

```

vars A B : Oid .          vars M N : Nat .
vars NONCE NONCE' : Nonce .  var IS : InitSessions .

```

The rule `send-1` models sending Message 1. The agent A has `notInitiated( $B$ )` in its `initSessions` attribute, which means that it wants to establish a connection with B . The agent A generates a fresh nonce `nonce( $A, N$ )` and sends the corresponding Message 1 to B . Agent A must also remember that it has initiated contact with B using nonce `nonce( $A, N$ )` and must increase its nonce counter:

```

rl [send-1] :
  < A : Initiator | initSessions : notInitiated(B) IS,
                    nonceCtr : N >
=>
  < A : Initiator | initSessions : initiated(B, nonce(A, N)) IS,
                    nonceCtr : N + 1 >
  msg (encrypt (nonce(A, N) ; A) with pubKey(B)) from A to B .

```

In rule `read-2-send-3` an agent A receives a Message 2 from B . If the first nonce (`NONCE`) in the message received (and decrypted) by A is the same as the nonce stored in A 's `initSessions` attribute for B , then agent A figures out that it has established an authenticated connection with B , and sends Message 3 (B 's nonce (`NONCE'`) encrypted with B 's public key) to B :

```

rl [read-2-send-3] :
  (msg (encrypt (NONCE ; NONCE') with pubKey(A)) from B to A)
  < A : Initiator | initSessions : initiated(B, NONCE) IS >
=>
  < A : Initiator | initSessions : trustedConnection(B) IS >
  msg (encrypt NONCE' with pubKey(B)) from A to B .

```

Modeling Responders. A responder is modeled as an object of class `Responder`:

```

class Responder | respSessions : RespSessions, nonceCtr : Nat .

```

The attribute `respSessions` keeps track of the sessions in which the agent is responder; a value `responded(A, N)` means that the agent has received Message 1 from `A` and has responded using its own nonce `N`:

```

sort RespSessions .
subsort Sessions < RespSessions .
op __ : RespSessions RespSessions -> RespSessions
      [ctor assoc comm id: emptySession] .
op responded : Oid Nonce -> RespSessions [ctor] .

```

The rule `read-1-send-2` models the reception of Message 1. The condition `not A inSession RS` ensures that the responder `B` is not already a responder in a session with the initiator `A`. When `B` receives the message, it creates its own nonce (`nonce(B, N)`) and sends this nonce together with the received nonce (`NONCE`), appropriately encrypted, back to `A`:

```

var RS : RespSessions .

crl [read-1-send-2] :
  (msg (encrypt (NONCE ; A) with pubKey(B)) from A to B)
  < B : Responder | respSessions : RS, nonceCtr : N >
=>
  < B : Responder | respSessions : responded(A, nonce(B, N)) RS,
                    nonceCtr : N + 1 >
  msg (encrypt (NONCE ; nonce(B, N)) with pubKey(A)) from B to A
  if not A inSession RS .

```

The second, and last, responder rule models the reception of Message 3 with the expected nonce from `A`:

```

rl [read-3] :
  (msg (encrypt NONCE with pubKey(B)) from A to B)
  < B : Responder | respSessions : responded(A, NONCE) RS >
=>
  < B : Responder | respSessions : trustedConnection(A) RS > .

```

Agents that are Both Initiators and Responders. An agent that may be both initiator and responder is modeled as an object instance of the class `InitAndResp`, which is a subclass of both `Initiator` and `Responder` and therefore inherits the union of the attributes of these classes, as well as their rewrite rules:

```

class InitAndResp .
  subclass InitAndResp < Initiator Responder .
endom)

```

14.3.1 Executing the NSPK Specification

To analyze NSPK in the absence of “bad guys” we define an initial state `init2` with three agents “a”, “Bank”, and “c”. The agents “a” and “c” may initiate a session with each other simultaneously (remember the “separation problem”?). Furthermore, “a” does *not* want to establish communication with “Bank”, so the “Bank” should never have a trusted connection with “a”.

```

(omod TEST-NSPK is including NSPK . protecting STRING .
  subsort String < Oid .

  op init2 : -> Configuration .
  eq init2
    = < "a" : InitAndResp | initSessions : notInitiated("c"),
                          respSessions : emptySession,
                          nonceCtr : 1 >
      < "Bank" : Responder | respSessions : emptySession,
                          nonceCtr : 1 >
      < "c" : InitAndResp | initSessions : notInitiated("Bank")
                          notInitiated("a"),
                          respSessions : emptySession,
                          nonceCtr : 1 > .

endom)

```

We quickly check all *final* states reachable from `init2`:

```
Maude> (search init2 =>! C:Configuration .)
```

Solution 1

```

C:Configuration -->
  < "Bank" : Responder | nonceCtr : 2,
                          respSessions : trustedConnection("c")>
  < "a" : InitAndResp | initSessions : trustedConnection("c"),
                      nonceCtr : 3,
                      respSessions : trustedConnection("c")>
  < "c" : InitAndResp | initSessions : trustedConnection("Bank")
                      trustedConnection("a"),
                      nonceCtr : 4,
                      respSessions : trustedConnection("a")>

```

No more solutions.

All behaviors lead to the single final state in which all the desired connections have been established: the protocol seems to be doing its job in the absence of “bad guys.”

14.4 Modeling Intruders

This section presents a model of an *intruder* (also called *attacker*, *adversary*, *enemy*, etc.) which allows us to analyze our protocol in the presence of “bad guys.”

Since messages may be transmitted over an unprotected network, we use the well-known “Dolev-Yao” intruder model [30, 79] where an intruder can:

- *Overhear* and/or *intercept* (steal) messages that are sent around in the system.
- Decrypt messages that are encrypted with its own public key.
- Introduce new messages into the system, using nonces that the intruder knows.
- Replay any message it has seen, even if it cannot understand the encrypted part of the message. The intruder may change the plaintext parts of such messages.

The intruders are assumed to be part of the computer network and can also take part in normal runs of the protocol [79]. (After all, an intruder must contact the bank as an ordinary agent to reap the benefits of his illegal activities.) This also means that an intruder knows the protocol being used.

The following specification defines *all possible* behaviors of an intruder, most of which make no sense whatsoever. The point is that *if* the protocol can withstand all possible attacks, then it is secure (under the perfect cryptography assumption).

The following variables are used to specify the intruder:

```
(omod NSPK-INTRUDER is
  including NSPK .    including OID-SET .

  vars NONCE NONCE' : Nonce .    var NSET : NonceSet .
  var ENCRMSG : EncrMsgContent . var ENCRMSGs : EncrMsgContentSet .
  var N : Nat .                var MSGC : PlainTextMsgContent .
  vars A B I O O' O'' : Oid .    var OS : OidSet .
  var IS : InitSessions .        var RS : RespSessions .
```

The intruder is modeled as an object instance of the following class `Intruder`:

```
class Intruder | initSessions : InitSessions,
                  respSessions : RespSessions,  nonceCtr : Nat,
                  agentsSeen : OidSet,
                  noncesSeen : NonceSet,
                  encrMsgsSeen : EncrMsgContentSet .
```

Since an intruder is also a normal actor, it has all the attributes of a normal agent. In addition, an intruder stores the information it gathers in three attributes:

- `agentsSeen` contains the set of agent identifiers known by the intruder;
- `noncesSeen` contains the set of nonces the intruder knows; and
- `encrMsgsSeen` contains the set of encrypted message contents which the intruder has seen without being able to decrypt.

The sort `NonceSet` is defined as expected:

```

sort NonceSet .
subsort Nonce < NonceSet .
op emptyNonceSet : -> NonceSet [ctor] .
op __ : NonceSet NonceSet -> NonceSet
                                     [ctor assoc comm id: emptyNonceSet] .
eq NONCE NONCE = NONCE .

```

The sort `EncrMsgContentSet` is defined in the same way.

Four rewrite rules describe the intruder’s “normal protocol behaviors.” These rules correspond to the two rules for initiators and the two rules for responders, except that the intruder also stores information about agents and nonces it sees. The following rule shows the one for receiving Message 1; the other intruder “protocol rules” are left for Exercise 197:

```

crl [intruder-receive-message-1] :
  (msg (encrypt (NONCE ; A) with pubKey(I)) from A to I)
  < I : Intruder | respSessions : RS,      nonceCtr : N,
                    agentsSeen : OS,      noncesSeen : NSET >
=>
  < I : Intruder | respSessions : responded(A, nonce(I,N)) RS,
                    nonceCtr : N + 1,      agentsSeen : OS ; A,
                    noncesSeen : NSET NONCE nonce(I, N) >
  msg (encrypt (NONCE ; nonce(I,N)) with pubKey(A)) from I to A
  if not A inSession RS .

```

That is, when receiving Message 1, the intruder responds to the message according to the NSPK protocol. In addition, it stores the identity of the sender (A) in its `agentsSeen` attribute, and stores the received nonce `NONCE` and its own newly created nonce `nonce(I, N)` in its `noncesSeen` attribute.

The following rule `intercept-but-not-understand` models the case when an intruder *intercepts* (steals) a message which is encrypted with another agent’s public key. (Since each message in NSPK is encrypted with the public key of the intended receiver, the intruder knows that the message is encrypted with o’s public key, even though it cannot decrypt the message.) The intruder cannot decrypt the message, but stores the *encrypted* message content and the sender and receiver names:

```

crl [intercept-but-not-understand] :
  (msg ENCRMSG from O' to O)
  < I : Intruder | agentsSeen : OS, encrMsgsSeen : ENCRMSGSGS >
=>
  < I : Intruder | agentsSeen : OS ; O ; O',
                    encrMsgsSeen : ENCRMSG ENCRMSGSGS >
  if O /= I .

```

Modeling *overhearing* a message is often omitted, since it can be mimicked by first intercepting the message and then sending out the intercepted message.

Three rules (two of which are shown below) model an intruder receiving a message sent to the intruder, but which the intruder will discard after extracting information. This could be because another intruder sent a fake message or because the intruder does not want to continue a normal run of the protocol (an intruder may for example initiate a run of the protocol with another agent to obtain its nonce):


```

rl [intercept-msg1-and-understand] :
  (msg (encrypt (NONCE ; A) with pubKey(I)) from O to I)
  < I : Intruder | agentsSeen : OS, noncesSeen : NSET >
=>
  < I : Intruder | agentsSeen : OS ; O ; A,
                    noncesSeen : NSET NONCE > .

rl [intercept-msg2-and-understand] :
  (msg (encrypt (NONCE ; NONCE') with pubKey(I)) from O to I)
  < I : Intruder | agentsSeen : OS, noncesSeen : NSET >
=>
  < I : Intruder | agentsSeen : OS ; O,
                    noncesSeen : NSET NONCE NONCE' > .

```

We next model an intruder's capabilities for sending *fake messages*, using the agent identities, the nonces, and the encrypted message contents it knows.

The rule `send-encrypted` models the case in which an intruder sends a fake message with a content that it has previously stored but could not decrypt. Since the content is encrypted with B's public key, the fake message will be sent to B. The claimed "sender" could be *any* agent A whose identity the intruder knows:

```

crl [send-encrypted] :
  < I : Intruder | encrMsgsSeen :
                        (encrypt MSGC with pubKey(B)) ENCRMSGs,
                        agentsSeen : A ; OS >
=>
  < I : Intruder | >
  (msg (encrypt MSGC with pubKey(B)) from A to B)
  if A /= B .

```

(A skeptic reader may wonder whether the intruder knows that the encrypted message is encrypted with the public key of B, since that knowledge is not given from the ciphertext itself. As mentioned above, the intruder can store this information when it intercepts the message, since it can read the receiver part of the message.)

Finally, an intruder may compose any Message 1, Message 2, or Message 3 (see Exercise 197) using the nonces and agent identifiers it knows:

```

crl [send-1-fake] :
  < I : Intruder | agentsSeen : A ; B ; OS,
                    noncesSeen : NONCE NSET >
=>
  < I : Intruder | >
  (msg (encrypt (NONCE ; A) with pubKey(B)) from A to B)
  if A /= B /\ B /= I .

crl [send-2-fake] :
  < I : Intruder | agentsSeen : A ; B ; OS,
                    noncesSeen : NONCE NONCE' NSET >
=>
  < I : Intruder | >
  (msg (encrypt (NONCE ; NONCE') with pubKey(A)) from B to A)
  if A /= B /\ A /= I .

```

This ends the modeling of the intruder capabilities.

Since the intruder may send the same fake message many times, there may be multiple copies of a message in the state. However, it is easy to see (Exercise 198) that any behavior possible when the state contains multiple copies of some message is also possible when multiple copies of the message are removed. To reduce the state space, we therefore add the following equation to remove copies of a message:

```
var MSG : Msg .
eq MSG MSG = MSG .
endom)
```

Exercise 197 Specify the “missing” rewrite rules:

1. The other three rules that model the intruder behavior when the intruder engages in a normal run of the protocol.
2. An intruder intercepting a Message 3 sent to itself.
3. An intruder overhearing a message whose content it cannot understand.
4. An intruder sending a fake Message 3.

Exercise 198 Why are no behaviors lost by adding the equation $\text{eq MSG MSG} = \text{MSG}$?

Exercise 199 Are the three rules in which the intruder intercepts a message to itself really necessary? Why/why not?

14.5 Analyzing NSPK with Intruders

This section uses Maude to analyze whether the Beagle Boys can fool the bank into thinking that it has an authenticated connection with Scrooge, who does *not* want to connect to the bank. We define the following initial state `intruderInit`:

```
op intruderInit : -> Configuration .
eq intruderInit
= < "Scrooge" : Initiator |
    initSessions : notInitiated("BeagleBoys"), nonceCtr : 1 >
< "Bank" : Responder |
    respSessions : emptySession, nonceCtr : 1 >
< "BeagleBoys" : Intruder |
    initSessions : emptySession, respSessions : emptySession,
    nonceCtr : 1, agentsSeen : "Bank" ; "BeagleBoys",
    noncesSeen : emptyNonceSet, encrMsgsSeen : emptyEncrMsg > .
```

The Beagle Boys do not know any other agent, except the bank, but hope to be contacted by some rich guys after creating an enticing web site promising ... Indeed, Scrooge wants to contact the Beagle Boys but *not* the bank. Therefore, if it is possible to reach a state where the bank thinks that it has established an authenticated connection with Scrooge, then the protocol is broken, and Scrooge’s wealth can be transferred to the Beagle Boys. The following search command checks whether such an undesired state is reachable from `intruderInit`:

```
Maude> (search [1] intruderInit =>*
      C:Configuration
      < "Bank" : Responder | respSessions :
        trustedConnection("Scrooge") RS:RespSessions > .)
```

After about hundred minutes execution on a 1,7 GHz laptop, Maude replies with:

Solution 1

```
C:Configuration -->
  < "Scrooge" : Initiator |
    initSessions : trustedConnection("BeagleBoys"),
    nonceCtr : 2 >
  < "BeagleBoys" : Intruder |
    agentsSeen : ("Bank" ; "Scrooge" ; "BeagleBoys"),
    encrMsgsSeen : encrypt nonce("Scrooge",1) ; nonce("Bank",1)
                  with pubKey("Scrooge"),
    initSessions : emptySession, nonceCtr : 1,
    noncesSeen : nonce("Bank",1) nonce("Scrooge",1),
    respSessions : emptySession > ;
...
```

The Beagle Boys have fooled the bank into thinking that it has a trusted connection with the unknowing Scrooge! The NSPK protocol is therefore insecure ... or our Maude model is incorrect. To be sure that NSPK can be broken, and to learn about the attack on NSPK, we need to obtain the path leading to the bad state. Using the technique in Sections 10.2.4.1 and 13.1.4.3, we obtain the following path:

```
Maude> show path 3443070 .
state 0, Configuration:
< "Bank" : Responder | nonceCtr : 1, respSessions : emptySession >
< "Scrooge" : Initiator | initSessions : notInitiated("BeagleBoys"),
  nonceCtr : 1 >
< "BeagleBoys" : Intruder | agentsSeen : ("Bank" ; "BeagleBoys"),
  encrMsgsSeen : emptyEncrMsg, initSessions : emptySession,
  nonceCtr : 1, noncesSeen : emptyNonceSet, respSessions : emptySession >
===[ rl ... [label start-send-1] . ]===>
state 1, Configuration:
< "Bank" : Responder | ... >      < "BeagleBoys" : Intruder | ... >
< "Scrooge" : Initiator | initSessions :
  initiated("BeagleBoys", nonce("Scrooge",1)),
  nonceCtr : 2 >
msg encrypt nonce("Scrooge", 1) ; "Scrooge" with pubKey("BeagleBoys")
  from "Scrooge" to "BeagleBoys"
===[ rl ... [label intercept-msg1-and-understand] . ]===>
state 2, Configuration:
< "Bank" : Responder | ... >      < "Scrooge" : Initiator | ... >
< "BeagleBoys" : Intruder | agentsSeen : ("Bank" ; "Scrooge" ; "BeagleBoys"),
  noncesSeen : nonce("Scrooge", 1), ... >
===[ crl ... [label send-1-fake] . ]===>
state 9, Configuration:
< "Bank" : Responder | ... >      < "Scrooge" : Initiator | ... >
< "BeagleBoys" : Intruder | ... >
msg encrypt nonce("Scrooge", 1) ; "Scrooge" with pubKey("Bank")
  from "Scrooge" to "Bank"
===[ crl ... [label read-1-send-2] . ]===>
state 66, Configuration:
< "Bank" : Responder | nonceCtr : 2,
  respSessions : responded("Scrooge", nonce("Bank",1)) >
< "Scrooge" : Initiator | ... >      < "BeagleBoys" : Intruder | ... >
msg encrypt nonce("Scrooge", 1) ; nonce("Bank", 1) with pubKey("Scrooge")
  from "Bank" to "Scrooge"
```

```

===[ crl ... [label intercept-but-not-understand] . ]===>
state 504, Configuration:
< "Bank" : Responder | ... >      < "Scrooge" : Initiator | ... >
< "BeagleBoys" : Intruder |
  encrMsgsSeen : encrypt nonce("Scrooge", 1) ; nonce("Bank", 1) with
                    pubKey("Scrooge"), ... >
===[ crl [label send-encrypted] . ]===>
state 3723, Configuration:
< "Bank" : Responder | ... >      < "Scrooge" : Initiator | ... >
< "BeagleBoys" : Intruder | ... >
msg encrypt nonce("Scrooge", 1) ; nonce("Bank", 1) with pubKey("Scrooge")
  from "BeagleBoys" to "Scrooge"
===[ rl ... [label read-2-send-3] . ]===>
state 24482, Configuration:
< "Bank" : Responder | ... >
< "Scrooge" : Initiator | initSessions : trustedConnection("BeagleBoys"), ... >
< "BeagleBoys" : Intruder | ... >
msg encrypt nonce("Bank", 1) with pubKey("BeagleBoys")
  from "Scrooge" to "BeagleBoys"
===[ rl ... [label intercept-msg3-and-understand] . ]===>
state 141220, Configuration:
< "Bank" : Responder | ... >      < "Scrooge" : Initiator | ... >
< "BeagleBoys" : Intruder | noncesSeen : nonce("Bank",1) nonce("Scrooge",1), ... >
===[ crl ... [label send-3-fake] . ]===>
state 726180, Configuration:
< "Bank" : Responder | respSessions : responded("Scrooge", nonce("Bank",1)), ... >
< "Scrooge" : Initiator | ... >      < "BeagleBoys" : Intruder | ... >
msg encrypt nonce("Bank", 1) with pubKey("Bank") from "Scrooge" to "Bank"
===[ rl ... [label read-3] . ]===>
state 3443070, Configuration:
< "Bank" : Responder | respSessions : trustedConnection("Scrooge"), ... >
< "Scrooge" : Initiator | ... >      < "BeagleBoys" : Intruder | ... >

```

The search encountered—and stored—3,443,070 *distinct* states until it found the (un)desired state. The path itself consists of only 10 rewrite steps.

Let us analyze this path to see if it actually corresponds to a valid attack on the NSPK protocol. In the initial state, Scrooge wants to communicate with the Beagle Boys, and sends Message 1 to the Beagle Boys, who then use it to impersonate Scrooge. The bank sends Message 2 to Scrooge in response to the fake request by the Beagle Boys. The Beagle Boys overhear this message, but cannot decrypt it, since it is intended for Scrooge. Nevertheless, the Beagle Boys store this message and in the next step replay it for Scrooge, with themselves as the sender. Scrooge, who is expecting a connection with the Beagle Boys, is happy to see this message and answers with a Message 3 to the Beagle Boys, *where he/it replays the bank's nonce* with the Beagle Boys' public key! In this way, the Beagle Boys have learnt the bank's nonce which the bank thinks that only Scrooge can read. Once they know this nonce for the bank/Scrooge-connection, they fake a Message 3, pretending to be Scrooge, to the bank, which is waiting for exactly this confirmation of its connection with Scrooge. The bank is therefore convinced it is talking to Scrooge, and the Beagle Boys can start transferring Scrooge's money to their own account.

This behavior can be given in the style of the informal specification as follows, where $S1$ means “session 1” of the protocol, $S1.M1$ means sending Message 1 in session $S1$, the agents are abbreviated B (bank), S (Scrooge), and BB (Beagle Boys), and $BB(S)$ means “ BB pretending to be S ” or “ BB reading a message meant for S ”:

$$\begin{array}{ll}
S1.M1 : S \rightarrow BB & : \quad S.BB.\{N_s.S\}_{PK_{BB}} \\
S2.M1 : BB(S) \rightarrow B & : \quad S.B.\{N_s.S\}_{PK_B} \\
S2.M2 : B \rightarrow BB(S) & : \quad B.S.\{N_s.N_b\}_{PK_S} \\
S1.M2 : BB \rightarrow S & : \quad BB.S.\{N_s.N_b\}_{PK_S} \\
S1.M3 : S \rightarrow BB & : \quad S.BB.\{N_b\}_{PK_{BB}} \\
S2.M3 : BB(S) \rightarrow B & : \quad S.B.\{N_b\}_{PK_B}
\end{array}$$

All steps are indeed valid steps: the bank and Scrooge follow the protocol, and the Beagle Boys only send things they know. This is therefore a valid attack on NSPK.

Exercise 200 Is the set of states reachable from `intruderInit` finite or infinite? (Remember the equation that removes copies of a message from the configuration.)

Exercise 201 A search for attacks often searches for compromised nonces or keys. Search for a state reachable from `intruderInit` where the bank has responded to a (perceived) request from Scrooge with nonce n_b , and where the intruder knows both Scrooge and the bank, and also knows the nonce n_b . (The intruder then has all the knowledge needed to send the appropriate fake Message 3 to the bank.)

Exercise 202 The *Handbook of Applied Cryptography* [79] presents the following version of the NSPK protocol that avoids encrypting the third message:

$$\begin{array}{lll}
\text{Message 1'.} & A \rightarrow B : & A.B.\{N_a.A.r_a\}_{PK_B} \\
\text{Message 2'.} & B \rightarrow A : & B.A.\{N_b.r_a.r_b\}_{PK_A} \\
\text{Message 3'.} & A \rightarrow B : & A.B.r_b
\end{array}$$

where r_a and r_b are “random numbers generated respectively by A and B.”

1. What do you think could be the role of the nonces N_a and N_b in this version?
2. Model this version of NSPK in Maude.
3. The handbook does not say anything about whether or not this modified version of NSPK can be broken. Can the attack on NSPK be modified step by step so that it becomes a valid and successful attack on the modified version of the protocol?

14.6 Discussion

The NSPK protocol, which was published in 1978, is discussed in the *Handbook of Applied Cryptography* [79] from 1996 without any comments that it is insecure. The protocol was also proved correct in the absence of intruders in 1989 [17].

The attack on the protocol was originally reported by Gavin Lowe in 1995 [72]. The attack was supposedly found during *formal analysis* using the FDR tool for the process algebra CSP [73], and is the same attack found in our Maude analysis.

Although it might look slightly disconcerting that the Maude search took hundred minutes, this reflects the complexity of the problem: after all, the attack had escaped the attention of experts for 17 years. In Lowe's analysis, the intruder model did not include rules for taking part in original runs of the protocol; if we ignore those four rules, Maude finds the attack in a few seconds. Another common way of speeding up the search is to search for *compromised* nonces/keys: it is sufficient to search for a state in which the bank is waiting for some nonce and the attacker has that nonce; this search takes about 15 seconds (see Exercise 201).

Lowe's work showed the need for automatic analysis of cryptographic protocols, since humans could no longer be expected to be able to manually verify their correctness. This led to the development of a number of successful formal tools for analyzing such protocols; examples include the TAMARIN Prover [78], Scyther [24], ProVerif [13], the Avispa toolset [5], and the Maude-based Maude-NPA tool [42].

14.7 The Corrected Protocol

Gavin Lowe also suggested a modification of the protocol to make it secure: The responder adds its own identity to the *encrypted* part of Message 2, which becomes

$$\text{Message 2.} \quad B \rightarrow A : \quad B.A.\{N_a.N_b.B\}_{PK_A}$$

Search is ultimately a technique for discovering errors. Nothing is *proved* about a system if a search goes well, since the search only analyzes all possible behaviors from a *single* initial state. That you cannot break the protocol with three agents does not necessarily mean that you cannot break it with four agents. However, in this particular case, Lowe proved that *if* the (modified) protocol can be broken, then it can be broken in a system with one initiator, one responder, and one intruder [73]. Furthermore, each honest agent only needs to create *one* nonce. In other words, if you cannot break the protocol with three agents you cannot break it with 58 agents either. This means that if you can use search to show that there is no undesired state reachable from a three-agent state, then you have *proved* the specification correct. The fact that each agent only needs to generate one nonce implies that the reachable state space is finite, which means that the search will always terminate, and hence in this case provides a *decision procedure* for the correctness of the modified protocol.

Exercise 203 Explain why the attack on NSPK no longer works (or can be easily modified to work) in the modified protocol.

Exercise 204 Modify the Maude specification of the protocol to model the new version of the protocol. Define an initial state with one honest initiator, one honest responder, and one intruder. Can you break the modified protocol?

Exercise 205 Explain why the reachable state space is finite if each honest agent in the three-agent setting above generates at most one nonce.

The previous chapters of this book explained how the *behaviors* of a system can be specified mathematically. Such a *system specification* must be complemented by a *requirement specification* defining the properties that the system must satisfy.

Example 15.1. In our building metaphor in Section 1.1, the system specification corresponds to a model of the building, e.g., a physical scale model, a set of drawings of the building, and/or a virtual model of the building that together describe (aspects of) the building to be constructed. The requirement specification, defining the requirements that the building must satisfy, could include properties such as:

- The building should be able to withstand an 8.0 magnitude earthquake.
- The building should be able to withstand winds of up to 95 knots.
- All rooms in the building must be wheelchair accessible.
- There must be at least one bathroom for every 12 bedrooms.



Example 15.2. This book has presented a number of system specifications. Some desired requirements of the respective systems are:

1. Two philosophers should never hold the same chopstick at the same time.
2. Each philosopher *must* eat infinitely often.
3. Each philosopher *could* eat infinitely often.
4. The receiver will sooner or later receive all the messages in the right order.
5. The Beagle Boys are able to establish a trusted connection with the bank.
6. The bank never has a trusted connection with Scrooge.
7. Two processes will not execute in their critical sections at the same time.
8. Each process will eventually execute in its critical section.
9. The processes will execute in their critical sections in the order in which they wanted to access their critical sections.
10. All nodes will eventually elect the same leader.



Given a system (represented by its model) and the requirements that the system should satisfy, the all-important question is whether *the system satisfies its requirements*. The answer to that question obviously depends on the initial states. Even a “correct” system will not satisfy a desired requirement from a bad initial state. For example, the requirement “two processes are never in their respective critical sections at the same time” is *not* satisfied by the specification `MUTEX-WITH-CENTRAL-SERVER` on page 223 if the initial state contains two nodes whose `state` attribute has the value `insideCS`. Therefore, the main question is:

Does the system S satisfy the requirement R when started from any initial state $s_0 \in I$ from a set I of *admissible* initial states?

To be able to reason precisely about this question, and, in particular, to make it possible for a computer to reason about it, we need:

1. a precise mathematical model of the system S (and the set of initial states I);
2. a precise mathematical specification of the requirement R ; and
3. methods for checking whether a system satisfies its requirements.

This book has shown how Maude can be used to define a precise mathematical model of a distributed system. Chapter 16 explains how system requirements can be formalized using *temporal logic* and how Maude’s temporal logic *model checker* can check whether a system satisfies its requirements. Another important benefit of formalizing requirements is to make them precise; for example, does Requirement 5 in Example 15.2 say that the Beagle Boys must be able to establish a connection with the bank in *all possible runs* of the system or that they must be able to do so in at least *one run* of the system? Formalizing requirements removes such doubts about what requirements must actually be satisfied.

In a static world, the notion of correctness is straightforward: the equational specification should be terminating, ground confluent, sufficiently complete, and the unique *normal form* of an expression must be the correct one. In dynamic systems, which may not be terminating or deterministic, the notion of “result” (or normal form) may not make much sense. Instead, the interesting system properties typically concern the states and actions encountered *during* the runs of the system.

This chapter first informally discusses some classes of requirements, or system properties, and then explains how one such class, *invariants*, can either be verified “manually” or analyzed automatically by Maude.

15.1 State-based and Action-based Properties

System requirements may be stated either in terms of the *actions* (or *events*) that are performed during system executions, or in terms of the *states* that are encountered during system executions, or both.

In some cases, action/event-based properties are more natural:

Example 15.3. Consider (American) football games (see Section 8.2.3). An important requirement (that is *not* satisfied by the module `ONE-FOOTBALL-GAME`) is that

an *extra point* or a *two-point conversion* by a team may only be performed *immediately* after that team has scored a touchdown.

In other words, a two-point conversion may not follow directly after a field goal, a safety, an extra point, a two-point conversion, or a touchdown by the *other* team. $\diamond$

Some other natural action-based requirements are:

- No person can be baptized more than once.
- Each `wedding` must be preceded by an `engagement` involving the same persons.
- Each philosopher can start eating infinitely many times.
- The order in which the nodes enter their critical section should equal the order in which the nodes perform the `requestAccessToCS` action.

Other system requirements are more conveniently expressed as properties of the states of the system:

- Two nodes are never in local state `insideCS` in any state.
- The population is never inconsistent: there is no state in which Bridget is married to Tom and Tom is married to Gisele.
- Two nodes do not have different leaders in any state.
- The value of the receiver's `msgsRcvd` attribute should eventually equal the value of the sender's `msgsToSend` attribute in the initial state.
- The bank should not have an established connection with Scrooge unless Scrooge has an established connection with the bank.
- Two neighboring philosophers should never eat at the same time.

Other requirements are most naturally given by combining actions and states:

- A person in state `baptized` should not be able to make a hajj (pilgrimage to Mecca) or undergo another baptism.
- No person older than 50 years old should be able to give birth.

A requirement that is naturally expressed using actions can often also be expressed (albeit less conveniently) using states, and vice versa. However, the requirement in Example 15.3 cannot be expressed in terms of states (unless the original specification is modified), because it is impossible to differentiate a *two-point conversion* from a *safety* by just looking at the states, since both are worth two points.

This book focuses on state-based requirements, which are more commonly used.

15.1.1 Actions/Events

Defining an “action” or “event” is not as easy as saying that an event corresponds to applying a certain rewrite rule. For example, the events “wedding involving persons *A* and *B*” and “engagement involving *A* and *B*” do not correspond to “applying the rules `wedding` and `engagement`,” but to applying those rules with (partial) substitution $\{x \mapsto A, x' \mapsto B\}$. Therefore, an action/event often corresponds to applying a rewrite rule with a partial substitution σ of the variables in the rule.

15.1.2 State Propositions

A *state proposition* is a statement about a *single* state. It is *not* a statement about a state and its successor or predecessor states, or about the path leading to/from the state. In principle, a state proposition p could be defined as a function

```
op p : State -> Bool [frozen (1)].
```

where *State* denotes the sort of the states. For example, for object-oriented systems, the *State* sort is *Configuration*.

Example 15.4. Examples of state propositions include:

- The "Seahawks" have more points than the "Patriots" in the state.
- Two nodes are in local state *insideCS*.
- The bank has `trustedConnection("Scrooge")` in its `respSessions` attribute.

The following are *not* state propositions, since they talk about a single state *and* following/preceding states:

- The number of points in the current state is 6 more than in the previous state.
- From the current state it is possible to reach a state in which the bank has a connection with Scrooge. ◇

A state-based property is then a statement about computations, where state propositions talk about properties of the individual states involved in the behaviors.

Exercise 206 Show a behavior that illustrates that the action-based requirement “philosopher 2 starts eating infinitely often” is different from the state-based requirement “philosopher 2 is in state *eating* infinitely often.” Are the requirements “each philosopher starts eating infinitely often” and “each philosopher is in state *eating* infinitely often” different (in terms of being satisfied by different behaviors)?

Exercise 207 For each system requirement in this section, decide whether it is satisfied by the corresponding specification(s) (for the obvious admissible initial states).

Exercise 208 Express each action-based requirement in this section as a state-based requirement, if possible, without changing the system specification. Vice versa: express each state-based requirement as an action-based requirement, if possible.

Exercise 209 For each state proposition appearing in a state-based requirement, define the corresponding function p .

15.2 Temporal Properties

System requirements can be given as (state-based) *temporal properties* that describe properties about all possible behaviors from an initial state. This section introduces some classes of temporal properties that are formally defined in Chapter 16.

15.2.1 Invariance: “Nothing Bad Will Happen”

A *state proposition* p is (an) *invariant* with respect to an initial state t_0 if and only if p holds in each state that can be reached (in zero or more rewrite steps) from t_0 : that is, $t_0 \longrightarrow t$ implies $p(t)$. The state proposition p is an invariant with respect to set I of initial states if and only if p is an invariant w.r.t. each initial state $t_0 \in I$. An invariant is often called a *safety* property, since it can be seen to mean that “nothing bad will happen.” Figure 15.1 illustrates invariants.

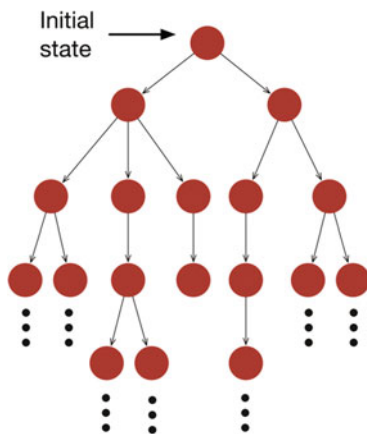


Fig. 15.1 “The state is red” is an invariant in the “tree” of possible system behaviors from the given initial state. (Each state is shown as a circle; an arrow means that there is a one-step sequential rewrite from the source state to the destination state.)

Example 15.5. A useful invariant p_{t_0} in the alternating bit protocol w.r.t. a “normal” initial state t_0 is that “the value of the receiver’s `msgsRcvd` attribute is a prefix of the value of the sender’s `msgsToSend` attribute in the initial state t_0 .” $\diamond$

Example 15.6. “At most one node has `state` attribute value `insideCS`” is a desired invariant in mutual exclusion algorithms. $\diamond$

Example 15.7. “Neighboring philosophers are not both in state `eating`” should be an invariant for our initial states in solutions to the dining philosophers problem. $\diamond$

A state may be “inconsistent” until some nodes receive a message. In these cases the invariant should take the messages traveling between nodes into account:

Example 15.8. The state proposition “if person A is married to person B , then also B is married to A ” is *not* an invariant (w.r.t. sensible initial states) in our model of populations. However, the state proposition “if A is married to B , then either B is married to A or there is a message (`msg` separate from B to A) in the state” should be invariant. $\diamond$

Example 15.9. The state proposition “either *all* nodes have `updated` equal `true` or all nodes have `updated` equal `false`” is not invariant (w.r.t. good initial states) in the two-phase commit protocol without node or communication failures. However, the state formula “either all nodes have `updated` equal `false`, or all nodes have `updated` equal `true`, or some nodes have `updated` equal `true` and there is a `commit` message in the state addressed to each node with `updated` equal `false`” should be an invariant in the two-phase commit protocol without failures. This invariant also implies that the databases are consistent when all messages have been consumed. ◇

15.2.2 Guarantee: “Something Good Must Eventually Happen”

Invariants say that something bad will not happen. We also want to be able to say that something good must eventually happen. A state proposition p is a *guarantee* (or *liveness*) property if a p -state can be reached in *all possible* computations from the initial state. That is, it is guaranteed that a p -state will be reached sooner or later, no matter how the rules are applied. Guarantee properties are illustrated in Fig. 15.2.

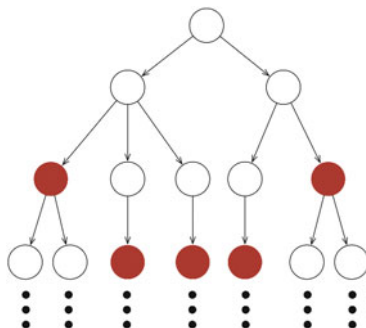


Fig. 15.2 “The state is red” is guaranteed

Example 15.10. The state proposition “process `node(4)` is executing inside its critical section” is *not* guaranteed in the central server algorithm (w.r.t. initial state `init(5)`) if all processes execute forever, alternating between executing outside and inside the critical section (see Exercise 184). The property *is* guaranteed by the ring-based mutual exclusion algorithm also when all nodes execute forever. ◇

Example 15.11. “Philosopher 3 is in state `eating`” is not guaranteed in any of the solutions to the dining philosophers problem (why not?). ◇

Example 15.12. “Each node has elected the highest-valued node as its leader” is guaranteed in both leader election algorithms in Section 13.3. ◇

Example 15.13. “The desired string has been stored in the receiver’s `msgsRcvd` attribute” is *not* guaranteed in our transport protocols. ◇

15.2.2.1 Fairness

It is often impossible to guarantee that a desired property (such as “philosopher 2 is eating” in the deadlock-free solutions, and “the receiver has received all strings in the desired order”) will be reached in all behaviors, since the model may allow extreme behaviors in which, e.g., all messages are lost, or only philosopher 1 does something. Such behaviors typically do not represent realistic system behaviors. Therefore, we can (and must) often assume *fairness* requirements on how the rewrite rules are applied in order to guarantee that a desired state will be reached. Since just imposing requirements on *which rules* are applied does not exclude unfair behaviors in which only philosopher 1 gets to execute, we generalize fairness to *events*.

Two classes of fairness requirements are:

- *Compassion* (or *strong fairness*): if an event is enabled (i.e., could take place) infinitely often, then the event must take place infinitely often.
- *Justice* (or *weak fairness*): an event cannot be *continuously* enabled from a certain point on without taking place.

Event fairness notions can state that the rule applications should be fair w.r.t. both *which objects* and *which rules* are executed. We also have to consider communication fairness. If messages can be lost, then there are (unrealistic) behaviors in which all messages are lost. One communication fairness assumption could be that “if an infinite number of copies of a certain message are sent, then an infinite number of such messages are *not* dropped.” Another fairness assumption is that no message is “overtaken” infinitely often by other messages. For example, the central server mutual exclusion algorithm in Section 13.2 with continuously executing processes does *not* guarantee that a given process p will be able to execute inside its critical section, since the `requestCS` message from p could be overtaken forever by messages from the other processes. We therefore need a “no infinite message overtaking” fairness assumption such as “a message cannot be available for reading continuously/infinitely often without being read.” Both of the above fairness assumptions can be seen as event fairness conditions: the event(s) in which the message m is read must be applied in a fair way.

There are many different notions and variations of fairness [45], and discussing them further is beyond the scope of this book.

15.2.3 Reachability: “Something Bad Could Happen”

A state proposition p is *reachable* w.r.t. an initial state t_0 if there exists some state t such that $t_0 \longrightarrow t$ and $p(t)$ holds. That is: it is possible to reach a p -state. The difference between a guaranteed property and a reachable property is that the former requires that a p -state is reached in *all possible runs*, whereas the latter only requires that a p -state is reached in *some* run, as illustrated in Fig. 15.3. There is a

big difference between being *guaranteed* to become a multimillionaire and having the *possibility* of becoming a multimillionaire by buying a lottery ticket every week.

Reachability is the dual property of invariance in the sense that

p is invariant if and only if not- p is not reachable.

Reachability properties are mostly used to analyze the possibility of reaching bad states in a specification.

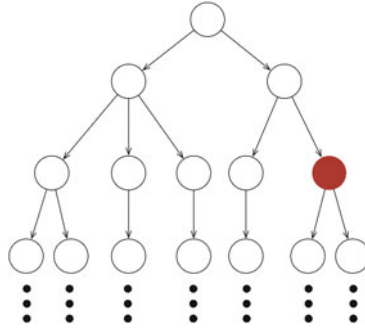


Fig. 15.3 “The state is red” is reachable

15.2.4 Response: “A Request Will Always be Answered”

An important task of a reactive system is to respond appropriately to stimuli from its “environment.” A pair of state propositions (p_1, p_2) is a *response (or reactivity) property* if and only if, for all possible behaviors from the initial state, a p_1 -state will be followed by a p_2 -state in zero or more steps. (The p_2 -state does not have to follow *immediately* after the p_1 -state.) Typical response properties could be:

- Every request must be followed by an acknowledgment.
- The airbags should be activated after the car control system detects a crash.
- Each state in which a process p wants to enter its critical section must eventually be followed by a state in which p is executing inside its critical section (Fig. 15.4).

15.2.5 Stability

A state proposition p is *stable* if it never stops holding after it first holds. For example, the property “the receiver’s `msgsRcvd` attribute equals the desired string s ” is

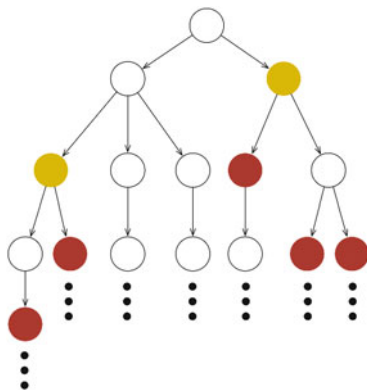


Fig.15.4 Response: Each yellow state must eventually be followed by a red state

a crucial stable property in the alternating bit protocol, which continues its execution even after the desired state has been reached. Stability, illustrated in Fig. 15.5, ensures that this result will not be destroyed by remaining actions of the system.

Likewise, “all nodes have the best-valued node as their leader” should be stable in a setting where nodes do not fail, so that new rounds of the leader election protocol do not destroy this property.

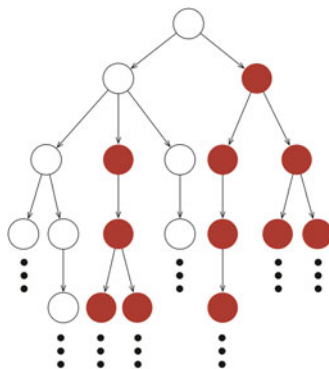


Fig. 15.5 “The state is red” is stable

15.2.6 Other Requirements

Until. The property “ p_1 until p_2 ” means that, in each behavior from the initial state(s), each state is a p_1 -state until a p_2 -state is reached. For example, “there is

a message in the state” until “all nodes have elected the best-valued node as their leader” should hold in a distributed leader election algorithm, and “there is a message in the state” until “all databases have the same `updated` value” holds in 2PC without failures. Two variations of the until property are shown in Fig. 15.6:

- *Weak until*: It is not necessary that a p_2 -state is eventually reached, in which case p_1 holds all the time. For example, in the Paxos consensus protocol “the nodes try to achieve consensus” weak-until “all nodes have agreed on a value” holds.
- *Strong until*: A p_2 -state must eventually be reached in all behaviors.

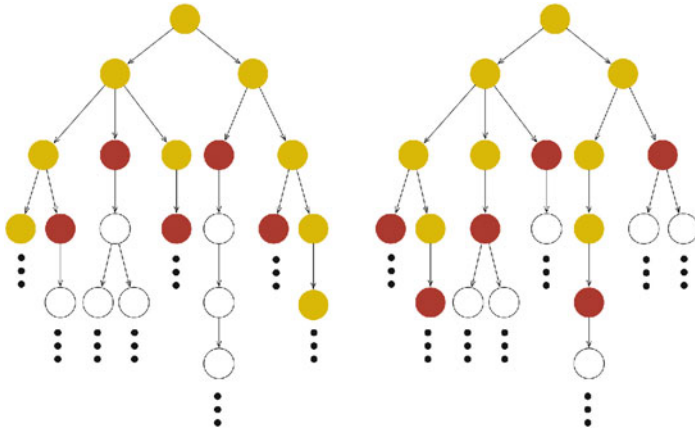


Fig. 15.6 “The state is yellow” weak-until “the state is red” (left), and “the state is yellow” strong-until “the state is red” (right)

Termination. The system does not allow any infinite behaviors.

Correct Final States. One could also require that all final states satisfy a given state proposition. For example, all databases should have the same `updated` value in all final states in the 2PC protocol, and all final states in the spanning-tree-based leader election protocol should have elected the same leader.

No Deadlocks. Yet another requirement is that there is *no* final, or “deadlocked,” state. For example, the dining philosophers should never stop eating/thinking, and hence any reachable state that cannot be rewritten is an undesired deadlock.

Exercise 210 Consider the second solution to the dining philosophers problem.

1. Show a behavior in which philosopher 2 could grab both chopsticks infinitely often, and never does so, but where (s)he cannot continuously grab both chopsticks from some point on.
2. Show a behavior in which philosopher 2 from a certain point on continuously can grab both chopsticks, but never gets to do so.

Which of these behaviors are illegal if we assume compassion w.r.t. the event “philosopher 2 grabs both chopsticks”? Which is illegal if we assume justice?

Exercise 211 Is the state proposition “philosopher 2 is in state *eating*” guaranteed in the deadlock-free solutions to the dining philosophers problem if we assume compassion w.r.t. “all events”? Is it guaranteed if we only assume justice?

Exercise 212 Consider the three solutions to the dining philosophers problem and the corresponding initial states.

1. In which solution(s) is “some philosopher is in state *eating*” guaranteed?
2. Is “two philosophers are in state *eating*” guaranteed in any of the solutions?
3. Is “two philosophers are in state *eating*” guaranteed in any of the solutions if we assume justice? How about if we assume compassion?

Exercise 213 1. Is the property “the receiver has received all the desired strings” guaranteed in the transport protocol *SEQNO-UNORDERED* (or the alternating bit protocol for that matter) under the “message loss fairness” assumption?

2. What additional fairness requirements (and for which events) are needed to guarantee that the above property will be reached? Is justice sufficient?
3. Assume message loss fairness, object/rule compassion, and no infinite message overtaking. Is the above property guaranteed in the sliding window protocol?

Exercise 214 Which is the state proposition whose reachability would show an error in the two-phase commit protocol? (Remember that a property of the form “... and the state is a final state” is not a state proposition.)

Exercise 215 The reachability of which state proposition would imply that the alternating bit protocol is incorrect?

Exercise 216 Consider the following statements about the Traveling Salesman problem with the parameters in Exercise 125:

1. The cost of the trip to now (i.e., stored in the current state) is greater than zero.
2. The (incomplete) trip up to the current state can be extended to a completed trip with total cost ≤ 45 .
3. The trip (stored in the current state) will sooner or later end in *PhnomPenh*.
4. The cost of the trip stored in the current state is greater than 12.
5. The cost of the trip stored in the current state is less than 22.

Which of these statements are state propositions? For each state proposition: is it invariant, guaranteed, reachable, and/or stable, for the obvious initial state?

Exercise 217 Assume that the initial state satisfies the state proposition p_1 . Explain why it is still not the case that “ p_2 is guaranteed” and “ (p_1, p_2) is a response property” are the equivalent for this initial state. Does one of these imply the other?

Exercise 218 Consider the following classes of requirements: invariance, guarantee, reachability, response, stability, and strong and weak until. What are the relationships between these properties? For example, are any of these special cases of others? Does any of them imply any others?

Exercise 219 Explain how the “no deadlock” requirement can be seen as a special case of the “all final states must satisfy p ” requirement.

15.3 Analyzing Invariants

It is easy to use Maude to analyze whether a state proposition p is an invariant in the system $\mathcal{R}$ with respect to a *single* initial state t_0 : Just check whether *one* state *not* satisfying p is reachable from t_0 . We have done this many times already:

- In the transport protocols in Chapter 12 we checked the desired invariant, that the receiver has received/stored a prefix of the list that the sender wanted to transmit, by searching for a reachable state in which the receiver has stored a sequence of messages which is *not* a prefix of what the sender wanted to transmit.
- In the mutual exclusion protocols we analyzed the desired invariant that “at most one process is inside its critical section” by searching for a reachable state in which at least two processes are in state `insideCS`.
- In the NSPK protocol we analyzed the desired invariant “the bank does *not* have a trusted connection with Scrooge” by searching for a reachable state in which the bank *has* such a “trusted” connection.

The outcome of such a Maude search is:

- If the property is *not* an invariant w.r.t. the given initial state, then Maude will eventually find a reachable bad state and the search command will terminate.
- If the property *is* invariant and the set of states reachable from t_0 is *finite*, then the search will terminate with `No solution` and we can conclude that the property is an invariant w.r.t. initial state t_0 .
- If the property *is* invariant w.r.t. initial state t_0 and the set of states reachable from t_0 is *infinite* then the search will not terminate. In this case we *cannot* conclude that the property is an invariant: maybe a bad state would be found if we just waited an hour/a year/a millennium longer? For example, if we stop the search in the NSPK protocol after half an hour, we may feel good and think that the protocol is safe; however, the bad behavior would have been discovered if we had waited 70 minutes longer.

One key to estimate the outcome is therefore to understand whether the reachable state space is finite. There are a number of ways in which the set of states reachable from t_0 can be an infinite set:

- Some value may grow beyond any bound. Examples include the football game specified in the module `ONE-FOOTBALL-GAME`, in which the scores may grow beyond any bound, and the dining philosophers solutions, in which the value of the attribute `#eats` may grow beyond any bound. If the `birthday` rule does *not* have an age limit, then also the `age` attribute could grow beyond any bound.
- The number of messages in the state may grow beyond any bound. This can happen, for example, in specifications with arbitrary duplication of messages.

- There is no bound on the number of objects that can be created. For example, in our population examples, there is no limitation on how many *new Person* objects can be created by the rule `birth`.

Using search to analyze invariance has some limitations:

- It cannot be used to prove that a state proposition is an invariant if the reachable state space is infinite.
- Invariance can only be analyzed for *single* initial states, not for infinite sets of admissible initial states.

For example, Maude search cannot even prove that “two neighboring philosophers do not eat at the same time” for the single initial state with 5 philosophers. Even if we remove the `#eats` attribute, which is the source of the infinite state space, search cannot prove this invariant for *any* number of philosophers. Search cannot even prove that “the total number of points scored is greater than 10” is an invariant when starting with initial state `"Steelers" vs "Ravens" 9 : 3`. If we limit the scoring in a football game, we would like to prove that the above property is invariant w.r.t. *all* initial states with at least 11 points scored. This is impossible using search.

We can instead prove inductively “by hand” that a state proposition p is an invariant w.r.t. to a set I of initial states by proving that:

- Each initial state $s_0 \in I$ satisfies p .
- For each rewrite rule r : if $t \longrightarrow t'$ is a one-step rewrite using rule r and t and t' are ground terms such that t is a p -state, then t' must also be a p -state.

Example 15.14. Let us prove that “the number of points scored is greater than 10” is an invariant w.r.t. *all* initial states (i.e., all ground terms of sort `Game`) where at least 11 points have been scored.

- Each initial state has at least 11 points scored, and therefore satisfies the property.
- Any ground term of sort `Game` has the form $a \text{ vs } b \text{ m} : n$. Any initial state then has this form, where, in addition, $m + n > 10$. Assume that we apply the rule `touchdown-home` to *any* such state; the resulting state is $a \text{ vs } b \text{ (m + 6)} : n$, which also satisfies the desired property since $m + 6 + n > 10$ holds when we assume that $m + n > 10$. In this same way, we can show that each rewrite rule preserves the desired formula.

We have therefore proved that the formula is an invariant in `ONE-FOOTBALL-GAME` for *all possible* initial states with at least 11 points scored. $\diamond$

If p is not *inductive*, that is, p is not strong enough to prove that $p(t) \wedge t \longrightarrow t'$ implies $p(t')$, we can strengthen p . The property p is then an invariant if: (i) the strengthened version p' is an invariant, and (ii) p' implies p . Exercise 224 is an exercise where the desired property must be strengthened in this way.

Exercise 220 Assume that the reachable state space from the single initial state is finite. Invariance and reachability can be analyzed using Maude’s search command. Explain why guarantee requirements cannot be analyzed using Maude’s search command. How about response, stability, and until requirements?

Exercise 221 Consider the following systems:

- Your specification of populations without rules for the birth of new persons.
- The *SEQNO-UNORDERED*, alternating bit, and the sliding windows protocols without message duplication.
- The dining philosophers solutions with the *#eats* attribute omitted.
- The central server mutual exclusion algorithm where processes execute forever.
- The ring-based mutual exclusion algorithm.

Which of these systems are terminating, and which of them have an infinite reachable state space, from the obvious initial states?

Exercise 222 Consider the following properties of the coffee bean game in Section 8.2.3:

- (i) “The state has 8 beans.”
- (ii) “The state has an odd number of beans.”
- (iii) “If the state has 5 beans then no following state will have more than 5 beans.”
- (iv) “The state has an even number of white beans.”
- (v) “If the state has an odd number of black beans, then we will end up with one (black) bean.”
- (vi) “The state has 8 or fewer beans.”
- (vii) “The state has an even number of black beans.”

1. Which of the above properties are state propositions?
2. Which of the state propositions are invariants w.r.t. the initial state in Exercise 122?
3. For each of the state propositions above, give the largest set of initial states for which the formula is an invariant.
4. Use Maude’s search command to check, for each state proposition, whether the state proposition is an invariant for the initial state in Exercise 122.
5. Prove inductively (“by hand”) that:
 - a. state proposition (vi) is an invariant for any initial state with 8 or fewer beans; and that
 - b. state proposition (vii) is an invariant for any initial state with an even number of black beans.

Exercise 223 Consider version 1 of the whiteboard game in Exercise 123.

1. Prove that “each number k on the whiteboard satisfies $\text{init}_{\min} \leq k \leq \text{init}_{\max}$ ” is an invariant for any initial state with smallest number $\text{init}_{\min}$ and largest number $\text{init}_{\max}$.
2. Use search to prove that the above property holds for the initial state with the numbers 9, 11, 21, 27, 77, and 85.

Exercise 224 (Slightly tricky?) Prove that “two neighboring philosophers are not eating” is an invariant in the first solution to the dining philosophers problem with respect to any appropriate initial state with $n \geq 3$ philosophers.

Chapter 15 discusses classes of requirements that a distributed system may have to satisfy. To make such requirements precise, and to be able to analyze whether a system S with initial state(s) I satisfies a requirement R , the requirement R must be defined mathematically.

There are a number of ways to formalize requirements of distributed systems. The most popular and intuitive way is to use (state-based) *linear temporal logic* (LTL), proposed by Amir Pnueli in 1977 [97], for which he was given the Turing Award, the equivalent of the Nobel Prize for computer science, in 1996.

If the set of states reachable from an initial state t_0 is a *finite* set, then *LTL model checkers* can automatically decide whether an LTL formula φ is satisfied by *all* possible system behaviors from state t_0 .¹ Maude has a high-performance explicit-state LTL model checker which provides a concrete counterexample (a “bad behavior”) if the requirement φ is not satisfied by all possible system behaviors from t_0 [21].

Section 16.1 introduces LTL; Section 16.2 explains how different properties can be formalized as LTL formulas; and Section 16.3 explains how Maude’s model checker can be used to check whether a system satisfies its requirements, and illustrates the techniques on the central server mutual exclusion algorithm in Section 13.2. Finally, Section 16.4 discusses extensions and variations of LTL.

16.1 Linear Temporal Logic

We use *linear temporal logic* (LTL) to formalize properties of rewrite specifications. A logic typically consists of:

¹Edmund Clarke, Allen Emerson, and Joseph Sifakis received the Turing Award in 2007 for their pioneering work on temporal logic model checking.

- a *syntax*, defining the *formulas* of the logic;
- a *semantics*, defining what it means that a formula holds in a specification; and
- a *proof system* that can be used to deduce/prove that a formula holds.

Example 16.1. In equational logic, formulas are equalities $t = u$; the semantics is defined by $E \models t = u$ if and only if the interpretation of $\sigma^*(t)$ and $\sigma^*(u)$ are the same element in each E -algebra/model, for each assignment σ of the variables in t and u ; and the proof system is the one given in Section 6.1. $\diamond$

Example 16.2. In rewriting logic, the formulas have the form $t \longrightarrow u$. As for the semantics, the models were just briefly mentioned in Section 8.6, and the proof system is given in Section 8.4. $\diamond$

This section presents the syntax of LTL (defining the set of LTL formulas) and its semantics. This book focuses on using model checking to *automatically* check whether a property is satisfied. We are less interested in coming up with a proof that a formula holds, and therefore do not provide a proof system for LTL, although sound and complete proof systems exist for LTL.

16.1.1 Behaviors

In this chapter we assume that each behavior from an initial state t_0 is an *infinite* sequence of one-step sequential rewrites. This assumption avoids having to define many concepts twice: one for finite behaviors and one for infinite behaviors. The point is that any *finite* behavior $t_0 \longrightarrow t_1 \longrightarrow \dots \longrightarrow t_n$, where t_n cannot be further rewritten, can be extended to an infinite sequence (also called a *path*) $t_0 \longrightarrow t_1 \longrightarrow \dots \longrightarrow t_n \longrightarrow t_n \longrightarrow \dots \longrightarrow t_n \longrightarrow \dots$ by just adding a self-loop from any deadlocked state t_n . Maude's model checker does this automatically.

Definition 16.1 A behavior from state t_0 in a specification $\mathcal{R}$ is an infinite sequence

$$t_0 \longrightarrow t_1 \longrightarrow t_2 \longrightarrow t_3 \longrightarrow \dots$$

of one-step sequential rewrites $t_i \longrightarrow t_{i+1}$ in $\mathcal{R}$. The set of all such behaviors starting with t_0 is denoted $\text{paths}_{\mathcal{R}}(t_0)$. If π is the above behavior and $k \in \mathbb{N}$, then

- $\pi(k) = t_k$ (the $(k+1)$ -th state in the path), and
- $\pi^k = t_k \longrightarrow t_{k+1} \longrightarrow t_{k+2} \longrightarrow \dots$ (the rest of the behavior from state t_k).

16.1.2 The Syntax of LTL

The basic building blocks in linear temporal logic formulas are *atomic propositions*. In a state-based logic, an atomic proposition p is a *state proposition*, which is either true or false in a state t of sort *State*, as explained in Section 15.1.2.

Example 16.3. Consider the specification `ONE-PERSON` in Section 8.2.3, which specifies the life of a single person. The designated sort *State* is `Person`, and some examples of state propositions are `alive`, `dead`, and `teenager`. $\diamond$

Example 16.4. Examples of atomic propositions in the dining philosophers systems are `noNeighborsEating`, `phil3eating`, `phil2eating`, and `phil2hasOneStick`. In an object-oriented specification, the designated sort *State* is `Configuration`. $\diamond$

Example 16.5. A useful family of state propositions, one for each pair of agents *a* and *b*, for NSPK is `a_hasTrustedConnectionWith_b`. $\diamond$

Linear temporal logic then adds to the atomic propositions the usual Boolean connectives $\neg$ (“not”), $\wedge$ (“and”), $\vee$ (“or”), $\rightarrow$ (“implies”), and $\leftrightarrow$ (“if and only if”), and the following temporal operators: $\square$, $\diamond$, $\bigcirc$, $\mathcal{U}$, and $\mathcal{W}$. Intuitively,

- the formula $\square \varphi$ holds in a path π if the formula φ holds *everywhere* in the path;
- the formula $\diamond \varphi$ holds in a path if φ holds *somewhere* in the path;
- the formula $\bigcirc \varphi$ holds if φ holds in the *next* position/state in the path;
- the formula $\varphi \mathcal{U} \psi$ holds in a path if the formula ψ holds *somewhere* in the path, and all positions in the path up to that point satisfy the formula φ ; and
- $\varphi \mathcal{W} \psi$ is similar to $\varphi \mathcal{U} \psi$, except that it is possible that ψ never holds in the path (in which case the formula φ must hold everywhere in the path).

Definition 16.2 Given a set *AP* of atomic propositions, the set of linear temporal logic (LTL) formulas are defined inductively as follows:

- *true* and *false* are LTL formulas;
- any state proposition $p \in AP$ is an LTL formula;
- if both φ and ψ are LTL formulas, then the following are also LTL formulas:
 - $\neg \varphi$ (*not* φ)
 - $\varphi \wedge \psi$ (φ and ψ)
 - $\varphi \vee \psi$ (φ or ψ)
 - $\varphi \rightarrow \psi$ (φ implies ψ)
 - $\varphi \leftrightarrow \psi$ (φ if and only if ψ)
 - $\square \varphi$ (*always* φ)
 - $\diamond \varphi$ (*eventually* φ)
 - $\varphi \mathcal{U} \psi$ (*“ φ (strong-) until ψ ”*)
 - $\varphi \mathcal{W} \psi$ (*“ φ weak-until ψ ”*)
 - $\bigcirc \varphi$ (*φ holds in the next state*)

Example 16.6. Examples of temporal logic formulas involving the atomic propositions in Examples 16.3–16.5 are:

1. $\square \text{alive}$ (the person is always alive).
2. $\diamond \text{dead}$ (sooner or later a state where the person is dead must be reached).
3. $\text{alive} \mathcal{U} \text{dead}$ (the person is continuously alive until she becomes dead).
4. $\text{alive} \mathcal{W} \text{dead}$ (as above, except that the person *could* live forever).

5. $\bigcirc_{\text{teenager}}$ (the person is, or becomes, a teenager in the *next* state).
6. The key property among the dining philosophers: $\Box \text{noNeighborsEating}$.
7. Philosopher 2 will eventually eat: $\Diamond \text{phil2eating}$.
8. Key NSPK property: $\Box \neg \text{"Bank"_hasTrustedConnectionWith_"Scrooge"}$ (notice the negation). $\Diamond$

The only operators needed are true , p , $\neg$, $\wedge$, $\mathcal{U}$, and $\bigcirc$. The other operators can be defined in terms of these. For example, $\varphi \vee \psi$ can be seen as an abbreviation of $\neg((\neg\varphi) \wedge (\neg\psi))$, and $\varphi \rightarrow \psi$ can be seen as an abbreviation of $(\neg\varphi) \vee \psi$, and $\varphi \leftrightarrow \psi$ can again be seen as an abbreviation of $(\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$. Likewise, as explained below, the temporal logic operators $\Box$, $\Diamond$, and $\mathcal{W}$ can be defined in terms of $\mathcal{U}$.

The formulas φ and ψ can themselves be LTL formulas, which means that we can have nested formulas such as $\Box(p \rightarrow \bigcirc \Diamond q)$. (What does this formula mean?)

16.1.3 The Semantics of LTL

To formally define the meaning of LTL; i.e., to define whether an LTL formula φ holds in a specification $\mathcal{R}$ with initial state t_0 , we must first define what it means that an atomic proposition holds in a state. A *labeling function* maps each state to those atomic propositions which hold in the state:

Definition 16.3 Given a rewrite theory $\mathcal{R} = (\Sigma, E, R)$, a sort *State* in Σ denoting the sort of the states, and a set *AP* of state propositions. Then a labeling function L is a function $L: \mathcal{T}_{\Sigma, \text{State}} \rightarrow \mathcal{P}(\text{AP})$ that assigns to each state t the set of state propositions holding in the state. Equivalent states must satisfy the same state propositions, so that $E \vdash t = t'$ must imply $L(t) = L(t')$.

Example 16.7. The obvious labeling function L in Example 16.3 gives us:

- $L(\text{person}(\text{"Peter"}, 46, \text{married})) = \{\text{alive}\}$ and
- $L(\text{person}(\text{"Joan of Arc"}, 19, \text{deceased})) = \{\text{dead}, \text{teenager}\}$. $\Diamond$

We also formalize the extension of a Maude module in which all deadlocked states are equipped with a self-loop, so that all computations become infinite paths:

Definition 16.4 Let $\mathcal{R}$ be a rewrite theory with a specific sort *State* denoting the sort of the states. Then $\mathcal{R}_{\text{State}}^\bullet$ is $\mathcal{R}$, except that there is a rewrite $t \longrightarrow t$ for each deadlocked state $t \in \mathcal{T}_{\Sigma, \text{State}}$.

As expected, a specification $\mathcal{R}$ with initial state t_0 satisfies an LTL formula φ if and only if each behavior from t_0 satisfies φ :

Definition 16.5 Given a set *AP* of atomic state propositions, a rewrite theory $\mathcal{R}$, a sort *State* denoting the sort of the system states in $\mathcal{R}$, and a labeling function L . Then $\mathcal{R}$ with initial state $t_0 \in \mathcal{T}_{\Sigma, \text{State}}$ satisfies an LTL formula φ , written

$$\mathcal{R}, \text{State}, L, t_0 \models \varphi$$

if and only if each path starting with t_0 satisfies φ ; that is:

$$\mathcal{R}, L, \pi \models \varphi \quad \text{holds for each path } \pi \in \text{paths}_{\mathcal{R}^{\bullet}_{\text{State}}}(t_0).$$

Notation: We often omit the state sort *State* and/or the labeling function *L* from $\mathcal{R}, \text{State}, L, t_0 \models \varphi$ and $\mathcal{R}, L, \pi \models \varphi$.

We define what it means that an (infinite) path π satisfies a formula φ inductively on the structure of φ :

Definition 16.6 $\mathcal{R}, L, \pi \models \varphi$ is defined inductively as follows:

- $\mathcal{R}, L, \pi \models \text{true}$ always holds;
- $\mathcal{R}, L, \pi \models \text{false}$ never holds;
- $\mathcal{R}, L, \pi \models p$, for $p \in AP$ an atomic proposition, holds if and only if the first state in the path π satisfies p ; i.e., $p \in L(\pi(0))$;
- $\mathcal{R}, L, \pi \models \neg\varphi$ holds if and only if $\mathcal{R}, L, \pi \models \varphi$ does not hold (which is written $\mathcal{R}, L, \pi \not\models \varphi$);
- $\mathcal{R}, L, \pi \models \varphi \wedge \psi$ holds if and only if both $\mathcal{R}, L, \pi \models \varphi$ and $\mathcal{R}, L, \pi \models \psi$ hold;
- $\mathcal{R}, L, \pi \models \varphi \vee \psi$ holds if and only if either $\mathcal{R}, L, \pi \models \varphi$ or $\mathcal{R}, L, \pi \models \psi$ (or both) hold;
- $\mathcal{R}, L, \pi \models \varphi \rightarrow \psi$ holds if and only if either $\mathcal{R}, L, \pi \models \varphi$ does not hold or $\mathcal{R}, L, \pi \models \psi$ holds (or both);
- $\mathcal{R}, L, \pi \models \varphi \leftrightarrow \psi$ holds if and only if ($\mathcal{R}, L, \pi \models \varphi$ holds if and only if $\mathcal{R}, L, \pi \models \psi$ holds);
- $\mathcal{R}, L, \pi \models \Box \varphi$ holds if and only if φ holds everywhere on the path π ; i.e., $\mathcal{R}, L, \pi^i \models \varphi$ for each $i \in \mathbb{N}$;
- $\mathcal{R}, L, \pi \models \Diamond \varphi$ holds if and only if φ holds somewhere on the path π ; i.e., $\mathcal{R}, L, \pi^k \models \varphi$ for some $k \in \mathbb{N}$;
- $\mathcal{R}, L, \pi \models \varphi \mathcal{U} \psi$ holds if and only if there exists a $k \in \mathbb{N}$ such that $\mathcal{R}, L, \pi^k \models \psi$ and such that $\mathcal{R}, L, \pi^i \models \varphi$ holds for each $0 \leq i < k$;
- $\mathcal{R}, L, \pi \models \varphi \mathcal{W} \psi$ holds if and only if either $\mathcal{R}, L, \pi \models \varphi \mathcal{U} \psi$ or $\mathcal{R}, L, \pi \models \Box \varphi$ (or both) hold; and
- $\mathcal{R}, L, \pi \models \bigcirc \varphi$ holds if and only if $\mathcal{R}, L, \pi^1 \models \varphi$ holds.

We can visualize this definition as follows, where we write below each state/position the subformula holding in the rest of the path starting in that state:

$$\begin{array}{ccccccc} t_0 & \longrightarrow & t_1 & \longrightarrow & t_2 & \longrightarrow & t_3 & \longrightarrow & t_4 & \longrightarrow & \cdots & \text{satisfies } \Box \varphi \\ \varphi & & \varphi & & \varphi & & \varphi & & \varphi & & \cdots & \end{array}$$

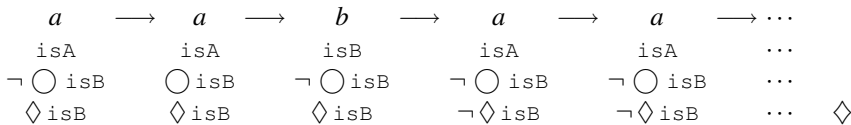
$$\begin{array}{ccccccc} t_0 & \longrightarrow & t_1 & \longrightarrow & t_2 & \longrightarrow & \cdots & \longrightarrow & t_k & \longrightarrow & \cdots & \text{satisfies } \Diamond \varphi \\ & & & & & & & & \varphi & & & \end{array}$$

$$\begin{array}{ccccccc} t_0 & \longrightarrow & t_1 & \longrightarrow & \cdots & \longrightarrow & t_{k-1} & \longrightarrow & t_k & \longrightarrow & t_{k+1} & \longrightarrow & \cdots & \text{satisfies } \varphi \mathcal{U} \psi \\ \varphi & & \varphi & & \cdots & & \varphi & & \psi & & & & & \end{array}$$

$$\begin{array}{ccccccc} t_0 & \longrightarrow & t_1 & \longrightarrow & t_2 & \longrightarrow & t_3 & \longrightarrow & \cdots & \text{satisfies } \bigcirc \varphi \\ & & \varphi & & & & & & & \end{array}$$

It is worth emphasizing that temporal formulas are not evaluated on *states*, but on (sub)paths starting at certain positions. We say that φ holds at position j in π if and only if φ holds in π^j . Notice that if the first position in the path satisfies φ , then $\Diamond \varphi$ and $\psi \mathcal{U} \varphi$ both hold.

Example 16.8. Consider a specification $\{a \longrightarrow b, a \longrightarrow a, b \longrightarrow a\}$, atomic propositions isA and isB , and labeling function L with $L(a) = \{\text{isA}\}$ and $L(b) = \{\text{isB}\}$. Different occurrences of a satisfy different formulas in the following path:



As already mentioned, the operators $\mathcal{W}$, $\Box$, and $\Diamond$ are not strictly necessary, since they can be defined in terms of $\mathcal{U}$:

- $\Diamond \varphi$ can be defined as $\text{true} \mathcal{U} \varphi$ (why?);
- $\varphi \mathcal{W} \psi$ can be defined as $(\varphi \mathcal{U} \psi) \vee \Box \varphi$; and
- $\Box \varphi$ can be defined in terms of $\mathcal{U}$ and the Boolean operators (see Exercise 227).

16.1.4 * Kripke Structures

We have defined the meaning of LTL formulas in terms of rewrite theories. However, LTL formulas can also talk about behaviors specified using other formalisms (such as Petri nets, automata, process algebras, etc.). The semantics of LTL is therefore usually defined on a more abstract model called a *Kripke structure*.

Definition 16.7 (Kripke structure) *Given a set AP of atomic propositions, a Kripke structure is a triple $(S, \rightarrow, L)$ where*

- S is a set (of states);
- $\rightarrow \subseteq S \times S$ is a binary relation, called the transition relation, that is total in the sense that for each $s \in S$ there is at least one $s' \in S$ such that $s \rightarrow s'$; and
- L is a labeling function $L: S \rightarrow \mathcal{P}(AP)$ assigning to each state the atomic propositions holding in that state.

A rewrite theory $\mathcal{R} = (\Sigma, E, R)$ with designated state sort *State* and labeling function L defines a Kripke structure $(\mathcal{T}_{\Sigma, E_{\text{State}}}, \longrightarrow^\bullet, L)$ in the obvious way, where:

- the set of states are the (E -equivalence classes of) ground terms of sort *State*;
- the transition relation $\longrightarrow^\bullet$ is the one-step sequential rewrite relation on the states extended with transitions $t \longrightarrow^\bullet t$ for deadlocked states; and
- L is the labeling function. Notice that for L to be a well-defined function (that is, assigning to each E -equivalence class of terms a single set of propositions holding in that equivalence class), E -equivalent states must be equivalent under L , which was assumed above.

- Exercise 225** 1. Explain why it could be the case that neither $\mathcal{R}, L, t_0 \models \varphi$ nor $\mathcal{R}, L, t_0 \models \neg\varphi$ holds.
2. Prove that it is always the case that either $\mathcal{R}, L, \pi \models \varphi$ or $\mathcal{R}, L, \pi \models \neg\varphi$ holds.

Exercise 226 Consider the formulas in Example 16.6. You can assume that the underlying specifications have been suitably completed, e.g., with rules for divorce.

1. Which of the formulas hold for the “standard” initial states?
2. For each formula, give the set of initial states for which the formula holds.
3. Give some examples of formulas φ and initial states t_0 such that neither $\mathcal{R}, L, t_0 \models \varphi$ nor $\mathcal{R}, L, t_0 \models \neg\varphi$ holds.
4. Define other useful atomic propositions and LTL formulas.

Exercise 227 Define $\Box\varphi$ in terms of $\mathcal{U}$ and the Boolean operators. Hint: Remember that $\Diamond$ can be defined by $\mathcal{U}$, and then define $\Box\varphi$ in terms of $\Diamond$.

16.2 Some LTL Formulas

This section discusses different LTL formulas, including the formalization of the different classes of properties mentioned in Chapter 15 and fairness assumptions.

16.2.1 Formalizing Classes of Requirements

This section formalizes the properties² in Chapter 15 and discusses other properties.

Invariance. Checking whether a (state) formula ς is invariant, that is, holding in each reachable state from the initial state t_0 , amounts to checking $\mathcal{R}, t_0 \models \Box\varsigma$.

Guarantee. Checking whether a state satisfying ς is guaranteed to be reached in all possible behaviors from t_0 amounts to checking whether $\mathcal{R}, t_0 \models \Diamond\varsigma$ holds.

Reachability. There is no LTL formula that formalizes reachability (i.e., it must be possible to reach a ς -state from t_0), since $\Diamond\varsigma$ requires that a ς -state is eventually reached in *all* possible behaviors. Nevertheless, LTL model checking can be used to check reachability: A ς -state is reachable from t_0 if and only if model checking $\mathcal{R}, t_0 \models \Box\neg\varsigma$ returns a counterexample, a “bad path” containing a ς -state.

²Those properties talk about *state formulas*, which are LTL formulas without temporal operators.

Response. That a “request” will always be followed by a “response” can be formalized by the LTL formula $\Box(\varphi \rightarrow \Diamond\psi)$, which says that each position in the path where φ holds must also satisfy $\Diamond\psi$, which again means that ψ must hold sometime later in the path. The ψ -position might be the same as the φ -position; Exercise 229 defines the response property when the response should come *after* the request.

Stability. Stability, which means that a property continues to hold forever once it starts holding, can be formalized as the property $\Box(\varphi \rightarrow \Box\varphi)$. That is, $\Box\varphi$ must hold whenever φ holds. For example: $\Box(\text{dead} \rightarrow \Box\text{dead})$.

A Property that Cannot be Checked. A useful requirement is that there is a *possibility* to reach a ϕ -state from a ζ -state. It is, for example, sensible to require from the state lottery agency that “whenever I buy a lottery ticket, there is a possibility that I will become a millionaire.” This is *not* a response property, since $\Box(\text{hasValidLotteryTicket} \rightarrow \Diamond\text{isMillionaire})$ *guarantees* that you become a millionaire after buying a lottery ticket. Likewise, a system modeling a person should satisfy the requirement that “a married person should be able to divorce.” This “may-lead-to” property *cannot* be formalized in LTL. Furthermore, it seems hard to use an LTL model checker to decide the property, since a counterexample obtained by model checking $\Box(\text{hasValidLotteryTicket} \rightarrow \Box\neg\text{isMillionaire})$ does not mean that the desired property holds (see Exercise 230).

Infinitely Often. It is sometimes needed to require that a property φ holds infinitely often in each path. For example, a main requirement of any correct solution to the dining philosophers problem is that “philosopher 2 should eat infinitely often.” The LTL formula $\Box\Diamond\varphi$ specifies exactly that φ holds infinitely often in each path from the initial state. Why is that? Assume that a path satisfies $\Box\Diamond\varphi$ (that is, this formula holds at the first position in the path) but that φ only holds *finitely* often in the path. If φ holds only finitely often in the path, then there must be a *last* position k where φ holds, and such that $\neg\varphi$ holds in all the following positions:

$$\begin{array}{ccccccccccc} t_0 & \longrightarrow & t_1 & \longrightarrow & \cdots & \longrightarrow & t_k & \longrightarrow & t_{k+1} & \longrightarrow & t_{k+2} & \longrightarrow & \cdots \\ & & & & & & \varphi & & \neg\varphi & & \neg\varphi & & \cdots \end{array}$$

However, this is impossible, since $\Diamond\varphi$ must hold everywhere, also in position $k+1$, which again means that φ *must* hold *somewhere* in $t_{k+1} \longrightarrow t_{k+2} \longrightarrow \cdots$:

$$\begin{array}{ccccccccccc} t_0 & \longrightarrow & \cdots & \longrightarrow & t_k & \longrightarrow & t_{k+1} & \longrightarrow & t_{k+2} & \longrightarrow & \cdots & \longrightarrow & t_l & \longrightarrow & \cdots \\ \Box\Diamond\varphi & & & & & & & & & & & & & & \text{which means} \\ \Diamond\varphi & & \cdots & & \Diamond\varphi & & \Diamond\varphi & & \Diamond\varphi & & \cdots & & \Diamond\varphi & & \cdots & \text{in particular:} \\ & & & & & & \Diamond\varphi & & & & & & & & & \text{which means} \\ & & & & & & & & & & & & \varphi & & & \text{for an } l > k. \end{array}$$

Therefore, $\Box\Diamond\text{phil2eating}$ formalizes the requirement that philosopher 2 gets to eat infinitely often.

Holds Continuously. A somewhat related property is that a property ϕ holds continuously from some point on. This can be formalized as the formula $\Diamond \Box \phi$, which means that somewhere along each path, $\Box \phi$ must hold; this also means that ϕ must hold from this point on. For example, a specification should satisfy $\Diamond \Box \text{dead}$ (although this is *not* satisfied by the module `ONE-PERSON` with divorce; why not?).

16.2.2 Fairness Assumptions

Recall that certain fairness assumptions are often necessary to prove that any kind of progress will be made by excluding obviously “unfair” behaviors in which, for example, messages are created and dropped all the time, or in which a person continuously marries and divorces all the time without even having time to celebrate her birthday. We mention two classes of fairness assumptions in Chapter 15:

- *Compassion:* If an event could be taken infinitely often, it should be taken infinitely often.
- *Justice:* An event cannot be enabled continuously from some point on without being taken infinitely often. (See also Exercise 232.)

If the formulas e_{enabled} and e_{taken} denote, respectively, that a certain event e is enabled and taken, then compassion fairness with respect to the event e can be expressed as the LTL formula

$$(\Box \Diamond e_{\text{enabled}}) \rightarrow \Box \Diamond e_{\text{taken}}$$

and justice fairness can be expressed by the LTL formula

$$(\Diamond \Box e_{\text{enabled}}) \rightarrow \Box \Diamond e_{\text{taken}}.$$

Example 16.9. For the dining philosophers, one compassion fairness condition on the application of the rules could be that if it happens infinitely often that philosopher number 2 already has one chopstick *and* the other chopstick is free, then this philosopher should be able to eat infinitely often:

$$(\Box \Diamond (\text{phil2hasOneStick} \wedge (\text{stick2free} \vee \text{stick3free}))) \rightarrow \Box \Diamond \text{phil2eating}.$$

Justice, however, would not help our philosopher much (why not?). In an unfair world, philosopher 2 may not even become hungry, since she could be thinking forever while other philosophers are doing stuff continuously. Justice is enough to ensure that philosopher 2 becomes hungry:

$$(\Diamond \Box \text{phil2thinking}) \rightarrow \Box \Diamond \text{phil2hungry},$$

where `phil2thinking` and `phil2hungry` are atomic propositions. ◇

If your LTL model checker does not support fairness, and you can encode your fairness assumptions as a formula ψ , you can model check the desired property φ under the fairness assumption by analyzing the formula $\psi \rightarrow \varphi$ instead.

One problem is that, since we use a state-based logic, e_{taken} cannot be defined directly, but must be defined by considering the effect of performing the event e , if possible. In Example 16.9 the event performed is “philosopher 2 applies the rule `grabSecond`,” and the effect of performing this event is that philosopher 2 is in state `eating`. In Section 16.3.5 we model check the central server mutual exclusion algorithm in Maude, and formalize all of its fairness assumptions in LTL.

Exercise 228 Why are the following formalizations of the response property wrong?

1. $\varphi \rightarrow \Diamond \psi$
2. $\Box(\varphi \rightarrow \psi)$

Exercise 229 Formalize the following properties as LTL formulas:

1. Each path contains only a finite number of φ -positions.
2. Every second position satisfies φ and every second position satisfies $\neg \varphi$.
3. As above, but in addition the first position must satisfy φ .
4. Each req-state will (eventually) be followed strictly later by a resp-state. (This is the strict version of the response/reactivity property, where the response cannot come at the same time as the request.)
5. Each request (req-state) must have gotten a response (resp-state) before the next request. Again, each response must come strictly after the request.
6. Two requests (req_a and req_b) must get the response ($resp_a$ and $resp_b$, respectively) in the order in which the requests took place. Assume for simplicity that two requests cannot happen at the same time; neither can two responses.

Exercise 230 Explain why obtaining a counterexample from model checking the formula $\Box(\text{hasValidLotteryTicket} \rightarrow \Box \neg \text{isMillionaire})$ does not imply that the desired “may-lead-to” requirement holds.

Exercise 231 Two LTL formulas φ and ψ are equivalent if they are evaluated in the same way in every possible path π . For example, $\neg \Diamond \varphi$ and $\Box \neg \varphi$ are equivalent:

- Assume that a path π satisfies $\neg \Diamond \varphi$. This means that a φ -position is never reached in the path, which of course means that all positions in the path satisfy $\neg \varphi$, which again means that the whole path satisfies $\Box \neg \varphi$.
- The other way: Assume that a path ρ satisfies $\Box \neg \varphi$. This means that all positions in the path are $\neg \varphi$ positions, and hence nowhere do we reach a φ -position, and therefore $\neg \Diamond \varphi$ holds.

For each the following pairs of LTL formulas (the last four of which are borrowed from [75]), determine whether the two formulas in the pair are equivalent. If not, show a path where one formula holds and the other formula does not hold. Does one of the formulas imply the other?

1. $\Box\Box\phi$ and $\Box\phi$
2. $\Diamond\Box\phi$ and $\Box\Diamond\phi$
3. $(\Box\phi) \rightarrow \Box\psi$ and $\Box(\phi \rightarrow \psi)$
4. $\Box((\Diamond\phi) \rightarrow \Diamond\psi)$ and $\Box(\phi \rightarrow \Diamond\psi)$
5. $(\Diamond\phi) \wedge (\Box\psi)$ and $\Diamond(\phi \wedge \Box\psi)$
6. $(\Diamond\Box\phi) \wedge (\Diamond\Box\psi)$ and $\Diamond((\Box\phi) \wedge (\Box\psi))$
7. $(\phi \mathcal{U} \psi) \wedge (\psi \mathcal{U} \theta)$ and $(\phi \mathcal{U} \theta)$
8. $(\Box\phi) \wedge (\Diamond\psi)$ and $\phi \mathcal{W} (\Diamond\psi)$

Exercise 232 The justice property is often (including in Chapter 15) defined “if, from a certain point on, an event is continuously enabled, then it must be taken,” which directly translates to the LTL formula $\Box((\Box e_{\text{enabled}}) \rightarrow \Diamond e_{\text{taken}})$. Is this formula equivalent to $(\Diamond\Box e_{\text{enabled}}) \rightarrow (\Box\Diamond e_{\text{taken}})$? Why/why not?

Exercise 233 (From [75]; tricky?) We can define the before operator $\mathcal{B}$ by $\phi \mathcal{B} \psi = (\neg\psi) \mathcal{W} (\phi \wedge \neg\psi)$. That is, the first occurrence of ϕ comes strictly before the first occurrence of ψ . Define $\mathcal{U}$ in terms of $\mathcal{B}$ and the Boolean connectives; that is, without using any temporal operator except $\mathcal{B}$.

16.3 Model Checking in Maude

Maude’s high-performance model checker can check whether a specification satisfies an LTL formula from an initial state, as long as the set of states reachable from the initial state is finite. If the formula is not satisfied by all paths from the initial state, Maude outputs a path which does not satisfy the formula. In contrast to many model checkers, Maude allows us to define *parametric* atomic propositions. Together with the possibility of using equations to define more complex formulas, this makes it easy to define fairly complex LTL properties.

The model checker can also check whether an LTL formula is *satisfiable*—that is, holds in *some* specification—and/or a *tautology*—i.e., holds in *all* specifications.

16.3.1 Getting Started

The model checker is declared in the file `model-checker.maude`, which is *not* loaded automatically and must therefore be loaded by the user. The main module in this file is `MODEL-CHECKER`. The module in which the atomic propositions (and possibly more complex formulas as well) are defined must therefore import both `MODEL-CHECKER` and the module in which you specify your system. In addition, you must define the built-in sort `State` to contain your system states by declaring `subsort s < State`, for *s* the sort of your system states:

```

mod MODEL-CHECK-MY-SPEC is
  protecting MY-SPEC . including MODEL-CHECKER .
  subsort s < State .
  --- declare and define atomic propositions
  --- and define complex formulas, if any
endm

```

When using Full Maude, this module should be enclosed between parentheses.

16.3.2 Defining Atomic Propositions

An atomic proposition is a term of the built-in sort `Prop`. For example:

```
ops dead alive teenager : -> Prop [ctor] .
```

We can also define *parametric* atomic propositions:

```
ops is_yearsOld olderThan : Nat -> Prop [ctor] .
```

Next we need to define the meaning of the atomic propositions; i.e., the labeling function L . This is done by defining the built-in function

```
op _|=_ : State Prop -> Bool [frozen] .
```

so that $t \models p$ evaluates to `true` whenever $p \in L(t)$. That is, we need to define the states in which p holds. It is not necessary to define explicitly the cases when the propositions do not hold. For example:

```

var X : String . vars M N : Nat . var S : Status .

eq person(X, N, S) |= alive = (S /= deceased) .
eq person(X, N, S) |= dead = (S == deceased) .
eq person(X, N, S) |= teenager = (N >= 13) and (N <= 19) .
eq person(X, N, S) |= is M yearsOld = (M == N) .
eq person(X, N, S) |= olderThan(M) = N > M .

```

These equations also define the `false` cases; since this is not strictly needed, the second and fourth equation could have been replaced by

```

eq person(X, N, deceased) |= dead = true .
eq person(X, N, S) |= is N yearsOld = true .

```

16.3.3 Defining LTL Formulas

LTL formulas are terms of the following sort `Formula`:


```

sorts Prop Formula .      subsort Prop < Formula .

ops True False : -> Formula [ctor ...] .
op ~_ : Formula -> Formula [ctor prec 53 ...] .
op _/\_ : Formula Formula -> Formula [comm ctor prec 55 ...] .
op _\/_ : Formula Formula -> Formula [comm ctor prec 59 ...] .
op O_ : Formula -> Formula [ctor prec 53 ...] .
op _U_ : Formula Formula -> Formula [ctor prec 63 ...] .
ops _->_ _<->_ : Formula Formula -> Formula [prec 65 ...] .
op <>_ : Formula -> Formula [prec 53 ...] .
op []_ : Formula -> Formula [prec 53 ...] .
op _W_ : Formula Formula -> Formula [prec 63 ...] .
...

```

The syntax of formulas is therefore pretty much the typewriter version of LTL formulas, with `True` for `true` (to avoid confusion with the Boolean `true`), `~` for $\neg$ (negation), `/\` for $\wedge$, `[]` for $\Box$, `<>` for $\Diamond$, and so on.

16.3.4 Performing Model Checking

The model checking is performed by giving the Maude command

```
red modelCheck(t0, φ) .
```

If the formula ϕ does not hold in all paths from the initial state t_0 , a counterexample

$$\text{counterexample}(\{t_0, r_1\} \{t_1, r_2\} \{t_2, r_3\} \cdots \{t_k, r_{k+1}\} , \\ \{t_{k+1}, r_{k+2}\} + \cdots \{t_n, r_{n+1}\}) +$$

is given. This path consists of an initial sequence $t_0 \longrightarrow t_1 \longrightarrow \cdots \longrightarrow t_k \longrightarrow t_{k+1}$ followed by a loop $t_{k+1} \longrightarrow \cdots \longrightarrow t_n \longrightarrow t_{k+1} \longrightarrow \cdots$, and where r_i is the rule used in the rewrite step $t_{i-1} \longrightarrow t_i$. The label of the self-loop that Maude adds from deadlocked states is `deadlock`.

Example 16.10. The following command checks whether the formula `alive U dead` holds in `ONE-PERSON` from initial state `person("Methuselah", 999, single)`:

```
Maude> red modelCheck(person("Methuselah", 999, single),
                        alive U dead) .
```

```

result ModelCheckResult:
  counterexample(
    {person("Methuselah", 999, single), 'birth-day}
    {person("Methuselah", 1000, single), 'birth-day}
    {person("Methuselah", 1001, single), 'successful-proposal} ,
    {person("Methuselah", 1001, engaged), 'marriage}
    {person("Methuselah", 1001, married), 'separation}
    {person("Methuselah", 1001, separated), 'divorce}
    {person("Methuselah", 1001, divorced), 'successful-proposal})

```

The counterexample shows that after becoming 1001 years old and engaged, Methuselah spends his remaining days marrying, separating, divorcing, proposing, remarrying, separating, and so on, forever and ever.

Although death may not be certain, it should be a stable property:

```
Maude> red modelCheck (person ("Peter", 46, single),
                        [] (dead -> [] dead)) .
```

```
result Bool: true
```

A person should be able to reach *any* age or be dead. This property does not hold in our specification due to possible loops of marriages, divorces, and remarriages. Since such behaviors are unrealistic, we can assume justice fairness on the application of the *birth-day* rule. However, one fairness condition is needed for each birthday event, so that the fairness assumption is formalized as the formula

```
((<> [] (alive /\ is 0 yearsOld) -> <> (is 1 yearsOld)) /\
(<> [] (alive /\ is 1 yearsOld) -> <> (is 2 yearsOld)) /\
(<> [] (alive /\ is 2 yearsOld) -> <> (is 3 yearsOld)) /\
...
((<> [] (alive /\ is 1000 yearsOld) -> <> (is 1001 yearsOld)) .
```

Fortunately, we can exploit the fact that Maude allows us to (i) specify parametric atomic propositions; and (ii) define more complex formulas equationally, to specify a function `fairBirthdays(currAge,desiredAge)`, which defines the fairness notions for all birthday events between age *currAge* and *desiredAge*:

```
op fairBirthdays : Nat Nat -> Formula .

vars M N : Nat .

ceq fairBirthdays(M, N) =
  = ((<> [] (alive /\ is M yearsOld)) -> <> (is M+1 yearsOld))
    /\ fairBirthdays(M+1, N)    if M < N .

eq fairBirthdays(N, N) = True .
```

The following command checks whether a 46-year-old person is guaranteed to either reach age 55 or die in-between:

```
Maude> red modelCheck (person ("Peter", 46, married),
                        fairBirthdays(46, 55) -> <> (is 55 yearsOld \/ dead)) .
```

```
result Bool: true
```

Why is the assumption `fairBirthdays(46, 55)` used instead of assuming that *all* birthdays events are applied fairly (i.e., `fairBirthdays(0, 1001)`)? The latter generates a very large LTL formula, and the model checking time complexity is linear in the number of reachable states, but exponential in the size of the LTL formula. Therefore, model checking large LTL formulas quickly becomes unfeasible. $\diamond$

16.3.5 Example: Analyzing Mutual Exclusion

In this section we analyze the central server mutual exclusion algorithm, but where each process loops forever (see Exercise 184; the only difference w.r.t. the specification in Section 13.2 is that a processor goes to state `beforeCS` instead of `afterCS` when exiting its critical section). Such distributed mutual exclusion should achieve: (i) two processes are never in the critical section at the same time; (ii) each process executes infinitely often in its critical section; and (iii) the processes access their critical sections in the order in which they wanted to enter it.

Requirement (i) is an invariant which is analyzed using search in Section 13.2. In this section we analyze the two other requirements.

Requirement (ii).

We start by checking whether each process is infinitely often in its critical section.

The following parametric state propositions `beforeCS(o)`, `waiting(o)`, and `inCS(o)` hold when node *o* is, respectively, executing outside its critical section, blocked waiting to enter its critical section, and executing inside its critical section:

```
(omod MODEL-CHECK-MUTEX-LOOP is
  protecting MUTEX-WITH-CENTRAL-SERVER-INITIAL-STATE .
  including MODEL-CHECKER .

  subsort Configuration < State .

  ops beforeCS waiting inCS : Oid -> Prop [ctor] .

  vars O O1 O2 : Oid .  var REST : Configuration .  var N : Nat .
  vars P Q : Formula .

  eq REST < O : Node | state : beforeCS >   |= beforeCS(O) = true .
  eq REST < O : Node | state : enteringCS > |= waiting(O) = true .
  eq REST < O : Node | state : insideCS >   |= inCS(O) = true .
```

Because of possible lack of fairness, a node, say `node(3)`, is not guaranteed to execute inside its critical section infinitely often:

```
Maude> (red modelCheck(init(3), [] <> inCS(node(3))) .)

result ModelCheckResult : counterexample(...)
```

We therefore only consider *just* paths, and must define the just use of the rewrite rules for an object *o*.

We first consider rule `requestAccessToCS` in Section 13.2, where a node in state `beforeCS` sends a request to the central server asking for access to the critical section. The result of applying this rule is that the node is `waiting`; the justice assumption therefore says that a node *o* cannot be continuously enabled (i.e., the node satisfies `beforeCS(o)`) without being taken infinitely often:

```

op justiceRule1 : Oid -> Formula .
eq justiceRule1(O) = (<> [] beforeCS(O)) -> [] <> waiting(O) .

```

The rules `grantAccess` and `putInWaitQueue` in Section 13.2 define how the central server reads request messages. The problem is that the server may always choose to read requests from other nodes, ignoring all requests from an unlucky node. In this case, not only is rule fairness required (is it really required?), but also fairness concerning *which* message the server reads.

We must define that if a request from object o is in the state forever, then it must eventually be read. The following formalization is based on the fact that there should never be more than one request message from the same node in the state. The desired communication fairness assumption `reqMsgFairness(o)` then just says that a request message from o cannot be in the state continuously from some point on:

```

op reqMsgFairness : Oid -> Formula .
op reqFrom_inState : Oid -> Prop [ctor] .
eq REST (msg requestCS from O to server) |= reqFrom O inState = true .
eq reqMsgFairness(O) = ~ (<> [] reqFrom O inState) .

```

Since this definition relies on the fact that there are not multiple requests from the same node in the state, we should first verify this fact:

```

Maude> (search [1] init(4) =>* (msg requestCS from O:Oid to server)
                                   (msg requestCS from O:Oid to server)
                                   REST:Configuration .)

```

No solution.

The justice assumptions for object o can therefore be defined as follows:

```

op justice : Oid -> Formula .
eq justice(O) = justiceRule1(O) /\ reqMsgFairness(O) .

```

We can now check Requirement (ii) for `node(3)`, assuming justice fairness for `node(3)` (the rules may be applied justly or unjustly w.r.t. `node(1)` and `node(2)`):

```

Maude> (red modelCheck(init(3),
                        justice(node(3)) -> [] <> inCS(node(3))) .)

```

result Bool : true

We can also check the requirement for all three nodes in one shot:

```

Maude> (red modelCheck(init(3),
                        (justice(node(1)) /\ justice(node(2))
                          /\ justice(node(3)))
                        -> (([] <> inCS(node(1))) /\ ([] <> inCS(node(2)))
                          /\ ([] <> inCS(node(3))))) .)

```

result Bool : true

Requirement (iii).

The nodes should access the critical section in the order in which they request it. This should also hold when the nodes execute forever.

We first define the `before` operator on LTL formulas (see Exercise 233):

```
op _before_ : Formula Formula -> Formula .
eq P before Q = (~ Q) W (P /\ ~ Q) .
```

We next define `rightOrder( $o_1, o_2$ )` to hold if it is always the case that if o_1 requests access to the critical section before o_2 , and none of them currently are in their critical section, then o_1 will access its critical section before o_2 :

```
op rightOrder : Oid Oid -> Formula .
eq rightOrder(O1, O2)
  = [] ((~ inCS(O1)) /\ (~ inCS(O2))
        /\ (waiting(O1) before waiting(O2))
        -> (inCS(O1) before inCS(O2))) .
```

Our specification does *not* satisfy even `rightOrder(node(1), node(2))`:

```
Maude> (red modelCheck(init(4), rightOrder(node(1), node(2))) .)

result ModelCheckResult : counterexample(...)
```

The point is that when two nodes send a request to the server, the server may not read the first request until the second arrives, and the server may then choose to read any of these multiple requests. Therefore, Requirement (iii) can only hold if the server reads request messages in the order in which they were sent.

The following formula `orderedReqRead( $o_1, o_2$ )` states that if there is a request from node o_1 , but not from node o_2 , in the state, then the request from o_1 will be read (i.e., disappear from the state) before a possible message from o_2 :

```
op orderedReqRead : Oid Oid -> Formula .
eq orderedReqRead(O1, O2) =
  = (reqFrom O1 inState /\ ~ reqFrom O2 inState)
    W (~ reqFrom O1 inState
        /\ ((reqFrom O1 inState /\ reqFrom O2 inState)
            W (~ reqFrom O1 inState))) .
```

The following formula `allReqsReadInOrder` then says that all requests in a three-node system are read in the order in which they were sent:

```
op allReqsReadInOrder : -> Formula .
eq allReqsReadInOrder
  = [] (orderedReqRead(node(1), node(2))
        /\ orderedReqRead(node(2), node(1)) /\ ...
        /\ orderedReqRead(node(3), node(2))) .
```

We can then check the desired property (for three pairs of nodes) when the server reads requests in order:

```
Maude> (red modelCheck (init (3),
  allReqsReadInOrder ->
    (rightOrder (node (1), node (2)) /\ rightOrder (node (2), node (1))
      /\ rightOrder (node (1), node (3)))) .)

result Bool : true
```

Exercise 234 Consider the Traveling Salesman setting in Exercise 125.

1. Define a parametric atomic proposition $\text{in}(c)$ which holds if the salesperson currently is in city c .
2. Which of the following formulas hold (in all paths) from the initial state where the journey starts in Phnom Penh? (a) $\bigcirc \bigcirc \bigcirc \bigcirc \bigcirc \text{in}(\text{PhnomPenh})$; (b) $\Diamond \text{in}(\text{Sisophon})$; (c) $\bigcirc \Diamond \text{in}(\text{SiemReap})$; (d) $\bigcirc \bigcirc \text{in}(\text{Battambang})$; (e) $\neg \bigcirc \bigcirc \text{in}(\text{Battambang})$; (f) $\Diamond \Box \text{in}(\text{PhnomPenh})$; (g) $\Box \Diamond \text{in}(\text{Sisophon})$.
3. Define and model check in Maude a formula which ensures that, in each path from the initial state, the person visits all cities and ends up in Phnom Penh.

Exercise 235 Explain why fairness assumptions on the application of the last four rewrite rules of the central server mutual exclusion algorithm are not needed to prove Requirement (ii).

Exercise 236 Consider the token ring mutual exclusion protocol in Exercise 185, item 6, where each process executes forever. Use LTL model checking to show that each process can start executing inside its critical section infinitely many times. What justice assumptions are needed? Are any compassion assumptions necessary?

Exercise 237 Consider the solution to the dining philosophers problem in Section 10.3.6.

1. Modify your specification from Exercise 157 by removing the `#eats` attribute (to obtain a finite reachable state space), and by making a philosopher leave the dining room immediately after eating.
2. It might be nontrivial to prove “by hand” that each philosopher is guaranteed to eat infinitely often. Therefore, use temporal logic model checking to show that any philosopher i can start eating infinitely often. Use as few and as mild fairness assumptions as possible: (i) do not make any fairness assumptions unless needed; (ii) use justice instead of compassion whenever possible; and (iii) only use the fairness assumptions for those philosophers needed to prove the property for philosopher i . (Remember that each fairness assumption adds to the size of the formula, and hence significantly to the model checking time.)
3. How can you check that there are behaviors satisfying the fairness conditions?

Note: Because of the large formulas involved, my verification took about 6 hours on a 1.7 GHz laptop. However, since the model checker returns once a bad behavior is found, counterexamples are typically provided much faster.

16.4 * Some More Temporal Logic

Satisfiability and Tautology Checking.

The model checker's SAT-SOLVER module provides a solver which can check whether an LTL formula is *satisfiable*, i.e., holding in *some* path in *some* Kripke structure, or is a *tautology*, i.e., holding in *all* paths in *all* Kripke structures.

The tautology checker can be used to decide whether two LTL formulas φ and ψ are equivalent by checking whether $\varphi \leftrightarrow \psi$ is a tautology. For example, we can check whether the pairs of formulas in Exercise 231, items 1 and 3, are equivalent:

```
load model-checker

mod CHECK-TAUT is including SAT-SOLVER .
  ops P Q : -> Formula .
endm

Maude> red tautCheck(([ ] P) <-> ([ ] P)) .

result Bool: true

Maude> red tautCheck((([ ] P) -> [ ] Q) <-> [ ] (P -> [ ] Q)) .

result TautCheckResult: counterexample((~ P) ; (P /\ ~ Q), True)
```

The counterexample shows a path which demonstrates that the formula does not hold; first the initial segment and then a loop: the first position satisfies $\neg P$, the second position satisfies $P \wedge \neg Q$, and the loop part does not matter (True).

Temporal Logic of Rewriting: Combining State and Action Propositions.

Sections 15.1 and 16.3.5 show that it is sometimes hard or impossible to express desired properties using only a state-based logic. In particular, fairness requirements quintessentially combine state-based properties (the enabledness of an action) with action-based properties (an action is “taken”).

The *temporal logic of rewriting* (TLR) [83] extends state-based atomic propositions with *action patterns*. An action pattern is a rule label l with a partial substitution σ of the variables in the rule. Furthermore, we can specify that the rewriting happens in a certain context (“position” or “part of the state”). A path satisfies an action pattern if the first rewrite step in the path conforms to the pattern. *Linear temporal logic of rewriting* (LTLR) is then LTL where the atomic propositions can be both state propositions and action patterns.

For example, “message-reading fairness” for a message m from an object o can be easily expressed using LTLR:

$$\begin{aligned} & \Diamond \Box \text{“message } m \text{ from } o \text{ is in the state”} \\ \rightarrow & \Box \Diamond (\text{“apply rule } l_1 \text{ with } o \mapsto o” \vee \dots \vee \text{“apply rule } l_k \text{ with } o \mapsto o”}). \end{aligned}$$

One problem encountered during my analysis of the dining philosophers in Exercise 237 is that, even when fairness assumptions can be expressed as an LTL or LTLR formula ψ , model checking $\psi \rightarrow \phi$ quickly becomes unfeasible due to the size of the formula ψ . Maude's LTLR model checker therefore builds in support for efficiently dealing with many kinds of fairness assumptions [7].

Branching Time Logics: CTL and CTL*.

LTL can express properties about *paths*. *Branching time logics* such as *computation tree logic* (CTL) [41] instead express properties about the *tree* of computations from a state. In CTL, the temporal operators are $\forall \square$, $\exists \square$, $\forall \diamond$, $\exists \diamond$, $\forall _ \mathcal{U} _$, and so on, where $\forall \square \phi$ and $\exists \square \phi$ mean that ϕ holds in all “positions” in, respectively, *all* paths and *some* path from the current state.

The “may-lead-to” property that I *may* become a millionaire after buying a lottery ticket, which is impossible to express in LTL, can be expressed by the CTL formula

$$\forall \square (\text{hasValidLotteryTicket} \rightarrow \exists \diamond \text{isMillionaire}).$$

On the other hand, CTL cannot formalize fairness assumptions (which concern *paths*); LTL and CTL are therefore incomparable in expressiveness and have different strengths and weaknesses [107]. The logic CTL* extends both CTL and LTL.

Temporal Logic with Past Temporal Operators.

The temporal operators $\square$, $\diamond$, and $\mathcal{U}$ concern the *future*. There are also corresponding *past* temporal operators such as $\blacksquare$, $\blacklozenge$, and $\mathcal{S}$. $\blacksquare \phi$ means that ϕ holds from the initial state until the current position; $\blacklozenge \phi$ means that a position satisfying ϕ occurred somewhere in the path *before* the current position; and $\phi \mathcal{S} \psi$ means that ϕ has held *since* ψ held. The property “every deployment of the airbag must have been preceded by a crash” can be formalized as $\square (\text{airbagDepl} \rightarrow \blacklozenge \text{crash})$.

The past operators are not really needed: any formula with past operators has a corresponding formula *without* past operators that is interpreted in the same way from the beginning of a path. Past operators are useful to express properties more succinctly: in the worst case, an LTL formula with past operators has an equivalent past-operator-less formula that is exponential in the size of the original formula [67].

Exercise 238 What is the difference between $\square (\text{airbagDepl} \rightarrow \blacklozenge \text{crash})$ and $\square (\text{crash} \rightarrow \diamond \text{airbagDepl})$?

Exercise 239 For each of the following formulas, define a formula without past operators that is evaluated in the same way from the beginning of a path.

$$\square (p \rightarrow \blacklozenge q) \quad \square (p \mathcal{S} q) \quad \diamond \blacklozenge p \quad \square \blacklozenge p.$$

Exercise 240 Specify all chess moves in Maude (if you want). Explain why you cannot express “white wins in two moves” in LTL. Can you express it in CTL?

The previous chapters abstract away timed and probabilistic aspects of distributed systems. This chapter briefly explains how real-time systems (Section 17.1) and probabilistic systems (Section 17.2) can be modeled and analyzed in rewriting logic.

17.1 Real-Time Systems

Real-time systems are systems where the *duration* of/between events affects the *functionality* of the system. This book abstracts from real-time features in its treatment of the two-phase commit protocol in Section 13.1, where, instead of using time-outs to determine whether a message has been lost, we assume that messages of certain types are never lost. More generally, message duplication is an abstraction for re-sending a message when the sender has not received feedback from the receiver for some time.

However, real-time features cannot be abstracted away in many distributed systems, for example because of the following reasons:

- *Fault-tolerant systems* must estimate whether messages are lost and/or whether other nodes are down, and must take appropriate action if so. However, it is impossible to check message loss and node crashes without taking time into account. Using time, a node can assume that a message was lost or that another node crashed if it has not gotten a reply within a certain time bound.
- Time is a key parameter in many distributed algorithms and protocols, for example to fine-tune performance.
- Most computer systems today, from those in toasters and cars to airplanes, are *embedded systems*, where processors interact with some physical devices/environments. Such systems tend to be time-critical: an action that happens at the wrong time could have unfortunate consequences.

- Timed models also enable reasoning about the *performance* of a system.
- We are often interested in *timed* properties. For example, “the airbag will *eventually* deploy after a crash” ($\Box(\text{crash} \rightarrow \Diamond \text{airbag})$ in temporal logic) is not an impressive guarantee. The system should satisfy the *timed* property “the airbag will deploy *within 10 milliseconds* after a crash.”

This section explains how real-time systems can be specified directly in rewriting logic as real-time rewrite theories [92]. The Real-Time Maude system [90,93] provides syntactic support and time-specific analysis methods for such theories.

17.1.1 Specifying Real-Time Systems in Rewriting Logic

Real-time systems can be specified in rewriting logic as *real-time rewrite theories* [92]. Such a theory has two kinds of rewrite rules:

- Ordinary rewrite rules, called *instantaneous rewrite rules* in the timed setting, which are assumed to take zero time.
- *Tick rewrite rules* of the form $\{t\} \xrightarrow{\tau} \{t'\}$ **if** *cond*, where τ is a term of sort `Time`, model the advance of time: it takes time $\tau\sigma$ to go from state $\{t\sigma\}$ to state $\{t'\sigma\}$. The operator $\{_ \}$ encloses the *entire* state, just as in Section 11.2.3.

The form of the tick rules ensures that time advances equally in all parts of the system. A tick rule, such as $\langle X : \text{Person} \mid \text{age} : N \rangle \xrightarrow{1} \langle X : \text{Person} \mid \text{age} : N+1 \rangle$, *without* the global-state operator, would lead to a sequence of rewrites taking the state $\langle \text{"Robert"} : \text{Person} \mid \text{age} : 6 \rangle < \langle \text{"Roland"} : \text{Person} \mid \text{age} : 8 \rangle$ to $\langle \text{"Robert"} : \text{Person} \mid \text{age} : 6 \rangle < \langle \text{"Roland"} : \text{Person} \mid \text{age} : 12 \rangle$, where one person has aged four years while the other person has not aged at all.

Real-time rewrite theories can be specified directly in Maude. Tick rewrite rules can be written

```
crl [tick] : {t} => {t'} in time  $\tau$  if cond .
```

The operator $\{_ \}$ can be defined as follows when the states are configurations:

```
(omod OO-TIMED-PRELUDE is protecting NAT-TIME .
  sorts GlobalState ClockedState .
  subsort GlobalState < ClockedState .
  op `[_]` : Configuration -> GlobalState [ctor] .
```

Assuming a sort `Time` for the time domain, the ‘in time’ part of tick rules can be modeled by the operator

```
op _in`time_ : GlobalState Time -> ClockedState [ctor] .
var CLS : ClockedState . vars T1 T2 : Time .
eq (CLS in time T1) in time T2 = CLS in time (T1 + T2) .
endom)
```

so that the “clocked state” of the system has the form $\{t\}$ in time r , where r is the total amount of time that has elapsed in the system since the start of the execution.

We can let the natural numbers be the time domain `Time`. It is often useful to have a superset `TimeInf` of `Time` with the additional value `oo` (for infinity) and a function `monus` denoting “minus down to 0”:

```
fmod NAT-TIME is protecting NAT .
  sorts Time NzTime TimeInf .  subsort NzTime < Time < TimeInf .
  subsort Nat < Time .  subsort NzNat < NzTime .

  op oo : -> TimeInf [ctor] .  --- infinity value

  vars T T' : Time .  var TI : TimeInf .  vars N N' : Nat .
  --- extend operators to infinity:
  op _<=_ : Time TimeInf -> Bool [ditto] .
  op min : TimeInf TimeInf -> TimeInf [ditto] .
  op _monus_ : TimeInf Time -> TimeInf .
  eq T <= oo = true .  eq min(oo, TI) = TI .
  eq N monus N' = if N' <= N then sd(N, N') else 0 fi .
  eq oo monus N = oo .
endfm
```

We extend these modules to model and execute some real-time systems in Maude.

Example 17.1. We model a stylish modern watch with only an *hour* hand/marker. The watch is *retrograde*, so that the hour hand must *jump* from 12 to 0, instead of time 12 being equal to time 0. The watch runs perfectly while it runs, but can break at any time (battery exhausted, dropped on bathroom floor, ...). Such a retrograde watch can be modeled as an object of the class `Clock` as follows:

```
(omod SINGLE-CLOCK is including OO-TIMED-PRELUDE .
  class Clock | state : ClockState, time : Time .
  sort ClockState .  ops running stopped : -> ClockState [ctor] .
  op genta : -> Oid [ctor] .
```

This specification has two instantaneous rewrite rules: At any time, a running watch may break, and when the watch shows 12, it must immediately jump to 0:

```
var C : Oid .  var T : Time .

rl [batteryDies] :
  < C : Clock | state : running > => < C : Clock | state : stopped > .

rl [jumpToZero] :
  < C : Clock | state : running, time : 12 >
=>
  < C : Clock | time : 0 > .
```

The following tick rule advances time by 1 in a running watch:

```
crl [tickOneRunning] :
  {< C : Clock | state : running, time : T >}
=>
  {< C : Clock | time : T+1 >} in time 1 if T < 12 .
```

The condition ensures that the watch is reset as soon as it reaches 12.

Finally, as we all know, time continues to fly even if your watch has stopped:

```
rl [tickOneStopped] :
  {< C : Clock | state : stopped >} => {< C : Clock | >} in time 1.
endom)
```

We can then simulate our watch for 100 rewrite steps:

```
Maude> (rew [100] {< genta : Clock | state : running, time : 0 >}.)
```

```
result ClockedState :
  {< genta : Clock | state : stopped, time : 12 >} in time 98
```

Since the state has the form $\{t\}$ in time r , where r can grow beyond any bound, the reachable state space is infinite. We therefore use bounded search to analyze the main safety requirement: the watch never shows a value greater than 12:

```
Maude> (search [1,1000]
  {< genta : Clock | state : running, time : 0 >} =>*
  {< genta : Clock | time : T:Time >} in time T2:Time
  such that T:Time > 12 .)
```

No solution. ◇

If the states have multiple objects and/or messages, the following tick rule has proved useful in most large Real-Time Maude applications [90]:

```
var CONF : Configuration .
cr1 [tick] :
  {CONF} => {timeEffect (CONF,  $\tau$ )} in time  $\tau$  if  $\tau \leq \text{mte}(\text{CONF})$  .
```

The function `timeEffect` defines what happens with a configuration when a certain amount of time has elapsed. It distributes over the elements in a configuration, so the user must define `timeEffect` only on single objects and messages:

```
vars CONF1 CONF2 : Configuration .   vars T T' : Time .

op timeEffect : Configuration Time -> Configuration [frozen (1)] .
eq timeEffect (none, T) = none .
ceq timeEffect (CONF1 CONF2, T)
  = timeEffect (CONF1, T)   timeEffect (CONF2, T)
  if CONF1 /= none and CONF2 /= none .
```

The function `mte`, for *maximum time elapse*, defines how much time can pass in the configuration before something must happen. This function also distributes over the elements in a configuration:

```
op mte : Configuration -> TimeInf [frozen (1)] .
eq mte (none) = oo .
ceq mte (CONF1 CONF2) = min(mte (CONF1), mte (CONF2))
  if CONF1 /= none and CONF2 /= none .
```

This infrastructure is used in all the subsequent examples in Section 17.1. The following example models a system with multiple retrograde watches.

Example 17.2. The state may now have multiple retrograde watches, each of which behaves as in Example 17.1. All running watches may not show the same time, since they can have different values initially.

The class `Clock` and the two instantaneous rules are as in Example 17.1 and are not shown. The tick rules in Example 17.1 are replaced by the above tick rule for object-oriented specifications, with the value 1 for τ . What remains is to define the functions `timeEffect` and `mte` on single `Clock` objects. Time elapse affects a running watch by increasing the time it shows by the amount of elapsed time, and passage of time does not affect a stopped watch at all:

```
eq timeEffect(< C : Clock | state : S, time : T >, T')
  = if S == running then < C : Clock | time : T+T' >
    else < C : Clock | > fi .
```

Time is allowed to advance until the moment when a running watch would show 12, and can advance forever when the watch is broken:

```
eq mte(< C : Clock | state : running, time : T >) = 12 monus T .
eq mte(< C : Clock | state : stopped >) = oo .
```

We can then simulate a system with three watches:

```
Maude> (rew [100] {< seiko : Clock | state:running, time: 0 >
                  < dubuis : Clock | state:running, time: 0 >
                  < ap : Clock | state:running, time: 0 >}.)

result ClockedState :
  {< ap : Clock | state:stopped, time: 0 >
   < dubuis : Clock | state:stopped, time: 0 >
   < seiko : Clock | state:stopped, time: 12 >} in time 94  ◇
```

Our watches keep perfect rate in Examples 17.1 and 17.2. It is more common that some watches are slow, while others are fast. However, *time* advances by the *same* amount in *all* parts of a distributed system,¹ even if the local clocks are imperfect.

Example 17.3. We now consider imperfect watches. Each watch has a *rate*, which tells how fast or slow it is. For example, a (fast) watch with rate $5/4$ increases its time value by 1.25 in an hour. A slow watch has rate < 1 . We use the non-negative rational numbers as the time domain. To ensure that each watch resets when it shows 12, we no longer advance time by one time unit in each tick step; we instead advance time to the next moment when some watch must be reset, and by 10 time units when all clocks have stopped:

```
(omod MANY-SKEWED-CLOCKS is including OO-TIMED-PRELUDE .
 protecting POSRAT-TIME .
 class Clock | state : ClockState, time : Time, rate : PosRat .
 ...
 crl [tick] : {CONF1} => {timeEffect(CONF1, min(10, mte(CONF1)))}
   in time min(10, mte(CONF1)) if mte(CONF1) /= 0 .
```

The instantaneous rules are as before, and it remains to define `timeEffect` and `mte` on single watches. Time affects a watch in the expected way:

¹Disregarding *time dilation* effects caused by the nature of spacetime.

```

eq timeEffect(< C : Clock | state : running, time : T, rate : RATE >, T')
  = < C : Clock | state : running, time : T + (T' * RATE) > .
eq timeEffect(< C : Clock | state : stopped >, T') = < C : Clock | > .

```

`mte` defines how much time can advance before a watch must be reset:

```

eq mte(< C : Clock | state : running, time : T, rate : RATE >)
  = (12 monus T) / RATE .
eq mte(< C : Clock | state : stopped >) = oo .

```

◇

The following example is a small network example with common timing features such as timers, clocks, and time-out-based message retransmissions.

Example 17.4. We consider a protocol for finding the *round trip time* (RTT) between two nodes; i.e., the time it takes for a message to travel from sender to receiver, and back. The sender sends a message `rttReq(t)`, where t is the value of the sender's local clock. When the receiver receives this message, it replies with the message `rttReq(t)`, with the same timestamp t . When the original sender receives `rttReq(t)`, it computes the RTT as $t_1 - t$, with t_1 its current clock value.

The message delay may be arbitrarily long, and messages could be lost. Therefore, if the original sender has not received the reply within 10 time units, it assumes that some message was lost or hopelessly delayed, and sends a new RTT request. This process goes on until the sender has recorded an RTT value smaller than 10.

The sender, receiver, and the messages are declared as follows:

```

(omod FIND-RTT is including OO-TIMED-PRELUDE .
  including MESSAGE-LOSS .    --- message wrapper and message loss
                               --- (from Section 11.2.5)
  class Sender | clock : Time, rtt : Time,
                  resendTimer : TimeInf, receiver : Oid .
  class Receiver .

  ops rttReq rttReply : Time -> MsgContent [ctor] .

```

The `clock` attribute denotes the value of the sender's local clock, and `rtt` stores the desired RTT value. `resendTimer` is a *timer*. A timer counts down, and when it reaches zero, time does not advance; this forces the application of an action which either resets or turns off the timer before time can advance further.

The following instantaneous rewrite rule starts an iteration of the RTT-finding process when the `resendTimer` expires (becomes zero). The sender then sends an `rttReq` message to the receiver with its current clock value as timestamp. The rule also resets the `resendTimer` to 10, so that the process will repeat itself in 10 time units from now, unless the `resetTimer` is turned off before then:

```

vars T T' T1 T2 : Time .   var TI : TimeInf .   vars S R O1 O2 : Oid .
vars CONF CONF1 CONF2 : Configuration .       var MC : MsgContent .

rl [sendRequest] :
  < S : Sender | clock : T, resendTimer : 0, receiver : R >
=>
  < S : Sender | resendTimer : 10 >
  (msg rttReq(T) from S to R) .

```

The receiver replies to a request with an `rttReply` message with the received timestamp `T`:

```
rl [reply] :
  (msg rttReq(T) from S to R)
  < R : Receiver | >
=>
  < R : Receiver | >
  (msg rttReply(T) from R to S) .
```

When the sender receives the reply, it checks whether this message is a response to its latest request, or to a previous request. If it is the former, the sender computes and stores the `rtt` value and turns off its timer by setting it to the infinity value `oo`. If the received message is a reply to an older request, it is just ignored:

```
rl [recReply] :
  (msg rttReply(T1) from R to S)
  < S : Sender | time : T2 >
=>
  if (T2 minus T1) < 10
  then < S : Sender | rtt : T2 minus T1, resendTimer : oo >
  else < S : Sender | > fi .
```

Those are all the instantaneous rewrite rules. The tick rule is the standard one:

```
crl [tick] :
  {CONF} => {timeEffect(CONF, 1)} in time 1 if 1 <= mte(CONF) .
```

`timeEffect` is defined on sender objects by increasing the local clock and *decreasing* the timer value according to the elapsed time:

```
eq timeEffect(< S : Sender | time : T1, resendTimer : TI >, T2)
  = < S : Sender | time : T1+T2, resendTimer : TI minus T2 > .
```

The elapse of time does not affect the receiver or the messages:

```
eq timeEffect(< R : Receiver | >, T) = < R : Receiver | > .
eq timeEffect(msg MC from O1 to O2, T) = (msg MC from O1 to O2) .
```

`mte` must ensure that time advance stops when the `resendTimer` expires, and that time cannot advance when the timer value is zero:

```
eq mte(< S : Sender | resendTimer : TI >) = TI .
```

The receiver and the messages do not place any restrictions on time advance:

```
eq mte(< R : Receiver | >) = oo .
eq mte(msg MC from O1 to O2) = oo .
```

Finally, we define a suitable initial state:

```
ops snd rec : -> Oid [ctor] .
op init : -> Configuration .
eq init = < snd : Sender | clock : 0, rtt : 0, resendTimer : 0,
                      receiver : rec >
          < rec : Receiver | > .
endom)
```

We test our specification, which fails to record an RTT value within 200 steps:

```
Maude> (rew [200] {init} .)
```

```
result ClockedState :
  {< rec : Receiver | none >
   < snd : Sender | resendTimer : 6, rtt : 0, clock : 154, ... >
   msg rttReq(150) from snd to rec} in time 154
```

We therefore check whether it is possible to record an `rtt` value 3:

```
Maude> (search [1] {init} =>*
        {< snd : Sender | rtt : 3 > C:Configuration} in time N:Nat.)
```

```
Solution 1
```

```
N:Nat --> 3 ; ...
```

◇

Message Delays.

The treatment of message delays (the time it takes for a message to travel from sender to receiver) in Example 17.4 is not very sophisticated. We briefly discuss how the following types of message delays can be specified in our methodology:

1. The message delay is *exactly* Δ time units.
2. The message delay is *at most* Δ time units.
3. The message delay is *at least* Δ time units.
4. The message delay is *any value* in the time interval $[\delta, \Delta]$.

To address the first three types, we introduce a message delay operator `dly`:

```
sort DlyMsg .      subsorts Msg < DlyMsg < Configuration .
op dly : Msg Time -> DlyMsg [ctor right id: 0] .
```

so that `dly( $m$ ,  $t$ )` denotes a message with remaining delay t . `right id: 0` means that a message m is considered identical to `dly( $m$ , 0)`. In the delay kinds 1–3 above,

- the sender sends a message of the form `dly( $m$ ,  $\Delta$ )`, and
- time advance decreases the remaining delay according to the elapsed time:

```
eq timeEffect(dly(M, T1), T2) = dly(M, T1 minus T2) .
```

No equation is needed to define the effect of time elapse on a “ripe” message; that is, a message with no remaining delay (why not?).

The only difference in the specification of the communication forms 1–3 is the way `mte` is defined on messages and the way the receiver reads messages:

1. If the message must be read exactly Δ time units after it was sent, the receiver must read *ripe* messages m (i.e., messages without the `dly` operator). `mte` must be defined so that time advance stops when a message is ripe, and so that time cannot advance when the state contains a ripe message. Since a ripe message m is identical to `dly( $m$ , 0)`, the following definition achieves all of this:

```
eq mte(dly(M, T)) = T .
```


2. If the message delay is *at most* Δ , the receiver should read a message of the form $\text{dly}(m, T)$, and the above definition $\text{mte}(\text{dly}(M, T)) = T$ ensures that the message is read no later than after its (maximal) delay has expired.
3. If the message delay is *at least* Δ , the receiver must read the undelayed message m , while the time advance does not need to stop when the message is ripe:

$\text{eq mte}(\text{dly}(M, T)) = \infty$.

4. Specifying message delays of type 4 above is left as Exercise 244.

Abstracting Away the System Clock.

Our specification methodology adds to the state a “system clock” which shows the total time elapse in the system. Since time typically can progress forever, the value of this system clock will grow beyond any bound, so that the reachable state space becomes infinite. Unbounded search for unreachable states and LTL model checking analysis will therefore fail to terminate. If we do not care about the total time elapse in the system, we can just abstract from the system clock by adding the following equation, which “removes” the ‘in time t ’ part of the state:

```
(omod SINGLE-CLOCK-NO-SYSTEM-CLOCK is including SINGLE-CLOCK .
  var GS : GlobalState .    var T : Time .
  eq GS in time T = GS .
endmod)
```

Unbounded search for unreachable states will now terminate:

```
Maude> (search {< genta : Clock | state : running, time : 0 >} =>*
      {< genta : Clock | time : T:Time >} such that T:Time > 12.)
```

No solution.

Time Advance in Tick Rules.

In most systems I have encountered in practice, an event takes place when a message arrives or a timer expires. The tick rule can then advance time all the way until the next event happens, namely by $\text{mte}(\text{CONF})$, without losing behaviors. This corresponds to *event-driven simulation*. If actions can happen at arbitrary times, as in the above examples, advancing time by one time unit in each application of the tick rule(s) covers all possible system behaviors when the time domain is discrete.

17.1.2 Timed Temporal Logics

Real-time system requirements are often *timed* properties, such as “the airbag will deploy *within 10 milliseconds* after a crash has been detected” or “the ventilator machine cannot be paused more than once, and for no longer than *two seconds*, *every ten minutes* during surgery.”

There are a number of timed extensions of temporal logics for specifying timed system requirements (see, e.g., [4]). The standard extension (also called *metric temporal logic*) equips the temporal operator $\mathcal{U}$ (and therefore also $\Box$, $\Diamond$, and $\mathcal{W}$) with a time interval I : $\phi_1 \mathcal{U}_I \phi_2$. A path π satisfies such a formula if it reaches a ϕ_2 -position in some time within the interval I , and all positions up to that point satisfy ϕ_1 .

The first property above can then be formalized as

$$\Box_{\geq 0} (\text{crash} \rightarrow \Diamond_{\leq 10ms} \text{airbag}).$$

The second property is formalized as $\Box_{\geq 0} (\text{paused} \rightarrow \Diamond_{\leq 2sec} (\Box_{\leq 10min} \neg \text{paused}))$.

17.1.3 Real-Time Maude

Real-Time Maude [90,93] supports the specification and analysis of real-time rewrite theories. It is implemented in Maude as an extension of Full Maude, and provides timed versions of Maude's analysis methods: simulate the system up to a certain time; search for states satisfying a certain pattern that are reachable in a certain time interval; and timed temporal logic model checking [68].

Real-Time Maude has been applied to a wide range of state-of-the-art applications, including wireless sensor networks and cloud storage systems, and also provides semantics and formal analysis to industrial modeling languages such as AADL and Ptolemy II [89,90]. It is worth remarking that Real-Time Maude randomized simulations could estimate the *performance* as well as dedicated simulation tools for wireless sensor networks [95].

Exercise 241 *Our watches only show the hours. Specify a watch, or a system with multiple watches, that display time in terms of hours, minutes, and seconds.*

Exercise 242 *Specify populations in a timed setting, so that when time advances by one time unit, a new year begins and everybody becomes one year older. Furthermore, an engaged couple should marry or break the engagement the same year, and a separated couple must be divorced within two years.*

Exercise 243 *Model the two-phase commit protocol as it was described: assume that `prepare`, `ok`, and `notOK` messages may be lost or much delayed, and that the coordinator assumes a “not OK” answer if it does not receive an answer from a node within 20 time units. Assume furthermore that `abort` and `commit` messages are not lost, and use Maude to check whether all “final” states are consistent.*

Exercise 244 *Assuming discrete time, how can we model that the message delay can be any time value in the interval $[\tau_1, \tau_2]$? What kind of message should the sender send, the receiver receive, and how should `timeEffect` and `mte` be defined?*

17.2 Probabilistic Systems

A rewrite $t \longrightarrow t'$ means that it is *possible* to go from state t to state t' . It says nothing about how *probable* it is to reach t' from t ; the probability could be 99%, or it could be 0.001%. For example, the message loss rule says that messages *can* be lost, but not how often this happens. Likewise, failures can happen, but how frequently? Abstracting away probabilities makes specifications less detailed; furthermore, model checking analysis covers *all possible* behaviors from the initial state.

There are, however, many reasons for explicitly modeling the probability of performing certain events—such as the probability of celebrating a `birthday` instead of dying—and of selecting certain values, such as message delays, probabilistically.

First of all, we are often interested in reasoning about the *probability* of something happening: What is the probability of winning at least \$200 after a night in the casino? A model which abstracts away probabilities cannot be used to analyze such questions; it can only be used to show that it is *possible* to win big and to lose big.

Even in highly safety-critical applications, such as aerospace and avionics applications, you typically cannot guarantee or require the absence of errors (which may happen due to fatigue of the physical components, fire, “bit flips” caused by cosmic radiation, etc.). Instead, certification authorities require that catastrophic failures occur with probability less than one per billion flight hours.

More generally, modeling probabilities allows us to predict the expected *performance*, or *quality of service*, of a system. To estimate key measures, such as the expected average latency of requests, the percentage of successful transactions, or the percentage of availability of a service, aspects like the expected distribution of message delays, the distribution of the workloads, etc. must be modeled.

Furthermore, many distributed algorithms are probabilistic, or randomized, in nature, for example to break symmetries (if all nodes do the same thing, that might be bad). Even the quicksort algorithm in Section 2.9.1 is supposed to have better expected performance in many cases when it is a randomized algorithm where the pivot element is chosen randomly instead of deterministically.

There is another important reason for considering probabilistic models: Precise (non-probabilistic or probabilistic) model checking quickly becomes unfeasible for large systems due to large state spaces involved and stored. If we do not need 100% confidence in the analysis results, a new world opens up: *statistical model checking* [102, 109]. Statistical model checking performs as many *randomized simulations* as needed to reach the user-defined confidence in the results. Obviously, the higher confidence you want, the more simulations are needed. Statistical model checking is a promising analysis method that scales up to large systems, since:

- Only simulations are performed: there is no need to explore and store all reachable states, so the use of memory is low.
- The individual simulations can run on different machines, so that statistical model checking is easy to parallelize.

Rewriting logic has been extended to specify probabilistic systems as *probabilistic rewrite theories* [1]. However, there is currently no tool that executes such

theories directly. Instead, a probabilistic rewrite theory has to be transformed into an ordinary rewriting logic specification, which can then be connected to statistical model checkers such as VESTA [103], its parallel version PVESTA [2], and MultiVeStA [101]. These tools are used to estimate:

- the expected (average) *value* of a certain expression of a run, such as the amount of money remaining after a long night at the blackjack table, and
- the expected *probability* that a run satisfies a certain property, such as having won at least \$200 at the end of the night in the casino.

Probabilistic rewrite theories and statistical model checking have been used, e.g., to evaluate the efficiency of mechanisms against denial-of-service attacks [3, 34], to estimate the performance of the Apache Cassandra data store and its proposed optimization [70] (this work also compared the performance estimates obtained by statistical model checking analysis of high-level models with the performance of running the actual Cassandra code), and evaluating and redesigning a state-of-the-art wireless sensor network algorithm [59].

17.2.1 Probabilistic Rewrite Theories

In *probabilistic rewrite theories* [1], a probabilistic rewrite rule has the form

$t \longrightarrow t'$ **with probability** $y_1 := \text{dist}_1(x_1, \dots, x_n) \wedge \dots \wedge y_k := \text{dist}_k(x_1, \dots, x_n)$ **if cond**

where $x_1, \dots, x_n$ are the variables in t , and $y_1, \dots, y_k$ are variables in t' that do *not* appear in t . These new variables could in principle be instantiated with *any* values. In the above rule, the value of y_j is sampled (“selected”) probabilistically from the *probability distribution* $\text{dist}_j(x_1, \dots, x_n)$, which is a function of the values of the x_i ; different matches yield different probability distributions.

Example 17.5. A system where a rewrites to b with 30% probability and to c with 70% probability, and where c rewrites to d with 40% probability and to e with 60% probability, can be specified with the following probabilistic rewrite rules, using a Maude-like syntax:

```
prl a => Y with probability Y := {b with prob 0.3; c with prob 0.7} .
```

```
prl c => Y with probability Y := {d with prob 0.4; e with prob 0.6} .◇
```

Example 17.6. Let us revisit our person/population example. There should be a certain probability of dying and of living one more year. This probability is a function of the age of a person: at early age the probability of celebrating a birthday should be much higher than that of dying. Estimating the probability of dying is far from my expertise, so I assume for illustration purposes that the probability of dying at age x is $x^4/120^4$. The following probabilistic rewrite rule specifies birthdays and deaths with these probabilities:

```

cprl [birthdayOrDeath] :
  < P : Person | age : X, state : S >
  =>
    if B then < P : Person | state : deceased >
      else < P : Person | age : X+1 > fi
    with probability B := bernoulli((X^4) / (120^4))
    if S /= deceased .

```

The *Bernoulli distribution* with probability p returns `true` with probability p and `false` with probability $1 - p$. The probability of assigning the value `true` to the new Boolean variable `B` in the right hand side of the rule is therefore $x^4/120^4$. If `true` is sampled, the person becomes `deceased`; otherwise, the person has dodged the Grim Reaper for another year and celebrates his/her birthday. $\diamond$

Example 17.7. Consider the blackjack example in Section 10.4. In a probabilistic theory, the next card “number” is sampled from the *uniform* distribution, where each (natural) number in the interval $[0, n]$ has the same probability of being chosen. For example, the rule `playerHit` in Section 10.4 becomes the probabilistic rule

```

cprl [playerHit] :
  < T : Table | shoe : CARDS, turn : P >
  < P : Player | hand : CARDS2 >
  =>
    < T : Table | shoe : remove(getNthCard(X, CARDS), CARDS) >
    < P : Player | hand : CARDS2 :: getNthCard(X, CARDS) >
    with probability X := uniform(size(CARDS) - 1)
    if not (leastValue(CARDS2) >= 15 or bestValue(CARDS2) >= 18) .

```

The probability distribution is again a function of the current state, namely, of the number of cards remaining in the shoe. $\diamond$

Transforming to Ordinary Rewrite Theories.

The direct execution of probabilistic rewrite theories is at the moment not supported by any tool. Such a theory must therefore be (manually) transformed into an ordinary rewrite theory. The key infrastructure provided by Maude for this purpose is the built-in function `random` described in Section 2.7.7, and the built-in constant `counter` with an (implicit) rewrite rule `counter => N:Nat`. The first time `counter` is rewritten, it rewrites to 0, the next time it rewrites to 1, and so on. Therefore, each time `random(counter)` rewrites, it rewrites to the next random number. We use this feature to transform the probabilistic rewrite rule in Example 17.6 into the ordinary Maude rewrite rule

```

crl [birthdayOrDeath] :
  < P : Person | age : X, state : S >
  =>
    if B then ... else ... fi
    if bernoulli((X^4) / (120^4)) => B /\ S /= deceased .

```

where `bernoulli` is defined

```

rl bernoulli(P)
=> if (random(counter) / max-rand) <= P then true else false fi .

```

where `max-rand` is the largest random number.

Obtaining Purely Probabilistic Models.

Current statistical model checking techniques require that there is *no nondeterminism* in the system that does not arise from probabilistic choices: the models must be *purely probabilistic*. Agha, Meseguer, and Sen propose the following method for resolving unquantified nondeterminism in object-oriented specifications in which each event is triggered by the arrival of a message [1]: Let the delay of each message be sampled from a *dense/continuous* distribution (such as intervals of rational or real numbers). The probability that two messages have the same delay, and therefore that two events happen at the same time, is then 0, eliminating nondeterminism.² These restrictions can be relaxed, as long as two events never happen at the same time.

Example 17.8. Assume that there are two persons, "Robert" and "Roland", in the state. The rule `birthdayOrDeath` could be applied on either person. If the system is a timed system, we can start by generating the messages

```

dly(firstBDay("Robert"), X)    dly(firstBDay("Roland"), Y)
with probability X := contDist(365) /\ Y := contDist(365)

```

where `contDist` is some dense/continuous distribution returning some rational/real number between 0 and 365, determining the “birthday” of the person down to the millisecond. When a person receives his/her `firstBDay` message, (s)he remembers the birthday. The key thing is that a person only applies the (modified) `birthdayOrDeath` rule each time (s)he has a birthday. The probability that these two persons have birthday the same millisecond is 0, and hence they never celebrate birthday/death at the same time, “resolving” nondeterminism, without having to create new messages every year. $\diamond$

17.2.2 Probabilistic Temporal Logics

Just as temporal logics can be extended with time, so can they be extended to *probabilistic temporal logics* [8, 55], many of which deal with both time and probabilities. For example, the formula $\mathcal{P}_{\geq 0.9}(\Box(\text{crash} \rightarrow \Diamond_{\leq 10ms} \text{airbag}))$ says that the airbag will deploy within 10 milliseconds after a crash, with probability 90% or higher.

There are a number of *probabilistic model checkers* which can check whether a probabilistic temporal logic formula holds in a model [56, 63]. The difference between such (precise) probabilistic model checking and statistical model checking is that the former guarantees that it gets the right answer, whereas the latter cannot

²That the probability of something happening is 0 does not imply that it cannot happen.

guarantee that its answer is correct, only that it is *likely* to be correct. For example, consider the property $\Psi \equiv a \models \mathcal{P}_{\geq 0.41}(\Diamond E)$ of the system in Example 17.5, with E an atomic proposition that holds only in state e . Since the probability of reaching e from a is exactly 42%, a probabilistic model checker will always answer that Ψ holds, whereas a statistical model checker *might* be very unlucky with its randomized simulations and could say that Ψ does not hold. On the other hand, as already mentioned, probabilistic model checking suffers from state space explosion and does not scale up well, in contrast to statistical model checking.

17.2.3 PVeSTA Analysis

The VESTA family of tools can be connected to discrete-event simulators to statistically evaluate a number of randomized simulations. In particular, they also interface to Maude. The tools use the QUATEX property specification language [1] to define properties/measures on single runs/simulations, and run simulations to statistically estimate either

1. the expected average *value* of an expression on a path, or
2. the expected *probability* that a run satisfies some property on paths.

Notice that checking probabilistic temporal logic properties is a special case of checking QUATEX expressions: $\mathcal{P}_{\geq p}(\phi)$ holds if the expected probability that a run satisfies ϕ is greater than or equal to p .

Example 17.9. In Example 17.6, the interesting *value* of a run is the value of the `age` attribute in the final state of the execution; in the blackjack game, the key value of a run is the value of the `money` attribute in the final state of the run.

The *probabilities* we might be interested in are:

1. What is the probability of becoming at least 65 years old?
2. Starting with \$1000, what is the probability of having at least \$1200 after 20 rounds of blackjack at the \$100 table? $\Diamond$

The VESTA tools perform as many simulations as needed to come up with an estimated average value/probability, with $(100 - \alpha)\%$ statistical confidence, and with interval size δ , where both α and δ are given by the user.

What are these parameters? α gives the acceptable error in the correctness of the result. We can obviously gain high statistical confidence that the expected average life time of a person lies in the interval $[2, 119]$ even with very few samples. But this interval is not very helpful. The parameter δ therefore gives us the size of the desired confidence interval. That is, if VESTA estimates that the average value of some measure is v , then it means that the expected *average* value of the measure is in the interval $[v - \delta/2, v + \delta/2]$, with $(100 - \alpha)\%$ statistical confidence. Obviously, the smaller α and δ we want, the more simulations must be performed.

Back to the essence:

1. Should I quit my day job and move to Las Vegas?
2. Should I play blackjack at the “dealer-must-hit-soft-17” table or at the “dealer-stands-on-all-17s” table?
3. How long can I expect to live?
4. What is the probability that a newborn lives until (s)he is at least 65 years old?

PVeSTA analysis shows that:

1. If I start with \$1000 and play 20 rounds of blackjack at a \$100 “dealer stands on all 17s” table with the amateurish strategy in Section 10.4, then:
 - the expected amount of money I have left at the end of the night is \$876; and
 - the expected probability that I can walk out of the casino with \$1200 or more is a promising 31%.
2. If I instead play at a “dealer must hit soft 17” table, then the expected amount of money I have left is \$826.³
3. The life expectancy of a newborn is 58 years in my crude model.
4. The probability that a newborn becomes at least 65 years is 34%.

The results are averages over multiple PVeSta statistical model checking sessions; each such statistical model checking session needs around 60 runs, for statistical confidence 99% and $\delta = 1$.

Exercise 245 *How would you model message communication where the message delay is chosen probabilistically by some distribution, and where the likelihood of a message being dropped is k%?*

Exercise 246 *Add suitable probabilities (and define the corresponding probabilistic rewrite rules) to other events in a population.*

Exercise 247 *Explain how some of your favorite probability distributions can be sampled in Maude, using `random` and `counter`.*

Exercise 248 *How would you model failures that occur with probability of one failure every 100 days?*

³It is known that “dealer must hit soft 17” is indeed better for the casino, even when faced with more sophisticated players.

Appendix A

Mathematical Preliminaries

This appendix gives some necessary background to the mathematical concepts used in this book, many of which are only needed in Chapter 7.

Sets. A *set* is a finite or infinite collection of elements. Examples of sets are the natural numbers $\mathbb{N} = \{0, 1, 2, 3, \dots\}$ and the set $\{a, b, c\}$. We write $a \in A$ if a is an element in the set A . If $A_1, \dots, A_n$ are sets, then $A_1 \times \dots \times A_n$ is another set, the *product* of those sets, whose elements are n -tuples $(a_1, \dots, a_n)$, where each $a_i \in A_i$. The *powerset* $\mathcal{P}(A)$ of A is the set whose elements are the *subsets* of A .

Functions. Given two sets A and B , a *function* $f : A \rightarrow B$ assigns to *each* element $a \in A$ a *unique* element $f(a) \in B$. It follows from this definition that: if $a_1 = a_2$, then $f(a_1) = f(a_2)$. Any function f is *total*: it can be applied to any value a in A ; and the result $f(a)$ must be defined and must be an element in the set B . A function f that cannot be applied to *all* elements in A is called a *partial* function.

To avoid having to find a name for a function, a function can also be defined using “lambda notation.” For example, $\lambda m, n . m + 2n$ denotes some function $h : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ defined by $h(m, n) = m + 2n$. A function can also be defined by showing the mapping of elements explicitly; for example, $\{a \mapsto 1, b \mapsto 3, c \mapsto 2\}$ defines some function $f_1 : \{a, b, c\} \rightarrow \{1, 2, 3\}$.

The *composition* $g \circ f$ of two functions $f : A \rightarrow B$ and $g : B \rightarrow C$ is a function $g \circ f : A \rightarrow C$ defined by $(g \circ f)(a) = g(f(a))$ for each $a \in A$.

A function $f : A \rightarrow B$ is *surjective* if for *each* element $b \in B$ there is *some* $a \in A$ such that $f(a) = b$. That is, the function f can “reach” all elements in the set B . For example, the above function h is surjective. The function $k : \mathbb{N} \rightarrow \mathbb{N}$ defined by $k(n) = 2n$ is *not* surjective, since the numbers 1, 3, 5, ... cannot be reached by k .

A function $f : A \rightarrow B$ is *injective* if $a_1 \neq a_2$ implies $f(a_1) \neq f(a_2)$; that is, f does not map different A -values to the same value in B . The above function h is *not* injective, since $(4, 2) \neq (2, 3)$, yet $h(4, 2) = h(2, 3) = 8$, whereas k is injective, since $m \neq n$ implies $k(m) = 2m \neq 2n = k(n)$.

A function that is both surjective and injective is *bijective*. A bijective function $f : A \rightarrow B$ has an *inverse* function $f^{-1} : B \rightarrow A$ such that $f^{-1} \circ f$ is the identity function id_A on A , and $f \circ f^{-1}$ is the identity function id_B on B , where, for any set S , the identity function $id_S : S \rightarrow S$ is defined by $id_S(s) = s$ for each $s \in S$. For example, the above function f_1 is bijective, and its inverse is the function $f_1^{-1} : \{1, 2, 3\} \rightarrow \{a, b, c\}$ with $f_1^{-1}(1) = a$, $f_1^{-1}(2) = c$, and $f_1^{-1}(3) = b$.

Relations. A relation R over a set S is a subset $R \subseteq S$. We usually write $R(s)$ for $s \in R$. If S is the product $A \times A$, then R is called a *binary relation* over A , and we often write $a_1 R a_2$ for $(a_1, a_2) \in R$.

Partial Orders. A binary relation R over A is a *partial order* (over A) if and only if it satisfies the following properties for all a, b, c in A :

- *Reflexivity:* aRa holds for all $a \in A$.
- *Antisymmetry:* If aRb and bRa both hold, then $a = b$.
- *Transitivity:* If aRb and bRc both hold, then aRc also holds.

Examples of partial orders include the relations $\leq$ and $\geq$ on numbers, and the (reflexive) subset relation $\subseteq$ on sets. The relations $<$ and $>$ on numbers (do not satisfy reflexivity), and “brother of” (does not satisfy antisymmetry) are *not* partial orders.

A binary relation R over A is *strict partial order* if and only if it satisfies:

- *Irreflexivity:* There is no $a \in A$ such that aRa .
- *Transitivity:* Defined as above.

Examples of strict partial orders are the relations $<$ and $>$ on numbers, the “forefather of” relation, and the proper subset relation $\subset$ on sets. Antisymmetry is not mentioned, since aRb and bRa cannot both hold in a strict partial order (why not?).

Equivalence and Congruence Relations. An *equivalence relation* $\approx$ over a set A is a binary relation over A that satisfies the following properties for all $a, b, c \in A$:

- *Reflexivity:* $a \approx a$ for all $a \in A$;
- *Symmetry:* If $a \approx b$ holds, then $b \approx a$ also holds.
- *Transitivity:* If $a \approx b$ and $b \approx c$ both hold, then $a \approx c$ also holds.

Examples of equivalence relations include:

1. Standard equality $=$ over the natural numbers (or any other set for that matter).
2. The relation $\equiv_k$ over $\mathbb{N}$, for any $k > 1$, defined by $m \equiv_k n$ if and only if $m \bmod k = n \bmod k$ (i.e., m and n have the same remainder when divided by k).
3. The relation $\equiv_{card}$ defined on sets of natural numbers by $s_1 \equiv_{card} s_2$ if and only if s_1 and s_2 have the same cardinality (the same number of distinct elements).
4. The relation *sameFather* on persons, where *sameFather*(p_1, p_2) holds if and only if p_1 and p_2 have the same father.

The relation $\leq$ on natural numbers is not an equivalence relation, since symmetry does not hold: $5 \leq 8$ holds, but not the symmetric $8 \leq 5$.

The *equivalence class* $[a]_{\approx}$ of a w.r.t. the equivalence relation $\approx$ is the set of elements that are $\approx$ -equivalent to a . Formally, $[a]_{\approx} = \{x \in A \mid x \approx a\}$. For example, the equivalence classes over the relation $\equiv_3$ are

$$\begin{aligned}
[0]_{\equiv_3} &= \{0, 3, 6, 9, \dots\} \\
[1]_{\equiv_3} &= \{1, 4, 7, 10, \dots\} \\
[2]_{\equiv_3} &= \{2, 5, 8, 11, \dots\},
\end{aligned}$$

and some equivalence classes over the relation $\equiv_{card}$ are:

$$\begin{aligned}
[\emptyset]_{\equiv_{card}} &= \{\emptyset\} \\
[\{8\}]_{\equiv_{card}} &= \{\{0\}, \{1\}, \{2\}, \{3\}, \dots\} \\
[\{4, 5\}]_{\equiv_{card}} &= \{\{0, 1\}, \{0, 2\}, \{0, 3\}, \{1, 2\}, \{2, 3\}, \{1, 4\}, \dots\} \\
&\vdots \qquad \qquad \qquad \vdots
\end{aligned}$$

The equivalence classes of an equivalence relation $\approx$ on A *partition* the set A .

An equivalence relation $\approx$ is a *congruence* on A w.r.t. to a set of functions F (on A) if and only if for each function $f \in F$: if $a_1 \approx a'_1, \dots, a_n \approx a'_n$ all hold, then $f(a_1, \dots, a_n) \approx f(a'_1, \dots, a'_n)$ also holds. For example:

1. Standard equality $=$ is a congruence for any function: If a_1 and a'_1 are the same element, then, by the definition of a function, $f(a_1)$ and $f(a'_1)$ must also be the same element.
2. The relation $\equiv_3$ is congruence w.r.t. the functions $+$, $-$, and $*$ (prove it!).
3. The relation $\equiv_{card}$ is not a congruence w.r.t. standard set operators such as union and intersection, since $\{1, 2\} \equiv_{card} \{3, 4\}$ and $\{1, 2, 3\} \equiv_{card} \{7, 8, 9\}$, whereas $\{1, 2\} \cup \{1, 2, 3\} \not\equiv_{card} \{3, 4\} \cup \{7, 8, 9\}$.
4. The relation *sameFather* is not congruent w.r.t. the function *mother* : *Person* $\rightarrow$ *Person*, since even though Aphrodite and Apollo have the same father, Zeus, their respective mothers Dione and Leto do not have the same fathers.

Mathematical Induction. Let $P(n)$ be a property about a natural number n . If you can prove the following:

- Basis: $P(0)$ holds, and
- Induction step: $P(k+1)$ holds, for *any* natural number k , assuming that $P(k)$ holds (the assumption that $P(k)$ holds is called the *induction hypothesis*).

Then you have proved that $P(n)$ holds for all natural numbers $n \in \{0, 1, 2, \dots\}$.

To prove that $P(n)$ holds for all $n \geq m$, the base case amounts to proving $P(m)$.

Another (equivalent) version of mathematical induction is: If for *any* natural number k , the property $P(k)$ holds when you can assume (as induction hypothesis) $P(k')$ for all $k' < k$, then $P(n)$ holds for all natural numbers n .

Exercise 249 Give an example of a function for which $\equiv_3$ is not a congruence.

Exercise 250 Prove that $0 + 1 + 2 + \dots + n = \frac{n \cdot (n+1)}{2}$ for all $n \in \mathbb{N}$.

Exercise 251 Prove that $n! \geq 2^n$ for all natural numbers $n \geq 4$.

Exercise 252 Show that the two versions of the induction principle for the natural numbers are equivalent.

References

1. G. Agha, J. Meseguer, and K. Sen. PMAude: Rewrite-based specification language for probabilistic object systems. *Electronic Notes in Theoretical Computer Science*, 153(2):213–239, 2006.
2. M. AlTurki and J. Meseguer. PVESTA: A parallel statistical model checking and quantitative analysis tool. In *Proc. Algebra and Coalgebra in Computer Science (CALCO 2011)*, volume 6859 of *Lecture Notes in Computer Science*. Springer, 2011.
3. M. AlTurki, J. Meseguer, and C. A. Gunter. Probabilistic modeling and analysis of DoS protection for the ASV protocol. *Electronic Notes in Theoretical Computer Science*, 234:3–18, 2009.
4. R. Alur and T. A. Henzinger. Logics and models of real time: A survey. In *Real-Time: Theory in Practice*, volume 600 of *Lecture Notes in Computer Science*. Springer, 1992.
5. A. Armando et al. The AVISPA tool for the automated validation of internet security protocols and applications. In *Proc. Computer Aided Verification (CAV 2005)*, volume 3576 of *Lecture Notes in Computer Science*. Springer, 2005.
6. F. Baader and T. Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
7. K. Bae and J. Meseguer. Model checking linear temporal logic of rewriting formulas under localized fairness. *Science of Computer Programming*, 99:193–234, 2015.
8. C. Baier, J.-P. Katoen, and H. Hermanns. Approximate symbolic model checking of continuous-time Markov chains. In *Proc. Concurrency Theory (CONCUR 1999)*, volume 1664 of *Lecture Notes in Computer Science*. Springer, 1999.
9. J. Baker et al. Megastore: Providing scalable, highly available storage for interactive services. In *Proc. Innovative Data Systems Research (CIDR 2011)*. www.cidrdb.org, 2011.
10. D. Benanav, D. Kapur, and P. Narendran. Complexity of matching problems. In *Proc. Rewriting Techniques and Applications (RTA 1985)*, volume 202 of *Lecture Notes in Computer Science*. Springer, 1985.
11. J. A. Bergstra and J. V. Tucker. A characterization of computable data types by means of a finite, equational specification method. CWI Technical Report IW 124/79, Stichting Mathematisch Centrum, Amsterdam, 1979.
12. J. A. Bergstra and J. V. Tucker. Algebraic specification of computable and semicomputable data types. *Theoretical Computer Science*, 50:137–181, 1987.

13. B. Blanchet. Automatic verification of security protocols in the symbolic model: The verifier ProVerif. In *Foundations of Security Analysis and Design VII (FOSAD 2012/2013)*, volume 8604 of *Lecture Notes in Computer Science*. Springer, 2014.
14. D. Bogdanas and G. Rosu. K-Java: A complete semantics of Java. In *Proc. Principles of Programming Languages (POPL 2015)*. ACM, 2015.
15. E. A. Brewer. Towards robust distributed systems (abstract). In *Proc. Principles of Distributed Computing (PODC 2000)*. ACM, 2000.
16. R. Bruni and J. Meseguer. Semantic foundations for generalized rewrite theories. *Theoretical Computer Science*, 360(1-3):386–414, 2006.
17. M. Burrows, M. Abadi, and R. M. Needham. A logic of authentication. *ACM Transactions on Computer Systems*, 8(1):18–36, 1990.
18. E. Chang and R. Roberts. An improved algorithm for decentralized extrema-finding in circular configurations of processes. *Communications of the ACM*, 22:281–283, 1979.
19. S. Chen, J. Meseguer, R. Sasse, H. J. Wang, and Y.-M. Wang. A systematic approach to uncover security flaws in GUI logic. In *Proc. IEEE Symposium on Security and Privacy*. IEEE Computer Society, 2007.
20. M. Clavel, F. Durán, S. Eker, S. Escobar, P. Lincoln, N. Martí-Oliet, J. Meseguer, and C. Talcott. *Maude Manual (Version 2.7.1)*, July 2016. <http://maude.cs.illinois.edu>.
21. M. Clavel, F. Durán, S. Eker, P. Lincoln, N. Martí-Oliet, J. Meseguer, and C. Talcott. *All About Maude – A High-Performance Logical Framework*, volume 4350 of *Lecture Notes in Computer Science*. Springer, 2007.
22. S. A. Cook. The complexity of theorem-proving procedures. In *Proc. ACM Symposium on Theory of Computing (STOC 1971)*. ACM, 1971.
23. G. Coulouris, J. Dollimore, and T. Kindberg. *Distributed Systems: Concepts and Design*. Addison-Wesley, third edition, 2001.
24. C. J. F. Cremers. The Scyther Tool: Verification, falsification, and analysis of security protocols. In *Proc. Computer Aided Verification (CAV 2008)*, volume 5123 of *Lecture Notes in Computer Science*. Springer, 2008.
25. M. Davis, Y. Matijasevič, and J. Robinson. Hilbert’s tenth problem. Diophantine equations: positive aspects of a negative solution. In *Mathematical Developments Arising from Hilbert Problems, Part 2*, volume 28.2 of *Proceedings of Symposia in Pure Mathematics*. American Mathematical Society, 1976.
26. N. Dershowitz. Orderings for term-rewriting systems. *Theoretical Computer Science*, 17:279–301, 1982.
27. N. Dershowitz. Termination of rewriting. *Journal of Symbolic Computation*, 3:69–116, 1987.
28. W. Diffie and M. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22:644–654, 1976.
29. E. W. Dijkstra. Two starvation free solutions to a general exclusion problem. EWD 625, Plataanstraat 5, 5671 Al Nuenen, The Netherlands, 1978.
30. D. Dolev and A. Yao. On the security of public-key protocols. *IEEE Transactions on Information Theory*, 29:198–208, 1983.
31. G. Dowek, C. A. Muñoz, and C. Rocha. Rewriting logic semantics of a plan execution language. In *Proc. Structural Operational Semantics (SOS 2009)*, volume 18 of *Electronic Proceedings in Theoretical Computer Science*, 2009.
32. F. Durán, S. Lucas, C. Marché, J. Meseguer, and X. Urbain. Proving operational termination of membership equational programs. *Higher-Order and Symbolic Computation*, 21(1-2):59–88, 2008.
33. F. Durán and J. Meseguer. On the Church-Rosser and coherence properties of conditional order-sorted rewrite theories. *Journal of Logic and Algebraic Programming*, 81(7-8):816–850, 2012.

34. J. Eckhardt, T. Mühlbauer, M. AlTurki, J. Meseguer, and M. Wirsing. Stable availability under denial of service attacks through formal patterns. In *Proc. Fundamental Approaches to Software Engineering (FASE 2012)*, volume 7212 of *Lecture Notes in Computer Science*. Springer, 2012.
35. H. Ehrig and B. Mahr. *Fundamentals of Algebraic Specifications I, Equations and Initial Semantics*, volume 6 of *EATCS Monographs on Theoretical Computer Science*. Springer, 1985.
36. S. Eker. Fast matching in combinations of regular equational theories. *Electronic Notes in Theoretical Computer Science*, 4:90–109, 1996.
37. S. Eker, M. Knapp, K. Laderoute, P. Lincoln, J. Meseguer, and K. Sonmez. Pathway logic: Symbolic analysis of biological signaling. In *Proc. Pacific Symposium on Biocomputing, Hawaii*, Jan 2002.
38. S. Eker, M. Knapp, K. Laderoute, P. Lincoln, and C. Talcott. Pathway logic: Executable models of biological networks. *Electronic Notes in Theoretical Computer Science*, 71:144–161, 2002.
39. C. Ellison and G. Rosu. An executable formal semantics of C with applications. In *Proc. Principles of Programming Languages (POPL 2012)*. ACM, 2012.
40. R. Elmasri and S. B. Navathe. *Fundamentals of Database Systems*. Addison-Wesley, sixth edition, 2011.
41. E. A. Emerson. Temporal and modal logic. In J. van Leeuwen, editor, *Handbook of Theoretical Computer Science*, volume B. Elsevier, 1990.
42. S. Escobar, C. A. Meadows, and J. Meseguer. Maude-NPA: Cryptographic protocol analysis modulo equational properties. In *Foundations of Security Analysis and Design V, (FOSAD 2007/2008/2009)*, volume 5705 of *Lecture Notes in Computer Science*. Springer, 2009.
43. A. Farzan, F. Chen, J. Meseguer, and G. Rosu. Formal analysis of Java programs in JavaFAN. In *Proc. Computer Aided Verification (CAV 2004)*, volume 3114 of *Lecture Notes in Computer Science*. Springer, 2004.
44. M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.
45. N. Francez. *Fairness*. Springer, 1986.
46. H. Garcia-Molina. Elections in distributed computer systems. *IEEE Transactions on Computers*, C-31(1):48–59, 1982.
47. M. R. Garey and D. S. Johnson. *Computers and Intractability. A Guide to the Theory of NP-Completeness*. Freeman and Company, 1979.
48. GMP home-page. <http://www.swox.com/gmp/>.
49. J. Goguen, J. Thatcher, E. Wagner, and J. Wright. Abstract data types as initial algebras and the correctness of data representations. In *Computer Graphics, Pattern Recognition, and Data Structure*, pages 89–93. IEEE, 1975.
50. J. A. Goguen and J. Meseguer. Order-sorted algebra I: equational deduction for multiple inheritance, overloading, exceptions and partial operations. *Theoretical Computer Science*, 105:217–273, 1992.
51. A. Goodloe, C. A. Gunter, and M.-O. Stehr. Formal prototyping in early stages of protocol design. In *Proc. Issues in the Theory of Security (WITS 2005)*. ACM, 2005.
52. M. T. Goodrich and R. Tamassia. *Data Structures and Algorithms in JAVA*. J. Wiley & Sons, first edition, 1997.
53. J. Grov and P. C. Ölveczky. Increasing consistency in multi-site data stores: Megastore-CGC and its formal analysis. In *Proc. Software Engineering and Formal Methods (SEFM 2014)*, volume 8702 of *Lecture Notes in Computer Science*. Springer, 2014.
54. R. Guerraoui and A. Schiper. Genuine atomic multicast in asynchronous distributed systems. *Theoretical Computer Science*, 254(1-2):297–316, 2001.
55. H. Hansson and B. Jonsson. A logic for reasoning about time and reliability. *Formal Aspects of Computing*, 6(5):512–535, 1994.

56. A. Hartmanns and H. Hermanns. The Modest toolset: An integrated environment for quantitative modelling and verification. In *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2014)*, volume 8413 of *Lecture Notes in Computer Science*. Springer, 2014.
57. J. Hendrix, J. Meseguer, and H. Ohsaki. A sufficient completeness checker for linear ordered-sorted specifications modulo axioms. In *Proc. Automated Reasoning (IJCAR 2006)*, volume 4130 of *Lecture Notes in Computer Science*. Springer, 2006.
58. S. Kamin and J.-J. Lévy. Two generalizations of the recursive path ordering. Unpublished Note, Department of Computer Science, University of Illinois, Urbana, IL, 1980.
59. M. Katelman, J. Meseguer, and J. Hou. Redesign of the LMST wireless sensor protocol through formal modeling and statistical model checking. In *Proc. Formal Methods for Open Object-Based Distributed Systems (FMOODS 2008)*, volume 5051 of *Lecture Notes in Computer Science*. Springer, 2008.
60. B. Kirkerud. Lecture notes on rewrite systems. Dept. of Informatics, University of Oslo, 1994. <http://heim.ifi.uio.no/~in307/notater/>.
61. T. Kleinjung et al. Factorization of a 768-bit RSA modulus. In *Proc. Advances in Cryptology (CRYPTO 2010)*, volume 6223 of *Lecture Notes in Computer Science*. Springer, 2010.
62. D. E. Knuth and P. B. Bendix. Simple word problems in universal algebras. In J. Leech, editor, *Computational Problems in Abstract Algebra*, pages 263–297. Pergamon Press, 1970.
63. M. Kwiatkowska, G. Norman, and D. Parker. PRISM 4.0: Verification of probabilistic real-time systems. In *Proc. Computer Aided Verification (CAV 2011)*, volume 6806 of *Lecture Notes in Computer Science*. Springer, 2011.
64. L. Lamport. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.
65. L. Lamport. Paxos made simple. *ACM SIGACT News*, 32:51–58, 2001.
66. B. Lampson and H. Sturgis. Crash recovery in a distributed data storage system. Technical report, Xerox Palo Alto Research Center, 1976.
67. F. Laroussinie, N. Markey, and P. Schnoebelen. Temporal logic with forgettable past. In *Proc. Logic in Computer Science (LICS 2002)*. IEEE Computer Society, 2002.
68. D. Lepri, E. Ábrahám, and P. C. Ölveczky. Sound and complete timed CTL model checking of timed Kripke structures and real-time rewrite theories. *Science of Computer Programming*, 99:128–192, 2015.
69. E. Lien and P. C. Ölveczky. Formal modeling and analysis of an IETF multicast protocol. In *Proc. Software Engineering and Formal Methods (SEFM 2009)*. IEEE Computer Society, 2009.
70. S. Liu, J. Ganhotra, M. R. Rahman, S. Nguyen, I. Gupta, and J. Meseguer. Quantitative analysis of consistency in NoSQL key-value stores. *Leibniz Transactions on Embedded Systems*, 4(1):03:1–03:26, 2017.
71. S. Liu, M. R. Rahman, S. Skeirik, I. Gupta, and J. Meseguer. Formal modeling and analysis of Cassandra in Maude. In *Proc. Formal Methods and Software Engineering (ICFEM 2014)*, volume 8829 of *Lecture Notes in Computer Science*. Springer, 2014.
72. G. Lowe. An attack on the Needham-Schroeder public-key authentication protocol. *Information Processing Letters*, 56:131–133, 1995.
73. G. Lowe. Breaking and fixing the Needham-Schroeder public-key protocol using FDR. In *Proc. Tools and Algorithms for Construction and Analysis of Systems (TACAS 1996)*, volume 1055 of *Lecture Notes in Computer Science*. Springer, 1996.
74. R. R. Lutz. Analyzing software requirements errors in safety-critical embedded systems. In *Proc. IEEE International Symposium on Requirements Engineering*. IEEE, 1993.
75. Z. Manna and A. Pnueli. *Temporal Verification of Reactive Systems: Safety*. Springer, 1995.
76. N. Marti-Oliet, M. Palomino, and A. Verdejo. Rewriting logic bibliography by topic: 1990–2011. *Journal of Logic and Algebraic Programming*, 81(7-8):782–815, 2012.

77. Y. Matijasevich. Simple examples of undecidable associative calculi. *Soviet Mathematics Doklady*, 8(2):555–557, 1967.
78. S. Meier, B. Schmidt, C. Cremers, and D. A. Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *Proc. Computer Aided Verification (CAV 2013)*, volume 8044 of *Lecture Notes in Computer Science*. Springer, 2013.
79. A. Menezes, P. van Oorschot, and S. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996. <http://www.cacr.math.uwaterloo.ca/hac>.
80. J. Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science*, 96:73–155, 1992.
81. J. Meseguer. A logical theory of concurrent objects and its realization in the Maude language. In *Research Directions in Concurrent Object-Oriented Programming*. MIT Press, 1993.
82. J. Meseguer. Membership algebra as a logical framework for equational specification. In *Proc. Recent Trends in Algebraic Development Techniques (WADT 1997)*, volume 1376 of *Lecture Notes in Computer Science*. Springer, 1998.
83. J. Meseguer. The temporal logic of rewriting: A gentle introduction. In *Concurrency, Graphs and Models*, volume 5065 of *Lecture Notes in Computer Science*. Springer, 2008.
84. J. Meseguer. Twenty years of rewriting logic. *Journal of Logic and Algebraic Programming*, 81(7-8):721–781, 2012.
85. J. Meseguer and J. A. Goguen. Initiality, induction and computability. In *Algebraic Methods in Semantics*, pages 460–541. Cambridge University Press, 1985.
86. J. Meseguer and G. Rosu. The rewriting logic semantics project. *Theoretical Computer Science*, 373(3):213–237, 2007.
87. J. Meseguer and G. Rosu. The rewriting logic semantics project: A progress report. *Information and Computation*, 231:38–69, 2013.
88. R. Needham and M. Schroeder. Using encryption for authentication in large networks of computers. *Communications of the ACM*, 21(12):993–999, 1978.
89. P. C. Ölveczky. Semantics, simulation, and formal analysis of modeling languages for embedded systems in Real-Time Maude. In *Formal Modeling: Actors, Open Systems, Biological Systems*, volume 7000 of *Lecture Notes in Computer Science*. Springer, 2011.
90. P. C. Ölveczky. Real-Time Maude and its applications. In *Proc. Rewriting Logic and Its Applications (WRLA'14)*, volume 8663 of *Lecture Notes in Computer Science*. Springer, 2014.
91. P. C. Ölveczky, A. Boronat, and J. Meseguer. Formal semantics and analysis of behavioral AADL models in Real-Time Maude. In *Proc. Formal Techniques for Distributed Systems (FORTE 2010)*, volume 6117 of *Lecture Notes in Computer Science*. Springer, 2010.
92. P. C. Ölveczky and J. Meseguer. Specification of real-time and hybrid systems in rewriting logic. *Theoretical Computer Science*, 285:359–405, 2002.
93. P. C. Ölveczky and J. Meseguer. Semantics and pragmatics of Real-Time Maude. *Higher-Order and Symbolic Computation*, 20(1-2):161–196, 2007.
94. P. C. Ölveczky, J. Meseguer, and C. L. Talcott. Specification and analysis of the AER/NCA active network protocol suite in Real-Time Maude. *Formal Methods in System Design*, 29(3):253–293, 2006.
95. P. C. Ölveczky and S. Thorvaldsen. Formal modeling, performance estimation, and model checking of wireless sensor network algorithms in Real-Time Maude. *Theoretical Computer Science*, 410(2-3):254–280, 2009.
96. L. L. Peterson and B. S. Davie. *Computer Networks: A Systems Approach*. Morgan Kaufmann, second edition, 2000.
97. A. Pnueli. The temporal logic of programs. In *Proc. Foundations of Computer Science (FOCS 1977)*. IEEE Computer Society, 1977.
98. R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

99. J. Rushby. Mechanized formal methods: Progress and prospects. In *Proc. Foundations of Software Technology and Theoretical Computer Science (FSTTCS 1996)*, volume 1180 of *Lecture Notes in Computer Science*. Springer, 1996.
100. R. Sasse, S. T. King, J. Meseguer, and S. Tang. IBOS: A correct-by-construction modular browser. In *Proc. Formal Aspects of Component Software (FACS 2012)*, volume 7684 of *Lecture Notes in Computer Science*. Springer, 2012.
101. S. Sebastio and A. Vandin. MultiVeStA: Statistical model checking for discrete event simulators. In *Proc. Performance Evaluation Methodologies and Tools (ValueTools 2013)*. ICST, Brussels, Belgium, 2013.
102. K. Sen, M. Viswanathan, and G. Agha. On statistical model checking of stochastic systems. In *Proc. Computer Aided Verification (CAV 2005)*, volume 3576 of *Lecture Notes in Computer Science*. Springer, 2005.
103. K. Sen, M. Viswanathan, and G. A. Agha. VESTA: A statistical model-checker and analyzer for probabilistic systems. In *Proc. Quantitative Evaluation of Systems (QEST 2005)*. IEEE Computer Society, 2005.
104. P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal of Computing*, 26(5):1484–1509, 1997.
105. Terese. *Term Rewriting Systems*, volume 55 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press, 2003.
106. Y. Toyama. Counterexamples to termination for the direct sum of term rewriting systems. *Information Processing Letters*, 25:141–143, 1987.
107. M. Y. Vardi. Branching vs. linear time: Final showdown. In *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2001)*, volume 2031 of *Lecture Notes in Computer Science*. Springer, 2001.
108. M. Wirsing. Algebraic specification. In J. van Leeuwen, editor, *Handbook of Theoretical Computer Science*, volume B. Elsevier, 1990.
109. H. L. S. Younes and R. G. Simmons. Probabilistic verification of discrete event systems using acceptance sampling. In *Proc. Computer Aided Verification (CAV 2002)*, volume 2404 of *Lecture Notes in Computer Science*. Springer, 2002.

Index

A

algebra, 110
 canonical term algebra, 118, 122
 computable, 24
 ground term algebra, 115
 initial algebra, 120
 isomorphic, 114
 many-sorted, 110
 normal form algebra, 118
 order-sorted, 112
 quotient algebra, 117
 (Σ, E) -algebra, 116
 $\mathcal{T}_{\Sigma, E}$, 117
 term algebra, 115
alternating bit protocol, 205
arity, 16
associativity, 42
atomic commit, 212
atomic multicast, 193
authentication, 1, 233

B

behavior, 142
Bernoulli distribution, 295
binary tree, 26, 106
BINTREE-NAT1, 26
Birkhoff's Completeness Theorem, 119
blackjack, 176
 statistical model checking of, 298
BOOL, 35
BOOLEAN, 14
broadcast, 189
 wireless, 190
built-in module, 35
 Boolean values, 35
 floating-point numbers, 39
 integers, 38

 natural numbers, 36
 random numbers, 40
 rational numbers, 38
 strings, 39

C

canonical form, 63
category, 144
choice operator, 131
class declaration, 164
class inheritance, 165
 multiple inheritance, 166
coffee bean game, 133, 149
comment, 12
communication, 183
 asynchronous, 160, 183
 ordered, 183, 193
 synchronous, 157, 183, 184
 unordered, 184
 unordered and asynchronous, 185
 unreliable, 191
commutativity, 41
computation, 19, 63
computation tree logic (CTL), 282
concurrency, 135
 nested concurrency, 138
 sideways concurrency, 136
CONFIGURATION, 163
configuration, 156
confluence, 63, 85, 90
 ground confluence, 85
 local confluence, 86
congruence, 301
connected component, 30
consensus, 231
 Paxos consensus algorithm, 232
consistency, 212

constant, 16
 constructor, 13, 17
 constructor ground term, 13, 20, 102
 CONVERSION, 40
 critical pair, 89
 critical section, 221
 cryptographic protocol, 1, 233
 CTL*, 282

D

deadlock, 173
 debugging, 58
 declarative program, 11
 definedness, 21
 denotational semantics, 109
 derivation, 19, 63
 digital signature, 234
 dining philosophers problem, 170
 LTL model checking of, 280
 distributed algorithm, 211
 distributed system, 128

E

embedding, 77
 empty sort, 124
 equation, 18
 conditional, 18
 equational attribute, 41
 equational completion, 90
 equational logic, 93, 94
 decidability, 99
 deduction rules, 94
 many-sorted, 124
 soundness and completeness, 118
 undecidability, 96
 equivalence class, 300
 of terms, 41
 equivalence relation, 300
 error sort, 34
 evaluation strategy, 57
 eager evaluation, 57
 lazy evaluation, 57
 event, 251
 event-based property, 250

F

FACTORIAL, 37
 fairness, 173, 255, 271
 compassion, 255
 justice, 255
 FLOAT, 39

football, 132, 250
 formatting, 58
`frew`, 145, 147, 148
 frozen operator, 144
 Full Maude, 155, 162
 obtaining search path, 169
 search, 168
 transform to core Maude, 169
 function, 299
 composition, 299
 identity, 300
 injective, 299
 inverse, 299
 lambda notation, 299
 partial, 31, 299
 surjective, 299
 function symbol, 16

G

GRAPH, 52
 ground term, 16
 group, 91, 93, 100, 112

H

Hilbert's Tenth Problem, 102
 homomorphism, 112

I

identity element, 43
 inductive theorem, 94, 101, 102
 associativity of addition, 104
 commutativity of addition, 105
 induction scheme, 105
 lemma, 104
 INT, 38
 integers, 31
 intended model, 109, 120
 interleaving semantics, 128
 intruder, 241
 Dolev-Yao model, 241
 isomorphism, 114

J

joinable, 89

K

kind, 34
 Knuth-Bendix completion, 100
 Kripke structure, 268
 Kruskal's Theorem, 77

L

label, 130
 language semantics, 6
 leader election, 226
 ring-based algorithm, 226
 spanning-tree-based algorithm, 227
 least sort, 30
 lexicographic comparison, 28, 76
 lexicographic path order (lpo), 79
 implementation, 83
 linear temporal logic (LTL), 263
 formula, 265
 model checking in Maude, 273
 satisfiability and tautology checking, 281
 semantics, 267
 LINK, 194
 link, 193
 limited capacity, 196
 unreliable, 195
 list, 24, 43
 LIST-INT, 44
 LIST-NAT1, 25
 livelock, 173
 looping, 72

M

many-sorted equational specification, 12, 18
 expressiveness, 23
 matching, 61
 modulo axioms, 65
 matching equation, 147
 mathematical induction, 301
 Maude, 4
 applications, 5
 comments, 12
 download, 5, 13
 errors, 14
 functional module, 12
 module importation, 15
 run, 13
 system module, 131
 Windows, 5
 membership equational logic, 34
 mergesort, 48
 parametric, 55
 message delay, 290
 message passing, 159
 message wrapper, 188

MESSAGE-LOSS, 191
 MESSAGE-LOSS-DUPLICATION, 191
 MESSAGE-WRAPPER, 188
 metric temporal logic, 292
 monotonic, 74
 MSET-INT, 44
 MULTICAST, 189
 multicast, 188
 multiset, 44
 multiset comparison, 76
 multiset path order (mpo), 80
 mutual exclusion, 221
 central server algorithm, 223
 Maekawa's voting algorithm, 222, 225
 temporal logic model checking of, 277
 token ring algorithm, 222, 225

N

NAT, 37
 NAT-ADD, 13
 NAT-EXP, 24
 NAT-MULT, 23
 NAT<, 15
 natural numbers, 13
 Needham-Schroeder protocol (NSPK), 1,
 233, 235
 attack on, 245
 Lowe's correction, 248
 Newman's Lemma, 86
 no confusion, 123
 no junk, 123
 nonce, 235
 nondeterminism, 128
 nontermination, 72
 Toyama's example, 73
 normal form, 21, 63
 NP-complete problem, 49
 Clique, 50
 Hamiltonian Circuit, 50, 52
 Integer Knapsack, 54
 Knapsack, 50
 Multiprocessor Scheduling, 50
 Partition, 50
 Satisfiability, 49
 Subgraph Isomorphism, 50
 Subset Sum, 50, 51, 57
 Traveling Salesman, 50, 54, 134
 NSPK, 237

O

object, 155
 creation and deletion, 158
 identifier, 164
 object-oriented module, 163
 ONE-PERSON, 133
 one-sorted, 59
 one-step concurrent rewrite, 141
 OO-POPULATION, 160
 operational semantics, 18, 59
 operator, 16
 operator attribute
 assoc, 42
 comm, 41
 ctor, 13
 ditto, 37
 format, 58
 frozen, 144
 id:, 43
 prec, 15
 special, 36
 strat, 57
 optimal proof system, 102
 order-sorted specification, 29
 overloaded, 29
 owise, 57

P

PARAM-SORT, 55
 parameterized module, 54
 partial order, 300
 past temporal operator, 282
 POPULATION, 137, 165
 position, 60
 powerset, 299
 precedence, 15, 79
 prelude.maude, 35
 preregular, 30
 probabilistic rewrite theory, 294
 to ordinary rewrite theory, 295
 probabilistic system, 293
 probabilistic temporal logic, 296
 process failure, 219
 Byzantine failure, 219
 crash failure, 218
 fault injection, 220
 recovery, 219
 protocol, 188
 public-key cryptography, 233, 234
 RSA algorithm, 234

Q

quicksort, 47

R

RANDOM, 40
 random numbers, 40
 RAT, 38
 reactive system, 127
 Real-Time Maude, 292
 real-time system, 283
 in rewriting logic, 284
 reduces, 59
 reducible, 63
 reduction, 59
 reduction sequence, 19, 63
 reduction step, 62
 relation, 300
 renaming, 87
 replicated databases, 212
 requirement specification, 249
 rew, 145, 147, 148
 rewrite condition, 131
 rewrite rule, 130
 rewrite theory, 131
 rewriting, 59
 rewriting logic, 127, 130
 concurrent steps, 140
 confluence, 142
 deduction rules, 140
 execution, 145
 model, 144
 semantics, 144
 sequent, 130
 specification, 131
 termination, 142
 run, 142

S

search, 150
 show path, 152
 search, 145, 150, 168
 self-embedding, 77
 separation problem, 185
 SEQNO-UNORDERED, 201
 sequence number, 200
 sequent, 94, 139
 sequential rewrite, 141
 shared variable, 184, 197
 signature
 many-sorted, 16
 order-sorted, 29

simplification, 59
simplification order, 76, 78, 81
simplification step, 62
simulation, 147
 randomized, 176, 293
sliding window protocol, 206
 with links, 209
sort, 12, 16
spacecraft, 4
starvation, 173
state proposition, 252
state-based property, 251
statistical model checking, 293
 PVESTA, 297
 QUATEX properties, 297
strict partial order, 75, 300
 well-founded, 75
STRING, 39
strings, 39
subclass, 165
subsort, 29
 semantic, 33
substitution, 61
subterm, 60
 proper, 61
sufficient completeness, 22
symmetric-key cryptography, 235

T

temporal logic of rewriting, 281
temporal properties, 252
 guarantee, 254
 invariance, 253
 analysis, 260
 reachability, 255
 response/reactivity, 256
 stability, 256

 until, 257
termination, 20, 67
 operational, 65
 undecidability, 68
 weakly terminating, 67
 weight function, 74, 81
terms, 17
tick rewrite rule, 284
TOTAL-ORDER, 55
tracing, 19, 58, 148
transaction, 211
 distributed, 211
transport protocol, 199
Turing machine, 68, 135
 configuration, 69
 universal Turing machine, 72
two-phase commit protocol (2PC), 211, 212
 with process failures, 220

U

unicast, 185
unification algorithm, 87
unifier, 87
 most general, 87
uniform distribution, 295
unsorted, 59

V

variable, 13, 17
 declaration, 17
variable assignment, 116
variable substitution, 61
view, 55

W

web browser attack, 5
whiteboard game, 134